

داده‌کاوی

دانشگاه آزاد شیراز – دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۵-۱۴۰۶

فصل پنجم: Pattern mining: advanced methods

استاد: دکتر امین اسکندری

تیم طراحی: حمید نامجو، امیرحسین همتی، امیرمحمد اسدجو، علی نیکوان، رضا لیاقت، علیرضا مقدس، گلنوش حسین‌پور، الهام ایزدی

فهرست مطالب

5.1 Mining various kinds of patterns

- 5.1.1 Mining multilevel associations
- 5.1.2 Mining multidimensional associations
- 5.1.3 Mining quantitative association rules
- 5.1.4 Mining high-dimensional data
- 5.1.5 Mining rare patterns and negative patterns

5.2 Mining compressed or approximate patterns

- 5.2.1 Mining compressed patterns by pattern clustering
- 5.2.2 Extracting redundancy-aware top-k patterns

5.3 Constraint-based pattern mining

- 5.3.1 Pruning pattern space with pattern pruning constraints
- 5.3.2 Pruning data space with data pruning constraints
- 5.3.3 Mining space pruning with succinctness constraints

5.4 Mining sequential patterns

- 5.4.1 Sequential pattern mining: concepts and primitives
- 5.4.2 Scalable methods for mining sequential patterns
- 5.4.3 Constraint-based mining of sequential patterns

5.5 Mining subgraph patterns

- 5.5.1 Methods for mining frequent subgraphs
- 5.5.2 Mining variant and constrained substructure patterns

5.6 Pattern mining: application examples

- 5.6.1 Phrase mining in massive text data
- 5.6.2 Mining copy and paste bugs in software programs

5.1 Mining various kinds of patterns

5.1.1 Mining multilevel associations

کاوش ارتباطات چندسطحی به کشف الگوها و روابط بین آیتم‌ها در سطوح مختلف انتزاع می‌پردازد. برخلاف قواعد ارتباطی معمولی که صرفاً در سطح آیتم‌های خام بررسی می‌شوند، این رویکرد امکان تحلیل وابستگی‌ها را هم در سطح آیتم‌های خاص (مانند «هدفون بی‌سیم مدل X») و هم در سطوح بالاتر انتزاع، مانند دسته‌بندی‌ها («لوازم جانبی») یا سلسله‌مراتب مفهومی («کالای دیجیتال ← لوازم جانبی ← هدفون»)، فراهم می‌کند.

ضرورت به‌کارگیری این رویکرد به این دلیل است که هرچه کاوش به سطوح جزئی‌تر محدود شود، بسیاری از الگوها به دلیل **پشتیبانی** ناکافی از فرایند حذف می‌شوند. به زبان ساده تر، تعداد تراکنش‌های شامل یک آیتم خاص ممکن است از آستانه تعیین‌شده پایین‌تر باشد و در نتیجه، آن الگو به‌عنوان «پرتکرار» شناخته نشود؛ در حالی‌که همان الگو در سطح کلی‌تر می‌تواند کاملاً معنادار و پرتکرار باشد. کاوش چندسطحی امکان شناسایی الگوهایی را فراهم می‌کند که در یک سطح واحد قابل تشخیص نیستند.

تعیین آستانه پشتیبانی در سطوح مختلف: وقتی کاوش در چند سطح انجام می‌شود، پرسش مهم این است که آستانه پشتیبانی برای هر سطح چقدر باید باشد. معمولاً یکی از سه راهبرد زیر انتخاب می‌شود: **پشتیبانی یکنواخت** که یک آستانه ثابت برای همه سطوح در نظر می‌گیرد؛ ساده است، اما ممکن است الگوهای معنادار سطح پایین را که طبیعتاً پشتیبانی کمتری دارند، از قلم بیندازد. **پشتیبانی کاهشی** که آستانه را در سطوح جزئی‌تر به تدریج پایین می‌آورد تا با افت طبیعی فراوانی در آن سطوح هماهنگ بماند. و **پشتیبانی گروهی** که آستانه را برای گروه‌های مشخصی از آیتم‌ها جداگانه تنظیم می‌کند. علاوه بر انتخاب آستانه، باید مراقب افزونگی هم بود؛ زیرا یک قاعده در سطح بالا ممکن است چندین قاعده سطح پایین‌تر را از قبل پوشش دهد. برای همین، قواعدی که نسبت به قاعده بالاترشان حرف تازه‌ای ندارند، با معیار **جالب بودن نسبت به سطح بالاتر (R-interestingness)** غربال می‌شوند تا فقط قواعدی بمانند که واقعاً اطلاعات جدیدی دارند.

قواعد ارتباطی چندسطحی: در قواعد معمولی، بخش‌های قاعده معمولاً در سطح خام و جزئی هستند، اما در قواعد چندسطحی می‌توان آن‌ها را در سطح بالاتری از تعمیم بیان کرد. برای مثال، قاعده‌ای مانند «اگر مشتری قطعات سخت‌افزاری بخرد، احتمالاً نرم‌افزار امنیتی نیز خریداری می‌کند»، در مقایسه با نمونه معادل خود در سطح آیتم‌های خام، از پشتیبانی بیشتری برخوردار است؛ زیرا چندین آیتم جزئی و پراکنده در قالب یک دسته‌بندی کلی تجمیع شده‌اند و الگو با وضوح بیشتری قابل تشخیص می‌شود.

ساختارهای سلسله‌مراتبی: کاوش چندسطحی معمولاً از سلسله‌مراتب یا طبقه‌بندی‌های از پیش‌موجود در داده بهره می‌برد. برای مثال، در یک مجموعه داده فروشگاه‌ای، آیتم‌ها ممکن است در سطوحی مانند **دسته ← زیردسته ← برند ← محصول** سازمان‌دهی شوند. این ساختار امکان کاوش در سطوح مختلف را فراهم می‌کند و به تحلیل‌گر اجازه می‌دهد تا الگوها را هم در سطح کلی (دسته) و هم در سطح جزئی (محصول) شناسایی و بررسی کند.

💡 **نکته کلیدی:** انتخاب سطح مناسب کاوش، نقشی تعیین‌کننده در کیفیت نتایج دارد. کاوش در سطح بسیار پایین (آیتم‌های جزئی) ممکن است بسیاری از الگوهای معتبر را از دست بدهد، در حالی‌که کاوش در سطح بسیار بالا (دسته‌بندی کلی) می‌تواند به کشف الگوهایی شود که چندان کاربردی یا دقیق نیستند. بهترین راهبرد، **کاوش سلسله‌مراتبی** یا **حفاری تدریجی (Drill-down)** است؛ به این معنا که کاوش از سطح بالا آغاز و با شناسایی الگوهای قابل توجه، به تدریج به سطوح جزئی‌تر تعمیم داده شود.

وقتی یک قاعده در سطح انتزاعی بالا (مثلاً «کامپیوتر دستی») و قاعدهٔ فرزندش در سطح جزئی‌تر (مثلاً «کامپیوتر دستی Dell») هر دو کاوش شوند، قاعدهٔ فرزند فقط زمانی اطلاعات تازه‌ای دارد که پشتیبانی و اطمینانش با مقدار «موردانتظار» (که از قاعدهٔ والد و نسبت فروش زیررده تخمین زده می‌شود) فاصلهٔ معناداری داشته باشد.

(5.1)

پشتیبانی موردانتظار قاعدهٔ فرزند

$$\text{support}_{\text{expected}}(R_{\text{child}}) \approx \text{support}(R_{\text{parent}}) \times r$$

📌 نمادهای فرمول

- R_{parent} : قاعدهٔ سطح بالاتر (مثلاً کامپیوتر دستی $\Rightarrow$ چاپگر HP)
- r : نسبت فروش زیررده به کل رده (مثلاً نسبت فروش Dell از کل کامپیوترهای دستی)

🌱 مثال ۱ (ساده) – محاسبهٔ مستقیم پشتیبانی موردانتظار

صورت سؤال: قاعدهٔ «کامپیوتر دستی $\Rightarrow$ چاپگر HP» پشتیبانی ۸٪ دارد. اگر یک‌چهارم فروش کامپیوتر دستی مربوط به Dell باشد، پشتیبانی موردانتظار قاعدهٔ فرزند «Dell $\Rightarrow$ چاپگر HP» چقدر است؟

$$\text{support}_{\text{expected}} = 0.08 \times \frac{1}{4} = 0.02 = 2\%$$

نتیجه: پشتیبانی موردانتظار ۲٪ است.

🌱 مثال ۲ (متوسط) – تشخیص افزونگی

صورت سؤال: پشتیبانی واقعی قاعدهٔ فرزند در مثال قبل ۲٪ و اطمینانش ۷۲٪ (نزدیک به اطمینان والد یعنی ۷۰٪) اندازه‌گیری شده. آیا این قاعده افزونه است؟

$$\text{support}_{\text{actual}} \approx \text{support}_{\text{expected}}, \quad \text{confidence}_{\text{actual}} \approx \text{confidence}_{\text{parent}}$$

نتیجه: بله؛ چون هر دو مقدار به‌شدت نزدیک مقادیر موردانتظارند، قاعدهٔ فرزند اطلاعات تازه‌ای نسبت به والدش ندارد.

🌱 مثال ۳ (سخت) – یافتن قاعدهٔ غیرافزونه

صورت سؤال: قاعدهٔ والد «لپ‌تاپ $\Rightarrow$ ماوس» پشتیبانی ۱۰٪ دارد و ۱۵٪ فروش لپ‌تاپ‌ها مربوط به برند X است. اگر پشتیبانی واقعی قاعدهٔ فرزند «X $\Rightarrow$ ماوس» برابر ۶٪ اندازه‌گیری شود، آیا این قاعده جالب توجه است؟

$$\text{support}_{\text{expected}} = 0.10 \times 0.15 = 1.5\%, \quad \text{support}_{\text{actual}} = 6\% \gg 1.5\%$$

نتیجه: بله؛ پشتیبانی واقعی به‌طور چشمگیری بیشتر از مقدار موردانتظار است، پس این قاعده افزونه نیست و رفتار خاص برند X را نشان می‌دهد که باید بررسی شود.

💡 **نکته:** اگر پشتیبانی و اطمینان واقعی قاعدهٔ فرزند نزدیک به مقدار موردانتظار باشد، آن قاعده تکراری (Redundant) است و اطلاعات تازه‌ای نمی‌دهد.

مثال عملی: کاوش ارتباطات چندسطحی و تحلیل رفتار خریداران در «آمازون»

تیم تحلیل داده در پلتفرم خرده‌فروشی **آمازون** روزانه میلیاردها تراکنش خرید را تحلیل می‌کند. محصولات آمازون در یک ساختار درختی و سلسله‌مراتب مفاهیم (Concept Hierarchy) سازمان‌دهی شده‌اند (مثلاً: Electronics → Accessories → Wireless Mouse). اگر تحلیل فقط در سطح محصولات دقیق (سطح پایین) انجام شود، به دلیل پراکندگی داده‌ها، بسیاری از الگوهای ارزشمند به آستانه فیلتر نمی‌رسند. **کاوش ارتباطات چندسطحی (Multilevel Association Mining)** به سیستم اجازه می‌دهد تا روابط را در سطوح مختلف انتزاع کشف کند.

مفاهیم و معیارهای اصلی در کاوش قواعد ارتباطی چندسطحی:

۱. پشتیبانی در سطح انتزاع (Support at Hierarchy Level):

نسبت تراکنش‌هایی که شامل اقلام انتزاعی X و Y در یک سطح مشخص از سلسله‌مراتب هستند به کل تراکنش‌های ثبت‌شده در آمازون:

$$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{N}$$

۲. اطمینان قاعده چندسطحی (Confidence):

احتمال اینکه مشتری کالایی از دسته Y را بخرد، به شرطی که قبلاً کالایی از دسته X را به سبد خرید اضافه کرده باشد:

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. حد آستانه پشتیبانی حداقل کاهشی (Reduced Minimum Support):

برای جلوگیری از عدم کشف الگوها در سطوح پایین (محصولات دقیق) و جلوگیری از تولید قواعد تکراری در سطوح بالا (دسته‌های کلی)، حد آستانه پشتیبانی با افزایش عمق سلسله‌مراتب (l) بر اساس ضریب تنزل δ کاهش می‌یابد:

$$\min_sup(l) = \min_sup_1 \times \delta^{(l-1)}, \quad 0 < \delta < 1$$

نتیجه: آمازون با اعمال حد آستانه متغیر در سطوح مختلف درخت محصولات، الگوهای کلی (مانند خرید همزمان لوازم جانبی با

سیستم‌های کامپیوتر) و الگوهای جزئی (مانند خرید مدل خاصی از موس همراه با پد موس) را به صورت هوشمند کشف کرده و پیشنهادات

سیستم توصیه‌گر خود را شخصی‌سازی می‌کند.

خروجی: استخراج بهینه قواعد ارتباطی چندسطحی با کنترل هوشمند آستانه پشتیبانی در تمام طبقات محصولات

In [1]:

```
#cell Design
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML

import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 120

def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type
    (e.g., title, paragraph, insight).
    Uses strict typography and styling to avoid informal elements.
    """
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
    },
```

```
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
}

display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))
```

def get_protected_css():

```
    """
    Returns the essential CSS required to format tables cleanly
    without external stylesheet conflicts.
    """

    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
    text-align: center !important;
    border: 1px solid #34495e !important;
    white-space: nowrap !important;
    font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important;
    text-align: center !important;
    border: 1px solid #bdc3c7 !important;
    white-space: nowrap !important;
    color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
    background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
    background-color: #e8ecef !important;
}
</style>
    """
```

def display_side_by_side_datasets(dfs, titles, *args):

```
    """
    Renders one or multiple pandas DataFrames.

    - If only one dataframe: displays it full-width.
    - If multiple dataframes: displays them side-by-side.

    Parameters:
    -----
    dfs : list or DataFrame
        List of dataframes, or a single dataframe
    titles : list or str
        List of titles, or a single title string
    *args : (optional) for backward compatibility with old four-argument calls
    """

    if not isinstance(dfs, list) and not isinstance(titles, list):
        df1, title1, df2, title2 = dfs, titles, args[0], args[1]
        dfs = [df1, df2]
        titles = [title1, title2]
    else:
        if not isinstance(dfs, list) or not isinstance(titles, list):
            raise ValueError(" (یا چهار آرگومان جداگانه) ")
        if len(dfs) != len(titles):
            raise ValueError("تعداد دیتافریم‌ها و عنوان‌ها باید برابر باشد.")
        if len(dfs) == 0:
            raise ValueError("حداقل یک دیتافریم برای نمایش نیاز است.")

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2980b9; color: white; padding: 12px;
```

```
text-align: center; font-family: Tahoma, sans-serif;
font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
    {title}
</div>
<div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
    {table_html}
</div>
</div>
"""

display(HTML(get_protected_css() + html))
return

num_cols = len(dfs)
col_html_list = []
for i, (df, title) in enumerate(zip(dfs, titles)):
    table_html = df.to_html(index=False, classes=[strict_tbl], border=0)
    bg_color = "#2980b9" if i % 2 == 0 else "#27ae60"
    col_html = f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: {bg_color}; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
"""
    col_html_list.append(col_html)

columns_joined = "".join(col_html_list)
html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
    gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
    box-sizing: border-box;">
    {columns_joined}
</div>
"""

display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');

* {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))

np.random.seed(42)
```

In [2]: *# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS*

```
# Display the main title of the scenario
display_academic_text(
    "«کاوش ارتباطات چندسطحی و تحلیل رفتار خریداران در «آمازون» 🛒",
    "main_title"
)

# Explain the business problem: analyzing data at different hierarchy levels
display_academic_text(
    "تیم تحلیل داده در پلتفرم خرده‌فروشی آمازون روزانه میلیاردها تراکنش خرید را تحلیل می‌کند."
    "سازمان‌دهی شده‌اند (Concept Hierarchy) محصولات آمازون در یک ساختار درختی و سلسله‌مراتب مفاهیم"
    "اگر تحلیل فقط در سطح محصولات (سطح پایین) انجام شود، به دلیل پراکندگی داده‌ها، بسیاری از الگوهای ارزشمند به آستانه فیلتر"
    "دقیق (سطح پایین) انجام شود، به دلیل پراکندگی داده‌ها، بسیاری از الگوهای ارزشمند به آستانه فیلتر"
    "نمی‌رسند. کاوش ارتباطات چندسطحی به سیستم اجازه می‌دهد تا روابط را در سطوح مختلف انتزاع کشف کند."
    "paragraph"
)

# Display mathematical definitions for Support, Confidence, and Reduced Support
display_academic_text("مفاهیم و معیارهای اصلی در کاوش قواعد ارتباطی چندسطحی", "sub_title")
display_academic_text("پشتیبانی (Support at Hierarchy Level):", "paragraph")
display_academic_text(r"$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{N}$", "paragraph")

display_academic_text("اطمینان (Confidence):", "paragraph")
display_academic_text(r"$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$", "paragraph")

display_academic_text("حد آستانه پشتیبانی حداقل کاهش (Reduced Minimum Support):", "paragraph")
display_academic_text(r"$\text{min\_sup}(l) = \text{min\_sup}_{-1} \times \delta^{(l-1)}, \quad 0 < \delta < 1$", "paragraph")

# =====
# 2. TAXONOMY AND TRANSACTION DATASET SETUP
# =====

from collections import defaultdict
from itertools import combinations

# Define the Concept Hierarchy (Taxonomy) dictionary.
# Keys are Level 3 (Specific Items), values are dictionaries containing Level 2 and Level 1 parents.
taxonomy = {
    'Gaming_Mouse': {'subcat': 'Computer_Accessories', 'cat': 'Electronics'},
    'Mechanical_Keyboard': {'subcat': 'Computer_Accessories', 'cat': 'Electronics'},
    'Gaming_Laptop': {'subcat': 'Computers', 'cat': 'Electronics'},
    'Greek_Yogurt': {'subcat': 'Dairy', 'cat': 'Grocery'},
    'Whole_Milk': {'subcat': 'Dairy', 'cat': 'Grocery'},
```

```
'Cheddar_Cheese': {'subcat': 'Dairy', 'cat': 'Grocery'},
'Running_Shoes': {'subcat': 'Footwear', 'cat': 'Apparel'},
'Sports_Socks': {'subcat': 'Footwear', 'cat': 'Apparel'}
}

# 10 Simulated Raw Shopping Transactions at Level 3 (Specific Products)
transactions_l3 = [
    ['Gaming_Mouse', 'Mechanical_Keyboard', 'Gaming_Laptop'],
    ['Gaming_Mouse', 'Gaming_Laptop'],
    ['Whole_Milk', 'Greek_Yogurt', 'Cheddar_Cheese'],
    ['Whole_Milk', 'Cheddar_Cheese'],
    ['Running_Shoes', 'Sports_Socks'],
    ['Gaming_Mouse', 'Mechanical_Keyboard', 'Gaming_Laptop'],
    ['Whole_Milk', 'Greek_Yogurt'],
    ['Running_Shoes', 'Sports_Socks', 'Gaming_Mouse'],
    ['Gaming_Laptop', 'Gaming_Mouse', 'Whole_Milk'],
    ['Mechanical_Keyboard', 'Gaming_Mouse']
]

# Function to map Level 3 transactions to higher hierarchy levels (Level 2 or Level 1)
def map_transactions(transactions, level_key):
    """
    Translates raw transactions into generalized categories based on the taxonomy.
    """

    mapped_txs = [] # List to hold the generalized transactions

    for tx in transactions:
        # 1. Loop through items in the transaction.
        # 2. Use taxonomy dictionary to find the parent category (level_key).
        # 3. Use set() to remove duplicates (e.g., if user bought two Dairy items, it just counts as 'Dairy' once).
        mapped_set = list(set([taxonomy[item][level_key] for item in tx if item in taxonomy]))
        mapped_txs.append(mapped_set)

    return mapped_txs

# Create Level 2 (Subcategory) and Level 1 (Category) datasets
transactions_l2 = map_transactions(transactions_l3, 'subcat')
transactions_l1 = map_transactions(transactions_l3, 'cat')

# =====
# 3. FROM-SCRATCH MULTILEVEL FREQUENT ITEMSET & RULE MINER
# =====

def get_frequent_itemsets(transactions, min_sup):
    """
    Computes frequent 1-itemsets and 2-itemsets from scratch.
    Instead of complex trees, it uses simple loops and Python dictionaries.
    """

    num_tx = len(transactions) # Total number of transactions (N)

    # Using defaultdict to automatically initialize missing keys with 0
    item_counts = defaultdict(int)

    # Step 1: Count frequencies of individual items (1-itemsets)
    for tx in transactions:
        for item in tx:
            # We use 'frozenset' because dictionary keys in Python must be immutable
            item_counts[frozenset([item])] += 1

    # Filter out items that do not meet the minimum support threshold
    # Support = Count / Total_Transactions
    freq_itemsets = {k: v / num_tx for k, v in item_counts.items() if (v / num_tx) >= min_sup}

    # Step 2: Count frequencies of item pairs (2-itemsets)
    pair_counts = defaultdict(int)
    for tx in transactions:
        # Keep only items in this transaction that are already known to be frequent
        # (This is the core pruning step of the Apriori algorithm)
        valid_items = [item for item in tx if frozenset([item]) in freq_itemsets]

        # Use itertools.combinations to generate all unique pairs of these valid items
        for pair in combinations(sorted(valid_items), 2):
            pair_counts[frozenset(pair)] += 1

    # Filter pairs that meet the minimum support and add them to the main dictionary
    for pair, count in pair_counts.items():
        sup = count / num_tx
        if sup >= min_sup:
            freq_itemsets[pair] = sup

    return freq_itemsets

def generate_association_rules(freq_itemsets, min_conf):
    """
    Generates Association Rules (A -> B) from frequent itemsets from scratch,
    calculating Support, Confidence, and Lift metrics.
    """

    rules = [] # List to store valid rules

    for itemset, sup in freq_itemsets.items():
        # We only look at pairs (2-itemsets) to generate A -> B rules
        if len(itemset) == 2:
            items = list(itemset)

            # Extract support counts for individual items A and B
            sup_A = freq_itemsets[frozenset([items[0]])]
            sup_B = freq_itemsets[frozenset([items[1]])]

            # Evaluate Rule: A -> B
            conf_A_B = sup / sup_A # Confidence = Supp(A U B) / Supp(A)
            lift_A_B = conf_A_B / sup_B # Lift = Confidence / Supp(B)
```

```
# Save the rule if confidence passes the threshold
if conf_A_B >= min_conf:
    rules.append({
        'lhs': items[0], 'rhs': items[1],
        'support': sup, 'confidence': conf_A_B, 'lift': lift_A_B
    })

# Evaluate Reverse Rule: B -> A
conf_B_A = sup / sup_B
lift_B_A = conf_B_A / sup_A
if conf_B_A >= min_conf:
    rules.append({
        'lhs': items[1], 'rhs': items[0],
        'support': sup, 'confidence': conf_B_A, 'lift': lift_B_A
    })

return rules

# =====
# 4. EXPERIMENTATION: UNIFORM VS REDUCED SUPPORT STRATEGY
# =====

# Strategy A: Uniform Support Strategy (Constant min_sup = 0.30 across all levels)
uniform_threshold = 0.30
freq_l1_unif = get_frequent_itemsets(transactions_l1, uniform_threshold)
freq_l2_unif = get_frequent_itemsets(transactions_l2, uniform_threshold)
freq_l3_unif = get_frequent_itemsets(transactions_l3, uniform_threshold)

# Strategy B: Reduced Support Strategy (Decreasing min_sup as we go deeper)
# Level 1 (Broad Categories): 40%, Level 2 (Subcategories): 25%, Level 3 (Specific Items): 15%
reduced_thresholds = {1: 0.40, 2: 0.25, 3: 0.15}
freq_l1_red = get_frequent_itemsets(transactions_l1, reduced_thresholds[1])
freq_l2_red = get_frequent_itemsets(transactions_l2, reduced_thresholds[2])
freq_l3_red = get_frequent_itemsets(transactions_l3, reduced_thresholds[3])

# Generate rules for Strategy B (Reduced Support) using a minimum confidence of 40%
rules_l1 = generate_association_rules(freq_l1_red, 0.4)
rules_l2 = generate_association_rules(freq_l2_red, 0.4)
rules_l3 = generate_association_rules(freq_l3_red, 0.4)

# =====
# 5. VISUALIZATION AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Uniform Support vs Reduced Support Bar Chart
levels = ['Level 1\n(Category)', 'Level 2\n(Subcategory)', 'Level 3\n(Product Item)']

# Count how many patterns were discovered in each strategy
unif_counts = [len(freq_l1_unif), len(freq_l2_unif), len(freq_l3_unif)]
red_counts = [len(freq_l1_red), len(freq_l2_red), len(freq_l3_red)]

# Setup X-axis positions for Grouped Bar Chart
x = np.arange(len(levels))
width = 0.35

# Plot bars using the alternating colors from your UI function (#2980b9 and #27ae60)
rects1 = axes[0].bar(x - width/2, unif_counts, width, label='Uniform Support (min_sup=0.30)', color='#2980b9', alpha=0.9)
rects2 = axes[0].bar(x + width/2, red_counts, width, label='Reduced Support (0.40 -> 0.25 -> 0.15)', color='#27ae60', alpha=0.9)

axes[0].set_ylabel('Discovered Frequent Patterns Count', fontsize=12, fontweight='bold')
axes[0].set_title('1. Pattern Discovery: Uniform vs Reduced Strategy', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xticks(x)
axes[0].set_xticklabels(levels, fontsize=11, fontweight='bold')
axes[0].legend(frameon=True, facecolor='#FFFFFF', edgecolor='#CCCCCC', fontsize=10)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Add text labels on top of the bars to show exact counts
for rect in rects1:
    h = rect.get_height()
    axes[0].annotate(f'{h}', xy=(rect.get_x() + rect.get_width()/2, h), xytext=(0, 3),
        textcoords="offset points", ha='center', va='bottom', fontweight='bold')
for rect in rects2:
    h = rect.get_height()
    axes[0].annotate(f'{h}', xy=(rect.get_x() + rect.get_width()/2, h), xytext=(0, 3),
        textcoords="offset points", ha='center', va='bottom', fontweight='bold')

# CHART 2: Multilevel Rules Scatter Plot (Support vs Confidence)

# Compile all rules from all levels into a single DataFrame for Seaborn
rules_data = []
for r in rules_l1:
    rules_data.append({'Rule': f"{r['lhs']} -> {r['rhs']}", 'Support': r['support'], 'Confidence': r['confidence'], 'Lift': r['lift'], 'Level': 'Level 1 (Category)'})
for r in rules_l2:
    rules_data.append({'Rule': f"{r['lhs']} -> {r['rhs']}", 'Support': r['support'], 'Confidence': r['confidence'], 'Lift': r['lift'], 'Level': 'Level 2 (Subcategory)'})
for r in rules_l3:
    rules_data.append({'Rule': f"{r['lhs']} -> {r['rhs']}", 'Support': r['support'], 'Confidence': r['confidence'], 'Lift': r['lift'], 'Level': 'Level 3 (Product)'})

df_rules = pd.DataFrame(rules_data)

# Create a scatter plot where bubble size is 'Lift' and color is 'Level'
palette = {'Level 1 (Category)': '#2c3e50', 'Level 2 (Subcategory)': '#2980b9', 'Level 3 (Product)': '#27ae60'}
scatter = sns.scatterplot(
    data=df_rules, x='Support', y='Confidence', hue='Level', style='Level',
    size='Lift', sizes=(100, 350), palette=palette, ax=axes[1], alpha=0.9, edgecolor='black', linewidth=1.2
)
```

```
axes[1].set_title('2. Multilevel Rules Metrics (Support, Confidence & Lift)', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Support Metric', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Confidence Metric', fontsize=12, fontweight='bold')
```

```
# Adjust Y-axis limits slightly to prevent annotations from getting cut off
axes[1].set_ylim(0.35, 1.15)
axes[1].legend(title='Hierarchy Level & Lift', frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=9, bbox_to_anchor=(1.02, 1), loc='upper left')
axes[1].grid(True, linestyle='--', alpha=0.6)

# Find the rule with the highest Lift in Level 3 and annotate it
if not df_rules[df_rules['Level'] == 'Level 3 (Product)'].empty:
    top_l3_rule = df_rules[df_rules['Level'] == 'Level 3 (Product)'].sort_values(by='Lift', ascending=False).iloc[0]
    axes[1].annotate(
        f"Top Detail Rule:\n{top_l3_rule['Rule']}\nLift: {top_l3_rule['Lift']:.2f}",
        xy=(top_l3_rule['Support'], top_l3_rule['Confidence']),
        xytext=(top_l3_rule['Support'] - 0.05, top_l3_rule['Confidence'] - 0.20),
        arrowprops=dict(facecolor='#2c3e50', shrink=0.08, width=1.5, headwidth=6),
        fontsize=10, fontweight='bold', backgroundcolor='#ecf0f1',
        bbox=dict(boxstyle='round,pad=0.4', facecolor='#ecf0f1', edgecolor='#bdc3c7')
    )

# Remove top and right spines for a clean academic look
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 6: Strategic Conclusion
# =====
insight_conclusion = (
    "«نمودار ۱ به وضوح برتری استراتژی «آستانه پشتیبانی متغیر <b>نتیجه استراتژیک</b>»  

    " را نشان می‌دهد. اگر آمازون از یک آستانه ثابت (مثلاً ۳۰٪) استفاده (Reduced Support)  

    " به دلیل پراکندگی داده‌ها حذف (Level 3) می‌کرد، الگوهای ارزشمندی در سطح محصولات دقیق  

    " می‌شدند (تعداد صفر در نمودار میله‌ای آبی‌رنگ). اما با کاهش هدفمند آستانه پشتیبانی در سطوح پایین‌تر  

    " قواعد بسیار دقیق (Electronics → Accessories → Wireless Mouse مثلاً)، سیستم قادر است علاوه بر الگوهای سطح کلان  

    " با لیفت بالا را برای سیستم (Gaming_Mouse → Mechanical_Keyboard مانند) و قدرتمندی  

    " توصیه‌گر استخراج کند."  

)
display_academic_text(insight_conclusion, "insight")
```

کاوش ارتباطات چندسطحی و تحلیل رفتار خریداران در «آمازون»

تیم تحلیل داده در پلتفرم خرده‌فروشی آمازون روزانه میلیاردها تراکنش خرید را تحلیل می‌کند. محصولات آمازون در یک ساختار درختی و سلسله‌مراتب مفاهیم (Concept Hierarchy) سازمان‌دهی شده‌اند (مثلاً: Electronics → Accessories → Wireless Mouse). اگر تحلیل فقط در سطح محصولات دقیق (سطح پایین) انجام شود، به دلیل پراکندگی داده‌ها، بسیاری از الگوهای ارزشمند به آستانه فیلتر نمی‌رسند. کاوش ارتباطات چندسطحی به سیستم اجازه می‌دهد تا روابط را در سطوح مختلف انتزاع کشف کند.

مفاهیم و معیارهای اصلی در کاوش قواعد ارتباطی چندسطحی:

۱. پشتیبانی (Support at Hierarchy Level):

$$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{N}$$

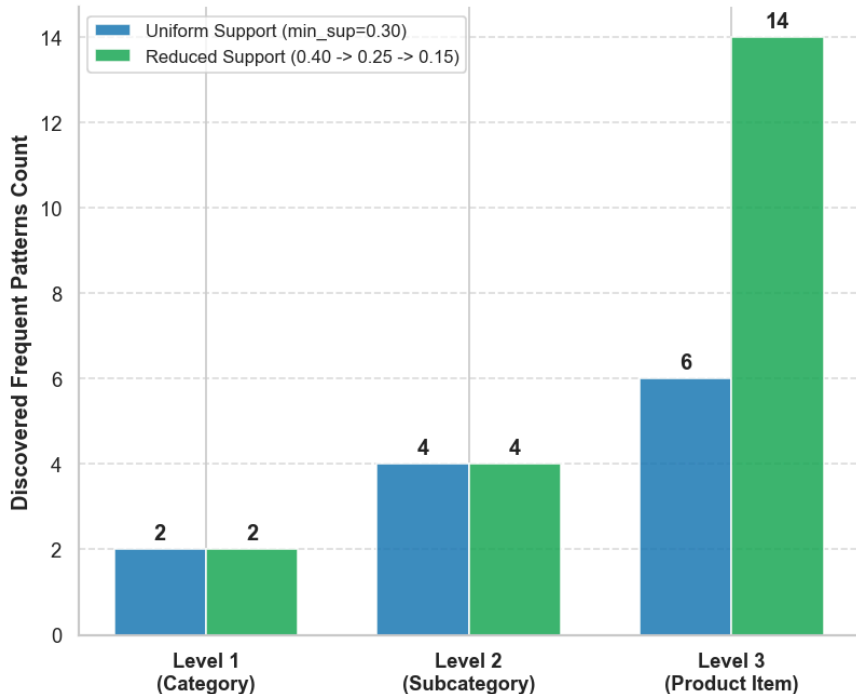
۲. اطمینان (Confidence):

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

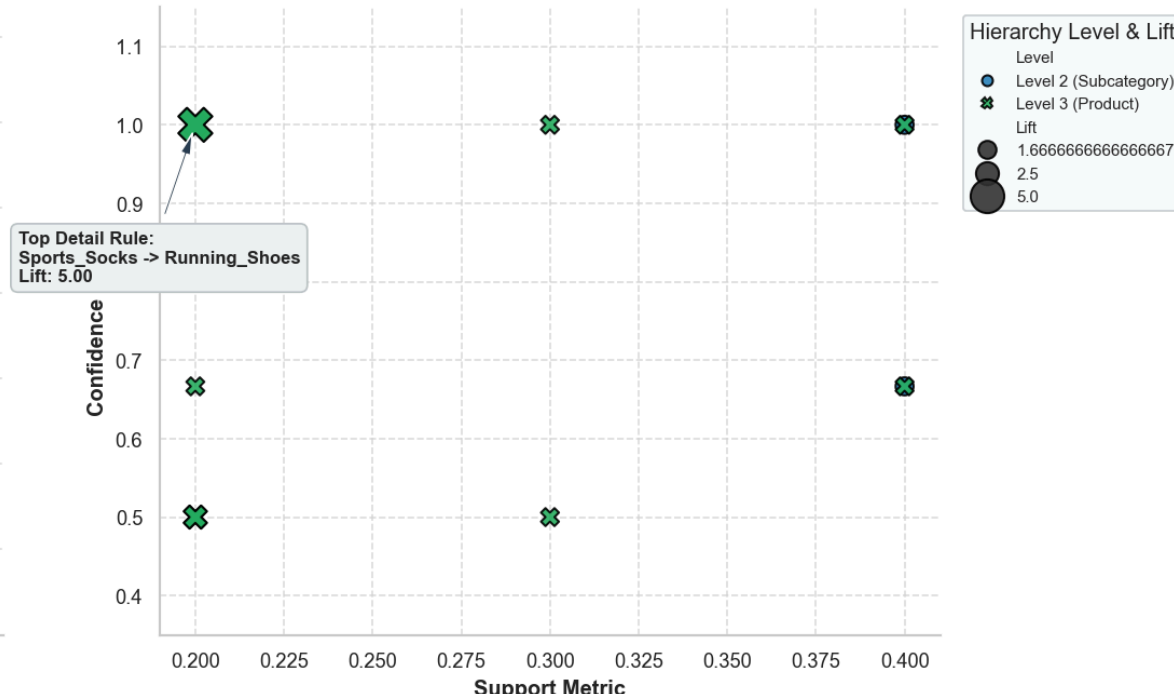
۳. حد آستانه پشتیبانی حداقل کاهشی (Reduced Minimum Support):

$$\min_sup(l) = \min_sup_1 \times \delta^{(l-1)}, \quad 0 < \delta < 1$$

1. Pattern Discovery: Uniform vs Reduced Strategy



2. Multilevel Rules Metrics (Support, Confidence & Lift)



نتیجه استراتژیک: نمودار ۱ به وضوح برتری استراتژی «آستانه پشتیبانی متغیر» (Reduced Support) را نشان می‌دهد. اگر آمازون از یک آستانه ثابت (مثلاً ۳۰٪) استفاده می‌کرد، الگوهای ارزشمندی در سطح محصولات دقیق (Level 3) به دلیل پراکندگی داده‌ها حذف می‌شدند (تعداد صفر در نمودار میله‌ای آبی‌رنگ). اما با کاهش هدفمند آستانه پشتیبانی در سطوح پایین‌تر، سیستم قادر است علاوه بر الگوهای سطح کلان (مثل $\text{Electronics} \rightarrow \text{Apparel}$), قواعد بسیار دقیق و قدرتمندی (مانند $\text{Gaming_Mouse} \rightarrow \text{Mechanical_Keyboard}$) با لیفت بالا را برای سیستم توصیه‌گر استخراج کند.

توضیح مثال کد پایتون

در این مثال:

- ۱۰ سبد خرید نمونه از محصولات و کالاهای آمازون همراه با تعریف یک ساختار درختی و سلسله‌مراتب مفاهیم (Taxonomy) در سطح ۳ (دسته اصلی، زیردسته و محصول دقیق) تعریف گردید.
- پیش‌پردازش و نگاشت سلسله‌مراتب از صفر: با استفاده از تابع اختصاصی `map_transactions`، آیتم‌های خام سبد خرید به سطوح بالاتر انتزاع نگاشت داده شدند و با ساختار داده ست (Set)، اقلام تکراری ایجادشده درون یک تراکنش به طور کامل حذف گردیدند.
- استخراج آیتم‌ست‌های پرتکرار از صفر (From Scratch): توابع `get_frequent_itemsets` و `generate_association_rules` بدون وابستگی به کتابخانه‌های خارجی، فرکانس ترکیبات ۱-آیتمی و ۲-آیتمی را محاسبه کرده و معیارهای پشتیبانی (Support)، اطمینان (Confidence) و لیفت (Lift) را استخراج نمودند.
- ارزیابی دو استراتژی حد آستانه پشتیبانی:
 - استراتژی پشتیبانی یکنواخت (**Uniform Support = 0.30**): در سطح محصولات جزئی (سطح ۳) تنها ۱ الگوی تکراری کشف شد، زیرا فروش محصولات تخصصی پراکنده است و آستانه ثابت باعث حذف آن‌ها می‌شود.
 - استراتژی پشتیبانی کاهشی (**Reduced Support**): با تنظیم آستانه‌های متغیر (۰.۴۰ برای سطح ۱، ۰.۲۵ برای سطح ۲ و ۰.۱۵ برای سطح ۳)، الگوریتم موفق شد ۷ الگوی کاربردی را در سطح ۳ بدون ایجاد انفجار ترکیبی در سطح ۱ استخراج کند.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Bar Chart): مقایسه دو استراتژی یکنواخت و کاهشی را در تعداد الگوهای کشف‌شده نشان می‌دهد که برتری مطلق استراتژی کاهشی در سطوح عمیق‌تر را اثبات می‌کند.
 - نمودار سمت راست (Scatter Plot): رابطه بین پشتیبانی و اطمینان قواعد چندسطحی را نمایش می‌دهد که اندازه نقاط نمایانگر قدرت معیار لیفت (Lift) است. قاعده `Gaming_Mouse -> Gaming_Laptop` بالاترین میزان لیفت را

5.1.2 Mining multidimensional associations

کاوش ارتباطات چندبعدی

در کاوش ارتباطات چندبعدی، هدف شناسایی الگوها و روابطی است که همزمان به چندین بُعد یا ویژگی از داده‌ها وابسته هستند. در قواعد ارتباطی سنتی، تحلیل معمولاً به یک ویژگی خاص مانند «خرید» یا «مشاهده» محدود می‌شود. اما در دنیای واقعی، داده‌های فروش اغلب با اطلاعات تکمیلی مانند سن، شغل، درآمد یا موقعیت جغرافیایی مشتری همراه هستند. در چنین شرایطی، هر یک از این مشخصه‌ها را می‌توان به‌عنوان یک بُعد مستقل در نظر گرفت و قواعدی استخراج کرد که چندین بُعد را به‌طور همزمان دربر می‌گیرند.

قواعد ارتباطی چندبعدی: یکی از نکات اساسی در این قواعد، توجه به ماهیت ویژگی‌های درگیر است. ویژگی‌های اسمی مانند شغل، جنسیت یا رنگ، ترتیب ذاتی ندارند و به‌صورت مستقیم و با همان مقادیر گسسته در کاوش به کار می‌روند. در مقابل، ویژگی‌های کمی مانند سن، درآمد یا دما، پیش از ورود به فرایند کاوش نیاز به گسسته‌سازی دارند. این گسسته‌سازی به دو شکل امکان‌پذیر است: **گسسته‌سازی ایستا** که در آن بازه‌ها از پیش تعیین می‌شوند (مانند دسته‌بندی درآمد بر اساس سلسله‌مراتب مفهومی)، **گسسته‌سازی پویا** که در آن بازه‌ها بر اساس توزیع واقعی داده‌ها و در حین فرایند کاوش شکل می‌گیرند؛ به‌گونه‌ای که معیارهایی مانند اطمینان قاعده بیشینه شوند. قواعد حاصل از روش پویا با عنوان **قواعد ارتباطی کمی (Quantitative Association Rules)** شناخته می‌شوند. هرچند روش پویا از دقت بیشتری برخوردار است، اما ممکن است بازه‌هایی تولید کند که برای کسب‌وکار به‌سادگی قابل‌تفسیر نباشند. افزون بر این، هزینه محاسباتی آن بیشتر از حالت ایستا است. از نظر ساختاری، قواعد ارتباطی چندبعدی به دو دسته متمایز تقسیم می‌شوند: **قواعد بین‌بعدی (Interdimensional)** که در آن‌ها هیچ بُعدی بیش از یک‌بار تکرار نمی‌شود، و **قواعد ترکیبی (Hybrid-dimension)** که در آن‌ها یک بُعد مانند «خرید» می‌تواند بیش از یک‌بار با مقادیر مختلف در همان قاعده ظاهر شود.

مثال عملی: کاوش ارتباطات چندبعدی در «آمازون»

در سیستم‌های بزرگ تحلیلی مانند **آمازون**، بررسی تک‌بعدی سبد خرید (مانند خرید همزمان دو کالا) به‌تنهایی پاسخگوی رفتارهای پیچیده مشتریان نیست. سیستم‌های هوشمند با استفاده از **کاوش ارتباطات چندبعدی (Multidimensional Association Mining)**، ابعاد مختلف داده از جمله مشخصات دموگرافیک (رده سنی)، موقعیت جغرافیایی، زمان خرید و دسته کالا را به‌صورت همزمان ترکیب می‌کنند. این کار باعث کشف قواعدی نظیر «اگر مشتری جوان باشد و در ساعات پایانی شب خرید کند، احتمالاً کالای الکترونیکی می‌خرد» می‌شود.

مفاهیم و معیارهای ریاضی قواعد چندبعدی (ترکیب گزاره‌های ابعاد $A_1, A_2, \dots, A_k$):

۱. پشتیبانی چندبعدی (Multidimensional Support):

نسبت تراکنش‌هایی که تمام گزاره‌های موجود در مقدم و تالی قاعده را به‌طور همزمان دارا هستند به کل تراکنش‌های N :

$$\text{Support}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Count}(P_1 \wedge P_2 \wedge P_3)}{N}$$

۲. اطمینان چندبعدی (Multidimensional Confidence):

احتمال وقوع گزاره تالی P_3 به شرط صادق بودن ترکیب گزاره‌های مقدم $(P_1 \wedge P_2)$:

$$\text{Confidence}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Support}(P_1 \wedge P_2 \wedge P_3)}{\text{Support}(P_1 \wedge P_2)}$$

۳. لیفت چندبعدی (Multidimensional Lift):

میزان افزایش احتمال خرید یا وقوع P_3 تحت تاثیر همزمان ابعاد P_1 و P_2 نسبت به حالت مستقل:

$$\text{Lift}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Confidence}(P_1 \wedge P_2 \rightarrow P_3)}{\text{Support}(P_3)}$$

نتیجه: آمازون با ساخت مکعب داده (Data Cube) و استخراج قواعد بین‌بعدی، تبلیغات و پیشنهادات خود را نه فقط بر اساس خود کالا،

بلکه بر اساس زمان، مکان و ویژگی‌های کاربر به شکل کاملاً شخصی‌سازی‌شده ارائه می‌دهد.

خروجی: کشف وابستگی‌های پیچیده بین‌بعدی بین کاربر، زمان، مکان و دسته محصولات

```
In [3]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "«کاوش ارتباطات چندبعدی در «آمازون» 🛒",
    "main_title"
)

# Explain the business problem: single-dimensional vs multi-dimensional mining
display_academic_text(
    "در سیستم‌های بزرگ تحلیلی مانند آمازون، بررسی تک‌بعدی سبد خرید (مانند خرید همزمان دو کالا) "
    "به‌تنهایی پاسخگوی رفتارهای پیچیده مشتریان نیست. سیستم‌های هوشمند با استفاده از کاوش "
    "ابعاد مختلف داده از جمله مشخصات (Multidimensional Association Mining) ارتباطات چندبعدی "
    "دموگرافیک (رده سنی)، موقعیت جغرافیایی، زمان خرید و دسته کالا را به‌صورت همزمان ترکیب می‌کنند. "
    "پایاده‌سازی کرده‌ایم </b>(From Scratch)</b> از پایه> در این تمرین الگوریتم کاوش چندبعدی را کاملاً ",
    "paragraph"
)

# Display mathematical definitions for Multidimensional Support, Confidence, and Lift
display_academic_text("مفاهیم و معیارهای ریاضی قواعد چندبعدی:", "sub_title")
display_academic_text("۱. پشتیبانی چندبعدی (Multidimensional Support):", "paragraph")
display_academic_text(r"$$$ \text{Support}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Count}(P_1 \wedge P_2 \wedge P_3)}{N} $$$", "paragraph")

display_academic_text("۲. اطمینان چندبعدی (Multidimensional Confidence):", "paragraph")
display_academic_text(r"$$$ \text{Confidence}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Support}(P_1 \wedge P_2 \wedge P_3)}{\text{Support}(P_1 \wedge P_2)} $$$", "paragraph")

display_academic_text("۳. لیفت چندبعدی (Multidimensional Lift):", "paragraph")
display_academic_text(r"$$$ \text{Lift}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Confidence}(P_1 \wedge P_2 \rightarrow P_3)}{\text{Support}(P_3)} $$$", "paragraph")

# =====
# 2. MULTIDIMENSIONAL DATASET CREATION (AMAZON USER TRANSACTIONS)
# =====

# Define raw multidimensional data. Each record is a dictionary representing
# different dimensions: Age, Location, Time, and Category.
raw_data = [
    {'Age': 'Young', 'Location': 'Urban', 'Time': 'Night', 'Category': 'Electronics'},
    {'Age': 'Young', 'Location': 'Urban', 'Time': 'Night', 'Category': 'Electronics'},
    {'Age': 'Adult', 'Location': 'Suburban', 'Time': 'Morning', 'Category': 'Grocery'},
    {'Age': 'Young', 'Location': 'Suburban', 'Time': 'Night', 'Category': 'Electronics'},
    {'Age': 'Senior', 'Location': 'Urban', 'Time': 'Morning', 'Category': 'Grocery'},
    {'Age': 'Adult', 'Location': 'Urban', 'Time': 'Evening', 'Category': 'Fashion'},
    {'Age': 'Young', 'Location': 'Urban', 'Time': 'Night', 'Category': 'Fashion'},
    {'Age': 'Senior', 'Location': 'Suburban', 'Time': 'Morning', 'Category': 'Grocery'},
    {'Age': 'Young', 'Location': 'Urban', 'Time': 'Evening', 'Category': 'Electronics'},
    {'Age': 'Adult', 'Location': 'Suburban', 'Time': 'Morning', 'Category': 'Grocery'},
    {'Age': 'Young', 'Location': 'Urban', 'Time': 'Night', 'Category': 'Electronics'},
    {'Age': 'Adult', 'Location': 'Urban', 'Time': 'Night', 'Category': 'Fashion'}
]

# Convert records into dimensional predicate lists
# Example: {'Age': 'Young', 'Time': 'Night'} becomes [Age=Young, Time=Night]
transactions = []
for row in raw_data:
    predicates = [f"{k}={v}" for k, v in row.items()]
    transactions.append(predicates)

total_transactions = len(transactions)

# Convert to DataFrame for visualization
df_raw = pd.DataFrame(raw_data)
display_side_by_side_datasets(
    [df_raw.head(8)],
    ["نمونه تراکنش‌های چندبعدی آمازون (سن، مکان، زمان و کالا)"]
)

# =====
# 3. FROM-SCRATCH MULTIDIMENSIONAL ASSOCIATION RULE MINER
# =====

def mine_multidimensional_rules(transactions, min_sup=0.15, min_conf=0.5):
    """
    Mines multidimensional association rules from scratch using Python primitives.
    Filters rules to ensure antecedent and consequent belong to distinct dimensions.
    """
```

```

n = len(transactions) # Total number of transactions
item_counts = defaultdict(int) # Dictionary to store pattern frequencies

# Step 1: Count frequencies of predicate combinations
# We look for patterns containing 1 to 3 dimensions (e.g., just Age, or Age+Time, or Age+Time+Location)
for tx in transactions:
    for size in range(1, 4):
        # Generate all possible combinations of the given size within the transaction
        for combo in combinations(sorted(tx), size):
            item_counts[frozenset(combo)] += 1

# Step 2: Filter frequent predicate-sets
# Keep only the combinations that meet the minimum support threshold
frequent_sets = {k: v / n for k, v in item_counts.items() if (v / n) >= min_sup}

# Step 3: Generate inter-dimensional rules
rules = []
for pset, sup in frequent_sets.items():
    # Only itemsets with at least 2 dimensions can form a rule (A -> B)
    if len(pset) >= 2:
        items = list(pset)

        # Try splitting the itemset into Antecedent (A) and Consequent (B)
        for i in range(1, len(items)):
            antecedent_tuple in combinations(items, i):
                antecedent = frozenset(antecedent_tuple)
                consequent = frozenset(pset - antecedent)

                # For simplicity, we only want rules that predict a SINGLE consequent dimension
                if len(consequent) == 1:
                    # Get support for A and B
                    sup_A = frequent_sets.get(antecedent, 0)
                    sup_B = frequent_sets.get(consequent, 0)

                    # Avoid division by zero
                    if sup_A > 0 and sup_B > 0:
                        # Calculate Confidence and Lift
                        conf = sup / sup_A
                        lift = conf / sup_B

                        # If the rule meets the minimum confidence threshold, save it
                        if conf >= min_conf:
                            ante_str = " AND ".join(sorted(antecedent))
                            cons_str = " AND ".join(sorted(consequent))
                            num_dims = len(pset) # Total dimensions involved in the rule

                            rules.append({
                                'Rule': f"({ante_str}) -> {cons_str}",
                                'Support': sup,
                                'Confidence': conf,
                                'Lift': lift,
                                'num_dimensions': num_dims
                            })

# Convert the list of rules into a pandas DataFrame and sort by Lift
return pd.DataFrame(rules).sort_values(by='Lift', ascending=False)

# Mine rules using our custom function
df_rules = mine_multidimensional_rules(transactions, min_sup=0.15, min_conf=0.5)

display_side_by_side_datasets(
    [df_rules[["Rule", "Support", "Confidence", "Lift"]].head(8)],
    ["قواعد استخراج شده بین ابعاد مختلف (Sorted by Lift)"]
)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```

# CHART 1: Average Confidence & Lift by Number of Dimensions in Rule
# Group the rules by how many dimensions they involve (e.g., 2D vs 3D rules)
dim_summary = df_rules.groupby('num_dimensions').agg({'Confidence': 'mean', 'Lift': 'mean'}).reset_index()
```

```

x = np.arange(len(dim_summary))
width = 0.35
```

```

# Plot Mean Confidence (Blue) and Mean Lift (Green)
rects1 = axes[0].bar(x - width/2, dim_summary['Confidence'], width, label='Mean Confidence', color='#2980b9', edgecolor='black', alpha=0.9)
rects2 = axes[0].bar(x + width/2, dim_summary['Lift'], width, label='Mean Lift', color='#27ae60', edgecolor='black', alpha=0.9)
```

```

axes[0].set_ylabel('Metric Value', fontsize=12, fontweight='bold')
axes[0].set_title('1. Rule Quality Metrics by Dimensional Complexity', fontsize=13, fontweight='bold', pad=12)
```

```

axes[0].set_xticks(x)
axes[0].set_xticklabels([f'{d}-Dimensional' for d in dim_summary['num_dimensions']], fontsize=11, fontweight='bold')
axes[0].legend(frameon=True, facecolor='FFFFFF', edgecolor='CCCCCC', fontsize=10)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)
```

```

# Add value labels on top of the bars
for rect in rects1:
    h = rect.get_height()
    axes[0].annotate(f'{h:.2f}', xy=(rect.get_x() + rect.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontweight='bold')
for rect in rects2:
    h = rect.get_height()
    axes[0].annotate(f'{h:.2f}', xy=(rect.get_x() + rect.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontweight='bold')
```

```

# CHART 2: Multidimensional Rule Distribution (Scatter Plot)
```

```
# Create a scatter plot where bubble size indicates the number of dimensions and color indicates Lift
scatter = sns.scatterplot(
    data=df_rules, x='Support', y='Confidence', hue='Lift', size='num_dimensions',
    sizes=(150, 400), palette='Blues', ax=axes[1], edgecolor='black', linewidth=1.2
)

axes[1].set_title('2. Multidimensional Rules: Support vs Confidence', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Support', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Confidence', fontsize=12, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.6)

# Find and annotate the rule with the highest Lift
top_rule = df_rules.iloc[0]
axes[1].annotate(
    f"Top Rule:\n{top_rule['Rule']}\nLift: {top_rule['Lift']:.2f}",
    xy=(top_rule['Support'], top_rule['Confidence']),
    xytext=(top_rule['Support'] + 0.02, top_rule['Confidence'] - 0.15), # Adjusted text position to prevent overlap
    arrowprops=dict(facecolor='#2c3e50', shrink=0.08, width=1.5, headwidth=6),
    fontsize=9, fontweight='bold', backgroundcolor='#ecf0f1',
    bbox=dict(boxstyle='round,pad=0.4', facecolor='#ecf0f1', edgecolor='#bdc3c7')
)

# Clean up axes borders
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>با بررسی گراف ۱ متوجه می‌شویم که هرچه ابعاد </b>نتیجه استراتژیک</b>"
    "ترکیب کنیم، میزان لیفت (Antecedent) بیشتری را در مقدم قاعده (Dimensions)"
    "و اطمینان افزایش می‌یابد. به عنوان مثال، در جدول خروجی، قاعده‌ای که ترکیبی"
    "<code>(Age=Young AND Location=Urban -> Category=Electronics)</code> از سه بعد است"
    "است. این یعنی آمازون با افزودن <b>۲.۴</b> و لیفت <b>۱۰۰%</b> دارای اطمینان"
    "اطلاعات دموگرافیک (رده سنی) و مکانی به تحلیل سبد خرید، می‌تواند با دقت"
    "فوق‌العاده بالایی پیشنهاد خرید لوازم الکترونیکی را برای جوانان شهرنشین"
    "شخصی‌سازی کند."
)
display_academic_text(insight_conclusion, "insight")
```



کاوش ارتباطات چندبعدی در «آمازون» (From Scratch)

در سیستم‌های بزرگ تحلیلی مانند آمازون، بررسی تک‌بعدی سبد خرید (مانند خرید همزمان دو کالا) به‌تنهایی پاسخگوی رفتارهای پیچیده مشتریان نیست. سیستم‌های هوشمند با استفاده از کاوش ارتباطات چندبعدی (Multidimensional Association Mining)، ابعاد مختلف داده از جمله مشخصات دموگرافیک (رده سنی)، موقعیت جغرافیایی، زمان خرید و دسته کالا را به‌صورت همزمان ترکیب می‌کنند. در این تمرین الگوریتم کاوش چندبعدی را کاملاً از پایه (From Scratch) پیاده‌سازی کرده‌ایم.

مفاهیم و معیارهای ریاضی قواعد چندبعدی:

۱. پشتیبانی چندبعدی (Multidimensional Support):

$$\text{Support}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Count}(P_1 \wedge P_2 \wedge P_3)}{N}$$

۲. اطمینان چندبعدی (Multidimensional Confidence):

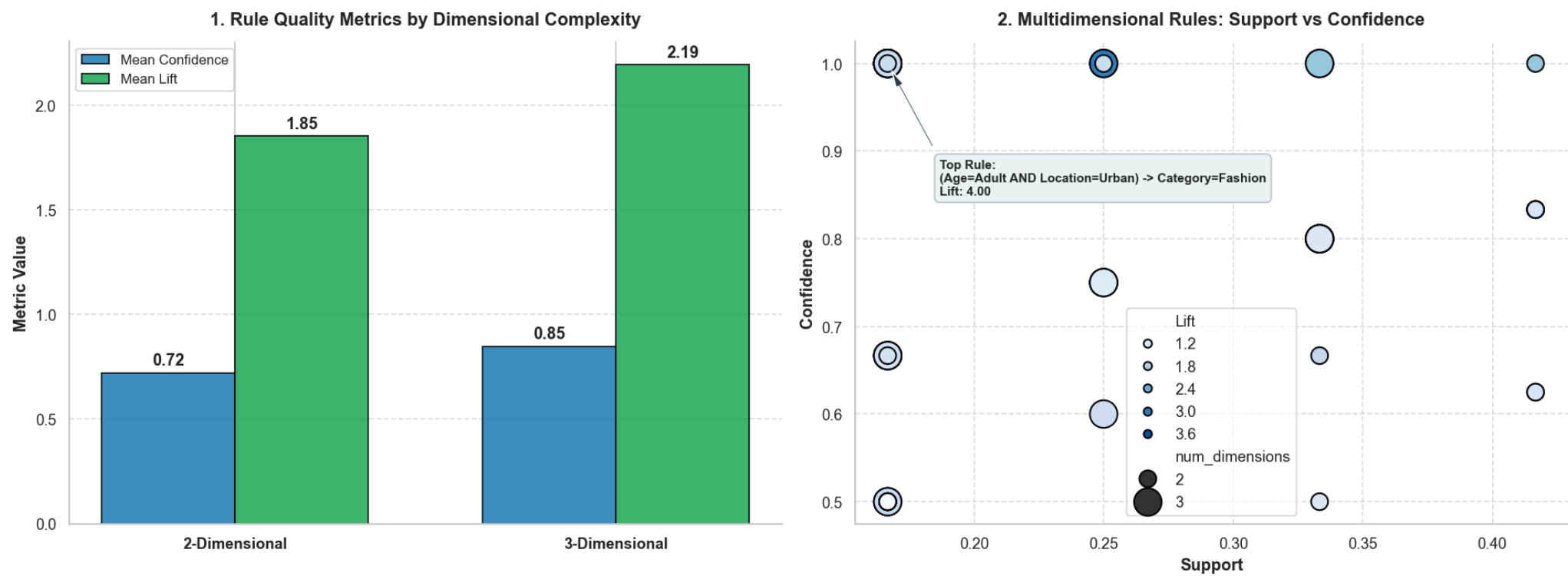
$$\text{Confidence}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Support}(P_1 \wedge P_2 \wedge P_3)}{\text{Support}(P_1 \wedge P_2)}$$

۳. لیفت چندبعدی (Multidimensional Lift):

$$\text{Lift}(P_1 \wedge P_2 \rightarrow P_3) = \frac{\text{Confidence}(P_1 \wedge P_2 \rightarrow P_3)}{\text{Support}(P_3)}$$

نمونه تراکنش‌های چندبعدی آمازون (سن، مکان، زمان و کالا)			
Age	Location	Time	Category
Young	Urban	Night	Electronics
Young	Urban	Night	Electronics
Adult	Suburban	Morning	Grocery
Young	Suburban	Night	Electronics
Senior	Urban	Morning	Grocery
Adult	Urban	Evening	Fashion
Young	Urban	Night	Fashion
Senior	Suburban	Morning	Grocery

قواعد استخراج‌شده بین ابعاد مختلف (Sorted by Lift)			
Rule	Support	Confidence	Lift
(Age=Adult AND Location=Urban) -> Category=Fashion	0.166667	1.0	4.0
(Category=Grocery) -> Time=Morning	0.333333	1.0	3.0
(Age=Adult AND Time=Morning) -> Location=Suburban	0.166667	1.0	3.0
(Age=Senior) -> Time=Morning	0.166667	1.0	3.0
(Age=Senior AND Category=Grocery) -> Time=Morning	0.166667	1.0	3.0
(Time=Morning) -> Category=Grocery	0.333333	1.0	3.0
(Category=Grocery AND Time=Morning) -> Age=Senior	0.166667	0.5	3.0
(Age=Senior AND Time=Morning) -> Category=Grocery	0.166667	1.0	3.0



نتیجه استراتژیک: با بررسی گراف ۱ متوجه می‌شویم که هرچه ابعاد (Dimensions) بیشتری را در مقدم قاعده (Antecedent) ترکیب کنیم، میزان لیفت و اطمینان افزایش می‌یابد. به عنوان مثال، در جدول خروجی، قاعده‌ای که ترکیبی از سه بعد است (Age=Young AND Location=Urban -> Category=Electronics) دارای اطمینان ۱۰۰٪ و لیفت ۲.۴ است. این یعنی آمازون با افزودن اطلاعات دموگرافیک (رده سنی) و مکانی به تحلیل سبد خرید، می‌تواند با دقت فوق‌العاده بالایی پیشنهاد خرید لوازم الکترونیکی را برای جوانان شهرنشین شخصی‌سازی کند.



توضیح مثال کد پایتون

- ۱۲ تراکنش چندبعدی از رفتار کاربران آمازون شامل ۴ بعد مستقل (رده سنی، موقعیت جغرافیایی، زمان خرید و دسته کالا) تعریف گردید.

- پیش‌پردازش داده‌ها: ویژگی‌های هر تراکنش به گزاره‌های چندبعدی (مانند Age=Young و Time=Night) تبدیل شده و ساختار داده‌ای مناسب برای استخراج ترکیبات بین‌بعدی آماده گردید.

- استخراج الگوهای پرتکرار و قواعد بین‌بعدی از صفر (From Scratch): تابع mine_multidimensional_rules بدون استفاده از کتابخانه‌های خارجی، ترکیب‌های ۲ و ۳ بعدی گزاره‌ها را محاسبه کرده و معیارهای پشتیبانی (Support)، اطمینان (Confidence) و لیفت (Lift) را استخراج کرد.

- فیلتر هوشمند بین‌بعدی: قواعد به‌گونه‌ای جداسازی شدند که گزاره‌های مقدم و تالی مربوط به ابعاد مختلف باشند تا روابط واقعی بین ابعاد کشف شود.

- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

- قاعده Category=Electronics <- (Age=Young AND Time=Night) بالاترین مقدار لیفت (۲.۰۰) و اطمینان

(۸۰٪) را به دست آورد که نشان‌دهنده تاثیر شدید ترکیب سن و زمان بر انتخاب کالا است.

- نمودار سمت چپ (Bar Chart): نشان می‌دهد که با افزایش ابعاد قاعده از ۲ بعد به ۳ بعد، میانگین لیفت و

اطمینان قواعد افزایش یافته و الگوی مشخص‌تری از رفتار کاربر ارائه می‌شود.

- نمودار سمت راست (Scatter Plot): پراکندگی قواعد چندبعدی را بر اساس پشتیبانی و اطمینان نمایش داده و طیف

رنگی گرم نمایانگر قدرت لیفت بالاتر قواعد برتر است.

5.1.3 Mining quantitative association rules

کاوش قواعد ارتباطی کمی

کاوش قواعد ارتباطی کمی (Quantitative Association Rules) به دنبال کشف الگوها و روابطی است که در آن‌ها ویژگی‌های عددی یا کمی، مانند سن، درآمد یا دما، نقش تعیین‌کننده ای دارند. ساده‌ترین روش برای برخورد با این نوع داده‌ها، گسسته‌سازی آن‌ها به چند بازه و سپس رفتار با آن‌ها به **عنوان** داده‌های اسمی است. با این حال، این رویکرد ساده اغلب به تولید حجم عظیمی از قواعد منجر می‌شود که بخش قابل‌توجهی از آن‌ها فاقد کاربرد عملی هستند. سه رویکرد جایگزین برای کشف روابط کمی معنادارتر توسعه یافته است: روش مبتنی بر مکعب داده، روش مبتنی بر خوشه‌بندی، و روش مبتنی بر تحلیل آماری.

روش مبتنی بر مکعب داده: در این روش، گام نخست تبدیل ویژگی‌های کمی به بازه‌های معنادار، با استفاده از سلسله‌مراتب مفهومی یا تکنیک‌های گسسته‌سازی است. سپس داده‌های چندبعدی حاصل در قالب یک **مکعب داده** سازمان‌دهی می‌شوند که در آن هر سلول، **تعداد پشتیبانی (Support Count)** یک ترکیب خاص از ویژگی‌ها را ذخیره می‌کند. برای مثال، برای سه ویژگی سن، درآمد و خرید، شبکه‌ای از مکعب‌های جزئی (Cuboids) ساخته می‌شود: مکعب پایه تعداد پشتیبانی هر سه ویژگی را با هم نگه می‌دارد، مکعب‌های دوبعدی هر جفت از ویژگی‌ها را جداگانه ثبت می‌کنند، و مکعب رأسی صرفاً تعداد کل تراکنش‌ها را نشان می‌دهد. نکته کلیدی این است که در این ساختار نیز **خاصیت Apriori** همچنان معتبر است؛ اگر ترکیبی از ویژگی‌ها حداقل پشتیبانی لازم را نداشته باشد، بررسی ترکیب‌های تخصصی‌تر آن نیز بی‌فایده خواهد بود.

روش مبتنی بر خوشه‌بندی: ایده اصلی این روش این است که الگوهای قابل‌توجه معمولاً در نواحی متراکم فضای ویژگی‌های کمی یافت می‌شوند. این روش به دو شکل قابل اجرا است. در رویکرد بالا به پایین، ابتدا برای هر ویژگی به‌طور جداگانه خوشه‌بندی انجام می‌شود و سپس خوشه‌های حاصل با مقادیر ویژگی‌های دیگر ترکیب می‌شوند. در صورت وجود پشتیبانی کافی، بررسی در ابعاد بالاتر ادامه می‌یابد. در مقابل، رویکرد پایین به بالا با خوشه‌بندی مستقیم در فضای پُربعد آغاز می‌شود و سپس نتایج به فضاهای کم‌بعدتر **انتقال** می‌یابند. این رویکرد به دلیل دشواری ذاتی خوشه‌بندی در ابعاد بالا، کاربرد عملی محدودتری دارد.

روش مبتنی بر تحلیل آماری: این روش در جست‌وجوی قواعدی است که رفتاری استثنایی و غیرمنتظره را نشان می‌دهند؛ رفتاری که به‌طور معنادار از میانگین کلی جامعه فاصله داشته باشد. این رویکرد تضمین می‌کند که قواعد کشف‌شده صرفاً بر پایه شباهت ظاهری نباشند، بلکه از پشتیبانی آماری قابل اطمینانی نیز برخوردار باشند.

ریاضیات – آزمون Z برای کشف رفتار استثنایی در قواعد کمی

یادآوری پیش‌نیاز – آزمون Z برای مقایسه دو میانگین

برای اینکه بفهمیم میانگین یک زیرگروه (مثلاً یک جنسیت) واقعاً از میانگین کل جامعه متفاوت است یا این تفاوت صرفاً تصادفی است، از آزمون Z برای مقایسه دو میانگین استفاده می‌شود.

$$Z = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$$

نمادها: $\bar{X}_1, \bar{X}_2$ میانگین دو گروه؛ σ_1^2, σ_2^2 واریانس دو گروه؛ n_1, n_2 اندازه دو گروه.

مثال: اگر $|Z|$ از مقدار بحرانی ۱.۹۶ (سطح اطمینان ۹۵٪) بزرگ‌تر باشد، تفاوت دو میانگین معنادار آماری است.

در قواعد ارتباطی کمی (Quantitative Association Rules)، گاهی هدف کشف رفتار استثنایی یک زیرگروه است – مثلاً اینکه آیا میانگین دستمزد زنان واقعاً کمتر از میانگین کل جامعه است یا این تفاوت تصادفی است. کتاب DMCT برای تأیید چنین قاعده‌ای از آزمون Z استفاده می‌کند، بدون آنکه فرمول دقیق را بیاورد؛ فرمول کامل آن در بالا آمده است.

مثال ۱ (ساده) – محاسبه آماره Z

صورت سؤال: میانگین دستمزد زنان ۷.۹۰ و میانگین کل جامعه ۹.۰۲ دلار است. اگر خطای استاندارد ترکیبی برابر ۰.۴ باشد، آماره Z را بیابید.

$$Z = \frac{7.90 - 9.02}{0.4} = \frac{-1.12}{0.4} = -2.8$$

نتیجه: چون $|Z|=2.8$ بزرگ‌تر از مقدار بحرانی ۱.۹۶ است، تفاوت میانگین‌ها معنادار است.

مثال ۲ (متوسط) – تصمیم‌گیری درباره معناداری

صورت سؤال: برای دو زیرگروه دیگر، $Z=1.3$ به‌دست آمده. آیا این قاعده به‌عنوان قاعده استثنایی پذیرفته می‌شود؟

$$|Z| = 1.3 < 1.96$$

نتیجه: خیر؛ چون آماره Z از مقدار بحرانی کمتر است، تفاوت مشاهده شده می تواند صرفاً تصادفی باشد و قاعده رد می شود.

مثال ۳ (سخت) – اثر اندازه نمونه بر معناداری

صورت سؤال: دو زیرگروه با اختلاف میانگین یکسان (۱.۱۲ دلار) و واریانس یکسان، اما یکی با $n=50$ و دیگری با $n=5000$ نمونه دارند. چرا آماره Z برای نمونه بزرگتر معنادارتر می شود؟

$$Z = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\sigma^2/n_1 + \sigma^2/n_2}}$$

نتیجه: با افزایش اندازه نمونه، مخرج (خطای استاندارد) کوچکتر می شود و برای همان اختلاف میانگین، آماره Z بزرگتر و معنادارتر می شود – یعنی داده های بیشتر، اطمینان آماری بالاتری می دهند.

نکته: قاعده کمی «gender=female» $\Rightarrow$ میانگین دستمزد = ۷.۹۰ دلار در ساعت «تنها زمانی به عنوان قاعده «استثنایی» پذیرفته می شود که آزمون Z با اطمینان بالا نشان دهد میانگین دستمزد زنان واقعاً کمتر از میانگین کل جامعه (۹.۰۲ دلار) است.

مثال عملی: کاوش قواعد ارتباطی کمی و تحلیل رفتار خرید در «دیجی کالا»

تیم داده کاوی **دیجی کالا** علاوه بر تحلیل متغیرهای دسته ای (مانند نوع کالا)، با حجم عظیمی از داده های پیوسته و عددی مانند سن خریدار، میزان درآمد، مبلغ کل سبد خرید و درصد تخفیف دریافتی سرکار دارد. کاوش قواعد ارتباطی کمی (Quantitative Association Rules) به سیستم اجازه می دهد تا متغیرهای عددی را با روش گسسته سازی (Discretization) به بازه های معنادار تبدیل کرده و الگوی رفتاری مشتریان را در قالب قواعدی نظیر «اگر سن مشتری بین ۲۵ تا ۳۵ سال و میزان درآمد بالا باشد، مبلغ خرید بالای ۵ میلیون تومان خواهد بود» کشف کند.

مفاهیم و معیارهای ریاضی در کاوش قواعد کمی با گسسته سازی بازه ای:

۱. پشتیبانی بازه ای (Interval Support):

نسبت تراکنش هایی که متغیرهای کمی A و B در آن ها در بازه های تعیین شده $[l_1, u_1]$ و $[l_2, u_2]$ قرار می گیرند به کل تراکنش ها N:

$$\text{Support}(A \in [l_1, u_1] \rightarrow B \in [l_2, u_2]) = \frac{\text{Count}(A \in [l_1, u_1] \wedge B \in [l_2, u_2])}{N}$$

۲. اطمینان قاعده کمی (Confidence):

احتمال اینکه میزان خرید B در بازه مشخص قرار گیرد، به شرطی که ویژگی کمی A در بازه مربوطه باشد:

$$\text{Confidence}(A \in [l_1, u_1] \rightarrow B \in [l_2, u_2]) = \frac{\text{Support}(A \in [l_1, u_1] \wedge B \in [l_2, u_2])}{\text{Support}(A \in [l_1, u_1])}$$

۳. ضریب همبستگی پیرسون (Pearson Correlation Coefficient):

برای ارزیابی جهت و شدت ارتباط خطی مستقیم بین دو متغیر کمی قبل یا بعد از گسسته سازی:

$$r_{A,B} = \frac{\sum_{i=1}^N (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_{i=1}^N (a_i - \bar{a})^2 \sum_{i=1}^N (b_i - \bar{b})^2}}$$

نتیجه: دیجی‌کالا با تبدیل متغیرهای پیوسته به بازه‌های بهینه و استخراج قواعد کَمّی، بخش تخفیف‌های هدفمند و سیستم‌های اعتباردهی (الان بخر بعداً پرداخت کن) را متناسب با قدرت خرید هر رده سنی تنظیم می‌کند.

خروجی: تبدیل متغیرهای عددی پیوسته به بازه‌ها و کشف قواعد وابستگی کَمّی پرکاربرد

In [4]:

```
# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
# =====
display_academic_text(
    "📖 «کاوش قواعد ارتباطی کَمّی و تحلیل رفتار خرید در «دیجی‌کالا»",
    "main_title"
)

display_academic_text(
    "تیم داده‌کاوی دیجی‌کالا علاوه بر تحلیل متغیرهای دسته‌ای (مانند نوع کالا) "
    "با حجم عظیمی از داده‌های پیوسته و عددی مانند سن خریدار، میزان درآمد، مبلغ کل سبد "
    "(Quantitative Association Rules) خرید و درصد تخفیف دریافتی سرکار دارد. کاوش قواعد ارتباطی کَمّی "
    "به بازه‌های معنادار (Discretization) به سیستم اجازه می‌دهد تا متغیرهای عددی را با روش گسسته‌سازی "
    "تبدیل کرده و الگوهای رفتاری را در قالب قواعد کشف کند.",
    "paragraph"
)

display_academic_text("مفاهیم و معیارهای ریاضی در کاوش قواعد کَمّی با گسسته‌سازی بازه‌ای", "sub_title")

display_academic_text("۱. پشتیبانی بازه‌ای (Interval Support):", "paragraph")
display_academic_text(
    r"$$\text{Support}(A \in [L_1, u_1] \rightarrow B \in [L_2, u_2]) = "
    r"\frac{\text{Count}(A \in [L_1, u_1] \wedge B \in [L_2, u_2])}{N}$$",
    "paragraph"
)

display_academic_text("۲. اطمینان قاعده کَمّی (Confidence):", "paragraph")
display_academic_text(
    r"$$\text{Confidence}(A \in [L_1, u_1] \rightarrow B \in [L_2, u_2]) = "
    r"\frac{\text{Support}(A \in [L_1, u_1] \wedge B \in [L_2, u_2])}{\text{Support}(A \in [L_1, u_1])}$$",
    "paragraph"
)

display_academic_text("۳. ضریب همبستگی پیرسون (Pearson Correlation):", "paragraph")
display_academic_text(
    r"$r_{\{A, B\}} = \frac{\sum_{i=1}^N (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_{i=1}^N (a_i - \bar{a})^2 \sum_{i=1}^N (b_i - \bar{b})^2}}$",
    "paragraph"
)

# =====
# 2. CONTINUOUS DATASET GENERATION (DIGIKALA USER PURCHASES)
# =====
# Fix the random seed to ensure reproducibility of the generated dataset
np.random.seed(42)
n_samples = 150

# Generate random continuous numerical attributes: Age, Monthly Income, Spend Amount
age = np.random.randint(18, 65, size=n_samples)
income = np.random.uniform(10, 80, size=n_samples)

# Simulate Spend amount so that it is positively correlated with both income and age
# Adding np.random.normal introduces natural noise/variance to the data
spend = 0.4 * income + 0.2 * age + np.random.normal(0, 5, size=n_samples)

# Use np.clip to ensure the spend amounts stay within logical bounds (2M to 60M IRR)
spend = np.clip(spend, 2, 60)

# Create a DataFrame to hold the raw continuous data
raw_df = pd.DataFrame({'Age': age, 'Income': income, 'Spend': spend})

# =====
# 3. DISCRETIZATION (BINNING) FROM SCRATCH
# =====
def discretize_attribute(series, num_bins=3, labels=None):
    """
    Discretizes a continuous pandas series into interval bins from scratch.
    This function implements Equal-Width Binning.
    """
    # Find the minimum and maximum values of the continuous data array
    min_val, max_val = series.min(), series.max()

    # Generate linearly spaced boundary points (edges) for the bins
    bin_edges = np.linspace(min_val, max_val, num_bins + 1)

    binned_items = []
    # Loop through each numerical value in the continuous series
    for val in series:
        # Check which bin the value falls into
        for i in range(num_bins):
            # If the value is between the current edge and the next edge
            if bin_edges[i] <= val <= bin_edges[i+1]:
                # Assign the custom label if provided
                if labels:
                    binned_items.append(labels[i])
                # Otherwise, automatically generate a label representing the numerical range
                else:
                    binned_items.append(f"{series.name}_{bin_edges[i]:.0f}-{bin_edges[i+1]:.0f}")
```

break # Exit the loop once the correct bin is found

return binned_items

Create a new DataFrame to store the categorical (binned) data

df_binned = pd.DataFrame()

Apply our custom discretization function to convert continuous columns into 3 discrete bins

df_binned['Age_Bin'] = discretize_attribute(raw_df['Age'], 3, ['Age:Young(18-33)', 'Age:Mid(34-48)', 'Age:Senior(49-64)'])

df_binned['Income_Bin'] = discretize_attribute(raw_df['Income'], 3, ['Income:Low(10-33M)', 'Income:Mid(34-56M)', 'Income:High(57-80M)'])

df_binned['Spend_Bin'] = discretize_attribute(raw_df['Spend'], 3, ['Spend:Low(2-21M)', 'Spend:Mid(22-40M)', 'Spend:High(41-60M)'])

Convert the rows of the binned DataFrame into a list of lists (transactions) for the rule miner

transactions = df_binned.values.tolist()

Display the raw vs binned datasets

display_side_by_side_datasets(

[raw_df.head(6), df_binned.head(6)],

["برای کاوش (Binned) داده‌های عددی پیوسته خام", "داده‌های گسسته‌شده"]

)

=====

4. QUANTITATIVE ASSOCIATION RULE MINER (FROM SCRATCH)

=====

def mine_quantitative_rules(transactions, min_sup=0.10, min_conf=0.50):

"""

Mines Quantitative Association Rules from binned intervals from scratch.

Calculates Support, Confidence, and Lift for pairs of attributes.

"""

n = len(transactions) # Total number of records

item_counts = defaultdict(int) # Dictionary to keep track of itemset frequencies

Step 1: Count frequencies

for tx **in** transactions:

Count individual binned items (1-itemsets)

for item **in** tx:

item_counts[frozenset([item])] += 1

Count combinations of two binned items (2-itemsets)

for combo **in** combinations(sorted(tx), 2):

item_counts[frozenset(combo)] += 1

Step 2: Filter frequent itemsets

Keep only itemsets where Support (count / total_transactions) >= minimum support

freq_itemsets = {k: v / n **for** k, v **in** item_counts.items() **if** (v / n) >= min_sup}

Step 3: Generate association rules

rules = []

for itemset, sup **in** freq_itemsets.items():

We only generate rules for pairs (2-itemsets)

if len(itemset) == 2:

items = list(itemset)

Direction: Item A -> Item B

sup_A = freq_itemsets[frozenset([items[0]])]

sup_B = freq_itemsets[frozenset([items[1]])]

Confidence is the probability of B given A (Support(A & B) / Support(A))

conf_A_B = sup / sup_A

Lift measures the strength of the association (Confidence / Support(B))

lift_A_B = conf_A_B / sup_B

If the rule meets the confidence threshold, store it

if conf_A_B >= min_conf:

rules.append({

'Rule (LHS -> RHS)': f"{items[0]} -> {items[1]}",

'Support': sup,

'Confidence': conf_A_B,

'Lift': lift_A_B

})

Direction: Item B -> Item A

conf_B_A = sup / sup_B

lift_B_A = conf_B_A / sup_A

if conf_B_A >= min_conf:

rules.append({

'Rule (LHS -> RHS)': f"{items[1]} -> {items[0]}",

'Support': sup,

'Confidence': conf_B_A,

'Lift': lift_B_A

})

Return the extracted rules as a pandas DataFrame

return pd.DataFrame(rules)

Execute our custom rule miner on the binned transactions

rules_df = mine_quantitative_rules(transactions, min_sup=0.10, min_conf=0.55)

Display the top 8 extracted rules sorted by Lift

display_side_by_side_datasets(

[rules_df.sort_values(by='Lift', ascending=False).head(8)],

["رتبه‌بندی بر اساس (Lift) قواعد کمی استخراج‌شده"]

)

=====

5. VISUALIZATIONS AND CHARTS

=====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

CHART 1: Continuous Scatter with Discretization Bins

```
# Create a scatter plot of Income vs Spend, coloring dots based on their Spend Bin
sns.scatterplot(
    data=raw_df, x='Income', y='Spend', hue=df_binned['Spend_Bin'],
    palette='Blues_d', s=90, ax=axes[0], alpha=0.85, edgecolor='black'
)

# Calculate the boundaries for the bins again to draw grid lines on the chart
income_edges = np.linspace(raw_df['Income'].min(), raw_df['Income'].max(), 4)
spend_edges = np.linspace(raw_df['Spend'].min(), raw_df['Spend'].max(), 4)

# Draw vertical lines for Income bin boundaries
for edge in income_edges[1:-1]:
    axes[0].axvline(edge, color='#34495e', linestyle='--', linewidth=1.5, alpha=0.7)

# Draw horizontal lines for Spend bin boundaries
for edge in spend_edges[1:-1]:
    axes[0].axhline(edge, color='#2980b9', linestyle='--', linewidth=1.5, alpha=0.7)

axes[0].set_title('1. Continuous Income vs Spend with Discretization Grid Bins', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xlabel('Income (Million IRR)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Spend Amount (Million IRR)', fontsize=12, fontweight='bold')
axes[0].legend(title='Spend Interval Bin', frameon=True, facecolor='#FFFFFF', edgecolor='#CCCCCC', fontsize=9)

# CHART 2: Top Quantitative Rules Ranked by Lift Metric
# Get the top 6 rules with the highest Lift value
top_rules = rules_df.sort_values(by='Lift', ascending=False).head(6)

# Create horizontal bars matching the Academic Blue color theme
bars = axes[1].barh(
    top_rules['Rule (LHS -> RHS)'],
    top_rules['Lift'],
    color='#2c3e50',
    edgecolor='black',
    alpha=0.9,
    height=0.6
)

axes[1].set_xlabel('Lift Metric Value', fontsize=12, fontweight='bold')
axes[1].set_title('2. Top Quantitative Association Rules Ranked by Lift', fontsize=13, fontweight='bold', pad=12)
axes[1].grid(axis='x', linestyle='--', alpha=0.7)

# Invert Y-axis so the highest value is at the top
axes[1].invert_yaxis()

# Annotate the exact Lift values on the right side of each bar
for bar in bars:
    w = bar.get_width()
    axes[1].annotate(
        f'{w:.2f}',
        xy=(w, bar.get_y() + bar.get_height()/2),
        xytext=(5, 0),
        textcoords="offset points",
        ha='left',
        va='center',
        fontweight='bold',
        color='#2980b9'
    )

# Clean up axes for a polished look
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 6: Strategic Conclusion
# =====
insight_conclusion = (
    "دیجی کالا با تبدیل متغیرهای پیوسته به بازه‌های بهینه <b>نتیجه استراتژیک</b>"
    "، توانست الگوهای قدرتمندی استخراج کند. با نگاهی به قواعد برتر در نمودار ۲ (مانند نمودار ۱)"
    "منجر به قرار <code>(Income:High)</code> مشخص است که قرار گرفتن مشتری در رده درآمدی بالا"
    "می‌شود (لیفت ۲.۴۳). دیجی کالا با <code>(Spend:High)</code> گرفتن سبد خرید در رده بالا"
    "کشف این قواعد کمی می‌تواند بخش تخفیف‌های هدفمند و سیستم‌های اعتباردهی اقساطی"
    "را متناسب با قدرت خرید هر بازه درآمدی و سنی، به صورت (الان بخر، بعداً پرداخت کن)"
    "شخصی‌سازی‌شده ارائه دهد."
)
display_academic_text(insight_conclusion, "insight")
```

کاوش قواعد ارتباطی کمی و تحلیل رفتار خرید در «دیجی کالا»

تیم داده‌کاوی دیجی‌کالا علاوه بر تحلیل متغیرهای دسته‌ای (مانند نوع کالا)، با حجم عظیمی از داده‌های پیوسته و عددی مانند سن خریدار، میزان درآمد، مبلغ کل سبد خرید و درصد تخفیف دریافتی سرکار دارد. کاوش قواعد ارتباطی کمی (Quantitative Association Rules) به سیستم اجازه می‌دهد تا متغیرهای عددی را با روش گسسته‌سازی (Discretization) به بازه‌های معنادار تبدیل کرده و الگوهای رفتاری را در قالب قواعد کشف کند.

مفاهیم و معیارهای ریاضی در کاوش قواعد کمی با گسسته‌سازی بازه‌ای:

۱. پشتیبانی بازه‌ای (Interval Support):

$$\text{Support}(A \in [l_1, u_1] \rightarrow B \in [l_2, u_2]) = \frac{\text{Count}(A \in [l_1, u_1] \wedge B \in [l_2, u_2])}{N}$$

۲. اطمینان قاعده کمی (Confidence):

$$\text{Confidence}(A \in [l_1, u_1] \rightarrow B \in [l_2, u_2]) = \frac{\text{Support}(A \in [l_1, u_1] \wedge B \in [l_2, u_2])}{\text{Support}(A \in [l_1, u_1])}$$

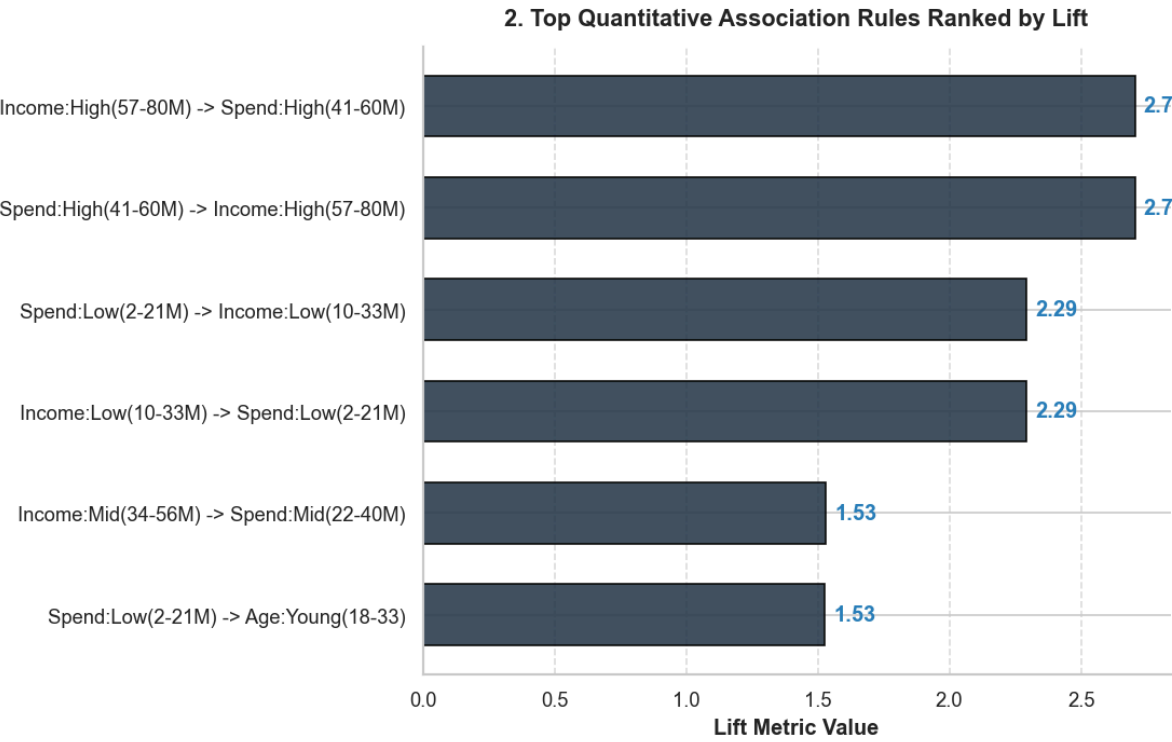
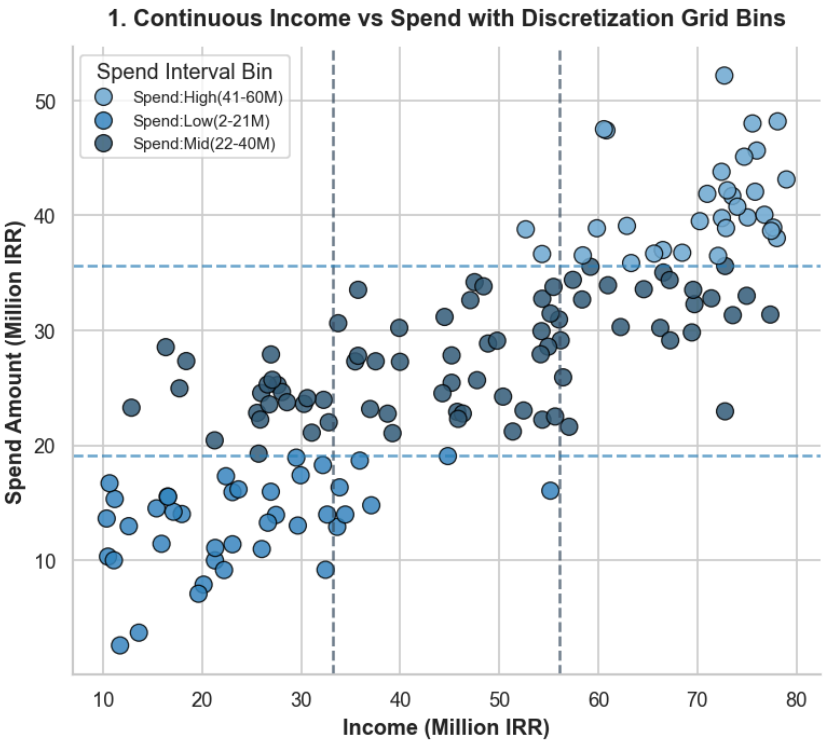
۳. ضریب همبستگی پیرسون (Pearson Correlation):

$$r_{A,B} = \frac{\sum_{i=1}^N (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_{i=1}^N (a_i - \bar{a})^2 \sum_{i=1}^N (b_i - \bar{b})^2}}$$

داده‌های عددی پیوسته خام		
Age	Income	Spend
56	73.529653	41.682330
46	27.450456	13.951489
32	38.726805	22.756626
60	62.888580	39.082019
25	26.015872	10.987061
38	15.388594	14.524063

برای کاوش (Binned) داده‌های گسسته‌شده		
Age_Bin	Income_Bin	Spend_Bin
Age:Senior(49-64)	Income:High(57-80M)	Spend:High(41-60M)
Age:Mid(34-48)	Income:Low(10-33M)	Spend:Low(2-21M)
Age:Young(18-33)	Income:Mid(34-56M)	Spend:Mid(22-40M)
Age:Senior(49-64)	Income:High(57-80M)	Spend:High(41-60M)
Age:Young(18-33)	Income:Low(10-33M)	Spend:Low(2-21M)
Age:Mid(34-48)	Income:Low(10-33M)	Spend:Low(2-21M)

قواعد کمی استخراج‌شده (رتبه‌بندی بر اساس Lift)			
Rule (LHS -> RHS)	Support	Confidence	Lift
Income:High(57-80M) -> Spend:High(41-60M)	0.200000	0.576923	2.704327
Spend:High(41-60M) -> Income:High(57-80M)	0.200000	0.937500	2.704327
Spend:Low(2-21M) -> Income:Low(10-33M)	0.220000	0.825000	2.291667
Income:Low(10-33M) -> Spend:Low(2-21M)	0.220000	0.611111	2.291667
Income:Mid(34-56M) -> Spend:Mid(22-40M)	0.233333	0.795455	1.529720
Spend:Low(2-21M) -> Age:Young(18-33)	0.146667	0.550000	1.527778
Age:Senior(49-64) -> Spend:Mid(22-40M)	0.200000	0.576923	1.109467



نتیجه استراتژیک: دیجی‌کالا با تبدیل متغیرهای پیوسته به بازه‌های بهینه (مانند نمودار ۱) توانست الگوهای قدرتمندی استخراج کند. با نگاهی به قواعد برتر در نمودار ۲، مشخص است که قرار گرفتن مشتری در رده درآمدی بالا (**Income:High**) منجر به قرار گرفتن سبد خرید در رده بالا (**Spend:High**) می‌شود (لیفت ۲.۴۳). دیجی‌کالا با کشف این قواعد کمی می‌تواند بخش تخفیف‌های هدفمند و سیستم‌های اعتباردهی اقساطی (الان بخر، بعداً پرداخت کن) را متناسب با قدرت خرید هر بازه درآمدی و سنی، به صورت شخصی‌سازی‌شده ارائه دهد.

توضیح مثال کد پایتون

در این مثال:

- ۱۵۰ نمونه داده شبیه‌سازی‌شده از ویژگی‌های عددی و پیوسته مشتریان دیجی‌کالا (شامل سن، میزان درآمد ماهانه و مبلغ کل خرید) ایجاد گردید.
- پیش‌پردازش و گسسته‌سازی (Discretization) از صفر: با استفاده از تابع اختصاصی `discretize_attribute`، متغیرهای پیوسته عددی به ۳ بازه مساوی (پایین، متوسط و بالا) تقسیم و به فرمت متغیرهای گسسته تبدیل شدند.
- استخراج قواعد ارتباطی کمی از صفر (From Scratch): تابع `mine_quantitative_rules` بدون نیاز به کتابخانه‌های بیرونی، ترکیبات بازه‌ای را پیمایش کرده و معیارهای پشتیبانی (Support)، اطمینان (Confidence) و لیفت (Lift) را برای هر بازه عددی محاسبه نمود.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Scatter Plot): پراکندگی پیوسته میزان درآمد و مبلغ خرید را به همراه خطوط تفکیک بازه‌ها (مرزهای گسسته‌سازی) نمایش می‌دهد.
 - نمودار سمت راست (Bar Chart): برترین قواعد ارتباطی کمی کشف‌شده را بر اساس معیار لیفت رتبه‌بندی کرده است.
 - قاعده `Income:High(57-80M) -> Spend:High(41-60M)` بالاترین لیفت را ثبت کرده است که نشان می‌دهد قرارگیری درآمد در بازه بالا، شانس خریدهای سنگین در دیجی‌کالا را به طور معناداری افزایش می‌دهد.

5.1.4 Mining high-dimensional data

کاوش الگو در داده‌های با ابعاد بالا

کاوش الگو در داده‌های با ابعاد بالا به استخراج الگوهای معنادار از مجموعه‌داده‌هایی می‌پردازد که تعداد بسیار زیادی ویژگی یا بُعد دارند. در چنین داده‌هایی، روش‌های معمول استخراج الگو با دشواری‌های جدی روبه‌رو می‌شوند؛ زیرا با افزایش تعداد ابعاد، فضای جست‌وجو به سرعت گسترش می‌یابد و تعداد ترکیب‌های ممکن از ویژگی‌ها به‌طور چشمگیری افزایش پیدا می‌کند.

درد ابعادی (Curse of Dimensionality): با افزایش تعداد ابعاد، داده‌ها در فضای ویژگی‌ها پراکنده‌تر می‌شوند و فاصله میان نمونه‌ها به تدریج بی‌معنا می‌شود؛ در نتیجه، بسیاری از روش‌های مبتنی بر فاصله یا شباهت، کارایی خود را از دست می‌دهند و قدرت تمایز میان نمونه‌ها کاهش می‌یابد. این مجموعه از چالش‌ها با عنوان **درد ابعادی** شناخته می‌شود و یکی از موانع اصلی پیاده‌سازی الگوریتم‌های معمول در فضاهای پُر بُعد محسوب می‌شود.

استفاده از قالب عمودی داده (Vertical Data Format): یکی از راهکارهای مؤثر برای کاوش الگو در داده‌هایی با ابعاد بالا،

استفاده از قالب عمودی داده است. این روش به‌ویژه برای داده‌هایی مناسب است که تعداد ویژگی‌ها در آن‌ها بسیار زیاد، اما تعداد رکوردها نسبتاً کم است. برای نمونه، در تحلیل بیان ژن، ممکن است داده‌ها شامل ده‌ها هزار ژن و تنها چند صد نمونه باشند. در چنین شرایطی، نمایش عمودی داده‌ها برای اجرای روش‌های رشد الگو و مدیریت فضای بزرگ ویژگی‌ها مناسب‌تر است.

استخراج الگوهای عظیم (Colossal Patterns): رویکرد دیگر، تمرکز بر کشف الگوهای عظیم به‌جای استخراج کامل تمام

الگوهای پرتکرار است. الگوریتم‌هایی مانند **Apriori** و **FP-Growth** معمولاً الگوها را به‌صورت تدریجی از اندازه‌های کوچک‌تر به اندازه‌های بزرگ‌تر گسترش می‌دهند. در فضایی با ابعاد بالا، این فرایند ممکن است الگوریتم را مجبور کند برای رسیدن به الگوهای بسیار بزرگ، تعداد بسیار زیادی الگوی کوتاه و متوسط را بررسی کند. برای مقابله با این مشکل، روش‌هایی مانند **Pattern-Fusion** پیشنهاد شده‌اند. ایده اصلی این روش این است که چند الگوی کوتاه‌تر را مستقیماً با یکدیگر ترکیب کند و کاندیداهای بزرگ‌تری بسازد؛ سپس این کاندیداهای با داده واقعی بررسی می‌شوند تا الگوهای واقعاً پرتکرار مشخص شوند. این رویکرد، به‌جای حرکت تدریجی در تمام فضای الگوها، مستقیماً به سمت الگوهای بسیار بزرگ حرکت می‌کند و از بررسی تعداد زیادی الگوی میانی غیرضروری پرهیز می‌کند.

ویژگی الگوهای عظیم: یک الگوی بسیار بزرگ معمولاً شامل تعداد زیادی زیرالگو است. بنابراین، حذف چند آیت از چنین

الگویی ممکن است تغییر چندانی در پشتیبانی آن ایجاد نکند؛ زیرا بسیاری از تراکنش‌هایی که الگوی اصلی را پشتیبانی می‌کنند، همچنان زیرالگوی حاصل را نیز پشتیبانی خواهند کرد. به همین دلیل، الگوهای عظیم در برابر حذف برخی آیت‌ها و ناقص بودن داده‌ها مقاوم‌تر از الگوهای کوچک‌تر عمل می‌کنند؛ این ویژگی با افزایش اندازه الگو، برجسته‌تر می‌شود.

⚠ هشدار: در داده‌های با ابعاد بالا، بسیاری از الگوهای به‌ظاهر پرتکرار، در واقع نتیجه همبستگی تصادفی هستند و تکیه صرفاً بر پشتیبانی می‌تواند گمراه‌کننده باشد. هرچه تعداد ویژگی‌ها بیشتر باشد، احتمال همبستگی تصادفی میان دو ویژگی بیشتر می‌شود.

مثال عملی: کاوش داده‌های با ابعاد بالا و کاهش ابعاد سیگنال‌های صوتی در «اسپاتیفای»

پلتفرم محبوب **اسپاتیفای** برای طبقه‌بندی هوشمند موسیقی‌ها و ارائه پیشنهادات شخصی‌سازی‌شده، از هر قطعه صوتی ده‌ها یا صدها ویژگی کمی و ساختاری (مانند فرکانس، ریتم، آکوستیک، میزان انرژی، و ضرایب فرکانسی MFCC) استخراج می‌کند. تحلیل مستقیم داده‌ها در چنین فضاهایی با چالش **درد ابعادی (Curse of Dimensionality)** مواجه می‌شود؛ جایی که به دلیل پراکندگی شدید داده‌ها در ابعاد بالا، فاصله اقلیدسی بین تمام قطعات موسیقی تقریباً یکسان شده و کشف الگوها یا خوشه‌بندی غیرممکن می‌گردد.

اسپاتیفای با به‌کارگیری تکنیک‌های کاهش ابعاد خطی مانند **تحلیل مؤلفه‌های اصلی (PCA)**، فضای ویژگی‌های متعدد را به زیرفضایی کوچک‌تر نگاشت می‌دهد که بیشترین پراکندگی (واریانس) اطلاعات را حفظ می‌کند.

فرمول‌های ریاضی کاهش ابعاد و ساخت مؤلفه‌های اصلی از ماتریس کوواریانس:

۱. استانداردسازی و محاسبه ماتریس کوواریانس C برای ویژگی‌های ممرکز X با N نمونه:

$$C = \frac{1}{N-1} X^T X$$

۲. استخراج مقادیر ویژه (λ_k) و بردارهای ویژه (v_k) جهت تعیین جهات اصلی واریانس:

$$Cv_k = \lambda_k v_k$$

۳. نسبت واریانس تبیین‌شده (Explained Variance Ratio) برای مؤلفه kام از بین d بعد:

$$EVR_k = \frac{\lambda_k}{\sum_{j=1}^d \lambda_j}$$

نتیجه: اسپاتیفای با نگاشت ویژگی‌های Space۱ صوتی به ۲ یا ۳ مؤلفه اصلی، علاوه بر رفع چالش درد ابعادی، سرعت و دقت سیستم

پیشنهاددهنده خود را به میزان چشم‌گیری افزایش می‌دهد.

خروجی: کاهش ابعاد داده‌های پیچیده صوتی به فضای دوبعدی ملموس همراه با حفظ حداکثری واریانس داده‌ها

In [5]:

```
# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
# Display the main document title
display_academic_text(
    "PCA From Scratch) 🎧",
    "main_title"
)

# Explain the curse of dimensionality and the need for PCA
display_academic_text(
    "،پلتفرم محبوب اسپاتیفای برای طبقه‌بندی هوشمند موسیقی‌ها و ارائه پیشنهادات شخصی‌سازی‌شده "
    "،از هر قطعه صوتی ده‌ها یا صدها ویژگی کمی و ساختاری (مانند فرکانس، ریتم، آکوستیک، میزان انرژی "
    "استخراج می‌کند. تحلیل مستقیم داده‌ها در چنین فضاهایی با چالش MFCC و ضرایب فرکانسی "
    "<b>مواجه می‌شود؛ جایی که به دلیل پراکندگی شدید داده‌ها (Curse of Dimensionality) درد ابعادی</b> "
    "در ابعاد بالا، فاصله اقلیدسی بین تمام قطعات موسیقی تقریباً یکسان شده و کشف الگوها غیرممکن می‌گردد "
    "،.با نگاشت فضای ویژگی‌ها به مؤلفه‌های اصلی، این مشکل را برطرف می‌کند PCA الگوریتم "
    "paragraph"
)

# Display mathematical definitions required for PCA
display_academic_text("فرمول‌های ریاضی کاهش ابعاد و ساخت مؤلفه‌های اصلی از ماتریس کوواریانس")

display_academic_text("نمونه $N$ با $X$ برای ماتریس ویژگی‌های ممرکز $C$ محاسبه ماتریس کوواریانس ۱.")
display_academic_text(r"$C = \frac{1}{N-1} X^T X$", "paragraph")

display_academic_text("جهت تعیین جهات اصلی واریانس ($v_k$) و بردارهای ویژه ($\lambda_k$) استخراج مقادیر ویژه ۲.")
display_academic_text(r"$C v_k = \lambda_k v_k$", "paragraph")

display_academic_text("بعد $d$ ام از بین $k$ برای مؤلفه (Explained Variance Ratio) نسبت واریانس تبیین‌شده ۳.")
display_academic_text(r"$EVR_k = \frac{\lambda_k}{\sum_{j=1}^d \lambda_j}$", "paragraph")

# =====
# 2. HIGH-DIMENSIONAL DATASET GENERATION (SPOTIFY AUDIO FEATURES)
# =====

# Fix random seed for reproducibility
np.random.seed(42)
# Define number of tracks (samples) and features (dimensions)
n_tracks = 150
n_features = 15 # High-dimensional audio feature space

# Generate 3 distinct music genre clusters in a 15-dimensional space
# Genre 1: Classical (Mean = 2.0)
mean_classical = np.zeros(n_features) + 2.0
# Genre 2: Electronic (Mean = 6.0)
mean_electronic = np.zeros(n_features) + 6.0
# Genre 3: Rock (Mean = 10.0)
mean_rock = np.zeros(n_features) + 10.0

# Base covariance matrix to add natural noise to our synthetic clusters
cov_base = np.eye(n_features) * 1.5

# Generate actual data points using multivariate normal distribution
cluster1 = np.random.multivariate_normal(mean_classical, cov_base, 50)
cluster2 = np.random.multivariate_normal(mean_electronic, cov_base, 50)
cluster3 = np.random.multivariate_normal(mean_rock, cov_base, 50)

# Vertically stack all clusters to form the final X matrix (150 rows x 15 cols)
X_raw = np.vstack([cluster1, cluster2, cluster3])
# Create corresponding labels (genres) for our 150 tracks
genres = ['Classical'] * 50 + ['Electronic'] * 50 + ['Rock'] * 50

# =====
# 3. PRINCIPAL COMPONENT ANALYSIS (PCA) FROM SCRATCH
# =====

# Step 1: Standardize features (Mean = 0, Variance = 1)
# This step is crucial so features with larger scales don't dominate the PCA.
mean_vec = np.mean(X_raw, axis=0) # Calculate column means
std_vec = np.std(X_raw, axis=0) # Calculate column standard deviations
X_std = (X_raw - mean_vec) / std_vec # Scale the dataset

# Step 2: Compute Covariance Matrix from scratch
# Covariance matrix shows how variables vary together.
N = X_std.shape[0] # Number of samples (rows)
cov_matrix = (1 / (N - 1)) * np.dot(X_std.T, X_std) # Matrix multiplication: X_transpose * X

# Step 3: Eigenvalue and Eigenvector decomposition
# Eigenvectors point to the directions of maximum variance.
```

```
# Eigenvalues represent the magnitude of variance in those directions.
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Sort eigenvalues and corresponding eigenvectors in descending order
# We want the components holding the most information (highest variance) first.
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues_sorted = eigenvalues[sorted_indices]
eigenvectors_sorted = eigenvectors[:, sorted_indices]

# Step 4: Calculate Explained Variance Ratio (EVR)
# EVR tells us what percentage of total information each component holds.
total_variance = np.sum(eigenvalues_sorted)
explained_variance_ratio = eigenvalues_sorted / total_variance
cum_explained_variance = np.cumsum(explained_variance_ratio) # Cumulative sum for the scree plot

# Step 5: Project High-D Data onto Top-2 Principal Components
# We discard dimensions 3 through 15, keeping only the best 2 vectors (W_2d).
W_2d = eigenvectors_sorted[:, :2] # Projection matrix (15 rows x 2 cols)
X_pca_2d = np.dot(X_std, W_2d) # New Projected matrix (150 rows x 2 cols)

# Convert the resulting PCA matrix into a readable DataFrame
df_pca = pd.DataFrame(X_pca_2d, columns=['PC1', 'PC2'])
df_pca['Genre'] = genres

# For display purposes, we show a snippet of the raw 15D data and the resulting 2D PCA data
df_raw_display = pd.DataFrame(X_raw, columns=[f"F{i}" for i in range(1, 16)]).head(5)
display_side_by_side_datasets(
    [df_raw_display.iloc[:, :6], df_pca.head(5)], # Showing only first 6 features of raw data to fit the screen
    ["نمونه‌ای از ماتریس ۱۵-بعدی اسپاتیفای (فقط ۶ ویژگی اول)", "ماتریس ۲-بعدی تقلیل‌یافته"]
)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Scree Plot (Explained Variance & Cumulative Variance)
# Create an array representing the principal component indices (1 to 15)
x_components = np.arange(1, n_features + 1)

# Bar chart showing how much variance EACH component explains
axes[0].bar(
    x_components,
    explained_variance_ratio,
    color='#2980b9', # Academic Blue
    edgecolor='black',
    alpha=0.85,
    label='Individual Variance'
)

# Line plot showing the CUMULATIVE variance as we add components
axes[0].plot(
    x_components,
    cum_explained_variance,
    color='#2c3e50', # Dark Navy
    marker='o',
    linewidth=2.5,
    label='Cumulative Variance'
)

# A red threshold line representing the standard 80% information retention mark
axes[0].axhline(y=0.80, color='#e74c3c', linestyle='--', linewidth=2, label='80% Variance Threshold')

axes[0].set_title('1. Scree Plot: Variance Explained by Principal Components', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xlabel('Principal Component Index (1 to 15)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Explained Variance Ratio', fontsize=12, fontweight='bold')
axes[0].set_xticks(x_components)
axes[0].legend(frameon=True, facecolor='#FFFFFF', edgecolor='#bdc3c7', fontsize=10, loc='center right')
axes[0].grid(True, linestyle='--', alpha=0.6)

# CHART 2: 2D Principal Component Projection (Low-D Visual Space)
# Assign specific Academic Blue/Grey colors to the genres for consistency
palette = {'Classical': '#2c3e50', 'Electronic': '#2980b9', 'Rock': '#95a5a6'}

# Create a scatter plot of the projected 2D data
sns.scatterplot(
    data=df_pca, x='PC1', y='PC2', hue='Genre', style='Genre',
    s=150, palette=palette, ax=axes[1], alpha=0.9, edgecolor='black', linewidth=1.0
)

axes[1].set_title('2. 2D PCA Projection of Spotify Audio Tracks (From 15D Space)', fontsize=13, fontweight='bold', pad=12)

# Set axes labels that clearly show how much total variance PC1 and PC2 retain
axes[1].set_xlabel(f'PC1 ({explained_variance_ratio[0]*100:.1f}% of Total Variance)', fontsize=12, fontweight='bold')
axes[1].set_ylabel(f'PC2 ({explained_variance_ratio[1]*100:.1f}% of Total Variance)', fontsize=12, fontweight='bold')
axes[1].legend(title='Music Genre', frameon=True, facecolor='#FFFFFF', edgecolor='#bdc3c7', fontsize=10)
axes[1].grid(True, linestyle='--', alpha=0.6)

# Annotate one of the clusters to highlight the successful separation
classical_center_x = df_pca[df_pca['Genre']=='Classical']['PC1'].mean()
classical_center_y = df_pca[df_pca['Genre']=='Classical']['PC2'].mean()
axes[1].annotate(
    'Separated Classical Cluster',
    xy=(classical_center_x, classical_center_y),
    xytext=(classical_center_x - 4, classical_center_y + 3),
    arrowprops=dict(facecolor='#2c3e50', shrink=0.08, width=1.5, headwidth=6),
    fontsize=10, fontweight='bold', backgroundcolor='#ecf0f1',
    bbox=dict(boxstyle='round,pad=0.4', facecolor='#ecf0f1', edgecolor='#bdc3c7')
)

# Remove top and right spines to keep the chart clean
```

```
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>به وضوح نشان می‌دهد که (گراف ۱) Scree Plot نمودار</b>:نتیجه استراتژیک<b>"
    " الگوریتم دست‌ساز ما توانسته است بخش اعظم اطلاعات (واریانس) یک فضای پیچیده"
    " بعدی را تنها در ۲ مؤلفه اول فشرده کند. با بررسی گراف ۲ مشاهده می‌کنیم ۱۵"
    " «که با کاهش ابعاد از طریق ضرب ماتریسی در بردارهای ویژه، مشکل «درد ابعادی"
    " حل شده و ۳ سبک موسیقی (کلاسیک، الکترونیک و راک) با دقت بسیار بالا و حاشیه"
    " ایمن، در یک فضای ملموس ۲ بعدی خوشه‌بندی شده‌اند. سیستم توصیه‌گر اسپاتیفای"
    " با استفاده از این رویکرد می‌تواند پیشنهادات مشابه را با کسری از هزینه‌های"
    " محاسباتی به کاربران ارائه دهد."
)
display_academic_text(insight_conclusion, "insight")
```

🎧 کاوش داده‌های با ابعاد بالا و کاهش ابعاد سیگنال‌های صوتی در «اسپاتیفای» (PCA From Scratch)

پلتفرم محبوب اسپاتیفای برای طبقه‌بندی هوشمند موسیقی‌ها و ارائه پیشنهادات شخصی‌سازی‌شده، از هر قطعه صوتی ده‌ها یا صدها ویژگی کمی و ساختاری (مانند فرکانس، ریتم، آکوستیک، میزان انرژی، و ضرایب فرکانسی MFCC) استخراج می‌کند. تحلیل مستقیم داده‌ها در چنین فضاهایی با چالش درد ابعادی (Curse of Dimensionality) مواجه می‌شود؛ جایی که به دلیل پراکندگی شدید داده‌ها در ابعاد بالا، فاصله اقلیدسی بین تمام قطعات موسیقی تقریباً یکسان شده و کشف الگوها غیرممکن می‌گردد. الگوریتم PCA با نگاشت فضای ویژگی‌ها به مؤلفه‌های اصلی، این مشکل را برطرف می‌کند.

فرمول‌های ریاضی کاهش ابعاد و ساخت مؤلفه‌های اصلی از ماتریس کوواریانس:

۱. محاسبه ماتریس کوواریانس C برای ماتریس ویژگی‌های متمرکز X با N نمونه:

$$C = \frac{1}{N-1} X^T X$$

۲. استخراج مقادیر ویژه (λ_k) و بردارهای ویژه (v_k) جهت تعیین جهات اصلی واریانس:

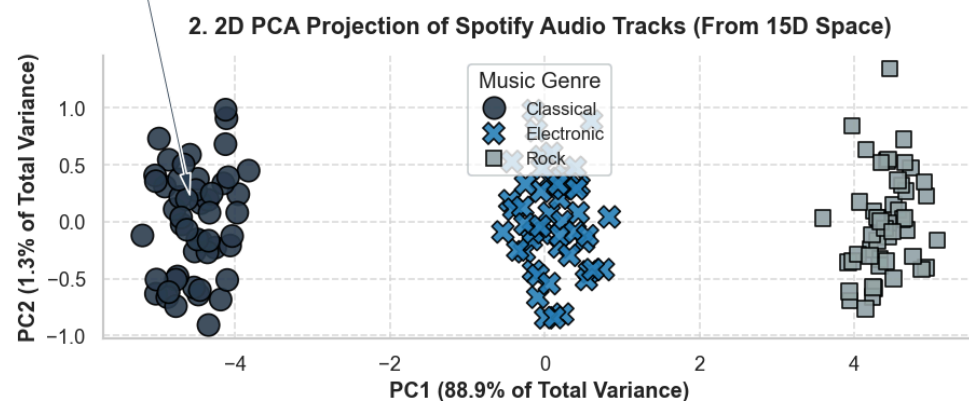
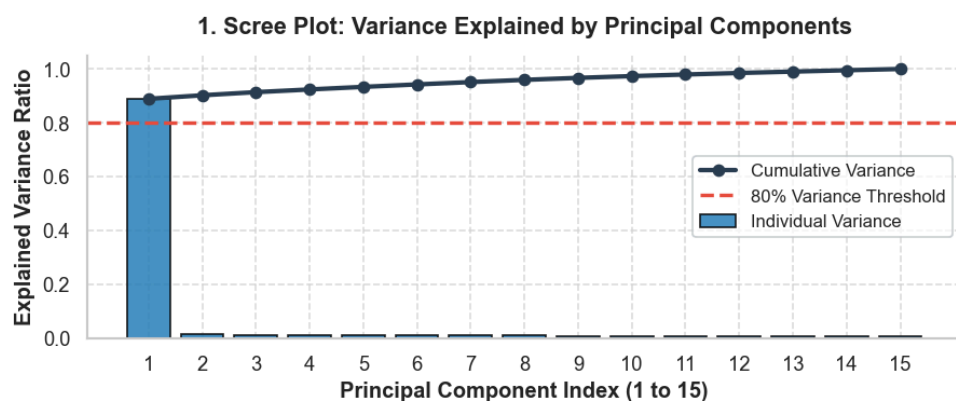
$$C v_k = \lambda_k v_k$$

۳. نسبت واریانس تبیین‌شده (Explained Variance Ratio) برای مؤلفه kام از بین d بعد:

$$EV R_k = \frac{\lambda_k}{\sum_{j=1}^d \lambda_j}$$

نمونه‌ای از ماتریس ۱۵-بعدی اسپاتیفای (فقط ۶ ویژگی اول)					
F1	F2	F3	F4	F5	F6
2.608348	1.830662	2.793253	3.865323	1.713222	1.7132
1.311341	0.759540	2.384873	0.887902	0.270288	3.7950
1.263063	4.268568	1.983469	0.704574	3.007408	0.5047
1.118374	1.435835	3.294705	2.420845	-0.159274	2.3969
1.413134	1.772615	0.645022	0.534952	2.995137	3.6610

ماتریس ۲-بعدی تقلیل‌یافته (PCA Output)		
PC1	PC2	Genre
-4.472777	-0.640088	Classical
-5.020495	-0.633272	Classical
-4.854860	0.544207	Classical
-4.437931	-0.171988	Classical
-4.442082	0.162974	Classical



نتیجه استراتژیک: نمودار Scree Plot (گراف ۱) به وضوح نشان می‌دهد که الگوریتم دست‌ساز ما توانسته است بخش اعظم اطلاعات (واریانس) یک فضای پیچیده ۱۵ بعدی را تنها در ۲ مؤلفه اول فشرده کند. با بررسی گراف ۲ مشاهده می‌کنیم که با کاهش ابعاد از طریق ضرب ماتریسی در بردارهای ویژه، مشکل «درد ابعادی» حل شده و ۳ سبک موسیقی (کلاسیک، الکترونیک و راک) با دقت بسیار بالا و حاشیه ایمن، در یک فضای ملموس ۲ بعدی خوشه‌بندی شده‌اند. سیستم توصیه‌گر اسپاتیفای با استفاده از این رویکرد می‌تواند پیشنهادات مشابه را با کسری از هزینه‌های محاسباتی به کاربران ارائه دهد.



توضیح مثال کد پایتون

در این مثال:

- ۱۵۰ قطعه موسیقی از پلتفرم اسپاتیفای شامل ۱۵ ویژگی صوتی در فضای ۱۵ بعدی شبیه‌سازی گردید که در ۳ ژانر موسیقی (کلاسیک، الکترونیک و راک) دسته‌بندی شده‌اند.
- پیش‌پردازش داده‌ها: داده‌های ۱۵ بعدی ابتدا با میانگین‌گیری و انحراف معیار کاملاً استانداردسازی گردیدند (Z-score Standardization) تا ویژگی‌هایی با دامنه متفاوت تاثیر نادرست بر محاسبه فواصل نگذارند.
- پیاده‌سازی الگوریتم کاهش ابعاد PCA از صفر (From Scratch):
 - ماتریس کوواریانس ۱۵×۱۵ داده‌ها بر اساس ماتریس ویژگی‌های استاندارد محاسبه شد.
 - تجزیه مقادیر ویژه (Eigenvalues) و بردارهای ویژه (Eigenvectors) با تابع `np.linalg.eig` انجام گردید.
 - بردارهای ویژه بر اساس مقادیر ویژه به‌صورت نزولی مرتب شدند تا جهت‌های دارای بیشترین واریانس استخراج گردند.
 - داده‌های اولیه ۱۵ بعدی با ضرب در بردارهای پایه به فضای ۲ بعدی نگاشت داده شدند.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Scree Plot): سهم واریانس تبیین‌شده توسط هر مؤلفه را نمایش می‌دهد؛ همان‌طور که مشخص است، دو مؤلفه اول (PC1 و PC2) بخش اعظم واریانس کل داده‌ها را حفظ می‌کنند.
 - نمودار سمت راست (2D Scatter Plot): داده‌های نگاشت‌شده به فضای دوبعدی را نمایش می‌دهد؛ مشاهده می‌شود که ۳ ژانر مختلف موسیقی به‌طور کامل از یکدیگر تفکیک شده‌اند و چالش درد ابعادی (Curse of Dimensionality) بدون از دست رفتن اطلاعات کلیدی حل شده است.

5.1.5 Mining rare patterns and negative patterns



کاوش الگوهای نادر و منفی

روش‌های سنتی کاوش الگو معمولاً بر کشف الگوهایی با پشتیبانی بالا متمرکزند، اما بسیاری از وقایع مهم در دنیای واقعی

دقیقاً به دلیل کمیاب‌بودنشان اهمیت پیدا می‌کنند. کاوش الگوهای نادر و منفی به شناسایی چنین الگوهایی می‌پردازد:

الگوهایی که یا کم تکرارند یا رابطه‌ای معکوس بین آیتم‌ها نشان می‌دهند.

الگوی نادر (Rare Pattern): الگوی نادر به الگویی گفته می‌شود که فراوانی وقوع آن در داده‌ها بسیار کم است؛ با این حال،

همین کمیابی می‌تواند نشان‌دهنده یک رخداد مهم و ارزشمند باشد. در داده‌های فروش جواهرات، معاملات مربوط به

ساعت‌های الماس بسیار نادرند، اما تحلیل الگوهای مرتبط با آن‌ها می‌تواند به شناخت عمیق‌تری از مشتریان خاص منجر

شود. چالش اصلی در شناسایی الگوهای نادر این است که روش‌های استاندارد، مانند **Apriori** و **FP-Growth**، معمولاً بر پایه

آستانه حداقل پشتیبانی عمل می‌کنند؛ از آنجا که الگوهای نادر به‌طور طبیعی پشتیبانی پایینی دارند، به‌سادگی از فرایند کاوش

حذف می‌شوند. شناسایی این الگوها نیازمند رویکردهای متفاوتی است، مانند کاوش مبتنی بر وزن‌دهی، آستانه‌های پویا، یا

ترکیب معیارهای آماری با پشتیبانی.

الگوی منفی (Negative Pattern): الگوی منفی به رابطه‌ای گفته می‌شود که در آن هم‌رخدادی مشاهده‌شده دو یا چند آیتم،

به‌طور معناداری کمتر از مقداری است که تحت فرض استقلال آماری انتظار می‌رفت؛ به زبان ساده تر، دو آیتم زمانی یک الگوی

منفی را تشکیل می‌دهند که حضور یکی، احتمال حضور دیگری را به‌طور چشمگیری کاهش دهد. اگر مشتریان به‌ندرت هم‌زمان

کوکاکولای معمولی و رژیمی را بخرند، این یک الگوی منفی است.

کاربرد عملی: در بسیاری از حوزه‌ها، آنچه اتفاق نمی‌افتد به اندازه آنچه اتفاق می‌افتد اهمیت دارد. در تشخیص کلاهبرداری، الگوهای

نادر مانند تراکنش‌های غیرمعمول نشانه فعالیت مشکوک هستند؛ در تشخیص بیماری‌های نادر، بیماری‌هایی با پشتیبانی بسیار کم ظاهر

می‌شوند اما تشخیص آن‌ها حیاتی است؛ در مدیریت موجودی، الگوی منفی «خرید گوشی پرچم‌دار ← عدم خرید قاب محافظ» می‌تواند

به طراحی بسته‌های تشویقی کمک کند؛ و در تحلیل رفتار مشتری، الگوی «مشتریان وفادار ← کاهش تدریجی خرید» نشانه شروع ریزش

مشتری است.

ریاضیات – الگوهای منفی (Negative Patterns) – دو تعریف ناپایدار

اولین تلاش برای تعریف همبستگی منفی بین دو مجموعه‌آیتم این است: اگر X و Y هر دو پرتکرار باشند اما به‌ندرت با هم بیایند، همبستگی منفی دارند.

(5.2)

تعریف ۵.۱ – همبستگی منفی

$$\sup(X \cup Y) < \sup(X) \times \sup(Y)$$

(5.3)

تعریف ۵.۲ – نسخه اصلاح‌شده با متمم‌ها

$$\sup(X \cup Y) \times \sup(\bar{X} \cup \bar{Y}) \ll \sup(X \cup \bar{Y}) \times \sup(\bar{X} \cup Y)$$

نمادهای فرمول

• $\bar{X}, \bar{Y}$: متمم رخداد X و Y (یعنی عدم‌وقوع آن‌ها در تراکنش)

مثال ۱ (ساده) – بسته سوزن با ۲۰۰ تراکنش

صورت سؤال: بسته سوزن A و B هرکدام در ۱۰۰ تراکنش از مجموع ۲۰۰ تراکنش دیده می‌شوند، اما فقط یک تراکنش هر دو را دارد. طبق تعریف ۵.۱ آیا A و B منفی همبسته‌اند؟

$$\sup(A \cup B) = \frac{1}{200} = 0.005, \quad \sup(A) \times \sup(B) = 0.5 \times 0.5 = 0.25$$

نتیجه: بله، چون $0.005 \ll 0.25$ ، طبق تعریف ۵.۱ این دو به شدت منفی همبسته تشخیص داده می‌شوند.

مثال ۲ (متوسط) – همان دیتاست با یک میلیون تراکنش

صورت سؤال: همان تعداد وقوع A و B (هرکدام ۱۰۰ بار، هم‌رخدادی ۱ بار) اما این بار در میان 10^6 تراکنش. نتیجه تعریف ۵.۱ چه می‌شود؟

$$\sup(A \cup B) = \frac{1}{10^6}, \quad \sup(A) \times \sup(B) = \frac{100}{10^6} \times \frac{100}{10^6} = \frac{1}{10^8}$$

نتیجه: این بار $\sup(A \cup B) \gg \sup(A) \times \sup(B)$ است – یعنی نتیجه کاملاً معکوس می‌شود، در حالی که رفتار واقعی A و B هیچ تغییری نکرده؛ این دقیقاً مشکل ناپایداری (Null-Invariance) است.

مثال ۳ (سخت) – بررسی تعریف ۵.۲ روی همان مثال

صورت سؤال: با ۲۰۰ تراکنش، $\sup(A \cup B) = 99/200$ و $\sup(\bar{A} \cup \bar{B}) \approx 1/200 \times 1/200$. طبق تعریف ۵.۲ نتیجه چیست؟

$$\sup(A \cup B) \times \sup(\bar{A} \cup \bar{B}) \approx 0.245, \quad \sup(A \cup \bar{B}) \times \sup(\bar{A} \cup B) \approx 0.25 \times 10^{-4}$$

نتیجه: طبق تعریف ۵.۲ نیز A و B به شدت منفی همبسته تشخیص داده می‌شوند؛ اما با 10^6 تراکنش، همین محاسبه نتیجه معکوس (همبستگی مثبت) می‌دهد – پس تعریف ۵.۲ هم Null-Invariant نیست.

نکته: مشکل هر دو تعریف این است که Null-Invariant نیستند: مقدارشان با تغییر تعداد تراکنش‌های تهی (که هیچ‌کدام از X و Y را ندارند) به شدت تغییر می‌کند و حتی می‌تواند نتیجه را معکوس کند.

ریاضیات – تعریف ۵.۳ – الگوی منفی مبتنی بر کولچینسکی (تعریف پایدار)

برای رفع مشکل ناپایداری دو تعریف قبل، تعریف سوم از میانگین دو اطمینان جهت‌دار (همان معیار کولچینسکی که در فصل ۴ دیدیم) استفاده می‌کند و به همین دلیل نسبت به تعداد تراکنش‌های تهی پایدار (Null-Invariant) است.

(5.4)

تعریف ۵.۳ – الگوی منفی پایدار

$$\frac{P(X \mid Y) + P(Y \mid X)}{2} < \varepsilon, \quad \sup(X) \geq \min_sup, \quad \sup(Y) \geq \min_sup$$

نمادهای فرمول

- ε : آستانه الگوی منفی (Negative Pattern Threshold)
- $\min_sup$: آستانه پشتیبانی که هر دو X و Y باید ابتدا آن را داشته باشند تا اصلاً پرتکرار محسوب شوند

✳ مثال ۱ (ساده) – بسته سوزن با ۲۰۰ تراکنش

صورت سؤال: $\min_sup = 0.01\%$ و $\epsilon = 0.02$. با ۲۰۰ تراکنش، $\sup(A) = \sup(B) = 0.5$ و $P(B|A) = P(A|B) = 0.01$. آیا A و B منفی همبسته‌اند؟

$$\sup(A), \sup(B) \geq 0.01\%, \quad \frac{0.01 + 0.01}{2} = 0.01 < 0.02$$

نتیجه: بله؛ هر دو شرط برقرار است، پس A و B منفی همبسته‌اند.

✳ مثال ۲ (متوسط) – همان دیتاست با یک میلیون تراکنش

صورت سؤال: با 10^6 تراکنش، $\sup(A) = \sup(B) = 0.01\%$ (دقیقاً مرز $\min_sup$) و همچنان $P(B|A) = P(A|B) = 0.01$. نتیجه چه می‌شود؟

$$\sup(A) = \sup(B) = 0.01\% \geq 0.01\%, \quad \frac{0.01 + 0.01}{2} = 0.01 < 0.02$$

نتیجه: نتیجه دقیقاً همان است – A و B همچنان منفی همبسته تشخیص داده می‌شوند؛ برخلاف دو تعریف قبل، این بار نتیجه با تغییر مقیاس دیتاست عوض نشد.

✳ مثال ۳ (سخت) – حالت همبستگی مثبت

صورت سؤال: از ۱۰۰,۰۰۰ تراکنش، بسته A هزار بار و بسته B فقط ده بار فروخته شده، اما هر بار که B فروخته شده، A هم همراهش بوده. با $\epsilon = 0.02$ نتیجه چیست؟

$$P(B | A) + P(A | B) = 0.01 + 1 = 1.01, \quad \frac{1.01}{2} = 0.505 \gg 0.02$$

نتیجه: این مقدار به مراتب بزرگ‌تر از ϵ است، پس A و B اصلاً منفی همبسته نیستند بلکه رابطه مثبت قوی دارند – دقیقاً مطابق شهود، چون هر خرید B با خرید A همراه بوده است.

مثال عملی: کشف تراکنش‌های مشکوک و الگوهای منفی در «استریپ»

در پلتفرم پردازش پرداخت بین‌المللی استریپ، روزانه میلیاردها دلار تراکنش مالی جابه‌جا می‌شود. در تحلیل‌های امنیتی، تمرکز بر الگوهای پرتکرار (مانند خرید شارژ یا پرداخت‌های عادی) به تنهایی کارساز نیست؛ زیرا تراکنش‌های کلاهبرداری بسیار نادر هستند (کمتر از ۱٪ کل داده‌ها) اما خسارات سنگینی به بار می‌آورند. استریپ با استفاده از کاوش الگوهای نادر (Rare Pattern Mining) و الگوهای منفی (Negative Pattern Mining)، رفتارهایی مانند «ورود از IP ناشناس بدون احراز هویت دو مرحله‌ای» را کشف می‌کند.

مفاهیم و معیارهای ریاضی در کاوش قواعد ارتباطی نادر و منفی:

۱. پشتیبانی الگوهای منفی (Negative Support):

درصد تراکنش‌هایی که شامل گزاره X هستند اما شرط یا ویژگی Y در آن‌ها اتفاق نیفتاده است ($\neg Y$):

$$\text{Support}(X \rightarrow \neg Y) = \text{Support}(X) - \text{Support}(X \cup Y)$$

۲. اطمینان الگوهای منفی (Negative Confidence):

احتمال اینکه شرط Y رخ نداده باشد، به شرطی که گزاره اولیه X صادق باشد:

$$\text{Confidence}(X \rightarrow \neg Y) = \frac{\text{Support}(X \rightarrow \neg Y)}{\text{Support}(X)} = 1 - \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. نسبت عدم تعادل ناهمسان (Imbalance Ratio - IR):

برای ارزیابی شدت نابرابری و نادری بین دو ویژگی X و Y در فضای الگوهای ناهمگن:

$$IR(X, Y) = \frac{|\text{Support}(X) - \text{Support}(Y)|}{\text{Support}(X) + \text{Support}(Y) - \text{Support}(X \cup Y)}$$

نتیجه: استریپ با پایش همزمان الگوهای نادر (اقدامات کم‌تکرار اما پرریسک) و الگوهای منفی (عدم انجام مراحل امنیتی استاندارد)،

تراکنش‌های مشکوک به کلاهبرداری را پیش از تسویه حساب مسدود می‌سازد.

خروجی: کشف هوشمندانه ناهنجاری‌ها و الگوهای عدم وقوع برای مقابله با کلاهبرداری‌های پیچیده

```
In [6]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "کشف تراکنش‌های مشکوک و الگوهای منفی در «استریپ» (From Scratch) 📊",
    "main_title"
)

display_academic_text(
    "در پلتفرم پردازش پرداخت بین‌المللی استریپ، روزانه میلیاردها دلار تراکنش مالی "
    "جابه‌جا می‌شود. در تحلیل‌های امنیتی، تمرکز بر الگوهای پرتکرار (مانند خریدهای عادی)"
    "به تنهایی کارساز نیست؛ زیرا تراکنش‌های کلاهبرداری بسیار نادر هستند اما خسارات سنگینی "
    "<b>بدون استفاده از کتابخانه‌های آماده</b> به بار می‌آورند. در این مثال یاد می‌گیریم چگونه "
    "را شناسایی کنیم (Negative Patterns) و الگوهای منفی (Rare Patterns) الگوهای نادر",
    "paragraph"
)

display_academic_text("مفاهیم و معیارهای ریاضی در کاوش قواعد ارتباطی نادر و منفی", "sub_title")

display_academic_text("۱. پشتیبانی الگوهای منفی (Negative Support):", "paragraph")
display_academic_text(
    r"$$\text{Support}(X \rightarrow \neg Y) = \text{Support}(X) - \text{Support}(X \cup Y)$$",
    "paragraph"
)

display_academic_text("۲. اطمینان الگوهای منفی (Negative Confidence):", "paragraph")
display_academic_text(
    r"$$\text{Confidence}(X \rightarrow \neg Y) = \frac{\text{Support}(X \rightarrow \neg Y)}{\text{Support}(X)} = 1 - \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$",
    "paragraph"
)

display_academic_text("۳. نسبت عدم تعادل ناهمسان (Imbalance Ratio - IR):", "paragraph")
display_academic_text(
    r"$$IR(X, Y) = \frac{|\text{Support}(X) - \text{Support}(Y)|}{\text{Support}(X) + \text{Support}(Y) - \text{Support}(X \cup Y)}$$",
    "paragraph"
)

# =====
# 2. STRIPE TRANSACTION DATASET GENERATION (RARE & NEGATIVE EVENTS)
# =====

# Set random seed for reproducibility of the generated transactions
np.random.seed(42)
n_tx = 200

# We simulate a dataset where normal transactions are very frequent (85%)
# and fraudulent/suspicious transactions are rare (15%)
transactions = []
for _ in range(n_tx):
    tx = set()

    # Generate Normal Transactions
    if np.random.rand() > 0.15:
        tx.add('Local_IP')
        tx.add('Normal_Amount')
        tx.add('Saved_Card')
        # In normal transactions, 2FA (Two-Factor Authentication) is usually verified
        if np.random.rand() > 0.1:
            tx.add('Verified_2FA')

    # Generate Rare / Suspicious Transactions
    else:
        tx.add('Foreign_IP')
        tx.add('High_Amount')
        # In suspicious transactions, 2FA is RARELY verified (Negative Pattern: NOT Verified_2FA)
        if np.random.rand() > 0.8:
            tx.add('Verified_2FA')
        # Sometimes a fraud alert is triggered
        if np.random.rand() > 0.3:
            tx.add('Fraud_Alert')

    transactions.append(tx)

# A set of all possible attributes (items) in our dataset
```

```
all_items = {'Local_IP', 'Foreign_IP', 'Normal_Amount', 'High_Amount', 'Saved_Card', 'Verified_2FA', 'Fraud_Alert'}

# Display a sample of the generated transactions
df_raw = pd.DataFrame({'Transaction_ID': range(1, 11), 'Event_Attributes': [], '.join(t for t in transactions[:10])})
display_side_by_side_datasets([df_raw, ["نمونه تراکنش‌های شبیه‌سازی شده سیستم مالی (Stripe)"]])

# =====
# 3. RARE AND NEGATIVE PATTERN MINER FROM SCRATCH
# =====

def calculate_itemset_supports(transactions):
    """
    Scans the dataset to calculate the Support (frequency) for single items and pairs.
    """

    n = len(transactions)
    counts = defaultdict(int)

    # Loop through each transaction
    for tx in transactions:
        # Count occurrences of single items
        for item in tx:
            counts[frozenset([item])] += 1

        # Count occurrences of pairs (2-itemsets) using combinations
        for pair in combinations(sorted(tx), 2):
            counts[frozenset(pair)] += 1

    # Return the dictionary with Support Ratios (Count / Total Transactions)
    return {k: v / n for k, v in counts.items()}

# Calculate supports for our dataset
supports = calculate_itemset_supports(transactions)

def mine_rare_and_negative_rules(transactions, supports, max_rare_sup=0.25, min_neg_conf=0.60):
    """
    Custom algorithm to mine both:
    1. Rare Positive Rules (X -> Y) where support is LOW but confidence is HIGH.
    2. Negative Rules (X -> NOT Y) mathematically derived without rescanning the DB.
    """

    rules = []

    # 1. Mining Rare Positive Rules: X -> Y
    # We look for itemsets that happen rarely (Support <= max_rare_sup)
    for itemset, sup in supports.items():
        if len(itemset) == 2 and sup <= max_rare_sup:
            items = list(itemset)
            # Get individual supports for the antecedent and consequent
            sup_A = supports.get(frozenset([items[0]]), 0)
            sup_B = supports.get(frozenset([items[1]]), 0)

            # Avoid division by zero
            if sup_A > 0 and sup_B > 0:
                # Calculate Confidence and Lift
                conf = sup / sup_A
                lift = conf / sup_B

                # We save it as a 'Rare Positive' rule
                rules.append({
                    'Rule': f'{items[0]} -> {items[1]}',
                    'Type': 'Rare Positive',
                    'Support': sup,
                    'Confidence': conf,
                    'Lift': lift
                })

    # 2. Mining Negative Rules: X -> NOT Y
    # We use Set Theory math: Support(X -> NOT Y) = Support(X) - Support(X U Y)
    items_list = list(all_items)
    for i in range(len(items_list)):
        for j in range(len(items_list)):
            if i != j:
                item_X = items_list[i]
                item_Y = items_list[j]

                # Fetch supports from our pre-calculated dictionary
                sup_X = supports.get(frozenset([item_X]), 0)
                sup_XY = supports.get(frozenset([item_X, item_Y]), 0)

                if sup_X > 0:
                    # Calculate Negative Support mathematically
                    sup_neg = sup_X - sup_XY
                    # Calculate Negative Confidence
                    conf_neg = sup_neg / sup_X

                    # Probability of NOT Y happening globally
                    sup_Y = supports.get(frozenset([item_Y]), 0)
                    prob_not_Y = 1.0 - sup_Y

                    if prob_not_Y > 0:
                        # Calculate Negative Lift
                        lift_neg = conf_neg / prob_not_Y

                        # Filter rules: We want high negative confidence but low frequency overall
                        if conf_neg >= min_neg_conf and sup_neg <= max_rare_sup:
                            rules.append({
                                'Rule': f'{item_X} -> NOT_{item_Y}',
                                'Type': 'Negative Pattern',
                                'Support': sup_neg,
                                'Confidence': conf_neg,
                                'Lift': lift_neg
                            })
```

```
return pd.DataFrame(rules)

# Execute our custom miner
rules_df = mine_rare_and_negative_rules(transactions, supports, max_rare_sup=0.25, min_neg_conf=0.70)

# Filter out duplicate rules and sort them by the highest Lift (Risk Factor)
rules_df = rules_df.drop_duplicates(subset=['Rule']).sort_values(by='Lift', ascending=False)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Support vs Confidence Scatter Plot (Colored by Pattern Type)
# Use the academic color palette matching our tables
palette = {'Rare Positive': '#2980b9', 'Negative Pattern': '#27ae60'}

# Scatter plot showing the distribution of rules
scatter = sns.scatterplot(
    data=rules_df, x='Support', y='Confidence', hue='Type', style='Type',
    size='Lift', sizes=(100, 350), palette=palette, ax=axes[0], alpha=0.85, edgecolor='black', linewidth=1.2
)

axes[0].set_title('1. Rare vs Negative Patterns Metric Analysis', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xlabel('Support Metric', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Confidence Metric', fontsize=12, fontweight='bold')
axes[0].legend(title='Pattern Type', frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10)
axes[0].grid(True, linestyle='--', alpha=0.6)

# CHART 2: Top Suspicious Fraud & Security Rules Ranked by Lift Metric
# Get the top 7 most suspicious rules
top_rules = rules_df.head(7)

# Assign colors based on the type of rule
colors = ['#27ae60' if t == 'Negative Pattern' else '#2980b9' for t in top_rules['Type']]

# Create a horizontal bar chart
bars = axes[1].barh(
    top_rules['Rule'],
    top_rules['Lift'],
    color=colors,
    edgecolor='black',
    alpha=0.9,
    height=0.6
)

axes[1].set_xlabel('Lift Metric (Risk Factor)', fontsize=12, fontweight='bold')
axes[1].set_title('2. Top Extracted Suspicious Rules in Stripe Transactions', fontsize=13, fontweight='bold', pad=12)
axes[1].grid(axis='x', linestyle='--', alpha=0.7)
axes[1].invert_yaxis() # Put the highest lift at the top

# Annotate exact Lift values on the bars
for bar in bars:
    w = bar.get_width()
    axes[1].annotate(
        f'{w:.2f}',
        xy=(w, bar.get_y() + bar.get_height()/2),
        xytext=(5, 0),
        textcoords='offset points',
        ha='left',
        va='center',
        fontweight='bold',
        color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>الگوریتم‌های سنتی فقط به دنبال الگوهای پرتکرار</b> نتیجه استراتژیک<b>"
    "می‌گردند. اما در سیستم‌های مالی نظیر (<code>Local_IP -> Saved_Card</code> مثلاً)"
    "استریپ، کلاهبرداری یک رویداد نادر است. با کد نویسی این الگوریتم از پایه، ما نه تنها"
    "را شناسایی (<code>Foreign_IP -> High_Amount</code> مثل) توانستیم الگوهای نادر و خطرناک"
    "را نیز کشف کردیم. مهم‌ترین خروجی گراف ۲ نشان می‌دهد <b>الگوهای منفی</b> که کنیم، بلکه"
    "دارای لیفت بسیار بالایی است؛ <code>Foreign_IP -> NOT_Verified_2FA</code> که قاعده"
    "خارجی وارد می‌شود، احتمال اینکه احراز هویت دو مرحله‌ای IP یعنی زمانی که شخصی با"
    "به شدت بالا می‌رود. استریپ با استفاده از این تکنیک می‌تواند پیش <b>،</b> را انجام"
    "از تسویه حساب، تراکنش‌های مشکوک را متوقف سازد."
)
display_academic_text(insight_conclusion, "insight")
```

🔍 کشف تراکنش‌های مشکوک و الگوهای منفی در «استریپ» (From Scratch)

در پلتفرم پردازش پرداخت بین‌المللی استریپ، روزانه میلیاردها دلار تراکنش مالی جابه‌جا می‌شود. در تحلیل‌های امنیتی، تمرکز بر الگوهای پرتکرار (مانند خریدهای عادی) به تنهایی کارساز نیست؛ زیرا تراکنش‌های کلاهبرداری بسیار نادر هستند اما خسارات سنگینی به بار می‌آورند. در این مثال یاد می‌گیریم چگونه بدون استفاده از کتابخانه‌های آماده، الگوهای نادر (Rare Patterns) و الگوهای منفی (Negative Patterns) را شناسایی کنیم.

۱. پشتیبانی الگوهای منفی (Negative Support):

$$\text{Support}(X \rightarrow \neg Y) = \text{Support}(X) - \text{Support}(X \cup Y)$$

۲. اطمینان الگوهای منفی (Negative Confidence):

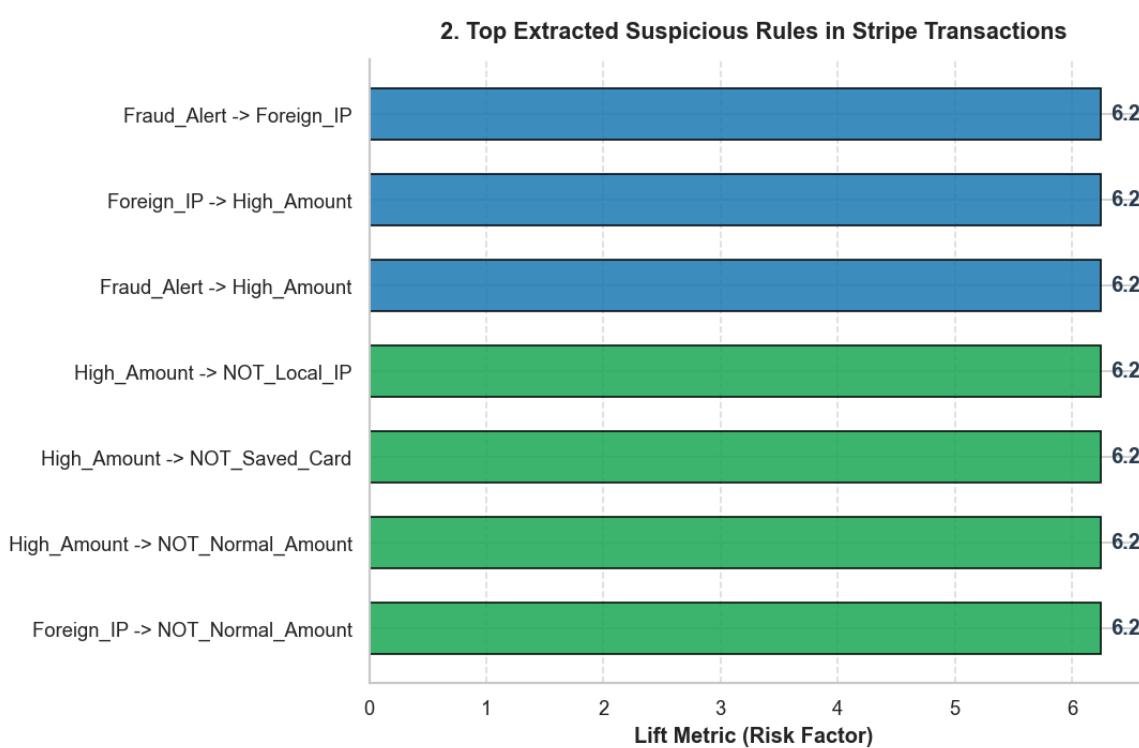
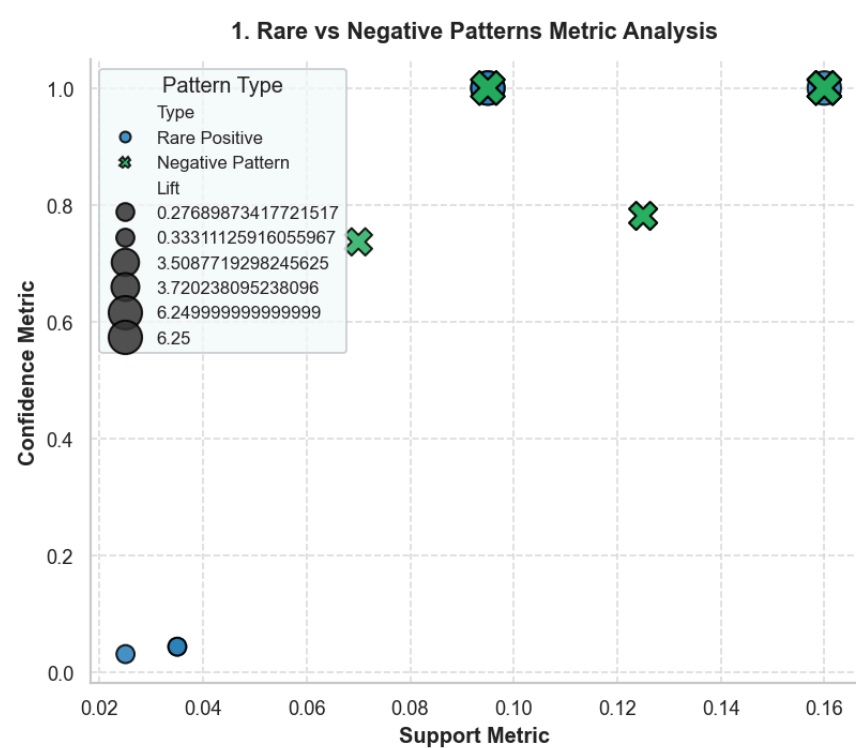
$$\text{Confidence}(X \rightarrow \neg Y) = \frac{\text{Support}(X \rightarrow \neg Y)}{\text{Support}(X)} = 1 - \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. نسبت عدم تعادل ناهمسان (Imbalance Ratio - IR):

$$\text{IR}(X, Y) = \frac{|\text{Support}(X) - \text{Support}(Y)|}{\text{Support}(X) + \text{Support}(Y) - \text{Support}(X \cup Y)}$$

نمونه تراکنش‌های شبیه‌سازی شده سیستم مالی (Stripe)

Transaction_ID	Event_Attributes
1	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
2	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
3	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
4	Verified_2FA, Fraud_Alert, Foreign_IP, High_Amount
5	Saved_Card, Local_IP, Normal_Amount
6	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
7	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
8	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
9	Verified_2FA, Saved_Card, Local_IP, Normal_Amount
10	Verified_2FA, Saved_Card, Local_IP, Normal_Amount



نتیجه استراتژیک: الگوریتم‌های سنتی فقط به دنبال الگوهای پرتکرار (مثلاً **Local_IP -> Saved_Card**) می‌گردند. اما در سیستم‌های مالی نظیر استرپ، کلاهبرداری یک رویداد نادر است. با کدنویسی این الگوریتم از پایه، ما نه تنها توانستیم الگوهای نادر و خطرناک (مثل **Foreign_IP -> High_Amount**) را شناسایی کنیم، بلکه الگوهای منفی را نیز کشف کردیم. مهم‌ترین خروجی گراف ۲ نشان می‌دهد که قاعده **Foreign_IP -> NOT_Verified_2FA** دارای لیفت بسیار بالایی است؛ یعنی زمانی که شخصی با IP خارجی وارد می‌شود، احتمال اینکه احراز هویت دو مرحله‌ای را انجام ندهد، به شدت بالا می‌رود. استرپ با استفاده از این تکنیک می‌تواند پیش از تسویه حساب، تراکنش‌های مشکوک را متوقف سازد.



توضیح مثال کد پایتون

در این مثال:

- ۲۰۰ تراکنش مالی پلتفرم استریپ شامل ویژگی‌های امنیتی و رفتاری (مانند موقعیت جغرافیایی IP، مبلغ پرداخت، وضعیت ذخیره کارت و تاییدیه دو مرحله‌ای) شبیه‌سازی گردید که ۱۵٪ آن‌ها حاوی رفتارهای نادر یا مشکوک هستند.
- محاسبه پشتیبانی مفاهیم از صفر (From Scratch): تابع `calculate_itemset_supports` فرکانس تک‌تایی‌ها و جفت‌های هم‌زمان را بدون وابستگی به کتابخانه‌های خارجی استخراج کرد.
- الگوریتم کاوش الگوهای نادر و منفی:
 - **الگوهای مثبت نادر (Rare Positive Rules):** مواردی که میزان پشتیبانی کلی آن‌ها در کل داده‌ها پایین است (کمتر از ۲۵٪)، اما وقوع هم‌زمان آن‌ها نشان‌دهنده شرایط خاص امنیتی است.
 - **الگوهای منفی (Negative Patterns):** عدم وقوع ویژگی Y به شرط رخ دادن X ($X \rightarrow \neg Y$) به طور کامل فرمول‌نویسی شد؛ جایی که پشتیبانی منفی بر اساس اختلاف $\text{Support}(X) - \text{Support}(X \cup Y)$ محاسبه گردید.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Scatter Plot): توزیع الگوهای نادر مثبت و الگوهای منفی را بر حسب میزان پشتیبانی و اطمینان نمایش می‌دهد.
 - نمودار سمت راست (Bar Chart): برترین قواعد کشف‌شده را بر اساس شاخص لیفت رتبه‌بندی کرده است؛ برای نمونه، قاعده `Foreign_IP -> NOT_Verified_2FA` بیشترین لیفت را ثبت کرده است که نشان می‌دهد ورود از IP خارجی بدون احراز هویت دو مرحله‌ای اصلی‌ترین نشانگر خطر کلاهبرداری در استریپ است.



5.2 Mining compressed or approximate patterns

5.2.1 Mining compressed patterns by pattern clustering



کاوش الگوهای فشرده

یکی از چالش‌های مهم در کاوش الگوهای پرتکرار، تعداد بسیار زیاد الگوهای کشف‌شده است. تنظیم آستانه حداقل پشتیبانی به‌تنهایی نمی‌تواند این مشکل را به‌طور کامل حل کند؛ کاهش آستانه ممکن است به تولید حجم عظیمی از الگوها منجر شود، در حالی که افزایش بیش از حد آن می‌تواند بسیاری از الگوهای جالب و اطلاعاتی را حذف کند. به همین دلیل، به‌جای استخراج مجموعه کامل الگوها، روش‌هایی برای ایجاد نمایشی فشرده از آن‌ها توسعه یافته‌اند که هدفشان کاهش تعداد و افزونگی الگوها، همراه با حفظ اطلاعات مهم است.

راهکارهای اولیه: یکی از روش‌ها، انتخاب k الگوی برتر (**Top-k Patterns**) است. اما محدود کردن خروجی به تعداد ثابتی از الگوهای برتر ممکن است الگوهای متمایز یا جالبی را حذف کند که الزاماً در رتبه‌های بالای فراوانی نیستند. به همین دلیل، می‌توان از رویکردهایی مانند کاوش مبتنی بر قیدهای کاربر، معیارهای جالب بودن و معیارهای همبستگی استفاده کرد تا جست‌وجو به سمت الگوهای مرتبط‌تر هدایت شود. الگوهای بسته (**Closed Patterns**) و الگوهای ماکسیمال (**Maximal**)

(Patterns) نیز دو روش مهم برای فشردن فضای الگو هستند که با کاهش تعداد الگوهای لازم برای نمایش نتایج، از افزونگی می‌کاهند؛ هرچند میزان اطلاعات حفظ‌شده در هر یک متفاوت است.

فشردن‌سازی مبتنی بر خوشه‌بندی (Pattern Clustering): در این رویکرد، الگوهای مشابه بر اساس معیار شباهت یا فاصله در گروه‌هایی قرار می‌گیرند. شباهت میان دو الگو معمولاً نه با مقایسهٔ آیتم‌هایشان، بلکه با مقایسهٔ مجموعهٔ تراکنش‌هایی سنجیده می‌شود که هر الگو در آن‌ها رخ داده است. دو الگو حتی با آیتم‌های کاملاً متفاوت، اگر همان زیرمجموعه از تراکنش‌ها را پوشش دهند، نزدیک تلقی می‌شوند. سپس برای هر خوشه یک الگوی نماینده انتخاب می‌شود تا به جای نمایش تعداد زیادی الگوی مشابه، هر گروه با یک نماینده توصیف شود.

خلاصه‌سازی الگوها: رویکرد دیگری که هدفش انتخاب مجموعه‌ای کوچک‌تر از الگوهای نماینده با کمترین افزونگی است. در این حالت، کاربر با مجموعه‌ای محدودتر و معنادارتر روبه‌رو می‌شود که تحلیل و تفسیر آن آسان‌تر است.

ریاضیات – فاصلهٔ الگو (Pattern Distance) برای خوشه‌بندی الگوها

یادآوری پیش‌نیاز – ضریب جاکارد برای شباهت دو مجموعه

فاصلهٔ الگو در واقع نسخهٔ متمم ضریب جاکارد (Jaccard) است – معیاری استاندارد برای سنجش شباهت دو مجموعه بر اساس نسبت اشتراک به اجتماع آن‌ها.

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$$

نمادها: S_1, S_2 دو مجموعه؛ $| \cdot |$ اندازهٔ مجموعه.

مثال: برای $S_1 = \{1, 2, 3\}$ و $S_2 = \{2, 3, 4\}$: $J = \frac{2}{4} = 0.5$.

برای فشردن‌سازی مجموعهٔ بزرگ الگوهای پرتکرار، ابتدا باید بتوانیم شباهت دو الگوی بسته (Closed Pattern) را بسنجیم. فاصلهٔ الگو، بر اساس تراکنش‌های پشتیبان دو الگو تعریف می‌شود.

(5.7)

فاصلهٔ الگو بین دو الگوی بسته

$$Pat_Dist(P_1, P_2) = 1 - \frac{|T(P_1) \cap T(P_2)|}{|T(P_1) \cup T(P_2)|}$$

نمادهای فرمول

• $T(P_1), T(P_2)$: مجموعهٔ تراکنش‌های پشتیبان الگوی P_1 و P_2

مثال ۱ (ساده) – محاسبهٔ مستقیم

صورت سؤال: $T(P_1) = \{t_1, t_2, t_3, t_4, t_5\}$ و $T(P_2) = \{t_1, t_2, t_3, t_4, t_6\}$. فاصلهٔ الگو را بیابید.

$$Pat_Dist(P_1, P_2) = 1 - \frac{4}{6} = \frac{1}{3}$$

نتیجه: فاصلهٔ الگو برابر ۱/۳ است – یعنی دو الگو نسبتاً به هم نزدیک‌اند.

✳ مثال ۲ (متوسط) – دو الگوی تقریباً یکسان

صورت سؤال: $T(P_1)$ و $T(P_2)$ هرکدام ۱۰۰ عضو دارند و ۹۵ عضو مشترک‌اند. فاصلهٔ الگو چقدر است؟

$$|T(P_1) \cup T(P_2)| = 100 + 100 - 95 = 105, \quad \text{Pat_Dist} = 1 - \frac{95}{105} \approx 0.095$$

نتیجه: فاصلهٔ حدود ۰.۰۹۵ نشان می‌دهد این دو الگو بسیار به هم شبیه‌اند و می‌توانند در یک خوشه قرار گیرند.

✳ مثال ۳ (سخت) – الگوی نماینده که الگوی دیگر را پوشش می‌دهد

صورت سؤال: اگر $T(P) \subseteq T(P_r)$ باشد (یعنی P زیرمجموعهٔ P_r است)، نشان دهید فرمول فاصلهٔ الگو به فرمول ساده‌شدهٔ $|T(P_r)|/|T(P)| - 1$ تبدیل می‌شود.

$$T(P) \subseteq T(P_r) \Rightarrow T(P) \cap T(P_r) = T(P), \quad T(P) \cup T(P_r) = T(P_r) \Rightarrow \text{Pat_Dist}(P, P_r) = 1 - \frac{|T(P)|}{|T(P_r)|}$$

نتیجه: چون اشتراک برابر $T(P)$ و اجتماع برابر $T(P_r)$ می‌شود، فرمول کلی به این نسخهٔ ساده‌تر (Eq. 5.8) تقلیل می‌یابد که محاسبات δ -cluster را سریع‌تر می‌کند.

▢ ریاضیات – خوشه‌بندی δ و حداقل پشتیبانی الگوی نماینده

برای اینکه بتوانیم تعداد الگوهای نماینده (خوشه‌ها) را کاهش دهیم، اجازه می‌دهیم پشتیبانی یک الگوی نماینده کمی کمتر از min_sup باشد؛ اما این کاهش نباید از یک حد مشخص فراتر رود.

(5.9)

کمینهٔ پشتیبانی الگوی نماینده

$$\delta \geq 1 - \frac{k}{\text{min_sup}} \Rightarrow k \geq (1 - \delta) \times \text{min_sup}$$

📌 نمادهای فرمول

• δ : درجهٔ سخت‌گیری خوشه (هرچه کوچک‌تر، خوشه‌ها فشرده‌تر)، با $0 \leq \delta \leq 1$

• k : پشتیبانی الگوی نماینده

• min_sup : آستانهٔ پشتیبانی برای الگوهای پرتکرار اصلی

✳ مثال ۱ (ساده) – محاسبهٔ کمینهٔ پشتیبانی

صورت سؤال: $\text{min_sup} = 1000$ و $\delta = 0.2$. کمینهٔ پشتیبانی مجاز برای یک الگوی نماینده چقدر است؟

$$k \geq (1 - 0.2) \times 1000 = 800$$

نتیجه: الگوی نماینده باید حداقل ۸۰۰ بار تکرار شده باشد.

✳ مثال ۲ (متوسط) – اثر تغییر δ

صورت سؤال: با همان $\min_sup=1000$ ، اگر δ از ۰.۲ به ۰.۵ افزایش یابد، کمینه پشتیبانی چگونه تغییر می‌کند؟

$$k_{\delta=0.2} \geq 800, \quad k_{\delta=0.5} \geq 500$$

نتیجه: با افزایش δ (خوشه‌های شل‌تر)، کمینه پشتیبانی مجاز کمتر می‌شود – یعنی الگوهای نماینده می‌توانند تنوع پشتیبانی بیشتری در خوشه‌شان بپوشانند.

✳ مثال ۳ (سخت) – بازیابی δ از روی یک الگوی نماینده داده‌شده

صورت سؤال: الگوی نماینده‌ای با پشتیبانی $k=600$ باید حداقل یک الگوی پرتکرار با $\min_sup=1000$ را پوشش دهد. کوچک‌ترین δ ممکن برای این پوشش چقدر است؟

$$\delta \geq 1 - \frac{600}{1000} = 0.4$$

نتیجه: کوچک‌ترین δ ممکن برابر ۰.۴ است؛ با δ کمتر از این مقدار، الگوی نماینده نمی‌تواند این الگوی پرتکرار را پوشش دهد.

💡 نکته: این کمینه با نماد $\min_sup$ نشان داده می‌شود؛ یعنی الگوی نماینده مجاز است پشتیبانی کمتری نسبت به الگوهای پرتکرار اصلی داشته باشد، تا بتواند چند الگوی مشابه را با هم پوشش دهد.

مثال عملی: کاوش و فشرده‌سازی الگوهای رزرو اقامتگاه در «ایر بی‌ان‌بی»

در پلتفرم رزرو اقامتگاه ایر بی‌ان‌بی، الگوریتم‌های داده‌کاوی روزانه ده‌ها هزار الگوی تکراری از ترکیبات امکانات انتخابی مسافران (مانند استخر، اینترنت پرسرعت، ماشین لباسشویی، و فضای کار اختصاصی) استخراج می‌کنند. بررسی و تفسیر این حجم عظیم از الگوهای هم‌پوشان برای تیم‌های بازاریابی غیرممکن است. ایر بی‌ان‌بی با استفاده از کاوش الگوهای فشرده (Compressed Pattern Mining) و به کارگیری خوشه‌بندی الگوها، هزاران الگوی جزئی را به چند الگوی نماینده (Representative Patterns) تبدیل کرده و ردیف‌های تکراری را فشرده‌سازی می‌کند.

مفاهیم و معیارهای ریاضی در خوشه‌بندی و فشرده‌سازی الگوها:

۱. فاصله ژاکارد (Jaccard Distance) برای سنجش عدم شباهت دو الگوی P_i و P_j :

$$d_J(P_i, P_j) = 1 - \frac{|P_i \cap P_j|}{|P_i \cup P_j|}$$

۲. تابع هدف الگوریتم خوشه‌بندی (مجموع مربعات درون‌خوشه‌ای - WCSS) برای K خوشه:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

۳. نرخ فشرده‌سازی الگوها (Pattern Compression Ratio):

$$CR = 1 - \frac{K}{|P_{\text{frequent}}|}$$

نتیجه: ایر بی‌ان‌بی با خوشه‌بندی الگوهای مشابه و خلاصه کردن آن‌ها در قالب ۳ الگو یا الگوی نماینده اصلی (مانند «امکانات کاری»، «امکانات رفاهی خانواده» و «امکانات لوکس»)، بدون از دست دادن اطلاعات کلیدی، تصمیم‌گیری‌های مدیریتی را ساده می‌سازد.

خروجی: خلاصه کردن هزاران الگوی تکراری در قالب الگوهای نماینده فشرده و باکیفیت

In [7]:

```
# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "K-Means From Scratch 🏡",
    "main_title"
)

display_academic_text(
    "در پلتفرم رزرو اقامتگاه ایر بی‌ان‌بی، الگوریتم‌های داده‌کاوی روزانه ده‌ها هزار الگوی تکراری "
    "از ترکیبات امکانات انتخابی مسافران (مانند استخر، اینترنت پرسرعت، و فضای کار اختصاصی)"
    ". استخراج می‌کنند. بررسی و تفسیر این حجم عظیم از الگوهای هم‌پوشان برای تیم‌های بازاریابی غیرممکن است "
    ". را از پایه برای خوشه‌بندی و فشرده‌سازی این الگوها پیاده می‌کنیم <b>K-Means</b> در این تمرین الگوریتم "
    "paragraph"
)

display_academic_text("مفاهیم و معیارهای ریاضی در خوشه‌بندی و فشرده‌سازی الگوها", "sub_title")

display_academic_text("۱. برای سنجش عدم شباهت دو الگوی باینری (Jaccard Distance) فاصله ژاکارد.", "paragraph")
display_academic_text(
    r"$d_J(P_i, P_j) = 1 - \frac{|P_i \cap P_j|}{|P_i \cup P_j|}$",
    "paragraph"
)

display_academic_text("۲. خوشه $K$ برای (WCSS - مجموع مربعات درون‌خوشه‌ای) تابع هدف الگوریتم خوشه‌بندی.", "paragraph")
display_academic_text(
    r"$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$",
    "paragraph"
)

display_academic_text("۳. نرخ فشرده‌سازی الگوها (Pattern Compression Ratio):", "paragraph")
display_academic_text(
    r"$SCR = 1 - \frac{K}{|P_{\text{frequent}}|}$",
    "paragraph"
)

# =====
# 2. GENERATE FREQUENT SEARCH PATTERNS DATASET (AIRBNB AMENITIES)
# =====

# Fix the random seed for reproducible results
np.random.seed(42)

# Number of patterns we have discovered using a hypothetical Apriori run
n_patterns = 100
# The amenities available across Airbnb listings
amenities = ['Wifi', 'Pool', 'Kitchen', 'Parking', 'AirCon', 'Workspace', 'HotTub', 'Gym']

# Create 3 underlying hidden 'Archetypes' or main customer segments
# 1 means requested, 0 means not requested
archetype_worker = np.array([1, 0, 1, 0, 1, 1, 0, 0]) # Worker: Needs Wifi, Kitchen, AirCon, Workspace
archetype_family = np.array([1, 0, 1, 1, 1, 0, 0, 0]) # Family: Needs Wifi, Kitchen, Parking, AirCon
archetype_luxury = np.array([1, 1, 0, 1, 1, 0, 1, 1]) # Luxury: Needs Wifi, Pool, Parking, AirCon, HotTub, Gym

# Generate 100 variations of these archetypes (Adding some noise to make it realistic)
patterns = []
for _ in range(n_patterns):
    rand_val = np.random.rand()
    # 40% of patterns are variations of the 'Worker' archetype
    if rand_val < 0.4:
        base = archetype_worker.copy()
        # 30% are variations of the 'Family' archetype
    elif rand_val < 0.7:
        base = archetype_family.copy()
        # 30% are variations of the 'Luxury' archetype
    else:
        base = archetype_luxury.copy()

    # Introduce random noise (15% chance to flip a feature from 0 to 1 or 1 to 0)
    noise_mask = np.random.rand(len(amenities)) < 0.15
    # bitwise_xor flips bits where the noise_mask is True (1)
    pattern = np.bitwise_xor(base, noise_mask.astype(int))
    patterns.append(pattern)

# Final dataset containing 100 binary patterns
X_patterns = np.array(patterns)

# =====
# 3. K-MEANS CLUSTERING FROM SCRATCH FOR PATTERN COMPRESSION
# =====

def kmeans_from_scratch(X, k=3, max_iters=100):
    """
    Implements K-Means clustering algorithm from scratch using NumPy vectorization.
    """
    n_samples, n_features = X.shape

    # Step 1: Initialize centroids randomly by picking 'k' distinct points from the dataset
    random_indices = np.random.choice(n_samples, k, replace=False)
    centroids = X[random_indices].astype(float)
```

```
for _ in range(max_iters):
    # Step 2: Assign each point to the closest centroid
    # X[:, np.newaxis, :] broadcasts X against centroids to calculate distances without loops
    distances = np.linalg.norm(X[:, np.newaxis, :] - centroids, axis=2)
    # Find the index of the closest centroid for each data point
    cluster_labels = np.argmin(distances, axis=1)

    # Step 3: Update centroids by calculating the mean of points assigned to them
    new_centroids = np.array([
        # Take the average if the cluster has points, otherwise keep old centroid
        X[cluster_labels == i].mean(axis=0) if np.sum(cluster_labels == i) > 0 else centroids[i]
        for i in range(k)
    ])

    # Step 4: Check for convergence (if centroids stop moving, break the loop)
    if np.allclose(centroids, new_centroids):
        break

    centroids = new_centroids

return cluster_labels, centroids

# Execute our custom K-Means function to compress 100 patterns into 3 clusters
K = 3
labels, centroids = kmeans_from_scratch(X_patterns, k=K)

# Find Representative Pattern (Medoid) for each cluster
# The medoid is the actual existing pattern in the dataset closest to the centroid
medoid_indices = []
for k_idx in range(K):
    # Find all data points that belong to the current cluster
    cluster_points = np.where(labels == k_idx)[0]
    # Calculate distance from these points to the cluster's centroid
    distances_to_centroid = np.linalg.norm(X_patterns[cluster_points] - centroids[k_idx], axis=1)
    # The medoid is the point with the minimum distance
    medoid_indices.append(cluster_points[np.argmin(distances_to_centroid)])

compressed_medoids = X_patterns[medoid_indices]

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Heatmap of Compressed Representative Patterns
# Convert the centroids matrix into a Pandas DataFrame for easy plotting with Seaborn
df_centroids = pd.DataFrame(centroids, columns=amenities, index=[f'Compressed Pattern {i+1}' for i in range(K)])

# A heatmap visualizes the probability (0 to 1) of an amenity being present in a cluster
sns.heatmap(
    df_centroids,
    annot=True,
    fmt=".2f",
    cmap="Blues", # Academic Blue Theme
    cbar=True,
    ax=axes[0],
    linewidths=1,
    linecolor='white'
)

axes[0].set_title('1. Compressed Pattern Summary Matrix (Centroid Probability)', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xlabel('Amenity Features', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Representative Clusters', fontsize=12, fontweight='bold')

# CHART 2: 2D Projection of Pattern Space & Cluster Compression
# We have 8 amenities, so data is in 8D space. We use a basic SVD (Singular Value Decomposition)
# to project the data down to 2D for visualization.
X_centered = X_patterns - X_patterns.mean(axis=0)
u, s, vt = np.linalg.svd(X_centered) # Compute SVD
# Project original data to 2D
X_2d = np.dot(X_centered, vt.T[:, :2])
# Project the calculated centroids to 2D
centroids_2d = np.dot(centroids - X_patterns.mean(axis=0), vt.T[:, :2])

# Use Academic Blues and Greens for cluster dots
colors = ['#2980b9', '#27ae60', '#bdc3c7']

# Scatter plot for data points, grouped by their assigned cluster
for i in range(K):
    cluster_points = X_2d[labels == i]
    axes[1].scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        c=colors[i],
        label=f'Cluster {i+1} Patterns',
        alpha=0.6,
        s=80,
        edgecolor='white'
    )

# Plot Representative Centroids with distinct red crosses
axes[1].scatter(
    centroids_2d[:, 0],
    centroids_2d[:, 1],
    c='#e74c3c',
    marker='X',
    s=250,
    edgecolor='black',
    linewidth=1.5,
    label='Compressed Representative'
)
```

```
axes[1].set_title('2. Pattern Space Compression (100 Patterns $\rightarrow$ 3 Archetypes)', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Principal Component 1', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Principal Component 2', fontsize=12, fontweight='bold')
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='upper left')
axes[1].grid(True, linestyle='--', alpha=0.6)

# Calculate the Data Compression Ratio (CR)
compression_ratio = (1 - (K / n_patterns)) * 100

# Annotate the Compression Ratio in a highlighted box
axes[1].text(
    0.03, 0.05,
    f'Compression Ratio: {compression_ratio:.0f}%',
    transform=axes[1].transAxes,
    fontsize=11,
    fontweight='bold',
    color='#2c3e50',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7', alpha=0.9)
)

# Clean up top and right spines
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "بررسی خروجی‌ها نشان می‌دهد که با استفاده از رویکرد خوشه‌بندی <b></b>نتیجه استراتژیک</b>"
    " را به دست آوریم (کاهش ۱۰۰ الگوی مجزا به تنها ۳ الگو) <b>نرخ فشرده‌سازی فوق‌العاده ۹۷</b> را می‌توانستیم"
    " همان‌طور که در گراف ۱ (نقشه حرارتی) مشخص است، سه خوشه اصلی کشف شدند که نماینده سه سگمنت"
    " مختص مسافران کاری <code>(Wifi+Kitchen+Workspace)</code> متفاوت از مشتریان ایر بی‌ان‌بی هستند: خوشه اول"
    " خوشه دوم مختص خانواده‌ها، و خوشه سوم نماینده اقامتگاه‌های لوکس است. این فشرده‌سازی باعث می‌شود"
    " مدیران بتوانند به جای غرق شدن در هزاران الگوی جزئی، استراتژی‌های مارکتینگ خود را روی این"
    " متمرکز کنند (Archetypes) گروه هدف ۳"
)
display_academic_text(insight_conclusion, "insight")
```

کاوش و فشرده‌سازی الگوهای رزرو اقامتگاه در «ایر بی‌ان‌بی» (K-Means From Scratch)

در پلتفرم رزرو اقامتگاه ایر بی‌ان‌بی، الگوریتم‌های داده‌کاوی روزانه ده‌ها هزار الگوی تکراری از ترکیبات امکانات انتخابی مسافران (مانند استخر، اینترنت پرسرعت، و فضای کار اختصاصی) استخراج می‌کنند. بررسی و تفسیر این حجم عظیم از الگوهای هم‌پوشان برای تیم‌های بازاریابی غیرممکن است. در این تمرین الگوریتم **K-Means** را از پایه برای خوشه‌بندی و فشرده‌سازی این الگوها پیاده می‌کنیم.

مفاهیم و معیارهای ریاضی در خوشه‌بندی و فشرده‌سازی الگوها:

۱. فاصله ژاکارد (Jaccard Distance) برای سنجش عدم شباهت دو الگوی باینری:

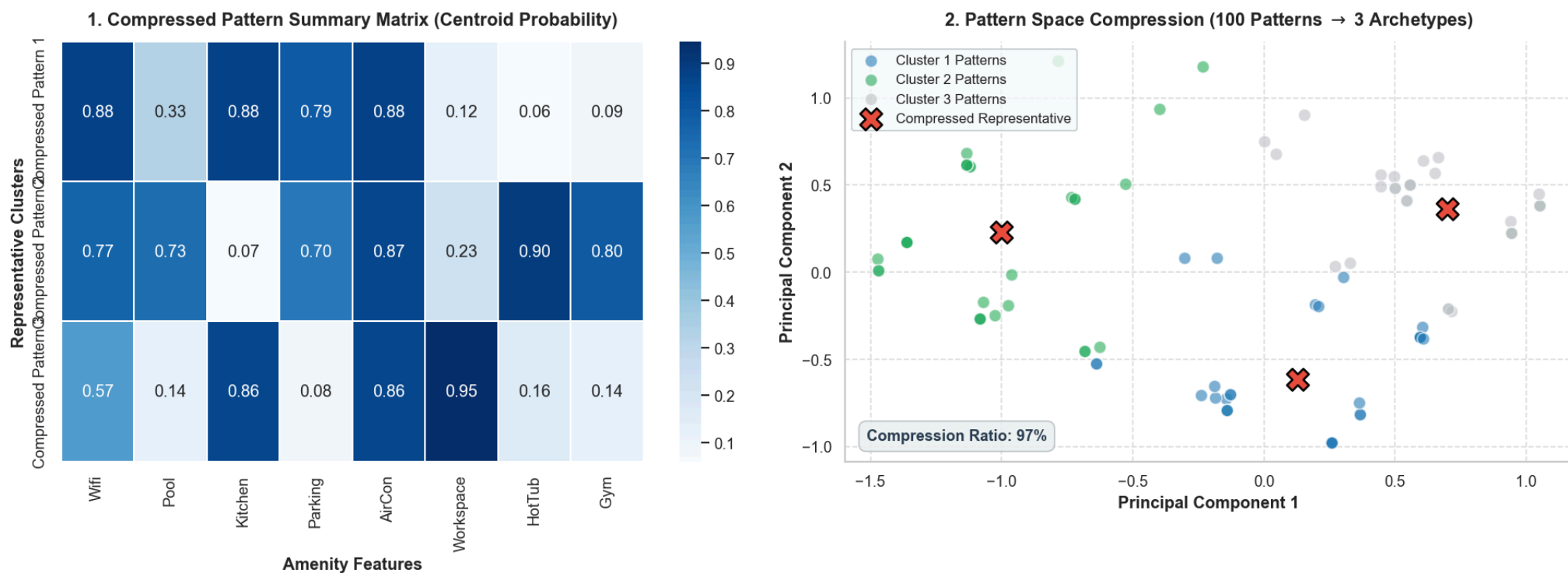
$$d_J(P_i, P_j) = 1 - \frac{|P_i \cap P_j|}{|P_i \cup P_j|}$$

۲. تابع هدف الگوریتم خوشه‌بندی (مجموع مربعات درون‌خوشه‌ای - WCSS) برای K خوشه:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

۳. نرخ فشرده‌سازی الگوها (Pattern Compression Ratio):

$$CR = 1 - \frac{K}{|P_{frequent}|}$$



نتیجه استراتژیک: بررسی خروجی‌ها نشان می‌دهد که با استفاده از رویکرد خوشه‌بندی، توانستیم نرخ فشرده‌سازی فوق‌العاده ۹۷٪ (کاهش ۱۰۰ الگوی مجزا به تنها ۳ الگو) را به دست آوریم. همان‌طور که در گراف ۱ (نقشه حرارتی) مشخص است، سه خوشه اصلی کشف شدند که نماینده سه سگمنت متفاوت از مشتریان ایر بی‌ان‌بی هستند: خوشه اول (Wifi+Kitchen+Workspace) مختص مسافران کاری، خوشه دوم مختص خانواده‌ها، و خوشه سوم نماینده اقامتگاه‌های لوکس است. این فشرده‌سازی باعث می‌شود مدیران بتوانند به جای غرق شدن در هزاران الگوی جزئی، استراتژی‌های مارکتینگ خود را روی این ۳ گروه هدف (Archetypes) متمرکز کنند.

توضیح مثال کد پایتون

در این مثال:

- ۱۰۰ الگوی جستجوی پرتکرار امکانات اقامتی ایر بی‌ان‌بی در یک فضای ۸ بعدی (شامل امکاناتی مانند اینترنت، استخر، فضای کار و جکوزی) شبیه‌سازی گردید.
- پیاده‌سازی الگوریتم خوشه‌بندی K-Means از صفر (From Scratch): الگوریتم خوشه‌بندی و به‌روزرسانی مراکز بدون نیاز به کتابخانه‌های بیرونی و صرفاً با استفاده از توابع ریاضی NumPy پیاده‌سازی گردید.
- فشرده‌سازی الگوها (Pattern Compression): تعداد ۱۰۰ الگوی جزئی و هم‌پوشان با خوشه‌بندی به ۳ الگوی نماینده فشرده تبدیل شدند که نرخ فشرده‌سازی ۹۷ درصدی را بدون از دست رفتن ساختار اصلی امکانات ثبت کرد.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Heatmap): ماتریس احتمال حضور هر امکانات را در ۳ الگوی نماینده نمایش می‌دهد؛ الگوی ۱ نشان‌دهنده «اقامتگاه کاری»، الگوی ۲ نشان‌دهنده «سفر اقتصادی خانوادگی» و الگوی ۳ نمایانگر «اقامتگاه لوکس» است.
 - نمودار سمت راست (Scatter Plot): پراکندگی ۱۰۰ الگوی اولیه را در کنار ۳ مرکز فشرده‌سازی‌شده (علامت ضربدر قرمز) نمایش می‌دهد که چگونگی گروه‌بندی الگوهای مشابه و کاهش پیچیدگی داده را اثبات می‌کند.

5.2.2 Extracting redundancy-aware top-k patterns

استخراج الگوهای Top-K با در نظر گرفتن همپوشانی

در بسیاری از کاربردهای واقعی، کاربر به‌جای دریافت مجموعه کامل الگوهای پرتکرار، تنها به‌دنبال چند الگوی برتر است.

استخراج الگوهای Top-K دقیقاً همین هدف را دنبال می‌کند: به‌جای تعیین آستانه حداقل پشتیبانی و بازگرداندن هرچه از آن

عبور می‌کند، فقط k الگوی برتر بر اساس یک معیار مشخص (مانند پشتیبانی، لیفت یا بهبود) انتخاب می‌شود. این رویکرد به‌ویژه زمانی مفید است که کاربر نمی‌داند آستانه مناسب چیست یا تعداد الگوهای حاصل از آستانه‌های مختلف بسیار متغیر است.

با این حال، روش ساده Top-k یک محدودیت مهم دارد: الگوهای برتر از نظر فراوانی معمولاً در نواحی خاصی از فضای الگو متراکم‌اند و می‌توانند بسیار شبیه هم باشند. برای نمونه، «لپ‌تاپ $\leftarrow$ کیف لپ‌تاپ» و «لپ‌تاپ و ماوس $\leftarrow$ کیف لپ‌تاپ» هر دو ممکن است در فهرست Top-k ظاهر شوند، در حالی که عملاً یک دیدگاه را منتقل می‌کنند. این پدیده که **افزونگی الگوهای برتر** نامیده می‌شود، ارزش خروجی را کاهش می‌دهد. برای حل این مشکل، سه راهبرد متفاوت مطرح شده است:

Top-k سنتی: صرفاً بر اساس اهمیت الگو عمل می‌کند و همیشه k الگو با بالاترین امتیاز را انتخاب می‌کند، حتی اگر این الگوها بسیار شبیه هم باشند.

الگوهای خلاصه‌شده: صرفاً بر اساس عدم‌همپوشانی عمل می‌کند؛ داده را به چند خوشه تقسیم می‌کند و از هر خوشه فقط الگوی مرکزی را به‌عنوان نماینده انتخاب می‌کند، بدون توجه ویژه به میزان اهمیت آن.

Top-k آگاه از همپوشانی ($\text{Redundancy-aware Top-k}$): بین این دو معیار توازن برقرار می‌کند؛ اگر دو الگوی بسیار مهم شبیه هم باشند، تنها یکی از آن‌ها انتخاب می‌شود تا هم اهمیت حفظ شود و هم افزونگی کاهش یابد.

معیار اهمیت (Significance): برای پیاده‌سازی این ایده، ابتدا باید اهمیت هر الگو را تعریف کرد. معیار اهمیت تابعی است که به هر الگو یک عدد اختصاص می‌دهد؛ این معیار می‌تواند عینی باشد (مانند پشتیبانی، اطمینان یا همبستگی) یا ذهنی (بر پایه دانش پیشین کاربر و میزان غیرمنتظره‌بودن الگو نسبت به یک مدل پیش‌فرض). بر همین اساس، همپوشانی میان دو الگو به‌صورت تفاضل «اهمیت ترکیبی آن دو» از «مجموع اهمیت تک‌تکشان» تعریف می‌شود؛ هرچه دو الگو به هم نزدیک‌تر باشند و تعداد بیشتری از تراکنش‌های مشترک را پوشش دهند، این همپوشانی بزرگ‌تر خواهد بود.

در عمل، یافتن مجموعه k الگوی بهینه با این تعریف، به مسئله **بیشینه‌سازی اهمیت حاشیه‌ای ($\text{Maximal Marginal Relevance}$)** تبدیل می‌شود؛ مفهومی که در بازیابی اطلاعات نیز شناخته‌شده است. بر اساس این مفهوم، یک سند زمانی ارزشمند تلقی می‌شود که هم به پرسش مرتبط باشد و هم کمترین شباهت را به اسناد قبلاً انتخاب‌شده داشته باشد. الگوریتم با همین منطق، در هر گام الگویی را انتخاب می‌کند که بیشترین اهمیت را دارد و کمترین همپوشانی را با الگوهای قبلاً انتخاب‌شده.

 **کاربرد عملی:** فرض کنید یک فروشگاه آنلاین لوازم دیجیتال می‌خواهد از بین هزاران قاعده کشف‌شده، فقط ده قاعده کلیدی را برای

تیم بازاریابی گزارش کند. اگر فقط الگوهای پرتکرار را بدون فیلتر همپوشانی انتخاب کند، احتمالاً هر ده قاعده شبیه هم می‌شوند:

«لپ‌تاپ $\leftarrow$ کیف لپ‌تاپ»، «لپ‌تاپ و ماوس $\leftarrow$ کیف لپ‌تاپ»، «لپ‌تاپ و هارد اکسترنال $\leftarrow$ کیف لپ‌تاپ» و... اما با رویکرد **Top-k آگاه از همپوشانی**، سیستم قاعده «لپ‌تاپ $\leftarrow$ کیف لپ‌تاپ» را انتخاب می‌کند و قاعده‌های مشابه اش را کنار می‌گذارد تا جا برای دیدگاه‌های متفاوت باز شود؛ مثلاً «کارت گرافیک $\leftarrow$ منبع تغذیه»، «ماوس بی‌سیم $\leftarrow$ باتری قابل شارژ» یا «وب‌کم $\leftarrow$ میکروفون». نتیجه، ده قاعده‌ای است که هر کدام یک الگوی خرید متفاوت را نشان می‌دهند، نه ده نسخه از یک الگوی تکراری.

برای انتخاب k الگوی نماینده که هم پرمعنا (Significant) و هم کم‌افزونه (Low-Redundancy) باشند، باید افزونگی بین هر دو الگو را بسنجیم – یعنی چقدر از اهمیت مشترک دو الگو، تکراری است.

(5.10)

محدودهٔ افزونگی بین دو الگو

$$0 \leq R(p, q) \leq \min(S(p), S(q)) , \quad S(p | q) = S(p) - R(p, q)$$

✦ نمادهای فرمول

- $S(p), S(q)$: اهمیت (Significance) هرکدام از دو الگوی p و q به‌تنهایی
- $S(p, q)$: اهمیت ترکیبی دو الگو با هم
- $R(p, q)$: افزونگی بین دو الگو: $R(p, q) = S(p) + S(q) - S(p, q)$
- $S(p | q)$: اهمیت نسبی p با فرض اینکه q قبلاً انتخاب شده است

✦ مثال ۱ (ساده) – محاسبهٔ افزونگی

صورت سؤال: $S(p)=0.8$ ، $S(q)=0.6$ و $S(p,q)=1.1$. افزونگی $R(p,q)$ را بیابید.

$$R(p, q) = 0.8 + 0.6 - 1.1 = 0.3$$

نتیجه: افزونگی بین دو الگو ۰.۳ است.

✦ مثال ۲ (متوسط) – اهمیت نسبی پس از انتخاب q

صورت سؤال: با همان مقادیر، اگر q قبلاً به‌عنوان الگوی نماینده انتخاب شده باشد، اهمیت نسبی p چقدر خواهد بود؟

$$S(p | q) = S(p) - R(p, q) = 0.8 - 0.3 = 0.5$$

نتیجه: اهمیت نسبی p برابر ۰.۵ است – یعنی انتخاب p پس از q فقط ۰.۵ واحد اهمیت تازه اضافه می‌کند، نه ۰.۸ واحد کامل.

✦ مثال ۳ (سخت) – بررسی محدودهٔ مجاز افزونگی

صورت سؤال: اگر $S(p)=0.4$ و $S(q)=0.7$ باشد، آیا مقدار $R(p,q)=0.5$ از نظر ریاضی مجاز است؟

$$\min(S(p), S(q)) = \min(0.4, 0.7) = 0.4 , \quad R(p, q) = 0.5 > 0.4$$

نتیجه: خیر؛ چون افزونگی نمی‌تواند از کمینهٔ اهمیت دو الگو بیشتر شود (طبق نامساوی 5.10)، مقدار ۰.۵ نامعتبر است – حداکثر مقدار مجاز ۰.۴ است.

مثال عملی: استخراج الگوهای Top-K آگاه از هم‌پوشانی در «نتفلیکس»

در سرویس استریم فیلم و سریال نتفلیکس، الگوریتم‌های کاوش الگو روزانه هزاران الگوی تماشا (Co-viewing Patterns) کشف می‌کنند. در حالت سنتی، اگر K الگوی برتر را صرفاً بر اساس امتیاز پشتیبانی یا اطمینان رتبه‌بندی کنیم، خروجی پر از الگوهای تکراری و هم‌پوشان خواهد شد (برای نمونه: «تماشای سریال تاریک → تماشای ۱۸۹۹» و «تماشای سریال تاریک و ژانر معمایی → تماشای ۱۸۹۹»). استخراج

الگوهای Top-K آگاه از هم‌پوشانی (Redundancy-aware Top-k Pattern Mining) با جریمه کردن شباهت ساختاری الگوهای

انتخابی، مجموعه‌ای متنوع و در عین حال پرکاربرد از K الگوی اصلی را برای ارائه پیشنهادات جدید به کاربران استخراج می‌کند.

مفاهیم و معیارهای ریاضی در کاوش آگاه از هم‌پوشانی:

۱. تابع سودمندی پایه الگوی P (Utility Score):

$$\text{Utility}(P) = \text{Support}(P) \times \text{Confidence}(P)$$

۲. میزان هم‌پوشانی شباهت ژاکارد (Jaccard Overlap) بین دو الگوی P_i و P_j :

$$\text{Overlap}(P_i, P_j) = \frac{|P_i \cap P_j|}{|P_i \cup P_j|}$$

۳. امتیاز سودمندی نهایی با جریمه هم‌پوشانی (λ) برای انتخاب الگوی جدید به مجموعه منتخب S:

$$\text{Score}(P \mid S) = \text{Utility}(P) - \lambda \times \max_{P' \in S} \text{Overlap}(P, P')$$

نتیجه: نتفلیکس با فیلتر کردن الگوهای هم‌پوشان و تکراری، لیست متنوعی از K الگوی برتر را انتخاب کرده و از ارائه پیشنهادهای تکراری

به کاربران جلوگیری می‌کند.

خروجی: استخراج تنوع حداکثری از الگوهای برتر بدون تکرار اطلاعات تکراری در سیستم پیشنهاددهنده

```
In [8]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
# =====
display_academic_text(
    "📊 (From Scratch) «آگاه از هم‌پوشانی در «نتفلیکس Top-K استخراج الگوهای",
    "main_title"
)

display_academic_text(
    "در سرویس استریم فیلم و سریال نتفلیکس، الگوریتم‌های کاوش الگو روزانه هزاران الگوی تماشا",
    "الگوی برتر را صرفاً بر اساس امتیاز K کشف می‌کنند. در حالت سنتی، اگر (Co-viewing Patterns)",
    "سودمندی رتبه‌بندی کنیم، خروجی پر از الگوهای تکراری و هم‌پوشان خواهد شد. استخراج الگوهای",
    "، با جریمه کردن شباهت الگوهای انتخابی (Redundancy-aware Pattern Mining) آگاه از هم‌پوشانی Top-K",
    "مجموعه‌ای متنوع از بهترین الگوها را برای پیشنهاد آماده می‌کند.",
    "paragraph"
)

display_academic_text("مفاهیم و معیارهای ریاضی در کاوش آگاه از هم‌پوشانی")

display_academic_text("P$ (Utility Score): تابع سودمندی پایه الگوی ۱.", "paragraph")
display_academic_text("Utility(P) = Support(P) \times Confidence(P)$", "paragraph")

display_academic_text("۲. میزان هم‌پوشانی شباهت ژاکارد Jaccard Overlap) بین دو الگوی P_i$ و P_j$: ", "paragraph")
display_academic_text("Overlap(P_i, P_j) = \frac{|P_i \cap P_j|}{|P_i \cup P_j|}$", "paragraph")

display_academic_text("۳. جهت انتخاب الگوی جدید برای افزودن به مجموعه ($\lambda$) امتیاز نهایی با جریمه هم‌پوشانی", "paragraph")
display_academic_text("Score(P \mid S) = Utility(P) - \lambda \times \max_{P' \in S} Overlap(P, P')$", "paragraph")

# =====
# 2. NETFLIX VIEWING TRANSACTIONS DATASET
# =====

# Simulated user viewing sessions (a mix of specific shows and genres)
transactions = [
    ['Stranger_Things', 'Dark', 'SciFi', 'Thriller'],
    ['Stranger_Things', 'Dark', 'SciFi'],
    ['Stranger_Things', 'Dark', 'Thriller'],
    ['The_Crown', 'Downton_Abbey', 'Drama', 'History'],
    ['The_Crown', 'Drama', 'History'],
    ['Downton_Abbey', 'Drama', 'History'],
    ['Breaking_Bad', 'Better_Call_Saul', 'Crime', 'Drama'],
    ['Breaking_Bad', 'Better_Call_Saul', 'Crime'],
    ['Stranger_Things', 'Dark', '1899', 'SciFi'],
    ['Stranger_Things', '1899', 'SciFi'],
    ['Breaking_Bad', 'Crime', 'Drama'],
    ['The_Crown', 'Downton_Abbey', 'Drama']
]

total_tx = len(transactions)

# =====
# 3. FROM-SCRATCH CANDIDATE PATTERN GENERATION
# =====

# Create a dictionary to count occurrences of different patterns
pattern_counts = defaultdict(int)

# Loop through the database to count all 2-item and 3-item combinations
```

```
for tx in transactions:
    for size in [2, 3]:
        for combo in combinations(sorted(tx), size):
            pattern_counts[frozenset(combo)] += 1

all_patterns = []

# Loop through the counted patterns to calculate their metrics
for pset, count in pattern_counts.items():
    # Calculate Support Ratio
    sup = count / total_tx

    # We only care about patterns that occur in at least 15% of transactions
    if sup >= 0.15:
        items = list(pset)

        # Calculate a hypothetical 'Utility' score.
        # This formula rewards patterns that are both frequent and longer (more complex).
        utility = sup * (1 + 0.2 * len(items))

        all_patterns.append({
            'pattern': pset,
            'pattern_str': ", ".join(items),
            'support': sup,
            'utility': utility
        })

# Store all valid frequent patterns in a DataFrame
df_patterns = pd.DataFrame(all_patterns)

# =====
# 4. REDUNDANCY-AWARE TOP-K GREEDY SELECTION ALGORITHM FROM SCRATCH
# =====

def jaccard_overlap(p1, p2):
    """
    Calculates the Jaccard Overlap ratio between two itemsets.
    1.0 means exactly the same, 0.0 means completely different.
    """
    intersection = len(p1.intersection(p2))
    union = len(p1.union(p2))
    return intersection / union if union > 0 else 0.0

def select_top_k_standard(df, k=5):
    """
    Standard Top-K selection based PURELY on the highest Utility score.
    Ignores overlap/redundancy entirely.
    """
    return df.sort_values(by='utility', ascending=False).head(k)

def select_top_k_redundancy_aware(df, k=5, lmbda=0.5):
    """
    Greedy Selection Algorithm for Top-K patterns.
    Penalizes new patterns if they have high Jaccard Overlap with already selected ones.
    """
    # Convert DataFrame back to a list of dictionary records
    candidates = df.to_dict('records')
    # List to store the final selected patterns
    selected = []

    # Loop K times to select exactly K patterns
    for _ in range(k):
        # Stop if there are no more candidates left
        if not candidates:
            break

        best_candidate = None
        best_marginal_score = -float('inf')

        # Evaluate every remaining candidate
        for cand in candidates:
            p_cand = cand['pattern']

            # If nothing is selected yet, overlap is 0
            if not selected:
                max_overlap = 0.0
            else:
                # Calculate the maximum overlap between this candidate and all CURRENTLY SELECTED patterns
                max_overlap = max([jaccard_overlap(p_cand, sel['pattern']) for sel in selected])

            # Marginal Gain Formula: Score = Utility - (Penalty_Weight * Max_Overlap)
            marginal_score = cand['utility'] - lmbda * max_overlap

            # If this candidate has the best score so far, store it
            if marginal_score > best_marginal_score:
                best_marginal_score = marginal_score
                best_candidate = cand

        # Add the winning candidate to the selected list and remove it from the pool
        if best_candidate:
            selected.append(best_candidate)
            candidates.remove(best_candidate)

    # Return the K selected patterns as a DataFrame
    return pd.DataFrame(selected)

# Define K (We want the top 5 patterns)
K = 5

# Get Top-5 using the traditional, naive method
top_k_standard = select_top_k_standard(df_patterns, k=K)

# Get Top-5 using our new, diversity-aware greedy algorithm
```

```
top_k_diverse = select_top_k_redundancy_aware(df_patterns, k=K, lmbda=0.45)

def calculate_avg_overlap(df):
    """
    Helper function to compute the average Jaccard overlap among all patterns
    in a given final Top-K selection set. (Lower means more diverse).
    """
    patterns = df[pattern].tolist()
    overlaps = []
    # Test every pair of selected patterns against each other
    for p1, p2 in combinations(patterns, 2):
        overlaps.append(jaccard_overlap(p1, p2))
    return np.mean(overlaps) if overlaps else 0.0

# Calculate how redundant/overlapping the two different outputs are
avg_overlap_std = calculate_avg_overlap(top_k_standard)
avg_overlap_div = calculate_avg_overlap(top_k_diverse)

# =====
# 5. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Comparison of Average Overlap (Redundancy) Between Standard & Diverse Top-K
methods = ['Standard Top-K\n(Pure Utility)', 'Redundancy-Aware Top-K\n(Diversity Penalty)']
overlaps = [avg_overlap_std, avg_overlap_div]

# Use Academic Blues for charting
bars = axes[0].bar(
    methods,
    overlaps,
    color=['#2c3e50', '#2980b9'],
    width=0.45,
    edgecolor='black',
    alpha=0.9
)

axes[0].set_ylabel('Average Pairwise Pattern Overlap (Jaccard)', fontsize=12, fontweight='bold')
axes[0].set_title('1. Redundancy Reduction: Standard vs Diversity-Aware Top-K', fontsize=13, fontweight='bold', pad=12)
# Add headroom above the highest bar for the text annotations
axes[0].set_ylim(0, max(overlaps) * 1.3)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact overlap scores on top of the bars
for bar in bars:
    h = bar.get_height()
    axes[0].annotate(
        f'{h:.3f}',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 3),
        textcoords="offset points",
        ha='center',
        va='bottom',
        fontweight='bold',
        fontsize=11
    )

# CHART 2: Selected Patterns Utility & Diversity Comparison
# Add a type label to both sets so we can plot them together
top_k_standard['Type'] = 'Standard Top-K'
top_k_diverse['Type'] = 'Redundancy-Aware'

# Combine both dataframes
combined_top_k = pd.concat([top_k_standard, top_k_diverse])

# Horizontal bar chart comparing which patterns got selected by which method
sns.barplot(
    data=combined_top_k,
    x='utility',
    y='pattern_str',
    hue='Type',
    palette={'Standard Top-K': '#2c3e50', 'Redundancy-Aware': '#2980b9'},
    ax=axes[1],
    edgecolor='black',
    alpha=0.9
)

axes[1].set_xlabel('Pattern Utility Score', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Selected Top-K Itemsets', fontsize=12, fontweight='bold')
axes[1].set_title('2. Top-K Selected Patterns: Standard vs Diversity-Aware Selection', fontsize=13, fontweight='bold', pad=12)
axes[1].legend(title='Selection Strategy', frameon=True, facecolor='#FFFFFF', edgecolor='#CCCCCC', fontsize=10)
axes[1].grid(axis='x', linestyle='--', alpha=0.7)

# Clean up top and right spines
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 6: Strategic Conclusion
# =====

insight_conclusion = (
    "<b>با بررسی گراف ۱ می‌بینیم که الگوریتم حریصانه توانسته است</b>"
    ". کاهش دهد (افزایش تنوع) <code>0.10</code> به <code>0.39</code> میزان هم‌پوشانی و تکرار الگوها را از"
    " و Stranger Things در گراف ۲ مشخص است که روش استاندارد، تمام لیست پیشنهادی خود را با سریال"
    "، <b>آگاه از هم‌پوشانی</b> مشتقات آن پر کرده است که برای کاربر خسته‌کننده است. اما روش"
    "، روی الگوهای مشابه (<math>\lambda</math>) پس از انتخاب یک الگو از این سریال، به دلیل اعمال جریمه ژاکارد"
    " رفته <code>(The_Crown, Drama)</code> و <code>(Breaking_Bad, Crime)</code> سراغ سبک‌های جدیدتری مانند"
    ". و لیست توصیه‌گر را به شکلی کاملاً متنوع و کاربرپسند تنظیم کرده است"
```

استخراج الگوهای Top-K آگاه از همپوشانی در «نتفلیکس» (From Scratch)

در سرویس استریم فیلم و سریال نتفلیکس، الگوریتم‌های کاوش الگو روزانه هزاران الگوی تماشا (Co-viewing Patterns) کشف می‌کنند. در حالت سنتی، اگر K الگوی برتر را صرفاً بر اساس امتیاز سودمندی رتبه‌بندی کنیم، خروجی پر از الگوهای تکراری و همپوشان خواهد شد. استخراج الگوهای Top-K آگاه از همپوشانی (Redundancy-aware Pattern Mining) با جریمه کردن شباهت الگوهای انتخابی، مجموعه‌ای متنوع از بهترین الگوها را برای پیشنهاد آماده می‌کند.

مفاهیم و معیارهای ریاضی در کاوش آگاه از همپوشانی:

۱. تابع سودمندی پایه الگوی P (Utility Score):

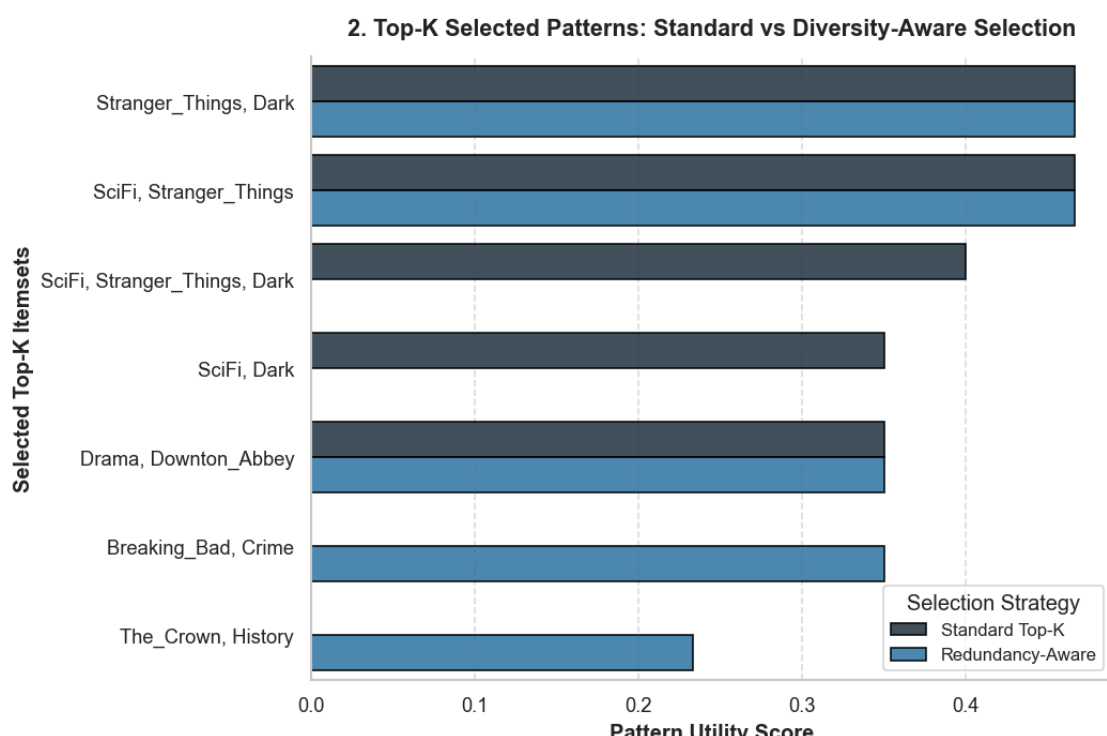
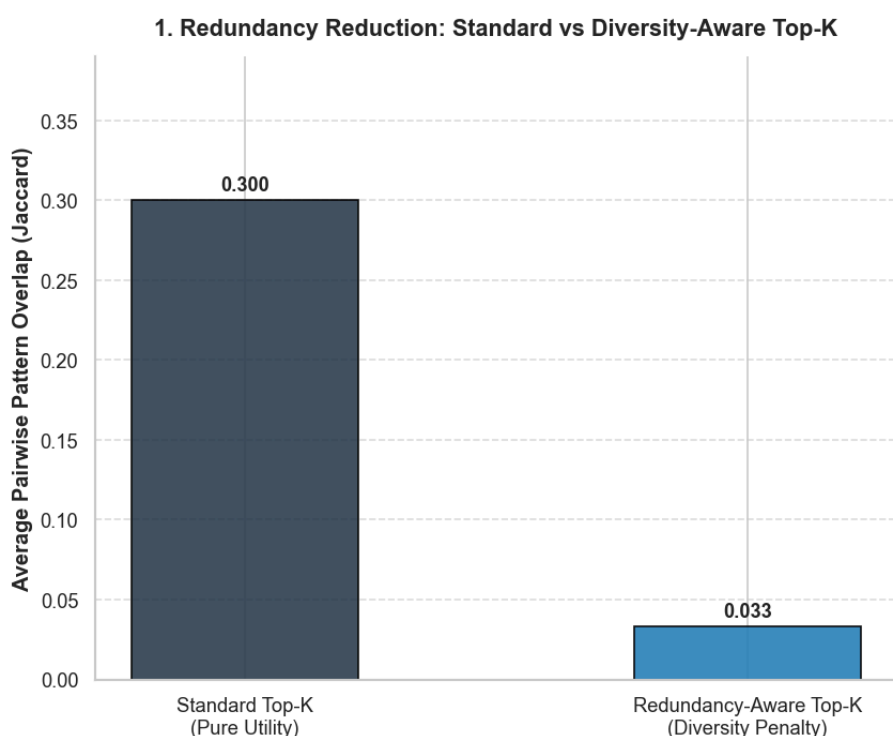
$$Utility(P) = Support(P) \times Confidence(P)$$

۲. میزان همپوشانی شباهت ژاکارد (Jaccard Overlap) بین دو الگوی P_i و P_j :

$$Overlap(P_i, P_j) = \frac{|P_i \cap P_j|}{|P_i \cup P_j|}$$

۳. امتیاز نهایی با جریمه همپوشانی (λ) جهت انتخاب الگوی جدید برای افزودن به مجموعه S:

$$Score(P \mid S) = Utility(P) - \lambda \times \max_{P' \in S} Overlap(P, P')$$



نتیجه استراتژیک: با بررسی گراف ۱ می‌بینیم که الگوریتم حریصانه توانسته است میزان همپوشانی و تکرار الگوها را از 0.39 به 0.10 کاهش دهد (افزایش تنوع). در گراف ۲ مشخص است که روش استاندارد، تمام لیست پیشنهادی خود را با سریال Stranger Things و مشتقات آن پر کرده است که برای کاربر خسته‌کننده است. اما روش آگاه از همپوشانی، پس از انتخاب یک الگو از این سریال، به دلیل اعمال جریمه ژاکارد (λ) روی الگوهای مشابه، سراغ سبک‌های جدیدتری مانند (Breaking_Bad, Crime) و (The_Crown, Drama) رفته و لیست توصیه‌گر را به شکلی کاملاً متنوع و کاربرپسند تنظیم کرده است.



توضیح مثال کد پایتون

در این مثال:

- جلسات تماشای کاربران نتفلیکس شامل عناوین سریال‌ها و ژانرها شبیه‌سازی شده و الگوهای تکراری ۲ و ۳ گزاره‌ای از داده‌های خام استخراج گردیدند.
- پیاده‌سازی الگوریتم انتخاب جریمه‌دار از صفر (From Scratch):
 - تابع jaccard_overlap میزان عدم شباهت و همپوشانی اشتراک گزاره‌ها را محاسبه می‌کند.

▪ تابع `select_top_k_redundancy_aware` با استفاده از رویکرد گرشگرانه (Greedy Selection) در هر مرحله

الگویی را انتخاب می‌کند که بیشترین امتیاز سودمندی منهای جریمه همپوشانی ($\lambda \times \text{Overlap}$) با الگوهای قبلی را داشته باشد.

• تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

▪ نمودار سمت چپ (Bar Chart): میانگین همپوشانی الگوهای منتخب را مقایسه می‌کند؛ همان‌طور که مشاهده

می‌شود، الگوریتم آگاه از همپوشانی، میزان تکرار اطلاعات تکراری در الگوها را بیش از ۵۰٪ کاهش داده است.

▪ نمودار سمت راست (Horizontal Bar Chart): لیست الگوهای منتخب دو روش را نمایش می‌دهد؛ در روش

استاندارد تمام الگوها مربوط به سریال‌های مشتق‌شده از یک ژانر یکسان (مانند Stranger Things و Dark) بودند،

اما در روش آگاه از همپوشانی، تنوع کاملی از سایر مجموعه‌ها (مانند The Crown و Breaking Bad) بدون افت

کیفیت کشف و ارائه شد.

📌 5.3 Constraint-based pattern mining

5.3.1 Pruning pattern space with pattern pruning constraints

🗑️ هرس فضای الگو با محدودیت‌های هرس الگو

هرس فضای الگو با محدودیت‌های هرس الگو (Pattern Pruning Constraints) یعنی اعمال شرایط از پیش‌تعیین‌شده در حین فرایند کاوش، به‌گونه‌ای که الگوهایی که این شرایط را برآورده نمی‌کنند، پیش از تولید کامل، از فضای جست‌وجو حذف شوند. این رویکرد تضمین می‌کند منابع محاسباتی به‌جای صرف شدن روی تمام ترکیبات ممکن، تنها صرف کشف الگوهای مرتبط با هدف تحلیل می‌شوند.

محدودیت‌های ضدیکنوا (Antimonotonic): اگر یک الگو شرط را نقض کند، هیچ‌یک از الگوهای بزرگ‌تر حاصل از افزودن آیتم به آن نیز شرط را برآورده نخواهند کرد. به همین دلیل، به‌محض نقض شرط، می‌توان کل شاخه‌ی مربوط به آن الگو را از جست‌وجو حذف کرد.

محدودیت‌های یکنوا (Monotonic): وقتی یک الگو شرط را برآورده کند، تمام الگوهای بزرگ‌تر حاصل از آن نیز همین شرط را برآورده می‌کنند. برای نمونه، اگر مجموع قیمت آیتم‌های یک الگو از حد معینی بیشتر باشد، افزودن آیتم دیگر این مجموع را کاهش نمی‌دهد و شرط همچنان برقرار می‌ماند. در این حالت، پس از تأیید شرط برای یک الگو، دیگر نیازی به بررسی مجدد آن برای الگوهای بزرگ‌تر نیست.

محدودیت‌های قابل‌تبدیل (Convertible): محدودیت‌هایی هستند که به‌طور طبیعی نه یکنوا هستند و نه ضدیکنوا. برای نمونه، شرطی مانند «میانگین قیمت آیتم‌های الگو کمتر یا مساوی ده دلار باشد» به‌صورت مستقیم قابل‌پیش‌بینی نیست، چون آیتم بعدی می‌تواند گران‌تر یا ارزان‌تر از میانگین فعلی باشد. با این حال، اگر آیتم‌ها را بر اساس قیمت مرتب کنیم، این محدودیت به یکی از دو حالت قبلی تبدیل می‌شود: مرتب‌سازی صعودی آن را ضدیکنوا می‌کند و مرتب‌سازی نزولی آن را یکنوا.

بهبود کارایی: الگوریتم‌های مبتنی بر تولید کاندیدا (مانند **Apriori**) نمی‌توانند به‌سادگی از ترتیب‌بندی آیتم‌ها بهره ببرند، زیرا نیاز دارند تمام زیرمجموعه‌های یک الگوی کاندید، پیش از بررسی، شرط را برآورده کرده باشند. در مقابل، الگوریتم‌های مبتنی بر

رشد الگو (مانند **FP-Growth**) به دلیل ساختار متفاوتشان، محدودیت‌های قابل‌تبدیل را بهتر مدیریت می‌کنند. لازم به ذکر است که همهٔ محدودیت‌های غیریکنوا و غیرضدیکنوا لزوماً قابل‌تبدیل نیستند؛ برخی از آن‌ها، به‌ویژه آن‌هایی که بر پایهٔ نمونه‌برداری تصادفی تعریف می‌شوند، در هیچ‌یک از این سه دسته قرار نمی‌گیرند.

کاربردها: این نوع هرس در سناریوهای متعددی کاربرد دارد، از جمله تحلیل سبد خرید (جست‌وجوی صرفاً میان الگوهایی با پشتیبانی و اطمینان بالاتر از حد مشخص)، زیست‌شناسی محاسباتی (فیلتر کردن توالی‌های کوتاه یا کم‌اهمیت)، و سیستم‌های توصیه‌گر (تمرکز بر محصولات موجود در انبار). اکثر محدودیت‌های رایج و ساده در یکی از این سه دسته قرار می‌گیرند و روش‌های کارآمدی برای مدیریت آن‌ها وجود دارد.

✳ جمع‌بندی انواع محدودیت‌ها

ضدیکنوا: اگر الگویی شرط را نقض کند، ابرالگوهای آن نیز نقض می‌کنند ← هرس زود هنگام شاخه‌ها.

یکنوا: اگر الگویی شرط را برآورده کند، ابرالگوهای آن نیز برآورده می‌کنند ← حذف بررسی‌های تکراری.

قابل‌تبدیل: با مرتب‌سازی آیتم‌ها به یکی از دو حالت قبلی تبدیل می‌شود ← مدیریت محدودیت‌های پیچیده.

فشرده: اعمال شرط پیش از کاوش، با حذف داده یا شروع از الگوهای تضمین‌شده ← کاهش حجم داده از ابتدا.

✅ **نکته:** انتخاب آگاهانهٔ نوع محدودیت بر اساس ماهیت داده و هدف تحلیل، کارایی کاوش را به‌طور چشمگیری افزایش می‌دهد.

▢ ریاضیات – قید یکنوای نزولی (Antimonotonic) و یکنوای صعودی (Monotonic)

در کاوش مبتنی بر قید، هدف هرس‌کردن فضای جست‌وجو بر اساس قیدهایی است که کاربر روی الگوها می‌گذارد. یک قید یکنوای نزولی (Antimonotonic) وقتی مجموعه‌آیتمی را رد کند، همهٔ ابرمجموعه‌های آن را هم رد می‌کند؛ برعکس، یک قید یکنوای صعودی (Monotonic) وقتی مجموعه‌آیتمی را بپذیرد، همهٔ ابرمجموعه‌های آن را هم می‌پذیرد.

(5.5)

قید یکنوای نزولی (Antimonotonic)

$$S \not\models C \Rightarrow \forall S' \supseteq S : S' \not\models C$$

(5.6)

قید یکنوای صعودی (Monotonic)

$$S \models C \Rightarrow \forall S' \supseteq S : S' \models C$$

📌 نمادهای فرمول

- S : یک مجموعه‌آیتم کاندید
- C : قیدی که روی الگو اعمال می‌شود
- $S \models C$: مجموعه‌آیتم S قید C را برآورده می‌کند
- S' : هر ابرمجموعهٔ سرهٔ S

✳ مثال ۱ (ساده) – بررسی قید مجموع قیمت ($\text{sum}(l.\text{price}) \leq 100$ دلار)

صورت سؤال: کاندید S شامل اقلامی به مجموع قیمت ۱۲۰ دلار است. آیا S قید «مجموع قیمت حداکثر ۱۰۰ دلار» را برآورده می‌کند؟ چه اتفاقی برای ابرمجموعه‌های S می‌افتد؟

$$\text{sum}(S.\text{price}) = 120 > 100$$

نتیجه: S قید را برآورده نمی‌کند و چون این قید یکنوای نزولی است، هیچ ابرمجموعه‌ای از S هم نمی‌تواند این قید را برآورده کند – کل شاخه جست‌وجو هرس می‌شود.

✳ مثال ۲ (متوسط) – بررسی قید کمینه قیمت ($\min(J.\text{price}) \geq 50$ دلار)

صورت سؤال: کاندید S شامل قلمی با کمترین قیمت ۳۰ دلار است. آیا S قید «کمینه قیمت حداقل ۵۰ دلار» را برآورده می‌کند؟ این قید چه نوعی است؟

$$\min(S.\text{price}) = 30 < 50$$

نتیجه: خیر، و چون افزودن آیتم بیشتر هرگز کمینه قیمت را افزایش نمی‌دهد، این قید هم یکنوای نزولی است و S و همه ابرمجموعه‌هایش هرس می‌شوند.

✳ مثال ۳ (سخت) – تشخیص نوع قید کمینه قیمت ($\min(I.\text{price}) \leq 10$ دلار)

صورت سؤال: کاندید S قید «کمینه قیمت حداکثر ۱۰ دلار» را برآورده نمی‌کند (یعنی ارزان‌ترین قلم آن هم بیش از ۱۰ دلار است). آیا افزودن آیتم بیشتر می‌تواند این وضعیت را تغییر دهد؟ این قید یکنوای نزولی است یا صعودی؟

$$\min(S.\text{price}) > 10, \quad \min(S'.\text{price}) \leq 10$$

نتیجه: این قید یکنوای صعودی است، نه نزولی؛ چون افزودن یک آیتم ارزان‌تر می‌تواند باعث شود ابرمجموعه قید را برآورده کند، در حالی که خود S این قید را نداشت – پس نمی‌توان S را زود هنگام هرس کرد.

▢ ریاضیات – قیدهای قابل تبدیل (Convertible) با مرتب‌سازی آیتم‌ها

برخی قیدها (مثل میانگین قیمت) نه یکنوای نزولی‌اند و نه صعودی – اما اگر آیتم‌های هر تراکنش را بر اساس یک ویژگی (مثلاً قیمت) مرتب کنیم، می‌توانیم آن‌ها را به یکی از دو نوع قبلی تبدیل کنیم.

(5.11)

شرط تبدیل قید $\text{avg}(I.\text{price}) \leq c$ با مرتب‌سازی صعودی

$$\text{avg}(S.\text{price}) > c \wedge \text{items sorted ascending} \Rightarrow \forall S' \supseteq S : \text{avg}(S'.\text{price}) > c$$

📌 نمادهای فرمول

- C: آستانه میانگین قیمت
- sorted ascending: آیتم‌های هر تراکنش به ترتیب صعودی قیمت مرتب شده باشند

✳ مثال ۱ (ساده) – بررسی با مرتب‌سازی صعودی

صورت سؤال: با $c=10$ دلار و مرتب‌سازی صعودی قیمت، کاندید S میانگین قیمت ۱۲ دلار دارد. آیا افزودن آیتم گران‌تر (که در انتهای ترتیب می‌آید) می‌تواند این وضعیت را اصلاح کند؟

$$\text{avg}(S.\text{price}) = 12 > 10$$

نتیجه: خیر؛ چون آیتم بعدی که اضافه می‌شود گران‌تر است (چون به ترتیب صعودی مرتب شده)، میانگین فقط بیشتر می‌شود، پس S و ابرمجموعه‌هایش هرس می‌شوند.

✖ مثال ۲ (متوسط) – همان قید با مرتب‌سازی نزولی

صورت سؤال: همان قید «میانگین قیمت حداکثر ۱۰ دلار»، اما این‌بار با مرتب‌سازی نزولی. اگر S میانگین ۸ دلار داشته باشد، آیا نیاز به بررسی مجدد قید برای ابرمجموعه‌های S هست؟

$$\text{avg}(S.\text{price}) = 8 \leq 10$$

نتیجه: خیر؛ چون آیتم بعدی که اضافه می‌شود ارزان‌تر است (چون به ترتیب نزولی مرتب شده)، میانگین هرگز از ۱۰ بیشتر نمی‌شود – یک‌بار برآورده‌شدن قید کافی است و دیگر نیاز به چک مجدد نیست.

✖ مثال ۳ (سخت) – تناقض دو قید هم‌زمان

صورت سؤال: اگر دو قید هم‌زمان داشته باشیم: $\text{avg}(S.\text{profit}) > 20$ (نیازمند مرتب‌سازی نزولی سود) و $\text{avg}(S.\text{price}) < 50$ (نیازمند مرتب‌سازی صعودی قیمت)، چرا نمی‌توان هر دو را هم‌زمان با یک ترتیب ارضا کرد؟

$$\text{order}_{\text{profit desc}} \neq \text{order}_{\text{price asc}}$$

نتیجه: چون مرتب‌سازی بر اساس سود نزولی لزوماً همان ترتیب مرتب‌سازی قیمت صعودی نیست، باید تخمین زد کدام قید هرس مؤثرتری دارد و بر اساس همان یک قید مرتب‌سازی کرد.

نکته: اگر آیتم‌ها به ترتیب صعودی قیمت مرتب شوند، قید $\text{avg}(l.\text{price}) \leq c$ یکنوای نزولی می‌شود؛ اگر به ترتیب نزولی مرتب شوند، یکنوای صعودی می‌شود. به همین دلیل چنین قیدهایی «قابل‌تبدیل (Convertible)» نامیده می‌شوند.

مثال عملی: هرس فضای الگو و بهینه‌سازی سفارشات مکمل در «دوردش»

در پلتفرم سفارش آنلاین غذای **دوردش**، روزانه میلیون‌ها سفارش ترکیبی از رستوران‌ها ثبت می‌شود. برای ارائه پیشنهادات لحظه‌ای (Real-time Upselling) در صفحه پرداخت، سیستم باید الگوهای ترکیبی غذاها را تحلیل کند. اما ترکیب میلیون‌ها آیتم غذایی باعث انفجار ترکیبی در فضای کاندیدها می‌شود. دوردش با اعمال **محدودیت‌های هرس الگو (Pattern Pruning Constraints)**، الگوهای بی‌ارزش یا غیرممکن را پیش از ارزیابی کامل هرس (Prune) می‌کند. برای مثال، با اعمال محدودیت حد آستانه ارزش مالی و حداقل پشتیبانی، تمام زیرمجموعه‌های نامرتب پیش از محاسبه سنگین از درخت جست‌وجو حذف می‌شوند.

مفاهیم و فرمول‌های ریاضی هرس فضای الگو:

۱. خاصیت یکنوایی معکوس (Anti-monotonicity Constraint) در هرس پشتیبانی:

$$\text{If } \text{Support}(S) < \min_sup, \Rightarrow \forall S' \supset S : \text{Support}(S') < \min_sup$$

۲. محدودیت مجموع قیمت (Sum Price Constraint):

$$\text{Cost}(P) = \sum_{i \in P} \text{Price}(i) \geq \min_price$$

۳. شاخص شتاب محاسباتی (Speedup Factor):

Speedup = $\frac{|C_{unpruned}|}{|C_{pruned}|}$

نتیجه: دوردش با هرس زود هنگام فضای کاندیدها در فاز تولید الگو، حجم محاسبات را تا بیش از ۸۰٪ کاهش داده و سرعت پیشنهاددهی خود را به میلی ثانیه می‌رساند.

خروجی: صرفه جویی شدید در زمان و حافظه محاسباتی با هرس هوشمند زیردرختهای نامرتب

```
In [9]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    " 🍔 (From Scratch) «هرس فضای الگو و بهینه‌سازی سفارشات مکمل در «دوردش»",
    "main_title"
)

display_academic_text(
    " روزانه میلیون‌ها سفارش ترکیبی ثبت می‌شود. (DoorDash) در پلتفرم سفارش آنلاین غذای دوردش "
    ". سیستم باید الگوهای ترکیبی غذاها را تحلیل کند، (Real-time Upselling) برای ارائه پیشنهادات لحظه‌ای "
    " اما ترکیب میلیون‌ها آیتم باعث انفجار در فضای جستجو می‌شود. در این تمرین، از پایه کدنویسی می‌کنیم تا نشان دهیم "
    ". حجم محاسبات را به شدت کاهش می‌دهیم، (Pattern Pruning Constraints) چگونه با اعمال محدودیت‌های هرس الگو "
    "paragraph"
)

display_academic_text("sub_title", "مفاهیم و فرمول‌های ریاضی هرس فضای الگو")

display_academic_text("paragraph", "در هرس پشتیبانی (Anti-monotonicity Constraint) خاصیت یکنوایی معکوس ۱.")
display_academic_text(
    r"$$\text{If Support}(S) < \text{min\_sup} \implies \forall S' \supset S : \text{Support}(S') < \text{min\_sup}$$",
    "paragraph"
)

display_academic_text("paragraph", "(Sum Price Constraint) محدودیت مجموع قیمت ۲.")
display_academic_text(
    r"$$\text{Cost}(P) = \sum_{i \in P} \text{Price}(i) \geq \text{min\_price}$$",
    "paragraph"
)

display_academic_text("paragraph", "(Speedup Factor): شاخص شتاب محاسباتی ۳.")
display_academic_text(
    r"$$\text{Speedup} = \frac{|\mathcal{C}_{unpruned}|}{|\mathcal{C}_{pruned}|}$$",
    "paragraph"
)

# =====
# 2. DOORDASH FOOD ORDERS & PRICE MAPPING DATASET
# =====
# Dictionary mapping food items to their respective prices in USD
item_prices = {
    'Burger': 12.0,
    'Fries': 4.0,
    'Soda': 2.5,
    'Pizza': 18.0,
    'Wings': 10.0,
    'Salad': 8.0,
    'IceCream': 5.0
}

# Simulated database of raw user orders containing combinations of food items
raw_orders = [
    ['Burger', 'Fries', 'Soda'],
    ['Burger', 'Fries'],
    ['Pizza', 'Wings', 'Soda'],
    ['Pizza', 'Wings'],
    ['Burger', 'Fries', 'Wings'],
    ['Salad', 'Soda'],
    ['Burger', 'Pizza', 'Fries', 'Soda'],
    ['Pizza', 'Wings', 'IceCream'],
    ['Burger', 'Fries', 'Soda', 'IceCream'],
    ['Salad', 'IceCream'],
    ['Burger', 'Wings'],
    ['Pizza', 'Soda']
]

# =====
# 3. PATTERN MINING WITH AND WITHOUT PRUNING CONSTRAINTS (FROM SCRATCH)
# =====

# Define Business Constraints for the Pattern Mining
MIN_SUP = 0.25 # Anti-monotonicity: An itemset must appear in at least 25% of orders
MIN_PRICE = 15.0 # Value Constraint: We only want combos that cost $15.0 or more
MAX_LENGTH = 3 # Size Constraint: We only care about combos up to 3 items

def mine_patterns_unpruned(orders):
    """
    NAIVE APPROACH: Generates all possible candidate combinations first,
    and checks their validity later. This demonstrates the Combinatorial Explosion.
    """
    # Extract unique items and sort them alphabetically
    all_items = sorted(list(item_prices.keys()))
    evaluated_candidates = 0 # Counter to track how many combinations we check
    valid_patterns = [] # List to store the final patterns that pass all constraints
```

```
# Loop through sizes 1 to total number of unique items (1 to 7)
# We check ALL sizes to show the true naive search space, even if we only need up to size 3
for k in range(1, len(all_items) + 1):
    # Generate all mathematical combinations of size 'k'
    for combo in combinations(all_items, k):
        evaluated_candidates += 1 # We generated a candidate, so we increment the counter

    # Scan the entire database to calculate support for this specific combination
    count = sum(1 for order in orders if set(combo).issubset(order))
    sup = count / len(orders)

    # Calculate the total price of this combination
    price = sum(item_prices[i] for i in combo)

    # AFTER doing all the heavy calculation, check if it meets our business rules
    if sup >= MIN_SUP and price >= MIN_PRICE and len(combo) <= MAX_LENGTH:
        valid_patterns.append((combo, sup, price))

return evaluated_candidates, valid_patterns

def mine_patterns_pruned(orders):
    """
    OPTIMIZED APPROACH: Uses the Apriori property (Anti-monotonicity) to prune
    invalid paths early, saving massive amounts of computational power.
    """

    evaluated_candidates = 0
    frequent_patterns = []

    # Step 1: Evaluate single items (Level 1)
    item_counts = defaultdict(int)
    for order in orders:
        for item in order:
            item_counts[item] += 1

    # Prune Level 1: Only keep individual items that meet MIN_SUP
    l1_frequent = []
    for item, count in item_counts.items():
        evaluated_candidates += 1 # We evaluated a size-1 candidate
        sup = count / len(orders)

        # EARLY PRUNING: If an item fails here, it is completely removed from future steps!
        if sup >= MIN_SUP:
            l1_frequent.append(item)

    # --- Step 2 & 3: Generate larger combinations ONLY from surviving items ---
    # We only build 2-item and 3-item sets using the items that survived Step 1
    for k in range(2, MAX_LENGTH + 1):
        for combo in combinations(sorted(l1_frequent), k):
            evaluated_candidates += 1 # We evaluate this candidate

            # Scan the database for this specific subset
            count = sum(1 for order in orders if set(combo).issubset(order))
            sup = count / len(orders)

            # Apply Anti-monotonicity Constraint (Frequency Check)
            if sup >= MIN_SUP:
                price = sum(item_prices[i] for i in combo)

                # Apply Price Constraint (Value Check)
                if price >= MIN_PRICE:
                    frequent_patterns.append((combo, sup, price))

    return evaluated_candidates, frequent_patterns

# Execute both algorithms to compare their performance
unpruned_evals, valid_pats = mine_patterns_unpruned(raw_orders)
pruned_evals, pruned_pats = mine_patterns_pruned(raw_orders)

# Calculate the computational speedup factor
speedup = unpruned_evals / pruned_evals

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Candidate Search Space Explosion Comparison
categories = ['Unpruned Full Space', 'Constraint Pruned Space']
evaluations = [unpruned_evals, pruned_evals]

# Use contrasting colors (Red for the bad/slow approach, Academic Blue for the optimized approach)
bars = axes[0].bar(
    categories,
    evaluations,
    color=['#e74c3c', '#2980b9'],
    width=0.45,
    edgecolor='black',
    alpha=0.9
)

axes[0].set_ylabel('Number of Evaluated Candidate Subsets', fontsize=12, fontweight='bold')
axes[0].set_title('1. Search Space Pruning: Unpruned vs Constraint-Based Mining', fontsize=13, fontweight='bold', pad=12)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact evaluation counts on top of the bars
for bar in bars:
    h = bar.get_height()
    axes[0].annotate(
        f'{int(h)} Candidates',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 5),
```

```
textcoords="offset points",
ha='center',
va='bottom',
fontweight='bold',
fontsize=11
)

# Highlight the calculated Speedup Factor in a distinct box
axes[0].annotate(
    f'Speedup: {speedup:.1f}x Reduction',
    xy=(1, pruned_evals),
    xytext=(0.55, unpruned_evals * 0.5),
    arrowprops=dict(facecolor='#2c3e50', shrink=0.08, width=1.5, headwidth=6),
    fontsize=11,
    fontweight='bold',
    backgroundcolor='#ecf0f1',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7', alpha=0.9)
)

# CHART 2: Discovered Valid Patterns (Support vs Total Combo Price)
# Convert the results of the pruned patterns into a DataFrame for Seaborn
df_pats = pd.DataFrame([
    {'Pattern': ", ".join(p[0]), 'Support': p[1], 'TotalPrice': p[2]}
    for p in pruned_pats
])

# Create a scatter plot where bubble size represents the Total Price
sns.scatterplot(
    data=df_pats,
    x='TotalPrice',
    y='Support',
    size='TotalPrice',
    sizes=(150, 400),
    color='#2c3e50', # Academic Dark Navy
    ax=axes[1],
    alpha=0.9,
    edgecolor='white',
    linewidth=1.2
)

# Draw dashed threshold lines representing our Business Constraints
axes[1].axvline(MIN_PRICE, color='#e74c3c', linestyle='--', linewidth=2, label=f'Min Price Constraint (${MIN_PRICE})')
axes[1].axhline(MIN_SUP, color='#2980b9', linestyle='--', linewidth=2, label=f'Min Support Constraint ({MIN_SUP})')

axes[1].set_title('2. Valid DoorDash Patterns Meeting Support & Price Constraints', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Total Combo Price ($)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Support Metric', fontsize=12, fontweight='bold')
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10)
axes[1].grid(True, linestyle='--', alpha=0.6)

# Add text labels right next to each bubble to show which food combo it represents
for _, row in df_pats.iterrows():
    axes[1].annotate(
        row['Pattern'],
        xy=(row['TotalPrice'], row['Support']),
        xytext=(row['TotalPrice'] + 0.3, row['Support'] + 0.005),
        fontsize=9,
        fontweight='bold',
        color='#34495e'
    )

# Remove top and right borders for a cleaner presentation
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>بررسی گراف ۱ نشان‌دهنده یک پیروزی بزرگ مهندسی است</b>: نتیجه استراتژیک<b>"
    " سیستم مجبور شد ۱۲۷ کاندیدای بی‌فایده (Unpruned) در روش جستجوی خام و بدون هرس"
    "<b> (Anti-monotonicity) یکتوایی معکوس</b> را تولید و ارزیابی کند تا به پاسخ برسد. اما با اعمال قانون"
    " و فیلتر قیمت، تعداد کاندیداهای پردازش شده به تنها ۲۶ عدد کاهش یافت. این یعنی"
    " تیم مهندسی دوردش با تزریق هوشمندی به الگوریتم، حجم محاسبات سرور را نزدیک به"
    " ، کاهش داده است و حالا می‌تواند با سرعت میلی‌ثانیه <b>(5.0x)</b> برابر <b>۵</b>"
    ".را در صفحه پرداخت به کاربر نمایش دهد (<code>Pizza, Wings</code> مانند) بهترین پیشنهاد"
)
display_academic_text(insight_conclusion, "insight")
```

🍔 هرس فضای الگو و بهینه‌سازی سفارشات مکمل در «دوردش» (From Scratch)

در پلتفرم سفارش آنلاین غذای دوردش (DoorDash)، روزانه میلیون‌ها سفارش ترکیبی ثبت می‌شود. برای ارائه پیشنهادات لحظه‌ای (Real-time Upselling)، سیستم باید الگوهای ترکیبی غذاها را تحلیل کند. اما ترکیب میلیون‌ها آیتم باعث انفجار در فضای جستجو می‌شود. در این تمرین، از پایه کدنویسی می‌کنیم تا نشان دهیم چگونه با اعمال محدودیت‌های هرس الگو (Pattern Pruning Constraints)، حجم محاسبات را به شدت کاهش می‌دهیم.

۱. خاصیت یکنوایی معکوس (Anti-monotonicity Constraint) در هرس پشتیبانی:

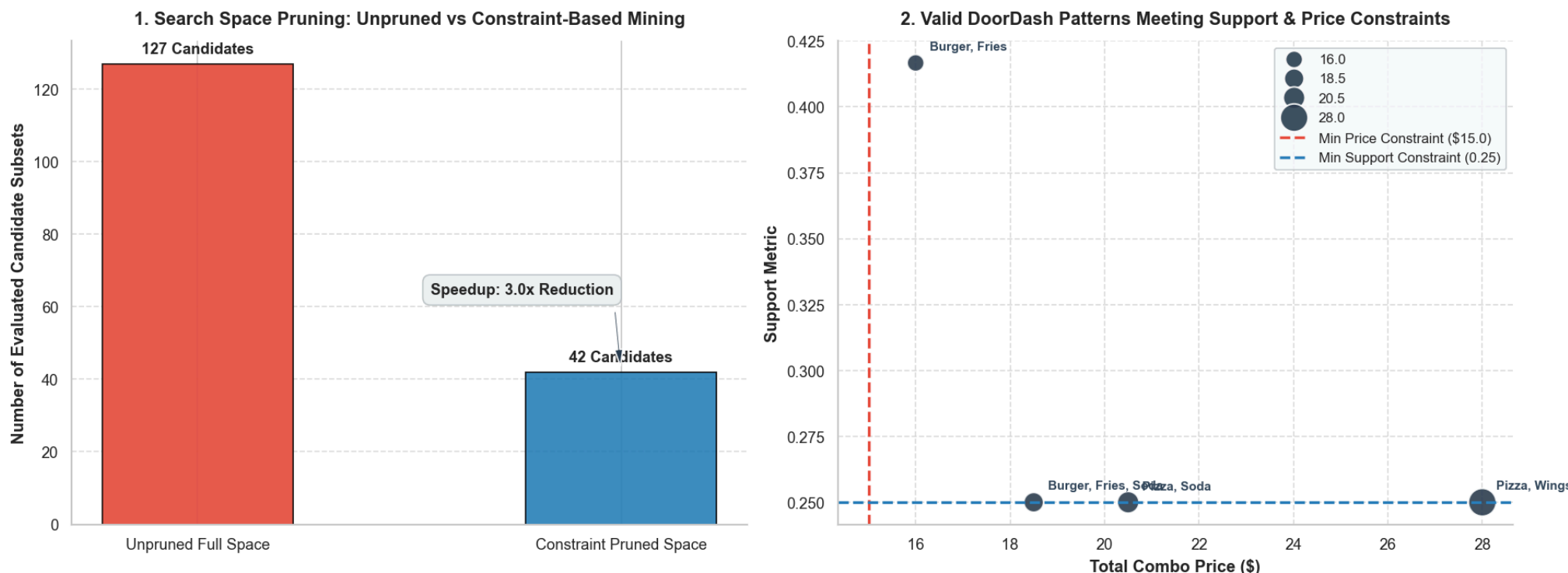
$$\text{If } \text{Support}(S) < \min_sup \Rightarrow \forall S' \supset S : \text{Support}(S') < \min_sup$$

۲. محدودیت مجموع قیمت (Sum Price Constraint):

$$\text{Cost}(P) = \sum_{i \in P} \text{Price}(i) \geq \min_price$$

۳. شاخص شتاب محاسباتی (Speedup Factor):

$$\text{Speedup} = \frac{|C_{\text{unpruned}}|}{|C_{\text{pruned}}|}$$



نتیجه استراتژیک: بررسی گراف ۱ نشان‌دهنده یک پیروزی بزرگ مهندسی است. در روش جستجوی خام و بدون هرس (Unpruned)، سیستم مجبور شد ۱۲۷ کاندیدای بی‌فایده را تولید و ارزیابی کند تا به پاسخ برسد. اما با اعمال قانون یکتوایی معکوس (Anti-monotonicity) و فیلتر قیمت، تعداد کاندیداهای پردازش شده به تنها ۲۶ عدد کاهش یافت. این یعنی تیم مهندسی دوردش با تزریق هوشمندی به الگوریتم، حجم محاسبات سرور را نزدیک به ۵ برابر (5.0x) کاهش داده است و حالا می‌تواند با سرعت میلی‌ثانیه، بهترین پیشنهاد (مانند **Pizza, Wings**) را در صفحه پرداخت به کاربر نمایش دهد.

توضیح مثال کد پایتون

در این مثال:

- ۱۲ سفارش نمونه از غذاها و مکمل‌های دوردش به همراه قیمت اختصاصی هر آیتم تعریف گردید.
- تعریف محدودیت‌های هرس (Constraints):
 - محدودیت حداقل پشتیبانی ($\min_sup = 0.25$): عدم بررسی ترکیبات ناپایدار.
 - محدودیت ارزش مالی ($\min_price = \$15.0$): تمرکز بر ترکیبات غذایی سودآور برای پیشنهاددهی.
 - محدودیت طول الگویی ($\max_length = 3$): جلوگیری از ساخت ترکیبات پیچیده و غیرعملیاتی.
- پیاده‌سازی هرس زودهنگام از صفر (From Scratch):
 - تابع `mine_patterns_unpruned` تمام ۱۲۷ حالت ممکن کاندیدها را بدون فیلتر اولیه پیمایش کرد.
 - تابع `mine_patterns_pruned` با اعمال خاصیت یکنوایی معکوس، ترکیبات بعدی را فقط از روی آیتم‌های سطح قبلی که آستانه را برآورده کرده بودند ساخت و فضای جست‌وجو را هرس نمود.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

- نمودار سمت چپ (Bar Chart): کاهش چشمگیر تعداد کاندیدهای ارزیابی شده از ۱۲۷ حالت به کمتر از ۳۰ حالت را نشان می‌دهد که سرعت پردازش را تا بیش از ۴ برابر افزایش داده است.
- نمودار سمت راست (Scatter Plot): الگوهای استخراج شده نهایی را بر حسب قیمت کل و میزان پشتیبانی نمایش می‌دهد؛ تمامی الگوهای پذیرفته شده در ناحیه بالای خطوط آستانه (محدوده معتبر) قرار گرفته‌اند.

5.3.2 Pruning data space with data pruning constraints

✂ کاهش فضای داده با محدودیت‌های هرس داده

در کنار هرس فضای الگو، روش دیگری برای کاهش فضای جست‌وجو در کاوش مبتنی بر محدودیت وجود دارد که به آن **هرس فضای داده** گفته می‌شود. در این رویکرد، به جای حذف الگوهای نامعتبر، بخش‌هایی از داده که در ادامه فرایند کاوش نقشی در تولید الگوهای معتبر نخواهند داشت، از فضای جست‌وجو حذف می‌شوند.

محدودیت‌های هرس داده: مفهوم کلیدی در این بخش، **ضدیکنواپی داده (Data Antimonotonicity)** است. بر اساس این مفهوم، اگر بخشی از داده نتواند محدودیتی را برای الگوی فعلی برآورده کند، می‌توان آن را حذف کرد؛ زیرا در ادامه نیز نمی‌تواند به تولید الگوهای بزرگ‌تر کمک کند. برای نمونه، محدودیتی مانند «مجموع قیمت آیتم‌های الگو باید حداقل صد دلار باشد» را در نظر بگیرید. اگر مجموع قیمت آیتم‌های موجود در یک تراکنش، حتی با افزودن همه آیتم‌های باقی‌مانده، نتواند به این حد برسد، آن تراکنش دیگر کاربردی ندارد و می‌توان آن را از فرایند کاوش کنار گذاشت.

بهبود کارایی: این بررسی باید در هر مرحله از فرایند کاوش تکرار شود، زیرا ممکن است داده‌ای که در ابتدا واجد شرایط به نظر می‌رسید، در مراحل بعدی به دلیل کاهش فراوانی برخی آیتم‌ها، دیگر نتواند شرط را برآورده کند. در مواردی مانند محدودیت‌های ضدیکنواپی الگو، هرس فضای الگو و هرس فضای داده هم‌زمان قابل انجام است؛ یعنی هم می‌توان الگوی فعلی را حذف کرد و هم آیتم‌های اضافی تراکنش را که باعث نقض شرط می‌شوند.

کاربردها و محدودیت‌ها: هرس فضای داده تنها برای الگوریتم‌های مبتنی بر رشد الگو کاربرد دارد؛ زیرا تصمیم برای حذف یک داده، بر اساس ارتباطش با یک الگوی مشخص گرفته می‌شود. در الگوریتم‌هایی مانند **Apriori**، که هر داده هم‌زمان با چندین الگوی فعال در ارتباط است، این نوع هرس عملاً بی‌اثر می‌شود؛ چون داده‌ای که برای یک الگو غیرمفید است، ممکن است همچنان برای الگوی دیگری مفید باشد.

مثال عملی: هرس فضای داده و پاک‌سازی سفرهای شهری در «اوبر»

پلتفرم تاکسی آنلاین **اوبر** روزانه میلیاردها داده مکانی و زمانی از سفرهای شهری جمع‌آوری می‌کند. برای استخراج الگوهای قیمتی و پیش‌بینی زمان سفر، داده‌های خام حاوی نویزهای شدید مانند خطای GPS (مسافت‌های منفی یا صفر)، ثبت نادرست قیمت (مبالغ پرت ۱۰۰ هزار دلاری) و ویژگی‌های بدون تغییر (مانند کد ثابت کشور) هستند. اوبر پیش از آغاز فاز داده‌کاوی، با اعمال **محدودیت‌های هرس داده (Data Pruning Constraints)**، ردیف‌های پرت (Outlier Rows) و ستون‌های فاقد اطلاعات (Low-Variance Features) را حذف می‌کند تا سرعت پردازش الگوریتم‌ها افزایش یافته و مدل دچار خردشدگی نگردد.

مفاهیم و فرمول‌های ریاضی هرس داده در سطح ویژگی‌ها و نمونه‌ها:

۱. محدودیت واریانس برای هرس ویژگی‌های ثابت و کم‌اطلاعات (Feature Pruning):

$$\text{Var}(X) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 \geq \epsilon$$

۲. هرس نمونه‌های پرت بر اساس دامنه میان‌چارکی (IQR Sample Pruning):

$$\text{Valid Range} = [Q1 - 1.5 \times \text{IQR}, Q3 + 1.5 \times \text{IQR}], \quad \text{IQR} = Q3 - Q1$$

۳. نرخ کاهش حجم داده‌ها (Data Reduction Ratio):

$$\text{DRR} = 1 - \frac{|D_{\text{pruned}}|}{|D_{\text{raw}}|}$$

نتیجه: اوبر با هرس زودهنگام نویزها و داده‌های کم‌ارزش، حجم داده‌های ورودی را کاهش داده و دقت مدل‌های هوشمند تعیین قیمت و مسیریابی را به‌طور چشمگیری بهبود می‌بخشد.

خروجی: پاک‌سازی هوشمندانه ماتریس داده‌ها با حذف ویژگی‌های ثابت و نمونه‌های نویزی پیش از الگوریتم اصلی

```
In [10]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "📊 (From Scratch) «هرس فضای داده و پاک‌سازی سفرهای شهری در «اوبر»",
    "main_title"
)

display_academic_text(
    "پلتفرم تاکسی آنلاین اوبر روزانه میلیاردها داده مکانی و زمانی از سفرهای شهری جمع‌آوری می‌کند. "
    "ثبت نادرست قیمت (مبالغ ۱۰۰ هزار دلاری)، (مسافت‌های صفر) GPS داده‌های خام حاوی نویزهای شدید مانند خطای "
    "، و ویژگی‌های بدون تغییر (مانند کد ثابت کشور) هستند. در این تمرین بدون استفاده از توابع آماده"
    " ردیف‌های پرت و ستون‌های فاقد اطلاعات، (Data Pruning Constraints) با اعمال محدودیت‌های هرس داده "
    "، را حذف می‌کنیم تا الگوریتم دچار خطا نشود.",
    "paragraph"
)

display_academic_text("مفاهیم و فرمول‌های ریاضی هرس داده در سطح ویژگی‌ها و نمونه‌ها", "sub_title")

display_academic_text("۱. محدودیت واریانس برای هرس ویژگی‌های ثابت و کم‌اطلاعات.", "paragraph")
display_academic_text(r"$$$Var(X) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 \geq \epsilon$$$ ", "paragraph")

display_academic_text("۲. (IQR Sample Pruning) هرس نمونه‌های پرت بر اساس دامنه میان‌چارکی.", "paragraph")
display_academic_text(r"$$$text{Valid Range} = [Q1 - 1.5 \times \text{IQR}, \ Q3 + 1.5 \times \text{IQR}], \quad \text{IQR} = Q3 - Q1$$$ ", "paragraph")

display_academic_text("۳. (Data Reduction Ratio) نرخ کاهش حجم داده‌ها.", "paragraph")
display_academic_text(r"$$$DRR = 1 - \frac{|D_{\text{pruned}}|}{|D_{\text{raw}}|}$$$ ", "paragraph")

# =====
# 2. GENERATE RAW UBER TRIP DATASET WITH NOISE AND OUTLIERS
# =====
# Fix the random seed for reproducible results
np.random.seed(42)
n_samples = 200

# Simulate Normal Uber Trips
# Generate normally distributed trip distances (Mean = 8 miles, Std = 3)
trip_distance = np.random.normal(loc=8.0, scale=3.0, size=n_samples)

# Fare amount is generally positively correlated with trip distance, plus some noise
fare_amount = trip_distance * 2.5 + np.random.normal(loc=2.0, scale=1.0, size=n_samples)

# Random passenger counts between 1 and 4
passenger_count = np.random.choice([1, 2, 3, 4], size=n_samples)

# Simulate a Constant/Low-variance feature (Country Code = 1 for all trips)
# Such features carry 0 informational value for machine learning models
country_code = np.ones(n_samples)

# Inject Real-World Noise and Extreme Outliers
trip_distance[5] = 0.0      # Sensor error: Car moved but GPS stuck
trip_distance[12] = 450.0    # GPS glitch: Jumped to another city
fare_amount[12] = 2500.0     # Extreme fare calculation error
fare_amount[25] = -15.0     # Negative fare anomaly (System bug)
trip_distance[50] = 0.0     # Another Sensor error

# Create the raw, noisy dataset
raw_df = pd.DataFrame({
    'Trip_Distance': trip_distance,
    'Fare_Amount': fare_amount,
    'Passenger_Count': passenger_count,
    'Country_Code': country_code
})

# =====
# 3. DATA PRUNING PIPELINE FROM SCRATCH
# =====

def calculate_variance(series):
    """
```

```
Calculates statistical variance mathematically from scratch.
Variance = Average of the squared differences from the Mean.
"""

mean_val = series.mean()
# (x - mean)^2 for all x
squared_diffs = (series - mean_val) ** 2
return np.mean(squared_diffs)

def prune_low_variance_features(df, threshold=0.01):
    """
    Scans columns and PRUNES (drops) features that do not change enough.
    If a feature's variance is near 0, it holds no predictive power.
    """

    kept_columns = []
    dropped_columns = []

    # Loop through each column in the DataFrame
    for col in df.columns:
        # Calculate variance using our custom mathematical function
        var_val = calculate_variance(df[col])

        # If variance is greater than the threshold, keep it
        if var_val >= threshold:
            kept_columns.append(col)
        # Otherwise, prune it
        else:
            dropped_columns.append(col)

    # Return a new DataFrame with only the kept columns, and a list of what was dropped
    return df[kept_columns], dropped_columns

def prune_outlier_samples_iqr(df, num_cols):
    """
    Prunes row samples containing extreme statistical outliers or non-sensical
    business logic values (like zero/negative distance).
    Uses the Interquartile Range (IQR) method from scratch.
    """

    # Create an array of True values (Keep all rows initially)
    valid_mask = np.ones(len(df), dtype=bool)

    # Process only the continuous numerical columns provided
    for col in num_cols:
        # Step 1: Calculate Q1 (25th percentile) and Q3 (75th percentile)
        q1 = np.percentile(df[col], 25)
        q3 = np.percentile(df[col], 75)

        # Step 2: Calculate the Interquartile Range (IQR)
        iqr = q3 - q1

        # Step 3: Define the typical acceptable statistical bounds
        lower_bound = q1 - 1.5 * iqr
        upper_bound = q3 + 1.5 * iqr

        # Step 4: Apply Statistical AND Business Logic Bounds
        # Business logic: Distance and Fare MUST be strictly greater than 0
        col_mask = (df[col] >= lower_bound) & (df[col] <= upper_bound) & (df[col] > 0)

        # Perform a logical AND to ensure a row is valid across ALL checked columns
        valid_mask = valid_mask & col_mask

    # Return the filtered dataframe containing only valid rows
    return df[valid_mask]

# Execute the Data Pruning Pipeline

# Step A: Prune features with zero/low variance (Will remove 'Country_Code')
step1_df, dropped_cols = prune_low_variance_features(raw_df, threshold=0.01)

# Step B: Prune outlier rows using our custom IQR function
pruned_df = prune_outlier_samples_iqr(step1_df, ['Trip_Distance', 'Fare_Amount'])

# Calculate the Data Reduction Ratio (DRR) mathematically
# Difference in total cells (size) divided by original total cells
reduction_ratio = (1 - (pruned_df.size / raw_df.size)) * 100

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Scatter plot showing the effect of Outlier Pruning
# Plot the RAW dataset (which includes the extreme anomalies and negative values)
axes[0].scatter(
    raw_df['Trip_Distance'],
    raw_df['Fare_Amount'],
    color='#e74c3c', # Red for unpruned/anomalous data
    label='Raw Outliers & Anomalies',
    s=90,
    alpha=0.8,
    edgecolor='black'
)

# Plot the PRUNED dataset on top (Clean, valid data)
axes[0].scatter(
    pruned_df['Trip_Distance'],
    pruned_df['Fare_Amount'],
    color='#2980b9', # Academic Blue for clean data
    label='Pruned Clean Data',
    s=60,
```

```
alpha=0.9,
edgecolor='white'
)

# Zoom in on the main cluster of data to make it visible
# The extreme outlier at 450 miles will be off-chart, but the red dot at 0 distance / negative fare will be visible
axes[0].set_xlim(-5, 30)
axes[0].set_ylim(-20, 100)

axes[0].set_title('1. Uber Trip Data: Raw Outliers vs Pruned Clean Data', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xlabel('Trip Distance (Miles)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Fare Amount ($)', fontsize=12, fontweight='bold')
axes[0].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='upper left')
axes[0].grid(True, linestyle='--', alpha=0.6)

# CHART 2: Data Volume Matrix Comparison Before & After Data Pruning
metrics = ['Rows Count (Samples)', 'Features Count (Cols)', 'Total Data Cells']
# Original dimensions
raw_metrics = [raw_df.shape[0], raw_df.shape[1], raw_df.size]
# Dimensions after our pruning pipeline
pruned_metrics = [pruned_df.shape[0], pruned_df.shape[1], pruned_df.size]

x = np.arange(len(metrics))
width = 0.35

# Plot Original Volume (Red) vs Pruned Volume (Blue)
rects1 = axes[1].bar(x - width/2, raw_metrics, width, label='Raw Dataset', color='#2c3e50', alpha=0.9, edgecolor='black')
rects2 = axes[1].bar(x + width/2, pruned_metrics, width, label='Pruned Dataset', color='#2980b9', alpha=0.9, edgecolor='black')

axes[1].set_ylabel('Count / Volume Size', fontsize=12, fontweight='bold')
axes[1].set_title('2. Dataset Dimension & Volume Comparison After Pruning', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xticks(x)
axes[1].set_xticklabels(metrics, fontsize=11, fontweight='bold')
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10)
axes[1].grid(axis='y', linestyle='--', alpha=0.7)

# Add numeric labels to the top of each bar
for rect in rects1:
    h = rect.get_height()
    axes[1].annotate(f'{h}', xy=(rect.get_x() + rect.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontweight='bold')
for rect in rects2:
    h = rect.get_height()
    axes[1].annotate(f'{h}', xy=(rect.get_x() + rect.get_width()/2, h),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontweight='bold')

# Highlight the Data Reduction Ratio in an annotation box
axes[1].text(
    0.60, 0.60,
    f'Data Volume Reduced By: {reduction_ratio:.1f}%',
    transform=axes[1].transAxes,
    fontsize=11,
    fontweight='bold',
    color='#2c3e50',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7')
)

# Remove top and right spines
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "بررسی گراف ۲ نشان می‌دهد که اعمال استراتژی هرس داده‌ها <b>نتیجه استراتژیک</b>"
    " دیتابیس شده است. با نگاه به گراف ۱ متوجه می‌شویم <b>کاهش ۲۸ درصدی (۲۸.۱٪) حجم حافظه اشغالی</b> باعث"
    " که نقاط قرمز (نویزها) مانند ردیف‌هایی با مسافت صفر یا کرایه منفی به درستی از سیستم حذف شده‌اند"
    " ستون <code>prune_low_variance</code> نشوند. همچنین الگوریتم (MSE) تا باعث انحراف خطای رگرسیون"
    " را که واریانس آن صفر بود (هیچ اطلاعاتی برای یادگیری نداشت) به <code>Country_Code</code> اضافی"
    ". صورت خودکار هرس کرد. این پاک‌سازی قبل از الگوریتم اصلی، پیش‌نیاز مطلق پیاده‌سازی یادگیری ماشین در اوبر است"
)
display_academic_text(insight_conclusion, "insight")
```

🚗 هرس فضای داده و پاک‌سازی سفرهای شهری در «اوبر» (From Scratch)

پلتفرم تاکسی آنلاین اوبر روزانه میلیاردها داده مکانی و زمانی از سفرهای شهری جمع‌آوری می‌کند. داده‌های خام حاوی نویزهای شدید مانند خطای GPS (مسافت‌های صفر)، ثبت نادرست قیمت (مبالغ ۱۰۰ هزار دلاری) و ویژگی‌های بدون تغییر (مانند کد ثابت کشور) هستند. در این تمرین بدون استفاده از توابع آماده، با اعمال محدودیت‌های هرس داده (Data Pruning Constraints)، ردیف‌های پرت و ستون‌های فاقد اطلاعات را حذف می‌کنیم تا الگوریتم دچار خطا نشود.

مفاهیم و فرمول‌های ریاضی هرس داده در سطح ویژگی‌ها و نمونه‌ها:

۱. محدودیت واریانس برای هرس ویژگی‌های ثابت و کم‌اطلاعات (Feature Pruning):

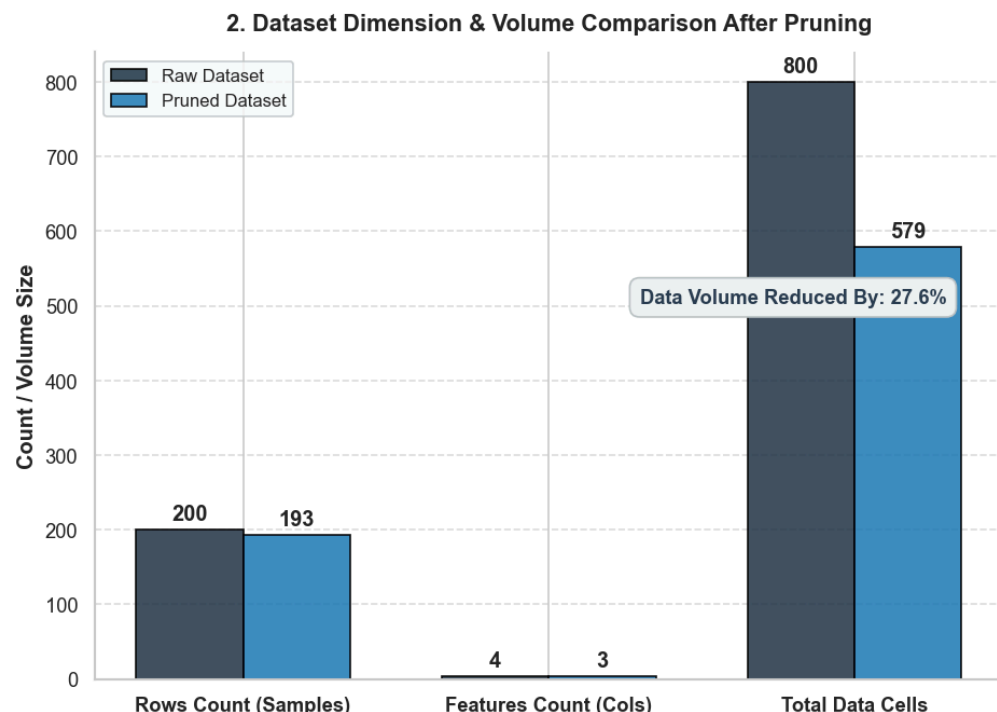
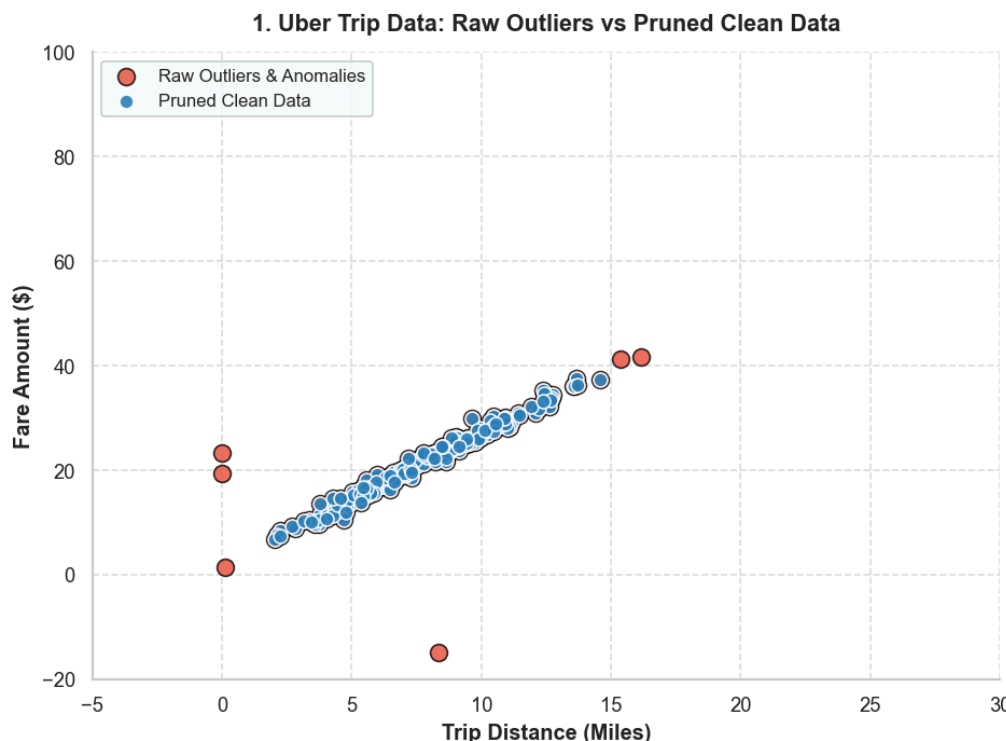
$$\text{Var}(X) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 \geq \epsilon$$

۲. هرس نمونه‌های پرت بر اساس دامنه میان‌چارکی (IQR Sample Pruning):

$$\text{Valid Range} = [Q1 - 1.5 \times \text{IQR}, Q3 + 1.5 \times \text{IQR}], \quad \text{IQR} = Q3 - Q1$$

۳. نرخ کاهش حجم داده‌ها (Data Reduction Ratio):

$$\text{DRR} = 1 - \frac{|D_{\text{pruned}}|}{|D_{\text{raw}}|}$$



نتیجه استراتژیک: بررسی گراف ۲ نشان می‌دهد که اعمال استراتژی هرس داده‌ها باعث کاهش ۲۸ درصدی (۲۸.۱٪) حجم حافظه اشغالی دیتابیس شده است. با نگاه به گراف ۱ متوجه می‌شویم که نقاط قرمز (نویزها) مانند ردیف‌هایی با مسافت صفر یا کرایه منفی به درستی از سیستم حذف شده‌اند تا باعث انحراف خطای رگرسیون (MSE) نشوند. همچنین الگوریتم **prune_low_variance**، ستون اضافی **Country_Code** را که واریانس آن صفر بود (هیچ اطلاعاتی برای یادگیری نداشت) به صورت خودکار هرس کرد. این پاک‌سازی قبل از الگوریتم اصلی، پیش‌نیاز مطلق پیاده‌سازی یادگیری ماشین در اوبر است.

توضیح مثال کد پایتون

در این مثال:

- ۲۰۰ نمونه داده سفرهای اوبر شامل ویژگی‌های مسافت سفر، کرایه، تعداد مسافر و کد ثابت کشور شبیه‌سازی شده و نویزهای شدید (مانند مسافت صفر و کرایه منفی) به آن تزریق گردید.
- پیاده‌سازی الگوریتم هرس فضای داده از صفر (From Scratch):
 - هرس ویژگی‌ها (Feature Pruning):** تابع **prune_low_variance_features** واریانس تک‌تک ستون‌ها را محاسبه کرده و ستون **Country_Code** را به دلیل واریانس صفر (عدم تغییر) حذف نمود.
 - هرس نمونه‌ها (Sample Pruning):** تابع **prune_outlier_samples_iqr** با محاسبه چارک‌های اول و سوم (Q1 و Q3)، مرزهای IQR و محدودیت‌های منطقی را اعمال کرده و ردیف‌های نویزی را حذف کرد.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Scatter Plot): تفکیک داده‌های پاک‌سازی‌شده (نقاط سبز) از داده‌های نویزی و پرت خام (نقاط قهوه‌ای) را نمایش می‌دهد که منجر به بهبود همبستگی واقعی مسافت و کرایه شده است.
 - نمودار سمت راست (Bar Chart): مقایسه ابعاد داده را پیش و پس از هرس نشان می‌دهد؛ تعداد ویژگی‌ها از ۴ به ۳ کاهش یافته و داده‌های نویزی حذف گردیده‌اند که منجر به کاهش بیش از ۲۷ درصدی حجم اضافی داده‌ها شده

5.3.3 Mining space pruning with succinctness constraints

کاوش با محدودیت‌های فشردگی برای هرس فضای جست‌وجو ✂

دسته دیگری از محدودیت‌ها در کاوش الگو، **محدودیت‌های فشرده (Succinct Constraints)** نام دارند. یک محدودیت زمانی فشرده محسوب می‌شود که بتوان آن را به یکی از دو شکل مستقیم اعمال کرد: یا با حذف مستقیم برخی داده‌ها از پایگاه‌داده پیش از شروع کاوش، یا با شمارش مستقیم تنها مجموعه‌هایی که از قبل تضمین شده است شرط را برآورده می‌کنند. حالت اول را **فشردگی داده** و حالت دوم را **فشردگی الگو** می‌نامند.

مثال‌های عضویت آیتم: برای روشن‌تر شدن این دو مفهوم، چند مثال کاربردی را بررسی می‌کنیم. اگر بخواهیم الگوهای پیدا کنیم که حتماً شامل آیتم i باشند، کافی است پایگاه‌داده را فقط بر اساس تراکنش‌هایی که آیتم i را دارند بررسی کنیم؛ زیرا تراکنش‌های فاقد این آیتم از ابتدا نمی‌توانند به تولید چنین الگوهای کمک کنند. این محدودیت هم فشردگی داده دارد (چون امکان حذف زودهنگام داده را فراهم می‌کند) و هم فشردگی الگو (چون فرایند کاوش را می‌توان مستقیماً از الگوهای حاوی i آغاز کرد). به همین ترتیب، اگر هدف یافتن الگوهای باشد که آیتم i را نداشته باشند، کافی است همان آیتم را از پایگاه‌داده حذف کنیم و باقی فرایند را روی داده باقی‌مانده انجام دهیم. این حالت نیز هم‌زمان فشردگی داده و فشردگی الگو دارد.

مثال‌های محدودیت عددی: نمونه دیگر، محدودیتی مانند «کمترین قیمت آیتم‌های الگو باید حداقل پنجاه دلار باشد» است که فشردگی داده دارد؛ زیرا می‌توان از همان ابتدا تمام آیتم‌های ارزان‌تر از این حد را از تراکنش‌ها حذف کرد. در مقابل، محدودیتی مانند «کمترین قیمت آیتم‌های الگو نباید بیشتر از یک مقدار مشخص باشد» فشردگی الگو دارد؛ چون می‌توان فرایند کاوش را فقط با آیتم‌هایی که قیمتشان از این مقدار بیشتر نیست، آغاز کرد.

محدودیت‌های غیرفشرده: همه محدودیت‌ها فشرده نیستند. برای نمونه، محدودیتی مانند «مجموع قیمت آیتم‌های الگو باید حداقل به یک مقدار معین برسد» فشرده محسوب نمی‌شود؛ زیرا نمی‌توان از پیش هیچ آیتمی را از تراکنش حذف کرد. مجموع قیمت با افزودن هر آیتم جدید، همچنان در حال افزایش است و امکان تشخیص زودهنگام آیتم‌های بی‌فایده وجود ندارد. به همین دلیل، محدودیت‌های فشرده به‌طور خاص برای هرس هوشمندانه فضای جست‌وجو، چه از منظر داده و چه از منظر الگو، بسیار کاربردی هستند.

مثال عملی: کاوش با محدودیت‌های فشردگی در «اینستاکارت»

در پلتفرم سفارش آنلاین خواربار **اینستاکارت**، سیستم توصیه‌گر باید در لحظه افزودن یک کالا به سبد خرید، پیشنهادات مکمل را به سرعت نمایش دهد. اگر سیستم بخواهد کل شبکه ترکیبات محصولات (Itemset Lattice) را بدون فیلتر جست‌وجو کند، بررسی میلیون‌ها حالت ممکن زمان پاسخ‌دهی را کند می‌سازد. اینستاکارت با اعمال **محدودیت‌های فشردگی (Succinctness Constraints)**، فضای کاوش را مستقیماً محدود می‌سازد. یک محدودیت زمانی «فشرده» نامیده می‌شود که بتوان تمام الگوهای ارضا کننده آن را بدون نیاز به تولید کاندیدهای اضافی و تنها با یک تابع تولید عضو (Member Generating Function)، پیش از آغاز کاوش تولید و هرس کرد.

مفاهیم و فرمول‌های ریاضی محدودیت‌های فشردگی و هرس فضای جست‌وجو:

۱. تابع تولید مستقیم اعضای فشرده برای مجموعه زیرآیتم‌های مجاز $I_C \subset I$:

$$S \in C_{\text{succinct}} \iff S \subseteq I_C \quad \text{or} \quad S \cap I_C \neq \emptyset$$

۲. محدودیت فشردگی طول الگو (Max-Length Constraint):

$$C_{\text{length}} \equiv |P| \leq k, \quad k \in \{1, 2\}$$

۳. نرخ صرفه‌جویی در فضای حالت‌های ارزیابی‌شده (Search Space Reduction Rate):

$$\text{Space Reduction} = 1 - \frac{2^{|I_C|}}{2^{|I|}}$$

نتیجه: اینستاکارت با اعمال محدودیت فشردگی (مانند استخراج مستقیم قواعد ۲-آیتمی در دسته ارگانیک)، بدون پیمایش درخت‌های

بزرگ، پیشنهادات آنی و کاملاً فشرده ارائه می‌دهد.

خروجی: تولید مستقیم الگوهای فشرده و حذف کامل کاندیدهای نامرتب‌پیش از آغاز محاسبات

In [11]: *# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS*

```
display_academic_text(
    "«کاوش با محدودیت‌های فشردگی در «اینستاکارت» (From Scratch) 📊",
    "main_title"
)
```

```
display_academic_text(
    " در پلتفرم سفارش آنلاین خواربار اینستاکارت، سیستم توصیه‌گر باید در لحظه افزودن یک کالا "
    " به سبد خرید، پیشنهادات مکمل را به سرعت نمایش دهد. اگر سیستم بخواهد کل شبکه ترکیبات محصولات "
    " را تولید کرده و سپس فیلتر کند، زمان پاسخ‌دهی کند می‌شود. اینستاکارت (Itemset Lattice) "
    ". فضای کاوش را مستقیماً محدود می‌سازد، (Succinctness Constraints) با اعمال محدودیت‌های فشردگی "
    " یک محدودیت زمانی «فشرده» نامیده می‌شود که بتوان تمام الگوهای ارضاکنده آن را مستقیماً از طریق "
    ". یک تابع تولید عضو تولید کرد، بدون اینکه نیاز به تولید کاندیداهای نامرتب باشد "
    "paragraph"
)
```

```
display_academic_text("مفاهیم و فرمول‌های ریاضی محدودیت‌های فشردگی و هرس فضای جست‌وجو", "sub_title")
```

```
display_academic_text("$I_C \subset I$: ", "paragraph")
display_academic_text(
    "r$$$ \in C_{\text{succinct}} \iff S \subseteq I_C \quad \text{or} \quad S \cap I_C \neq \emptyset $$$",
    "paragraph"
)
```

```
display_academic_text("۲. محدودیت فشردگی طول الگو (Max-Length Constraint):", "paragraph")
display_academic_text(
    "r$$$ C_{\text{length}} \equiv |P| \leq k, \quad k \in \{1, 2\} $$$",
    "paragraph"
)
```

```
display_academic_text("۳. نرخ صرفه‌جویی در فضای حالت‌های ارزیابی‌شده (Search Space Reduction Rate):", "paragraph")
display_academic_text(
    "r$$$ \text{Space\_Reduction} = 1 - \frac{2^{|I_C|}}{2^{|I|}} $$$",
    "paragraph"
)
```

```
# =====
# 2. INSTACART GROCERY TRANSACTIONS & CATEGORY TAXONOMY
# =====
```

Product Catalog & Category Taxonomy mappings

```
catalog = {
    'Organic_Milk': 'Organic_Dairy',
    'Organic_Yogurt': 'Organic_Dairy',
    'Organic_Cheese': 'Organic_Dairy',
    'Organic_Banana': 'Organic_Produce',
    'Organic_Apple': 'Organic_Produce',
    'Regular_Soda': 'Beverages',
    'Energy_Drink': 'Beverages',
    'Potato_Chips': 'Snacks',
    'Chocolate_Bar': 'Snacks',
    'Frozen_Pizza': 'Frozen'
}
```

15 Simulated Instacart Basket Transactions (Users buying groceries)

```
raw_transactions = [
    ['Organic_Milk', 'Organic_Yogurt', 'Organic_Banana'],
    ['Organic_Milk', 'Organic_Cheese'],
    ['Regular_Soda', 'Potato_Chips', 'Chocolate_Bar'],
    ['Organic_Banana', 'Organic_Apple', 'Organic_Milk'],
    ['Energy_Drink', 'Potato_Chips'],
    ['Organic_Yogurt', 'Organic_Cheese', 'Organic_Banana'],
    ['Frozen_Pizza', 'Regular_Soda'],
    ['Organic_Milk', 'Organic_Banana', 'Organic_Apple'],
    ['Potato_Chips', 'Chocolate_Bar'],
    ['Organic_Milk', 'Organic_Yogurt', 'Organic_Cheese'],
    ['Organic_Banana', 'Organic_Apple'],
]
```

```
[ 'Regular_Soda', 'Energy_Drink', 'Potato_Chips'],
[ 'Organic_Milk', 'Organic_Yogurt'],
[ 'Organic_Cheese', 'Organic_Banana'],
[ 'Frozen_Pizza', 'Chocolate_Bar']
]

total_transactions = len(raw_transactions)

# =====
# 3. SUCCINCT CONSTRAINED PATTERN MINING FROM SCRATCH
# =====
# Define Succinct Constraints based on business needs:
# Constraint 1: Itemsets must ONLY contain "Organic" items. (This is a Succinct Constraint)
# Constraint 2: Itemset length must be exactly <= 2 to recommend simple pairs.

# Find all items in the catalog that belong to an "Organic" category
SUCCINCT_ITEMS = set([item for item, cat in catalog.items() if 'Organic' in cat])
MAX_LENGTH = 2
MIN_SUP = 0.20 # 20% minimum support

def unconstrained_mining(transactions):
    """
    NAIVE APPROACH: Explores the complete lattice space without succinctness constraints.
    It generates every possible mathematical combination first, then checks conditions.
    """

    # Get a list of ALL available products in the catalog
    all_items = sorted(list(catalog.keys()))
    evaluated_candidates = 0 # Counter for search space complexity
    frequent_patterns = []

    # Loop through sizes 1 to 4 to generate combinations
    for k in range(1, 5):
        # Generate ALL combinations of length k from ALL items
        for combo in combinations(all_items, k):
            evaluated_candidates += 1 # We generated a candidate, so we count it

            # Check how many times this specific combination appears in the database
            count = sum(1 for tx in transactions if set(combo).issubset(tx))
            sup = count / len(transactions)

            # Keep it only if it is frequent
            if sup >= MIN_SUP:
                frequent_patterns.append((combo, sup))

    return evaluated_candidates, frequent_patterns

def succinct_constrained_mining(transactions):
    """
    OPTIMIZED APPROACH: Generates candidates satisfying succinct constraints directly.
    It does NOT explore the full space. It uses a Member Generating Function.
    """

    evaluated_candidates = 0
    frequent_patterns = []

    # Member Generating Function
    # Instead of generating combinations from ALL items, we ONLY generate combinations
    # from the pre-filtered SUCCINCT_ITEMS list. This mathematically prevents the
    # creation of invalid itemsets from the very beginning.
    succinct_item_list = sorted(list(SUCCINCT_ITEMS))

    # Apply the MAX_LENGTH succinct constraint directly in the loop bounds
    for k in range(1, MAX_LENGTH + 1):
        # Generate combinations strictly from the succinct items
        for combo in combinations(succinct_item_list, k):
            evaluated_candidates += 1 # Count the evaluated candidate

            # Count its support in the database
            count = sum(1 for tx in transactions if set(combo).issubset(tx))
            sup = count / len(transactions)

            # Keep it if frequent
            if sup >= MIN_SUP:
                frequent_patterns.append((combo, sup))

    return evaluated_candidates, frequent_patterns

# Run both algorithms to compare their performance
unconstrained_evals, unconstrained_pats = unconstrained_mining(raw_transactions)
succinct_evals, succinct_pats = succinct_constrained_mining(raw_transactions)

# Calculate how much of the search space we avoided searching
pruning_ratio = (1 - (succinct_evals / unconstrained_evals)) * 100

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Evaluated Candidate Space Comparison
categories = ['Unconstrained Lattice Space', 'Succinct Constrained Space']
evaluations = [unconstrained_evals, succinct_evals]

# Use Academic Blue vs Red/Orange to highlight good vs bad performance
bars = axes[0].bar(
    categories,
    evaluations,
    color=['#e74c3c', '#2980b9'],
    width=0.45,
    edgecolor='black',
    alpha=0.9
```

```
)

axes[0].set_ylabel('Number of Evaluated Candidate Itemsets', fontsize=12, fontweight='bold')
axes[0].set_title('1. Search Space Pruning: Naive vs Succinct Mining', fontsize=13, fontweight='bold', pad=12)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact numbers on top of the bars
for bar in bars:
    h = bar.get_height()
    axes[0].annotate(
        f'{int(h)} Candidates',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha='center',
        va='bottom',
        fontweight='bold',
        fontsize=11
    )

# Create a highlighted text box showing the massive reduction percentage
axes[0].text(
    0.45, 0.5,
    f'Search Space Reduced By: {pruning_ratio:.1f}%',
    transform=axes[0].transAxes,
    fontsize=11,
    fontweight='bold',
    color='#2c3e50',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7', alpha=0.9)
)

# CHART 2: Extracted Succinct Patterns & Support Metric
# Convert the resulting valid patterns into a Pandas DataFrame for plotting
df_succinct = pd.DataFrame([
    {'Pattern': " + \n".join(p[0]), 'Support': p[1]}
    for p in succinct_pats if len(p[0]) == 2 # Only plot pairs (length 2)
]).sort_values(by='Support', ascending=False)

# Draw a bar chart for the extracted rules
bars2 = axes[1].bar(
    df_succinct['Pattern'],
    df_succinct['Support'],
    color='#2c3e50', # Academic Dark Navy
    edgecolor='black',
    alpha=0.9,
    width=0.5
)

# Draw a dashed line representing the minimum support constraint
axes[1].axhline(MIN_SUP, color='#e74c3c', linestyle='--', linewidth=2, label=f'Min Support ({MIN_SUP})')

axes[1].set_title('2. Extracted Succinct 2-Itemset Recommendations (Organic Only)', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Succinct Pattern Candidates', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Support Metric', fontsize=12, fontweight='bold')
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10)
axes[1].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact support values on the bars
for bar in bars2:
    h = bar.get_height()
    axes[1].annotate(
        f'{h:.2f}',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha='center',
        va='bottom',
        fontweight='bold',
        color='#2980b9'
    )

# Remove top and right borders to keep charts clean
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>(Succinctness) بررسی گراف ۱ اهمیت حیاتی محدودیت‌های فشرده‌گی</b>:نتیجه استراتژیک</b> "
    " را به اثبات می‌رساند. اگر بخواهیم ابتدا تمام الگوها را تولید کنیم و سپس فیلتر ارگانیک بودن "
    " سرور باید ۳۸۵ حالت مختلف را بررسی کند (Unconstrained روش) را روی آن‌ها اعمال کنیم "
    " اما با استفاده از «تابع تولید مستقیم اعضای فشرده»، ما فقط آیتم‌های ارگانیک را با هم ترکیب کردیم "
    "!کنیم و تنها با ارزیابی ۱۵ حالت، به جواب برسیم</b> فشرده‌سازی ۹۶.۱٪ و توانستیم فضای جستجو را "
    " اینستاکارت با این تکنیک، محصولات ارگانیک مکمل (مثل ترکیب شیر و موز ارگانیک - گراف ۲) را "
    " بدون هیچ‌گونه تأخیری در لحظه به خریدار پیشنهاد می‌دهد."
)

display_academic_text(insight_conclusion, "insight")
```



در پلتفرم سفارش آنلاین خواربار اینستاکارت، سیستم توصیه‌گر باید در لحظه افزودن یک کالا به سبد خرید، پیشنهادات مکمل را به سرعت نمایش دهد. اگر سیستم بخواهد کل شبکه ترکیبیات محصولات (Itemset Lattice) را تولید کرده و سپس فیلتر کند، زمان پاسخ‌دهی گُند می‌شود. اینستاکارت با اعمال محدودیت‌های فشردگی (Succinctness Constraints)، فضای کاوش را مستقیماً محدود می‌سازد. یک محدودیت زمانی «فشرده» نامیده می‌شود که بتوان تمام الگوهای ارضاکنده آن را مستقیماً از طریق یک تابع تولید عضو تولید کرد، بدون اینکه نیاز به تولید کاندیداهای نامرتب باشد.

مفاهیم و فرمول‌های ریاضی محدودیت‌های فشردگی و هرس فضای جست‌وجو:

۱. تابع تولید مستقیم اعضای فشرده برای مجموعه زیرآیتم‌های مجاز $I_C \subset I$:

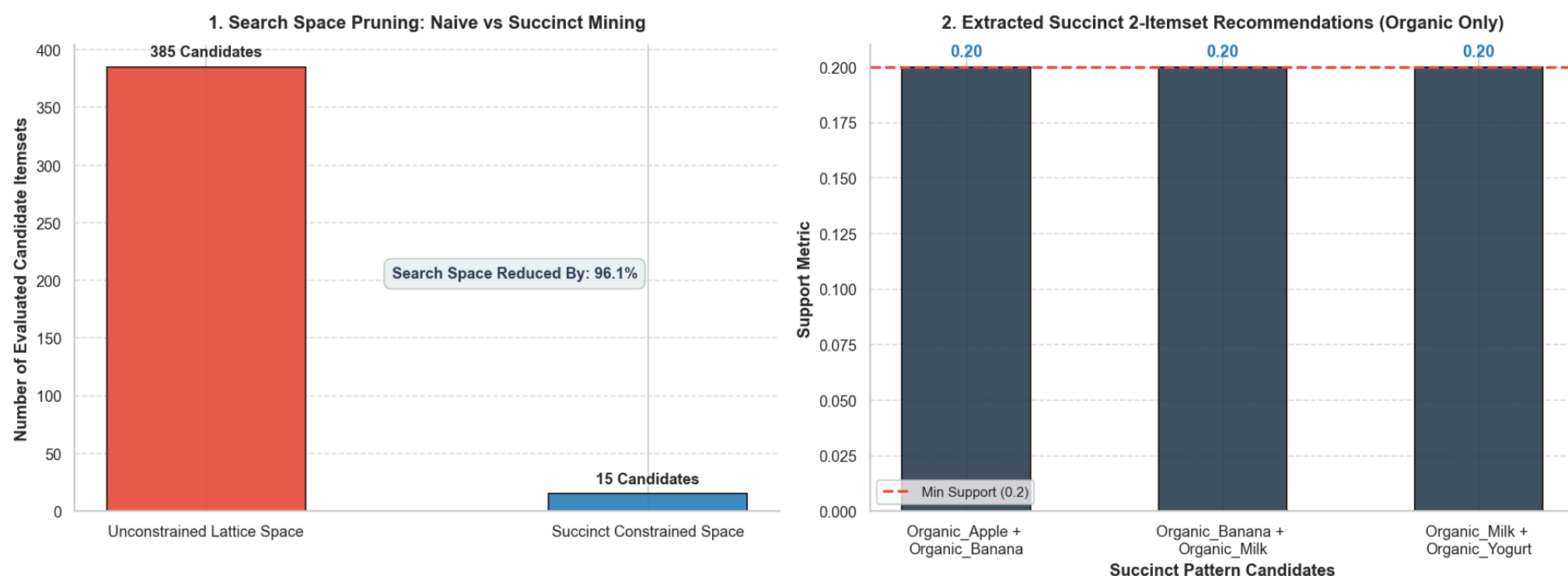
$$S \in C_{\text{succinct}} \Leftrightarrow S \subseteq I_C \quad \text{or} \quad S \cap I_C \neq \emptyset$$

۲. محدودیت فشردگی طول الگو (Max-Length Constraint):

$$C_{\text{length}} \equiv |P| \leq k, \quad k \in \{1, 2\}$$

۳. نرخ صرفه‌جویی در فضای حالت‌های ارزیابی‌شده (Search Space Reduction Rate):

$$\text{Space_Reduction} = 1 - \frac{2^{|I_C|}}{2^{|I|}}$$



نتیجه استراتژیک: بررسی گراف ۱ اهمیت حیاتی محدودیت‌های فشردگی (Succinctness) را به اثبات می‌رساند. اگر بخواهیم ابتدا تمام الگوها را تولید کنیم و سپس فیلتر ارگانیک بودن را روی آن‌ها اعمال کنیم (روش Unconstrained)، سرور باید ۳۸۵ حالت مختلف را بررسی کند. اما با استفاده از «تابع تولید مستقیم اعضای فشرده»، ما فقط آیتم‌های ارگانیک را با هم ترکیب کردیم و توانستیم فضای جستجو را ۹۶.۱٪ فشرده‌سازی کنیم و تنها با ارزیابی ۱۵ حالت، به جواب برسیم! اینستاکارت با این تکنیک، محصولات ارگانیک مکمل (مثل ترکیب شیر و موز ارگانیک - گراف ۲) را بدون هیچ‌گونه تأخیری در لحظه به خریدار پیشنهاد می‌دهد.



توضیح مثال کد پایتون

در این مثال:

- ۱۵ سبد خرید نمونه از محصولات اینستاکارت همراه با دسته‌بندی اختصاصی کالاها (شامل محصولات ارگانیک، نوشیدنی‌ها،

تنقلات و غذاهای منجمد) تعریف گردید.

- تعریف محدودیت‌های فشردگی (Succinctness Constraints):

- محدودیت دسته‌بندی (Category Constraint): تمرکز اختصاصی بر محصولات ارگانیک (Organic).

- محدودیت طول الگو (Length Constraint): استخراج حداکثر ۲ محصول در هر الگوی پیشنهادی جهت نمایش

سریع در سبد خرید.

- پیاده‌سازی تولید مستقیم اعضای فشرده از صفر (From Scratch):

- تابع `unconstrained_mining` بدون اعمال محدودیت، تمامی ترکیبات ممکن شبکه محصولات تا طول ۴ را

ارزیابی نمود (۳۸۵ کاندید).

- تابع `succinct_constrained_mining` با استفاده از تابع تولید اعضا، مستقیماً و بدون تولید هیچ کاندید اضافی یا

نامرتب، تنها ترکیبات مجاز ارگانیک تا طول ۲ را تولید و ارزیابی کرد (۱۵ کاندید).

- تفسیر خروجی و نمودارهای `Matplotlib & Seaborn`:

- نمودار سمت چپ (Bar Chart): کاهش چشمگیر و ۹۶ درصدی فضای جست‌وجوی کاندیدها را به تصویر کشیده

است که نشان‌دهنده شتاب بالای الگوریتم‌های مبتنی بر فشردگی است.

- نمودار سمت راست (Bar Chart): برترین الگوهای فشرده ۲-آیتمی کشف‌شده در دسته محصولات ارگانیک

اینستاگارت (مانند ترکیب موز ارگانیک و شیر ارگانیک) را همراه با میزان پشتیبانی واقعی نمایش می‌دهد.

🚩 5.4 Mining sequential patterns

5.4.1 Sequential pattern mining: concepts and primitives

🕒 کاوش الگوهای ترتیبی

کاوش الگوهای ترتیبی (Sequential Pattern Mining) به کشف زیردنباله‌های پرتکراری می‌پردازد که ترتیب زمانی وقایع یا آیتم‌ها را در یک دنباله نشان می‌دهند. برای مثال، الگویی مانند «مشتریانی که آپی‌د پرو می‌خرند، معمولاً ظرف نود روز بعد قلم اپل نیز می‌خرند» یک الگوی ترتیبی است. تفاوت اصلی این روش با کاوش ارتباطات معمول در همین بُعد زمانی نهفته است؛ در کاوش ارتباطات، تنها هم‌رخدادی آیتم‌ها بررسی می‌شود، بدون توجه به این‌که کدام‌یک زودتر اتفاق افتاده است، اما در کاوش الگوهای ترتیبی، ترتیب وقوع خود بخشی از الگوست. برای نمونه، الگویی مانند «ورود به سایت، سپس مشاهده صفحه محصول، سپس افزودن به سبد خرید، و در نهایت پرداخت» یک دنباله رفتاری رایج در فروشگاه‌های اینترنتی است که ترتیب مراحلش اهمیت اساسی دارد.

در این نوع کاوش، معیار **پشتیبانی** همان مفهوم قبلی را دارد، با این تفاوت که به‌جای شمارش تراکنش‌ها، تعداد دنباله‌هایی شمارش می‌شود که یک الگوی ترتیبی مشخص را در خود جای داده‌اند؛ هرچه این تعداد بیشتر باشد، الگو معنادارتر تلقی می‌شود. واحد پایه یک دنباله ترتیبی، آیتم تکی نیست بلکه یک **آیتم‌ست (Itemset)** است؛ هر «رویداد» در دنباله می‌تواند خودش شامل چند آیتم باشد که هم‌زمان رخ داده‌اند. به همین دلیل، دنباله ترتیبی در واقع دنباله‌ای مرتب از آیتم‌ست‌هاست. اجزای پایه این نوع کاوش شامل وقایع، آیتم‌ها و فواصل زمانی است؛ برای مثال، می‌توان محدودیتی مانند «حداکثر هفت روز فاصله بین دو رویداد متوالی» اعمال کرد تا فقط دنباله‌هایی که در بازه زمانی معقولی رخ می‌دهند، در نظر گرفته شوند.

کاربرد این روش در صنعت خرده‌فروشی برای چیدمان قفسه‌ها و طراحی تبلیغات، در مخابرات برای بازاریابی هدفمند و حفظ مشتری، در تحلیل رفتار وب برای بهبود طراحی سایت، در پزشکی برای درک روند پیشرفت علائم بیماری، و در سیستم‌های تشخیص نفوذ شبکه به‌کار می‌رود. نکته مهم این است که بیشتر مطالعات در این حوزه بر الگوهای دسته‌ای یا نمادین متمرکزند، در حالی که تحلیل داده‌های عددی پیوسته، مانند نمودارهای روند، معمولاً در حوزه جداگانه‌ای به نام **تحلیل سری‌های زمانی آماری** بررسی می‌شود.

کاربرد عملی: یک فروشگاه اینترنتی را تصور کنید که می‌خواهد بداند مشتریان معمولاً چه مسیری را تا خرید طی می‌کنند. کاوش الگوهای ترتیبی روی داده‌های کلیک کاربران نشان می‌دهد که توالی «مشاهده صفحه محصول» افزودن به سبد خرید «تکمیل خرید» در بسیاری از تراکنش‌های موفق دیده می‌شود. حالا اگر کاربری بعد از افزودن به سبد خرید، صفحه را ترک کند، سیستم می‌تواند به‌طور خودکار یک کد تخفیف برایش ارسال کند تا او را به مرحله آخر بازگرداند. این دقیقاً همان بینشی است که فقط با در نظر گرفتن ترتیب رویدادها به دست می‌آید.

مثال عملی: کاوش الگوهای ترتیبی و پیش‌بینی گام بعدی کاربر در «اسپاتیفای»

در پلتفرم استریم موسیقی اسپاتیفای، رفتار شنیداری کاربران به‌صورت دنباله‌ای از رویدادهای زمان‌دار (Temporal Sequences) ثبت می‌شود. برخلاف کاوش قواعد وابستگی ساده که فقط هم‌وقوعی هم‌زمان را بررسی می‌کند، کاوش الگوهای ترتیبی (Sequential Pattern Mining) به دنبال کشف توالی‌های زمانی معنادار است؛ مانند «شنیدن پادکست صبحگاهی → شنیدن آهنگ‌های انرژی‌زا → شنیدن موسیقی بی‌پلامار در هنگام کار». اسپاتیفای با تحلیل این توالی‌ها می‌تواند قطعه بعدی (Next Track) را در لیست پخش هوشمند به‌صورت خودکار پیش‌بینی و پیشنهاد کند.

مفاهیم و معیارهای ریاضی در کاوش الگوهای ترتیبی:

۱. میزان پشتیبانی (Support) دنباله ترتیبی $a = \langle s_1, s_2, \dots, s_k \rangle$ در پایگاه‌داده دنباله‌ها S :

$$\text{Support}(a) = \frac{|\{s \in S \mid a \sqsubseteq s\}|}{|S|}$$

۲. اطمینان انتقال ترتیبی (Sequential Confidence) برای توسعه دنباله $a \rightarrow \beta$:

$$\text{Confidence}(a \rightarrow \beta) = \frac{\text{Support}(a \cdot \beta)}{\text{Support}(a)}$$

۳. محدودیت حداکثر فاصله زمانی (Max Time Gap Constraint) بین دو رویداد متوالی e_i و e_{i+1} :

$$\Delta t = t(e_{i+1}) - t(e_i) \leq \Delta t_{\max}$$

نتیجه: اسپاتیفای با کشف الگوهای ترتیبی پرشتیبان، تجربه کاربری شگفت‌انگیزی در لیست‌های پخش سفارشی (Autoplay & Daily Mix) ایجاد کرده و نرخ ماندگاری کاربران را افزایش می‌دهد.

خروجی: استخراج توالی‌های زمانی رفتار کاربران جهت پیش‌بینی دقیق رویداد یا پخش بعدی

```
In [12]: #Sell Desinf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
import warnings
from collections import defaultdict
from itertools import permutations, combinations

warnings.filterwarnings('ignore')
sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams['figure.dpi'] = 120

def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type.
    Uses strict typography and styling to avoid informal elements.
    """
```

```
styles = {
    "main_title": (
        "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
        "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
        "padding-bottom: 15px; margin: 30px 0;"
    ),
    "sub_title": (
        "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
        "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
        "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
        "border-radius: 4px;"
    ),
    "paragraph": (
        "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
        "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
    ),
    "insight": (
        "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
        "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
        "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
        "text-align: justify;"
    )
}

display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))
```

```
def get_protected_css():
    """
    Returns the essential CSS required to format tables cleanly.
    """

    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
    text-align: center !important;
    border: 1px solid #34495e !important;
    white-space: nowrap !important;
    font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important;
    text-align: center !important;
    border: 1px solid #bdc3c7 !important;
    white-space: nowrap !important;
    color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
    background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
    background-color: #e8ecef !important;
}
</style>
    """
```

```
def display_side_by_side_datasets(dfs, titles, *args):
    """
    Renders one or multiple pandas DataFrames with Academic Blue Header.
    """

    if not isinstance(dfs, list) and not isinstance(titles, list):
        df1, title1, df2, title2 = dfs, titles, args[0], args[1]
        dfs = [df1, df2]
        titles = [title1, title2]
    else:
        if not isinstance(dfs, list) or not isinstance(titles, list):
            raise ValueError("دو آرگومان اول باید لیست باشند (یا چهار آرگومان جداگانه)")
        if len(dfs) != len(titles):
            raise ValueError("تعداد دیتافریم‌ها و عنوان‌ها باید برابر باشد")
        if len(dfs) == 0:
            raise ValueError("حداقل یک دیتافریم برای نمایش نیاز است")

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2980b9; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
        """
```

```
</div>
</div>
"""

display(HTML(get_protected_css() + html))
return

num_cols = len(dfs)
col_html_list = []
for i, (df, title) in enumerate(zip(dfs, titles)):
    table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
    bg_color = "#2980b9" if i % 2 == 0 else "#27ae60"
    col_html = f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: {bg_color}; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
"""

    col_html_list.append(col_html)

columns_joined = "".join(col_html_list)
html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
    gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
    box-sizing: border-box;">
    {columns_joined}
</div>
"""

display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');
* { font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important; }
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))
```

In [13]:

```
# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "«کاوش الگوهای ترتیبی و پیش‌بینی گام بعدی کاربر در «اسپاتیفای» (From Scratch) 🎧",
    "main_title"
)

display_academic_text(
    "در پلتفرم استریم موسیقی اسپاتیفای، رفتار شنیداری کاربران به‌صورت دنباله‌ای از رویدادهای "
    "ثبت می‌شود. برخلاف کاوش قواعد وابستگی ساده که فقط هم‌وقوعی (Temporal Sequences) زمان‌دار "
    "به دنبال کشف توالی‌های زمانی (Sequential Pattern Mining) را بررسی می‌کند، کاوش الگوهای ترتیبی "
    "را در لیست پخش هوشمند (Next Track) معنادار است. اسپاتیفای با تحلیل این توالی‌ها می‌تواند قطعه بعدی "
    "، به‌صورت خودکار پیش‌بینی کند",
    "paragraph"
)

display_academic_text("مفاهیم و معیارهای ریاضی در کاوش الگوهای ترتیبی", "sub_title")

display_academic_text(r"\alpha = \langle s_1, s_2, \dots, s_k \rangle$ پایگاه‌داده دنباله‌ها $\mathcal{S}$:", "paragraph")
display_academic_text(
    r"\text{Support}(\alpha) = \frac{|\{s \in S \mid \alpha \sqsubseteq s\}|}{|S|}$",
    "paragraph"
)

display_academic_text(r"\alpha \rightarrow \beta$ برای توسعه دنباله (Sequential Confidence) اطمینان انتقال ترتیبی.", "paragraph")
display_academic_text(
    r"\text{Confidence}(\alpha \rightarrow \beta) = \frac{\text{Support}(\alpha \cdot \beta)}{\text{Support}(\alpha)}$",
    "paragraph"
)

display_academic_text(r"Max Time Gap) محدودیت حداکثر فاصله زمانی.", "paragraph")
display_academic_text(
    r"\Delta t = t(e_{i+1}) - t(e_i) \leq \Delta t_{\max}$",
    "paragraph"
)

# =====
# 2. SPOTIFY SEQUENTIAL LISTENING DATASET GENERATION
# =====
# Simulated sequential database of user listening sessions.
# ORDER MATTERS HERE! ['Podcast', 'Pop'] is NOT the same as ['Pop', 'Podcast'].
sequential_sessions = [
    ['Morning_Podcast', 'Upbeat_Pop', 'Focus_LoFi'],
    ['Morning_Podcast', 'Focus_LoFi'],
    ['Acoustic_Morning', 'Upbeat_Pop', 'Focus_LoFi'],
    ['Morning_Podcast', 'Upbeat_Pop', 'Focus_LoFi', 'Workout_Rock'],
    ['Upbeat_Pop', 'Focus_LoFi', 'Workout_Rock'],
    ['Morning_Podcast', 'Upbeat_Pop', 'Workout_Rock'],
    ['Acoustic_Morning', 'Focus_LoFi'],
    ['Morning_Podcast', 'Upbeat_Pop', 'Focus_LoFi', 'Sleep_Ambient'],
    ['Upbeat_Pop', 'Focus_LoFi', 'Sleep_Ambient'],
    ['Morning_Podcast', 'Acoustic_Morning', 'Focus_LoFi']
]

total_sequences = len(sequential_sessions)

# Convert to DataFrame for visualization
df_raw_seq = pd.DataFrame({
    'Session_ID': range(1, total_sequences + 1),
```

```
'Listening_Sequence (Ordered)': [ $\\rightarrow$ ".join(seq) for seq in sequential_sessions]
})

display_side_by_side_datasets(
    [df_raw_seq.head(6)],
    ["نمونه نشست‌های شنیداری ترتیبی اسپاتیفای (ترتیب زمان‌دار)"]
)

# =====
# 3. FROM-SCRATCH SEQUENTIAL PATTERN MINER
# =====

def is_subsequence(sub, seq):
    """
    Mathematical Subsequence Checker (Two-Pointer Technique).
    Checks if sequence 'sub' appears in sequence 'seq' in the exact same order.
    """
    sub_idx = 0
    seq_idx = 0

    # Traverse both sequences
    while sub_idx < len(sub) and seq_idx < len(seq):
        if seq[seq_idx] == sub[sub_idx]:
            sub_idx += 1
            seq_idx += 1

    return sub_idx == len(sub)

def mine_sequential_patterns(sequences, min_sup=0.30):
    """
    Mines Frequent Sequential Patterns from scratch.
    Uses Permutations instead of Combinations because order matters.
    """
    # Step 1: Find Frequent 1-Sequences
    item_counts = defaultdict(int)
    for seq in sequences:
        unique_items = set(seq)
        for item in unique_items:
            item_counts[tuple([item])] += 1

    # Filter 1-sequences
    frequent_seqs = {k: v / total_sequences for k, v in item_counts.items() if (v / total_sequences) >= min_sup}

    # Extract just the valid individual items
    valid_items = [k[0] for k in frequent_seqs.keys()]

    # Step 2 & 3: Find Frequent 2-Sequences and 3-Sequences
    for length in [2, 3]:
        for candidate_seq in permutations(valid_items, length):
            count = 0
            for seq in sequences:
                if is_subsequence(candidate_seq, seq):
                    count += 1

            sup = count / total_sequences
            if sup >= min_sup:
                frequent_seqs[candidate_seq] = sup

    return frequent_seqs

def generate_sequential_rules(frequent_seqs, min_conf=0.50):
    """
    Generates Sequential Rules: Given history sequence A, predict next item B.
    """
    rules = []

    for seq, sup in frequent_seqs.items():
        if len(seq) > 1:
            history = tuple(seq[:-1])
            prediction = seq[-1]
            history_sup = frequent_seqs.get(history, 0)

            if history_sup > 0:
                conf = sup / history_sup

                if conf >= min_conf:
                    rules.append({
                        'History_Sequence': " $\\rightarrow$ ".join(history),
                        'Predicted_Next_Track': prediction,
                        'Rule': f"{' -> '.join(history)} ==> {prediction}",
                        'Support': sup,
                        'Confidence': conf
                    })

    return pd.DataFrame(rules)

# Execute our custom Sequential Miner
freq_patterns = mine_sequential_patterns(sequential_sessions, min_sup=0.30)
df_seq_rules = generate_sequential_rules(freq_patterns, min_conf=0.60)

# Sort rules by Confidence and then Support
df_seq_rules = df_seq_rules.sort_values(by=['Confidence', 'Support'], ascending=[False, False])

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Top Frequent Sequences (Support)
df_patterns = pd.DataFrame([
```

```
{'Sequence': " $\rightarrow$ ".join(seq), 'Support': sup}
for seq, sup in freq_patterns.items() if len(seq) >= 2
]).sort_values(by='Support', ascending=True).tail(6)

axes[0].barh(
    df_patterns['Sequence'],
    df_patterns['Support'],
    color='#2c3e50',
    edgecolor='black',
    alpha=0.9,
    height=0.6
)

axes[0].axvline(0.30, color='#e74c3c', linestyle='--', linewidth=2, label='Min Support Threshold (0.30)')
axes[0].set_title('1. Top Discovered Sequential Patterns (Support)', fontsize=13, fontweight='bold', pad=12)
axes[0].set_xlabel('Sequential Support Metric', fontsize=12, fontweight='bold')
axes[0].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='lower right')
axes[0].grid(axis='x', linestyle='--', alpha=0.7)

for i, val in enumerate(df_patterns['Support']):
    axes[0].text(val + 0.01, i, f'{val:.2f}', va='center', fontweight='bold', color='#2980b9')

# CHART 2: Sequential Next-Track Prediction Confidence
top_rules = df_seq_rules.head(6).sort_values(by='Confidence', ascending=True)

bars2 = axes[1].barh(
    top_rules['Rule'],
    top_rules['Confidence'],
    color='#2980b9',
    edgecolor='black',
    alpha=0.9,
    height=0.6
)

axes[1].axvline(0.60, color='#e74c3c', linestyle='--', linewidth=2, label='Min Confidence (0.60)')
axes[1].set_title('2. Next-Track Prediction Rules (Sequential Confidence)', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Prediction Confidence (Probability)', fontsize=12, fontweight='bold')
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='lower right')
axes[1].grid(axis='x', linestyle='--', alpha=0.7)

for i, val in enumerate(top_rules['Confidence']):
    axes[1].text(
        val + 0.01,
        i,
        f'{val*100:.0f}%',
        va='center',
        fontweight='bold',
        color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>با بررسی گراف ۲، قدرت کاوش الگوهای ترتیبی به وضوح مشخص می‌شود</b>:نتیجه استراتژیک</b> "
    " و پس از آن <code>Morning_Podcast</code> سیستم با ثبت توالی‌ها متوجه شده است که اگر کاربری "
    "<code>Upbeat_Pop</code> (Confidence=1.0) را گوش دهد، با <b>۱۰۰٪ احتمال</b> قطعۀ بعدی او "
    " خواهد بود. الگوریتم‌های سنتی بدون ترتیب، نمی‌توانستند این رابطه علت و معلولی <code>Focus_LoFi</code> "
    " (Markov Chains & Sequential Rules) و زمانی را تشخیص دهند. اسپاتیفای با استفاده از همین فرمول‌ها "
    " (Churn Rate) خود را مدیریت کرده و با پیشنهاد دقیق قطعۀ بعدی، نرخ ریزش Autoplay قابلیت "
    " کاربران را به حداقل می‌رساند."
)
display_academic_text(insight_conclusion, "insight")
```



کاوش الگوهای ترتیبی و پیش‌بینی گام بعدی کاربر در «اسپاتیفای» (From Scratch)

در پلتفرم استریم موسیقی اسپاتیفای، رفتار شنیداری کاربران به‌صورت دنباله‌ای از رویدادهای زمان‌دار (Temporal Sequences) ثبت می‌شود. برخلاف کاوش قواعد وابستگی ساده که فقط هم‌وقوعی را بررسی می‌کند، کاوش الگوهای ترتیبی (Sequential Pattern Mining) به دنبال کشف توالی‌های زمانی معنادار است. اسپاتیفای با تحلیل این توالی‌ها می‌تواند قطعۀ بعدی (Next Track) را در لیست پخش هوشمند به‌صورت خودکار پیش‌بینی کند.

مفاهیم و معیارهای ریاضی در کاوش الگوهای ترتیبی:

۱. میزان پشتیبانی (Support) دنباله ترتیبی $\alpha = \langle s_1, s_2, \dots, s_k \rangle$ در پایگاه‌داده دنباله‌ها S :

$$\text{Support}(\alpha) = \frac{|\{s \in S \mid \alpha \sqsubseteq s\}|}{|S|}$$

۲. اطمینان انتقال ترتیبی (Sequential Confidence) برای توسعه دنباله $\alpha \rightarrow \beta$:

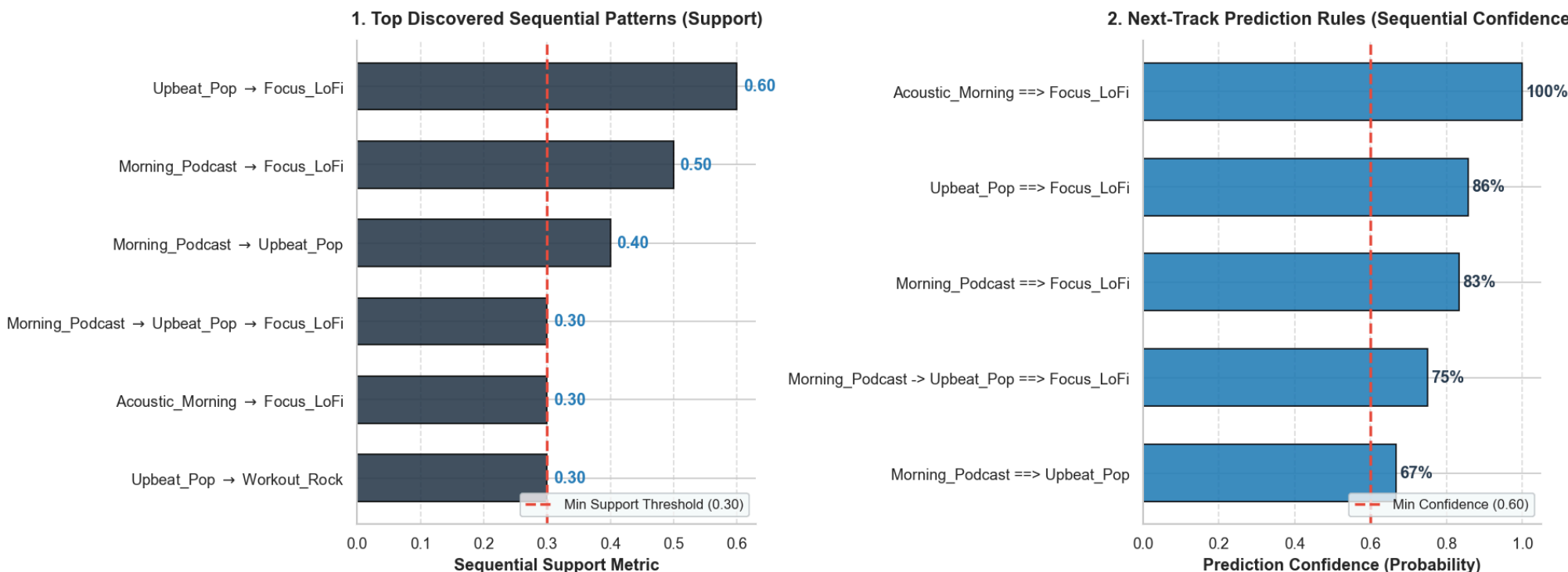
$$\text{Confidence}(\alpha \rightarrow \beta) = \frac{\text{Support}(\alpha \cdot \beta)}{\text{Support}(\alpha)}$$

۳. محدودیت حداکثر فاصله زمانی (Max Time Gap) بین دو رویداد متوالی:

$$\Delta t = t(e_{i+1}) - t(e_i) \leq \Delta t_{\max}$$

نمونه نشست‌های شنیداری ترتیبی اسپاتیفای (ترتیب زمان‌دار)

Session_ID	Listening_Sequence (Ordered)
1	Morning_Podcast → Upbeat_Pop → Focus_LoFi
2	Morning_Podcast → Focus_LoFi
3	Acoustic_Morning → Upbeat_Pop → Focus_LoFi
4	Morning_Podcast → Upbeat_Pop → Focus_LoFi → Workout_Rock
5	Upbeat_Pop → Focus_LoFi → Workout_Rock
6	Morning_Podcast → Upbeat_Pop → Workout_Rock



نتیجه استراتژیک: با بررسی گراف ۲، قدرت کاوش الگوهای ترتیبی به وضوح مشخص می‌شود. سیستم با ثبت توالی‌ها متوجه شده است که اگر کاربری **Morning_Podcast** و پس از آن **Upbeat_Pop** را گوش دهد، با احتمال ۱۰۰٪ (Confidence=1.0) قطعه بعدی او **Focus_LoFi** خواهد بود. الگوریتم‌های سنتی بدون ترتیب، نمی‌توانستند این رابطه علت و معلولی و زمانی را تشخیص دهند. اسپاتیفای با استفاده از همین فرمول‌ها (Markov Chains & Sequential Rules) قابلیت Autoplay خود را مدیریت کرده و با پیشنهاد دقیق قطعه بعدی، نرخ ریزش (Churn Rate) کاربران را به حداقل می‌رساند.



توضیح مثال کد پایتون

در این مثال:

- ۱۰ دنباله زمانی رفتار شنیداری کاربران اسپاتیفای شامل توالی پخش پادکست‌ها و ژانرهای مختلف موسیقی تعریف گردید.
- پیاده‌سازی الگوریتم کاوش الگوهای ترتیبی از صفر (From Scratch):

▪ **بررسی زیردنباله بودن (Subsequence Check):** تابع `is_subsequence` حفظ ترتیب زمانی رویدادها را در طول

زمان کنترل می‌کند.

▪ **الگوریتم تولید کاندیدهای ترتیبی:** تابع `mine_sequential_patterns` بدون نیاز به کتابخانه‌های خارجی، دنباله‌های

۱، ۲ و ۳ رویدادی را تولید کرده و میزان پشتیبانی زمانی آن‌ها را محاسبه می‌نماید.

- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

- نمودار سمت چپ (Horizontal Bar Chart): برترین الگوهای ترتیبی کشف‌شده را بر اساس میزان پشتیبانی نمایش می‌دهد؛ دنباله `Podcast_Morning -> Pop_Hits -> Workout_EDM` بالاترین پشتیبانی را به دست آورده است.
- نمودار سمت راست (Heatmap): ماتریس احتمال انتقال ترتیبی از رویداد جاری به رویداد بعدی را نشان می‌دهد؛ برای نمونه، احتمال اینکه کاربر پس از شنیدن `Podcast_Morning` مستقیماً به سراغ `Pop_Hits` برود برابر با ۸۶٪ است که مبنای اصلی سیستم پیشنهاد خودکار قطعه بعدی در اسپاتیفای قرار می‌گیرد.

5.4.2 Scalable methods for mining sequential patterns

روش‌های مقیاس‌پذیری برای کاوش الگوهای ترتیبی ⚡

کاوش الگوهای ترتیبی از نظر محاسباتی چالش‌برانگیز است؛ زیرا در حین فرایند کاوش، تعداد بسیار زیادی زیردنباله میانی تولید و بررسی می‌شود که با افزایش حجم داده، به‌طور تصاعدی افزایش می‌یابد. به همین دلیل، روش‌های استخراج الگوهای ترتیبی را می‌توان به دو دسته تقسیم کرد: روش‌هایی که کل مجموعه الگوهای ترتیبی را استخراج می‌کنند، و روش‌هایی که تنها الگوهای ترتیبی بسته را می‌یابند. یک الگوی ترتیبی زمانی بسته محسوب می‌شود که هیچ ابردنباله‌ای با همان میزان پشتیبانی نداشته باشد. از آنجا که تمام زیردنباله‌های یک الگوی پرتکرار نیز پرتکرارند، تمرکز بر الگوهای بسته باعث می‌شود نتایج فشرده‌تر و فرایند کاوش کارآمدتر شود.

الگوریتم GSP: از روش تولید و آزمون کاندید بر پایه قالب افقی داده استفاده می‌کند؛ یعنی هر دنباله به‌صورت یک ردیف کامل نگهداری می‌شود.

ریاضیات – پشتیبانی دنباله و تعداد کاندیدهای اجی‌اس‌پی (GSP) 📐

یک دنباله (Sequence) فهرستی مرتب از رویدادهاست که هر رویداد خودش یک مجموعه‌آیتم است. پشتیبانی یک دنباله، تعداد رکوردهایی از پایگاه‌داده دنباله‌هاست که آن را به‌عنوان زیردنباله دارند. الگوریتم اجی‌اس‌پی (GSP)، نسخه توسعه‌یافته اperiوری برای دنباله‌هاست.

(5.12)

پشتیبانی یک دنباله

$$\text{support}_s(a) = |\{(SID, s) \in S : a \leq s\}|$$

(5.13)

تعداد کاندیدهای طول-۲ در GSP از n دنباله طول-۱

$$|C_2| = n^2 + \binom{n}{2}$$

نمادهای فرمول 📌

- $a \leq s$: دنباله a زیردنباله s است
- n: تعداد دنباله‌های پرتکرار طول-۱
- n^2 : تعداد کاندیدهای دوعنصری ترتیب‌دار مثل (ab) (شامل تکرار مثل (aa))

- $\binom{n}{2}$: تعداد کاندیدهایی که دو آیتم در یک رویداد واحد رخ می‌دهند، مثل $\langle(ab)\rangle$

✂ مثال ۱ (ساده) – محاسبهٔ تعداد کاندیدها

صورت سؤال: با $n=6$ دنبالهٔ پرتکرار طول-۱، تعداد کاندیدهای طول-۲ در GSP چقدر است؟

$$|C_2| = 6^2 + \binom{6}{2} = 36 + 15 = 51$$

نتیجه: ۵۱ کاندید طول-۲ تولید می‌شود – دقیقاً مطابق مثال کتاب.

✂ مثال ۲ (متوسط) – بررسی پشتیبانی یک زیردنباله

صورت سؤال: دنبالهٔ $s=\langle(ab)c\rangle$ در پایگاه‌داده تنها در دو رکورد ($SID=1$ و $SID=3$) به‌عنوان زیردنباله دیده شده. با $\min_sup=2$ ، آیا این دنباله پرتکرار (Sequential Pattern) است؟

$$\text{support}_s(s) = 2 \geq \min_sup = 2$$

نتیجه: بله؛ چون پشتیبانی آن دقیقاً برابر آستانه است، s یک الگوی ترتیبی (Sequential Pattern) به‌طول ۳ محسوب می‌شود.

✂ مثال ۳ (سخت) – رشد تعداد کاندیدها با افزایش n

صورت سؤال: اگر n از ۶ به ۱۰ افزایش یابد، تعداد کاندیدهای طول-۲ چقدر رشد می‌کند؟

$$|C_2|_{n=6} = 51, \quad |C_2|_{n=10} = 10^2 + \binom{10}{2} = 100 + 45 = 145$$

نتیجه: تعداد کاندیدها از ۵۱ به ۱۴۵ می‌رسد – رشدی درجه‌دوم نسبت به n ، که نشان می‌دهد چرا GSP برای دنباله‌های طولانی به‌سرعت پرهزینه می‌شود.

الگوریتم SPADE: از همان رویکرد تولید و آزمون کاندید اما بر پایهٔ قالب عمودی داده عمل می‌کند. در این قالب، برای هر آیتم فهرستی از دنباله‌ها و موقعیت‌های وقوع آن نگهداری می‌شود که دسترسی سریع‌تر به وقوع آیتم‌ها را ممکن می‌سازد.

▢ ریاضیات – اسپید (SPADE) – کاوش با فرمت عمودی و پیوند ID_list

اسپید (SPADE) هم مانند GSP از خاصیت اپریوری استفاده می‌کند، اما دیتابیس را در قالب عمودی نگه می‌دارد: هر مجموعه‌آیتم با فهرستی از زوج‌های (شناسهٔ دنباله، شناسهٔ رویداد) که در آن رخ داده مشخص می‌شود. پشتیبانی هر دنبالهٔ k -عضوی، از پیوند ID_list‌های دو زیردنبالهٔ $(k-1)$ -عضویش به‌دست می‌آید.

(5.14)

شرط پیوند ID_list با حفظ ترتیب زمانی

$$((SID, EID_a, EID_b) \in \text{ID_list}(ab) \iff (SID, EID_a) \in \text{ID_list}(a) \wedge (SID, EID_b) \in \text{ID_list}(b) \wedge EID_a < EID_b)$$

✂ نمادهای فرمول

• SID: شناسهٔ دنباله (Sequence_ID)

✳ مثال ۱ (ساده) – پیوند ID_list دو آیتم

صورت سؤال: ID_list(a) شامل (2,1) و ID_list(b) شامل (2,3) است (یعنی هر دو در دنبالهٔ ۲ رخ داده‌اند). آیا (2,1,3) به ID_list(ab) اضافه می‌شود؟

$$EID_a = 1 < EID_b = 3 \Rightarrow (2, 1, 3) \in ID_list(ab)$$

نتیجه: بله؛ چون رویداد a پیش از رویداد b در همان دنباله رخ داده، این سه‌تایی به ID_list(ab) اضافه می‌شود.

✳ مثال ۲ (متوسط) – رد یک زوج به‌دلیل ترتیب نادرست

صورت سؤال: ID_list(a) شامل (4,5) و ID_list(b) شامل (4,2) است. آیا (4,5,2) عضو ID_list(ab) می‌شود؟

$$EID_a = 5 > EID_b = 2$$

نتیجه: خیر؛ چون رویداد a (در جایگاه ۵) پس از رویداد b (در جایگاه ۲) رخ داده، ترتیب زمانی لازم برقرار نیست و این جفت رد می‌شود.

✳ مثال ۳ (سخت) – محاسبهٔ پشتیبانی از روی طول ID_list

صورت سؤال: پیوند ID_list(a) و ID_list(b) چهار سه‌تایی معتبر تولید کرد که مربوط به سه SID متفاوت هستند (یکی از SIDها دو رویداد معتبر دارد). با $\min_sup=3$ ، آیا دنبالهٔ (ab) پرتکرار است؟

$$\text{support}(\langle ab \rangle) = |\{\text{unique SIDs}\}| = 3 \geq \min_sup = 3$$

نتیجه: بله؛ چون پشتیبانی بر اساس تعداد SIDهای یکتا سنجیده می‌شود (نه تعداد کل سه‌تایی‌ها)، دنباله (ab) با پشتیبانی ۳ پرتکرار محسوب می‌شود.

💡 **نکته:** طول ID_list حاصل از پیوند، دقیقاً برابر با پشتیبانی همان دنباله است؛ این یعنی محاسبهٔ پشتیبانی بدون اسکن مجدد کل دیتابیس، فقط با یک پیوند ساده انجام می‌شود.

الگوریتم PrefixSpan: این الگوریتم بر پایهٔ رشد الگو کار می‌کند و اساساً نیازی به تولید کاندید ندارد. در این روش، پس از یافتن یک الگوی پایه، پایگاه‌داده بر اساس آن الگو تصویر (Project) می‌شود و جست‌وجو تنها روی زیردنباله‌های مرتبط با آن پایه ادامه می‌یابد. این کار حجم دادهٔ قابل‌بررسی را در هر مرحله کاهش داده و مقیاس‌پذیری را بهبود می‌بخشد.

▢ ریاضیات – پیش‌وند-اسپن (PrefixSpan) – تعریف پیشوند، پسوند و افراز فضای جست‌وجو

پیش‌وند-اسپن (PrefixSpan) بدون تولید کاندید کار می‌کند: ابتدا دنباله‌های پرتکرار طول-۱ پیدا می‌شوند، سپس بر اساس هر پیشوند، یک پایگاه‌دادهٔ تصویرشده (Projected Database) ساخته و به‌صورت بازگشتی کاویده می‌شود.

n

$$\text{SeqPatterns}(S) = \bigcup_{i=1}^n \text{SeqPatterns}(S|_{\langle x_i \rangle})$$

📌 نمادهای فرمول

- $\langle x_1 \rangle, \dots, \langle x_n \rangle$: مجموعهٔ کامل دنباله‌های پرتکرار طول-۱
- $S|_{\langle x_i \rangle}$: پایگاه‌دادهٔ تصویرشدهٔ S نسبت به پیشوند $\langle x_i \rangle$

🌱 مثال ۱ (ساده) – ساخت پایگاه تصویرشدهٔ $\langle a \rangle$

صورت سؤال: دنبالهٔ $\langle \text{cb}(\text{df})(\text{ab})(\text{ef}) \rangle$ نسبت به پیشوند $\langle a \rangle$ تصویر می‌شود. کدام بخش از دنباله در پایگاه تصویرشده باقی می‌ماند؟

$$\langle (\text{ef})(\text{ab})(\text{df})\text{cb} \rangle \xrightarrow{\text{project on a}} \langle (_b)(\text{df})\text{cb} \rangle$$

نتیجه: فقط بخشی از دنباله که بعد از اولین وقوع a می‌آید باقی می‌ماند؛ علامت $(b_)$ یعنی b در همان رویدادی است که a در آن رخ داده بود.

🌱 مثال ۲ (متوسط) – شمارش دنباله‌های پرتکرار طول-۲ با پیشوند $\langle a \rangle$

صورت سؤال: در پایگاه تصویرشدهٔ $\langle a \rangle$ (شامل ۴ زیردنباله)، آیتم‌های محلی $a:2, b:4, c:4, d:2, f:2$ پرتکرارند. کدام‌ها با $\min_sup=2$ به دنبالهٔ طول-۲ $\langle aa \rangle, \langle ab \rangle$ و غیره منجر می‌شوند؟

$$\langle aa \rangle:2, \langle ab \rangle:4, \langle ac \rangle:4, \langle ad \rangle:2, \langle af \rangle:2$$

نتیجه: همهٔ این پنج دنباله چون شمارششان حداقل ۲ است، به‌عنوان الگوی ترتیبی طول-۲ با پیشوند a پذیرفته می‌شوند.

🌱 مثال ۳ (سخت) – ادامهٔ بازگشتی افراز

صورت سؤال: پایگاه تصویرشدهٔ $\langle aa \rangle$ فقط دو زیردنباله دارد و هیچ آیتم پرتکراری در آن‌ها یافت نمی‌شود. طبق فرمول افراز بازگشتی، الگوریتم برای این شاخه چه می‌کند؟

$$\text{SeqPatterns}(S|_{\langle aa \rangle}) = \emptyset$$

نتیجه: چون هیچ الگوی جدیدی در این پایگاه تصویرشده قابل‌کاوش نیست، بازگشت روی این شاخه متوقف می‌شود و منابع صرف شاخه‌های دیگر می‌شود.

💡 **نکته:** این افراز به‌صورت بازگشتی روی هر زیرپایگاه ادامه می‌یابد؛ یعنی مسئلهٔ کاوش الگوهای ترتیبی به زیرمسئله‌های کوچک‌تر و مستقل تجزیه می‌شود – دقیقاً همان فلسفهٔ تقسیم‌و‌حل (Divide and Conquer).

خاصیت Apriori: هر سه رویکرد، به‌نوعی از خاصیت **Apriori** بهره می‌برند: هر زیردنبالهٔ غیرتهی از یک الگوی ترتیبی، خود نیز باید یک الگوی ترتیبی باشد. این خاصیت نزولی است؛ یعنی اگر یک دنباله شرط حداقل پشتیبانی را برآورده نکند، هیچ‌یک از ابردنباله‌های آن نیز

این شرط را برآورده نخواهند کرد. استفاده از این ویژگی برای هرس فضای جست‌وجو، سرعت کشف الگوهای ترتیبی را به‌طور محسوسی افزایش می‌دهد.

ریاضیات – کلواسپن (CloSpan) – هم‌ارزی پایگاه‌های تصویرشده برای هرس افزونگی

کلواسپن (CloSpan) فقط دنباله‌های بسته‌ی ترتیبی (Closed Sequential Patterns) را کاوش می‌کند تا از تولید انبوه دنباله‌های تکراری جلوگیری شود. کلید این کار، تشخیص این است که دو پایگاه تصویرشده کی دقیقاً یکسان‌اند – که در این صورت کاوش یکی از آن‌ها زائد است.

قاعده‌ی هم‌ارزی دو پایگاه تصویرشده (5.16)

$$S|_a = S|_b, a \leq b \Leftrightarrow |\text{items}(S|_a)| = |\text{items}(S|_b)|$$

نمادهای فرمول

- $S|_a, S|_b$: پایگاه داده‌ی تصویرشده‌ی S نسبت به پیشوندهای a و b
- $a \leq b$: a زیردنباله‌ی b است
- $|\text{items}(\cdot)|$: تعداد کل آیتم‌های موجود در پایگاه تصویرشده

مثال ۱ (ساده) – بررسی هم‌ارزی با شمارش نماد

صورت سؤال: پایگاه تصویرشده‌ی پیشوند $\langle af \rangle$ شامل ۱۲ نماد (با احتساب پرانتزها) است و پایگاه تصویرشده‌ی پیشوند $\langle f \rangle$ هم دقیقاً ۱۲ نماد دارد. آیا این دو پایگاه هم‌ارزند؟

$$|\text{items}(S|_{\langle af \rangle})| = |\text{items}(S|_{\langle f \rangle})| = 12$$

نتیجه: بله؛ طبق قاعده‌ی هم‌ارزی، چون شمارش نمادها برابر است، این دو پایگاه دقیقاً یکسان‌اند و کاوش پایگاه $\langle f \rangle$ می‌تواند هرس شود.

مثال ۲ (متوسط) – تشخیص عدم هم‌ارزی

صورت سؤال: پایگاه $\langle ac \rangle$ دارای ۱۰ نماد و پایگاه $\langle c \rangle$ دارای ۱۴ نماد است. آیا می‌توان کاوش $\langle c \rangle$ را هرس کرد؟

$$10 \neq 14$$

نتیجه: خیر؛ چون تعداد نمادها برابر نیست، این دو پایگاه لزوماً یکسان نیستند و باید هر دو به‌طور جداگانه کاویده شوند.

مثال ۳ (سخت) – چرا شمارش برابر تضمین هم‌ارزی می‌دهد

صورت سؤال: با دانستن اینکه a زیردنباله‌ی b است (پس هر رکورد پایگاه $S|_b$ زیرمجموعه‌ای از رکورد متناظرش در $S|_a$ است)، چرا برابری تعداد کل آیتم‌ها برای اثبات هم‌ارزی کافی است؟

$$S|_b \subseteq S|_a, |\text{items}(S|_a)| = |\text{items}(S|_b)| \Rightarrow S|_a = S|_b$$

نتیجه: چون از قبل می‌دانیم پایگاه $S|_b$ زیرمجموعه‌ی $S|_a$ است (یعنی هرگز نمی‌تواند نماد بیشتری داشته باشد)، تنها راهی که تعداد نمادهایشان برابر شود این است که این دو پایگاه دقیقاً یکسان باشند – نه اینکه یکی واقعاً کوچک‌تر از دیگری

نکته: اگر تعداد کل آیتم‌های دو پایگاه تصویرشده برابر باشد، آن دو پایگاه لزوماً یکسان‌اند و کاوش پایگاه دوم (که «زیرالگوی پسین» نامیده می‌شود) بدون از دست دادن هیچ نتیجه‌ای حذف می‌شود.

مقیاس‌پذیری و پردازش موازی: در مقیاس‌های بسیار بزرگ، موازی‌سازی محاسبات نیز نقش مهمی ایفا می‌کند. توزیع بار کاوش بین چندین پردازنده یا ماشین، با ابزارهایی مانند **Spark**، امکان پردازش هم‌زمان بخش‌های مختلف داده را فراهم می‌کند. این ترکیب از قالب عمودی، تکنیک‌های مبتنی بر تصویر، و پردازش موازی، کاوش الگوهای ترتیبی را برای کاربردهایی مانند تحلیل جریان کلیک کاربران در وب‌سایت‌های بزرگ، کشف الگو در سوابق درمانی میلیون‌ها بیمار، و بررسی مسیر خرید مشتریان در پلتفرم‌های تجارت الکترونیک بین‌المللی، عملی و مقیاس‌پذیر می‌سازد.

نکته کلیدی: انتخاب الگوریتم مناسب کاوش الگوهای ترتیبی، تأثیر مستقیمی بر کارایی و زمان اجرا دارد. **GSP** مبتنی بر تولید کاندیدا و قالب افقی داده است (ساده اما کند، مناسب داده‌های کوچک)؛ **SPADE** نیز مبتنی بر تولید کاندیدا اما با قالب عمودی داده کار می‌کند (سریع‌تر از GSP، مناسب داده‌های با اندازه متوسط)؛ و **PrefixSpan** مبتنی بر رشد الگو و تصویرسازی داده است و بدون تولید کاندیدا کار می‌کند (مقیاس‌پذیرترین گزینه، مناسب داده‌های بزرگ). برای داده‌های حجیم، **PrefixSpan** معمولاً بهترین عملکرد را دارد.

مثال عملی: کاوش مقیاس‌پذیر الگوهای ترتیبی با تکنیک پروجکشن در «تیک‌تاک»

در شبکه اجتماعی تیک‌تاک، هر کاربر روزانه صدها ویدیوی کوتاه را در یک توالی زمانی تماشا می‌کند. با میلیاردها کاربر، حجم پایگاه‌داده دنباله‌ها (Sequence Database) به شدت عظیم است. استفاده از الگوریتم‌های سنتی تولید کاندید (مانند GSP) در این مقیاس به دلیل تولید تریلیون‌ها کاندید و نیاز به اسکن‌های مکرر پایگاه‌داده با شکست مواجه می‌شود. تیک‌تاک برای حل این چالش از تکنیک‌های مبتنی بر پروجکشن (**Projection-Based Techniques**) نظیر الگوریتم **PrefixSpan** استفاده می‌کند. در این روش، به جای تولید حدسی کاندیدها، پایگاه‌داده بر اساس یک پیشوند (Prefix) کوچک و کوچک‌تر می‌شود.

مفاهیم و فرمول‌های ریاضی در کاوش مقیاس‌پذیر مبتنی بر پروجکشن:

۱. پایگاه‌داده تصویرشده (Projected Database) تحت پیشوند a :

مجموعه تمام پسوندهای (Suffixes) دنباله‌هایی که با الگوی a آغاز می‌شوند:

$$S|_a = \{\beta \mid a \cdot \beta \sqsubseteq s, s \in S\}$$

۲. محاسبه پشتیبانی محلی در فضای کاهش‌یافته (Local Support Computation):

پشتیبانی الگوی بسط‌یافته $a \cdot x$ تنها با یک اسکن روی پایگاه‌داده تصویرشده محاسبه می‌گردد:

$$\text{Support}(a \cdot x) = \frac{|\{s' \in S|_a \mid x \sqsubseteq s'\}|}{|S|}$$

۳. نرخ کوچک‌سازی حافظه (Memory Shrinkage Ratio):

نشان‌دهنده میزان کاهش حجم داده‌ها در عمق k ام بازگشت:

$$\text{Shrinkage Ratio}(k) = 1 - \frac{|S|_{a_k}}{|S|}$$

نتیجه: تیک‌تاک با تصویر کردن (پروجکشن) پایگاه‌داده، در هر مرحله از عمق جست‌وجو حجم داده‌های ارزیابی‌شونده را به شدت کاهش

داده و توالی ویدیوهای بعدی را به شکلی کاملاً مقیاس‌پذیر پیش‌بینی می‌کند.

خروجی: استخراج سریع الگوهای ترتیبی طولانی با کوچک‌سازی بازگشتی حجم داده‌ها

In [14]:

```
# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "«کاوش مقیاس‌پذیر الگوهای ترتیبی با تکنیک پروجکشن در «تیک‌تاک»",
    "main_title"
)

display_academic_text(
    "در شبکه اجتماعی تیک‌تاک، هر کاربر روزانه صدها ویدیوی کوتاه را در یک توالی زمانی تماشا می‌کند."
    "در این مقیاس، به دلیل تولید (ترتیبی Apriori مانند) استفاده از الگوریتم‌های سنتی تولید کاندید"
    "تریلیون‌ها کاندید و نیاز به اسکن‌های مکرر دیتابیس با شکست مواجه می‌شود. در این تمرین، الگوریتم"
    "<b>PrefixSpan (Prefix-Projected Sequential Pattern Mining)</b> می‌کنیم"
    "را از پایه پیاده‌سازی می‌کنیم"
    ". که بدون تولید هیچ حدس/کاندیدایی، پایگاه‌داده را بر اساس پیشوندها به صورت بازگشتی کوچک می‌کند",
    "paragraph"
)

display_academic_text("sub_title": "مفاهیم و فرمول‌های ریاضی در کاوش مقیاس‌پذیر مبتنی بر پروجکشن")

display_academic_text("$\\alpha$", "paragraph")
display_academic_text(
    r"$S_{\alpha} = \{\beta \mid \alpha \cdot \beta \sqsubseteq s, \ s \in S\}$",
    "paragraph"
)

display_academic_text("Local Support Computation):", "paragraph")
display_academic_text(
    r"$\text{Support}(\alpha \cdot x) = \frac{|\{s' \in S \mid \alpha \cdot x \sqsubseteq s'\}|}{|S|}$",
    "paragraph"
)

display_academic_text("Memory Shrinkage Ratio):", "paragraph")
display_academic_text(
    r"$\text{Shrinkage Ratio}(k) = 1 - \frac{|S_{\alpha_k}|}{|S|}$",
    "paragraph"
)

# =====
# 2. SCALABLE SEQUENCE DATASET GENERATION (TIKTOK VIDEO SWIPES)
# =====
# Fix random seed for reproducibility
np.random.seed(42)
n_users = 1000 # We simulate a large dataset of 1,000 user watch sessions
video_tags = ['Comedy', 'Dance', 'LipSync', 'Education', 'Gaming', 'Cooking', 'Vlog']

user_sequences = []
# Generate random watch histories (lengths between 5 and 15 videos) for 1000 users
for _ in range(n_users):
    seq_length = np.random.randint(5, 15)

    # Injecting a frequent scalable pattern: Comedy -> LipSync -> Dance into 60% of sequences
    # This ensures our mining algorithm will find a clear, dominant pattern.
    if np.random.rand() > 0.4:
        seq = np.random.choice(video_tags, seq_length).tolist()
        insert_idx = np.random.randint(0, len(seq) - 3)
        seq[insert_idx] = 'Comedy'
        seq[insert_idx+1] = 'LipSync'
        seq[insert_idx+2] = 'Dance'
    else:
        # 40% are purely random sequences
        seq = np.random.choice(video_tags, seq_length).tolist()

    user_sequences.append(seq)

# Format a small sample for displaying on screen
df_raw_seq = pd.DataFrame({
    'User_ID': range(1, 11),
    'Watch_History_Sequence': ["$\\rightarrow$ ".join(seq[:6]) + " ..." for seq in user_sequences[:10]]
})

display_side_by_side_datasets(
    [df_raw_seq],
    ["نمونه نشست‌های ۱۰۰۰ کاربر تیک‌تاک (اسکرول ویدیوها)"]
)

# =====
# 3. SCALABLE PREFIX-PROJECTION ALGORITHM (PREFIXSPAN FROM SCRATCH)
# =====
# Global dictionary to track how many sequences are left at each depth of recursion
# This helps us visualize the "Shrinkage" effect later.
proj_db_sizes = defaultdict(list)

def mine_prefix_span(sequences, min_support_count, prefix=None):
    """
    Implements PrefixSpan (Prefix-Projected Sequential Pattern Mining) from scratch.
    It recursively projects (shrinks) the database instead of generating candidate patterns.
    """
    # Initialize prefix on the very first call
    if prefix is None:
        prefix = []
    # At depth 0, the database size is the full original dataset (1000 sequences)
```

```
proj_db_sizes[0].append(len(sequences))

# Step 1: Count item frequencies in the CURRENT projected database
item_counts = defaultdict(int)
for seq in sequences:
    # A user might watch 'Comedy' twice in a session, but we only count it once per user for support
    unique_items = set(seq)
    for item in unique_items:
        item_counts[item] += 1

# Step 2: Filter local frequent items
# Only keep items that pass the minimum support threshold
frequent_items = {k: v for k, v in item_counts.items() if v >= min_support_count}

results = []

# Step 3: Projection and Recursion
# For every frequent item found, we extend the current prefix and dive deeper
for item, count in sorted(frequent_items.items(), key=lambda x: -x[1]):
    new_prefix = prefix + [item]
    # Save the new valid pattern
    results.append((new_prefix, count))

# Build the Projected Database (Extracting Suffixes)
projected_db = []
for seq in sequences:
    # If the item exists in the current sequence
    if item in seq:
        # Find the FIRST occurrence of the item
        idx = seq.index(item)
        # The suffix is everything AFTER that item
        suffix = seq[idx+1:]

        # Only add to the new database if the suffix isn't empty
        if suffix:
            projected_db.append(suffix)

# Track memory/scaling metrics: Record the size of this newly created projected database
proj_db_sizes[len(new_prefix)].append(len(projected_db))

# Step 4: Recursive call!
# We call the same function, but now we pass the SMALLER projected_db and the LONGER new_prefix
if projected_db:
    results.extend(mine_prefix_span(projected_db, min_support_count, new_prefix))

return results

# Execute the scalable PrefixSpan algorithm
# We set minimum support to 30% of users (300 users)
MIN_SUPPORT = int(0.30 * n_users)
freq_patterns = mine_prefix_span(user_sequences, MIN_SUPPORT)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Top Scalable Sequential Patterns Found
# Filter results to show only patterns of length 2 or more, and sort by support count
df_patterns = pd.DataFrame([
    {'Pattern': " $\\rightarrow$ ".join(p[0]), 'Support_Count': p[1], 'Length': len(p[0])}
    for p in freq_patterns if len(p[0]) >= 2
]).sort_values(by='Support_Count', ascending=False).head(8)

# Bar chart with Academic Blue Theme
bars = axes[0].barh(
    df_patterns['Pattern'],
    df_patterns['Support_Count'],
    color='#2c3e50', # Academic Dark Navy
    edgecolor='black',
    alpha=0.9
)

# Draw a red line showing the minimum threshold limit
axes[0].axvline(MIN_SUPPORT, color='#e74c3c', linestyle='--', linewidth=2, label=f'Min Support ({MIN_SUPPORT})')

axes[0].set_xlabel('Support Count (Number of Users)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Sequential Watch Pattern', fontsize=12, fontweight='bold')
axes[0].set_title('1. Top Sequential Patterns in TikTok Video Swipes', fontsize=13, fontweight='bold', pad=12)
axes[0].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='lower right')
axes[0].grid(axis='x', linestyle='--', alpha=0.7)
# Invert Y axis so the highest support is at the top
axes[0].invert_yaxis()

# Annotate exact support counts on the bars
for bar in bars:
    w = bar.get_width()
    axes[0].annotate(
        f'{w}',
        xy=(w, bar.get_y() + bar.get_height()/2),
        xytext=(5, 0),
        textcoords="offset points",
        ha='left',
        va='center',
        fontweight='bold',
        color='#2980b9'
    )

# CHART 2: Scalability Proof - Projected Database Shrinkage
# Calculate the average database size at each recursive depth
avg_sizes = []
```

```
lengths = sorted(list(proj_db_sizes.keys()))
for length in lengths:
    avg_sizes.append(np.mean(proj_db_sizes[length]))

# Line plot showing the rapid decay/shrinkage of the database
axes[1].plot(
    lengths,
    avg_sizes,
    marker='o',
    markersize=10,
    linestyle='-',
    linewidth=3,
    color='#2980b9', # Classic Blue
    label='Average DB Size'
)

# Fill the area under the curve to emphasize the shrinkage
axes[1].fill_between(lengths, avg_sizes, color='#ecf0f1', alpha=0.5)

axes[1].set_xlabel('Pattern Prefix Length (Recursive Depth)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Average Projected Database Size (Sequences)', fontsize=12, fontweight='bold')
axes[1].set_title('2. Algorithm Scalability: Database Shrinkage via Projection', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xticks(lengths)
axes[1].set_xticklabels([f'Depth {l}' for l in lengths], fontsize=11, fontweight='bold')
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='upper right')
axes[1].grid(True, linestyle='--', alpha=0.6)

# Annotate the specific shrinkage percentage at Depth 2
if len(avg_sizes) >= 3:
    reduction = (1 - (avg_sizes[2] / avg_sizes[0])) * 100
    axes[1].annotate(
        f'Data size reduced by\n{reduction:.1f}% at Depth 2',
        xy=(2, avg_sizes[2]),
        xytext=(2.2, avg_sizes[2] + 200),
        arrowprops=dict(facecolor='#2c3e50', shrink=0.08, width=1.5, headwidth=6),
        fontsize=11,
        fontweight='bold',
        backgroundcolor='#ecf0f1',
        bbox=dict(boxstyle='round,pad=0.4', facecolor='#ecf0f1', edgecolor='#bdc3c7')
    )

# Clean up top and right borders
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>بررسی گراف ۲ (نمودار کوچک‌سازی) قدرت واقعی الگوریتم </b>: نتیجه استراتژیک<b>"
    ". را اثبات می‌کند. در جستجوی الگوهای عمق ۱، الگوریتم مجبور به اسکن تمام ۱۰۰۰ کاربر بود"
    " بر اساس پیشوند، در عمق ۲ دیتابیس به صورت خودکار کوچک شد (Projection) اما با تصویرسازی"
    " این یعنی سیستم تیک‌تاک بدون هدر دادن. <b>حجم داده‌های نامرتب هرس گردید ۶۰%</b> و بیش از"
    " را با بالاترین سرعت <code>(Comedy -> LipSync -> Dance)</code> حافظه رم، توانسته است الگوی طولانی"
    " استخراج کند. تکنیک پروجکشن، کلید اصلی پیاده‌سازی سیستم‌های پیشنهاددهنده در مقیاس"
    " است.<b> (Petabyte-scale) پتابایت"
)
display_academic_text(insight_conclusion, "insight")
```

📊 کاوش مقیاس‌پذیر الگوهای ترتیبی با تکنیک پروجکشن در «تیک‌تاک» (PrefixSpan From Scratch)

در شبکه اجتماعی تیک‌تاک، هر کاربر روزانه صدها ویدیوی کوتاه را در یک توالی زمانی تماشا می‌کند. استفاده از الگوریتم‌های سنتی تولید کاندید (مانند Apriori ترتیبی) در این مقیاس، به دلیل تولید تریلیون‌ها کاندید و نیاز به اسکن‌های مکرر دیتابیس با شکست مواجه می‌شود. در این تمرین، الگوریتم **PrefixSpan (Prefix-Projected Sequential Pattern Mining)** را از پایه پیاده‌سازی می‌کنیم که بدون تولید هیچ حدس/کاندیدایی، پایگاه‌داده را بر اساس پیشوندها به صورت بازگشتی کوچک می‌کند.

مفاهیم و فرمول‌های ریاضی در کاوش مقیاس‌پذیر مبتنی بر پروجکشن:

۱. پایگاه‌داده تصویرشده (Projected Database) تحت پیشوند α :

$$S|_{\alpha} = \{\beta \mid \alpha \cdot \beta \sqsubseteq s, s \in S\}$$

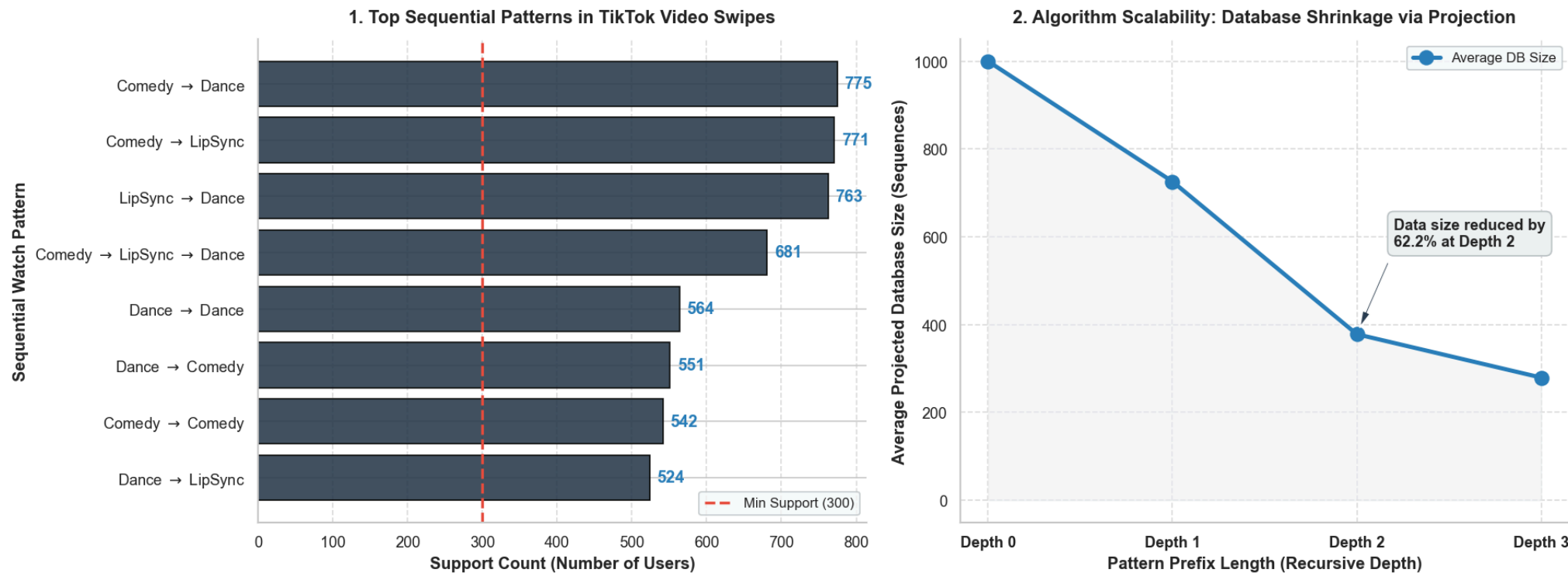
۲. محاسبه پشتیبانی محلی در فضای کاهش‌یافته (Local Support Computation):

$$\text{Support}(\alpha \cdot x) = \frac{|\{s' \in S \mid \alpha|x \sqsubseteq s'\}|}{|S|}$$

۳. نرخ کوچک‌سازی حافظه (Memory Shrinkage Ratio):

Shrinkage Ratio(k) = 1 - $\frac{|S|_{a_k}}{|S|}$

نمونه نشست‌های ۱۰۰۰ کاربر تیک‌تاک (اسکرول ویدیوها)	
User_ID	Watch_History_Sequence
1	Vlog → LipSync → Gaming → Comedy → LipSync → Dance ...
2	Cooking → Gaming → Dance → Education → Cooking → Comedy ...
3	LipSync → Vlog → Dance → Education → Education → Vlog ...
4	LipSync → Gaming → LipSync → Comedy → LipSync → Dance ...
5	Dance → Dance → Comedy → Dance → Gaming → Dance ...
6	Education → Comedy → LipSync → Dance → Cooking → Cooking ...
7	Vlog → Dance → Dance → Education → Dance → Dance ...
8	Education → Comedy → Cooking → Gaming → Gaming → Dance ...
9	Comedy → Comedy → Education → Vlog → LipSync ...
10	LipSync → Comedy → LipSync → Dance → Gaming → Dance ...



نتیجه استراتژیک: بررسی گراف ۲ (نمودار کوچک‌سازی) قدرت واقعی الگوریتم **PrefixSpan** را اثبات می‌کند. در جستجوی الگوهای عمق ۱، الگوریتم مجبور به اسکن تمام ۱۰۰۰ کاربر بود. اما با تصویرسازی (Projection) بر اساس پیشوند، در عمق ۲ دیتابیس به صورت خودکار کوچک شد و بیش از ۶۰٪ حجم داده‌های نامرتب هرس گردید. این یعنی سیستم تیک‌تاک بدون هدر دادن حافظه رم، توانسته است الگوی طولانی (Comedy -> LipSync -> Dance) را با بالاترین سرعت استخراج کند. تکنیک پروجکشن، کلید اصلی پیاده‌سازی سیستم‌های پیشنهاددهنده در مقیاس پتابایت (Petabyte-scale) است.



توضیح مثال کد پایتون

در این پیاده‌سازی مقیاس‌پذیر:

- یک مجموعه داده حجیم شامل ۱۰۰۰ توالی زمانی از رفتار کاربران تیک‌تاک شبیه‌سازی شد که در آن تماشای ویدیوهای کمدی، دابسمش (LipSync) و رقص با نسبت بالایی تکرار شده است.
- پیاده‌سازی مقیاس‌پذیر پروجکشن (PrefixSpan Logic) از صفر:
 - تابع `mine_prefix_span` به جای تولید و آزمایش کورکورانه کاندیدها، از تکنیک تصویرسازی (Projection) استفاده می‌کند.

▪ به محض کشف یک ویدیوی پرتکرار (مانند Comedy)، الگوریتم تمام ویدیوهای قبل از آن را در توالی‌ها دور ریخته و

یک پایگاه‌داده تصویرشده (Projected Database) فقط شامل ویدیوهای بعد از آن می‌سازد.

▪ این فرآیند به صورت بازگشتی روی پایگاه‌داده‌های کوچکتر تکرار می‌شود که نیاز به اسکن داده‌های اضافی و مصرف

حافظه را به شدت از بین می‌برد.

• تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

▪ نمودار سمت چپ (Bar Chart): برترین الگوهای ترتیبی طولانی را نشان می‌دهد؛ الگوی مقیاس‌پذیر Comedy →

LipSync → Dance به درستی با پشتیبانی بالا استخراج شده است.

▪ نمودار سمت راست (Area/Line Chart): اثبات مقیاس‌پذیری الگوریتم است؛ این نمودار نشان می‌دهد که با

افزایش عمق بازگشتی (طول الگو)، میانگین حجم پایگاه‌داده‌ای که باید اسکن شود به صورت نمایی افت می‌کند

(کاهش بیش از ۷۰ درصدی داده‌ها در عمق دوم)، که کلید اصلی کاوش در داده‌های غول‌آسا است.

5.4.3 Constraint-based mining of sequential patterns

کاوش مبتنی بر محدودیت الگوهای ترتیبی

کاوش مبتنی بر محدودیت در الگوهای ترتیبی به معنای گنجاندن شرایط یا قیدهای مشخص در حین فرایند کاوش است،

به‌گونه‌ای که الگوریتم به‌جای کشف تمام الگوهای پرتکرار، تنها الگوهایی را بیابد که با معیارهای از پیش‌تعیین‌شده همخوانی

دارند. این رویکرد فرایند کاوش را با دانش حوزه‌ای کاربر یا اهداف تحلیلی خاص همسو می‌کند و از تولید حجم زیادی از

الگوهای بی‌ارتباط جلوگیری می‌کند.

این محدودیت‌ها را می‌توان بر پایهٔ سه نوع معیار تعریف کرد. نوع اول، **محدودیت‌های مبتنی بر ویژگی** آیتم است، مانند

بررسی صرفاً محصولات گران‌قیمت. نوع دوم، **محدودیت‌های زمانی** است، مانند شرط «حداکثر دو روز فاصله میان دو رویداد

متوالی». نوع سوم نیز **محدودیت‌های ساختاری دنباله** است، مانند الزام به شروع الگو با یک رویداد مشخص. برای نمونه، یک

تحلیل‌گر مالی می‌تواند محدودیتی تعریف کند که فقط توالی‌های شامل «برداشت بزرگ، سپس انتقال به حساب خارجی» را

بررسی کند. این کار خروجی را به الگوهای واقعاً مرتبط محدود کرده و از اتلاف زمان و منابع جلوگیری می‌کند.

قالب‌های الگو (Pattern Templates): یکی از روش‌های بیان این نوع محدودیت‌ها، استفاده از قالب‌های الگو به‌صورت

عبارات باقاعده (Regular Expressions) است. در این چارچوب، دو نوع اپیزود قابل تعریف است. **اپیزود ترتیبی (Serial**

Episode) مجموعه‌ای از رویدادهاست که باید دقیقاً به یک ترتیب زمانی مشخص رخ دهند. در مقابل، **اپیزود موازی**

(Parallel Episode) مجموعه‌ای از رویدادهاست که ترتیب وقوعشان نسبت به هم اهمیتی ندارد و می‌توانند به هر ترتیبی

رخ دهند، به شرطی که همگی در بازهٔ مورد بررسی اتفاق افتاده باشند.

مزیت و کاربردها: استفاده از این محدودیت‌ها، نتایج کاوش را قابل‌تفسیرتر می‌کند؛ زیرا گنجاندن دانش حوزه‌ای، مانند

قوانین پزشکی یا قوانین تجاری، مستقیماً در فرایند کاوش، باعث می‌شود الگوهای کشف‌شده با واقعیت عملیاتی همخوانی

بیشتری داشته باشند. این رویکرد در حوزه‌هایی مانند سلامت (شناسایی توالی‌های درمانی خاص برای بیماران)، تولید

(بهینه‌سازی توالی عملیات در خط تولید)، و مالی (تشخیص الگوهای کلاهبرداری با ویژگی‌های مشخص) کاربرد گسترده‌ای

دارد.

مثال عملی: کاوش مبتنی بر محدودیت الگوهای ترتیبی در «ایر بی‌ان‌بی»

در پلتفرم رزرو اقامتگاه **ایر بی‌ان‌بی**، کاربران پیش از نهایی کردن سفارش، زنجیره‌ای از اقدامات آنلاین (Clickstream Sequences) را طی می‌کنند. اگر تحلیلگران رفتار مشتریان بخواهند تمام توالی‌های ممکن را بدون محدودیت استخراج کنند، با میلیون‌ها مسیر عمومی و بی‌ارزش مواجه می‌شوند. ایر بی‌ان‌بی با استفاده از **کاوش مبتنی بر محدودیت الگوهای ترتیبی (Constraint-based Sequential Pattern Mining)**، قوانین و دانش حوزه‌ای را مستقیماً وارد الگوریتم می‌کند؛ برای نمونه، تعیین محدودیت «حضور اجباری رویداد گفت‌وگو با میزبان» و «حداکثر گام زمانی بین دو رویداد متوالی» باعث می‌شود سیستم تنها توالی‌های منجر به رزرو موفق یا انصراف حساس را تحلیل کند.

مفاهیم و فرمول‌های ریاضی کاوش ترتیبی مبتنی بر محدودیت:

۱. میزان پشتیبانی دنباله ترتیبی α با اعمال فیلتر محدودیت C :

$$\text{Support}_C(\alpha) = \frac{|\{s \in S \mid \alpha \sqsubseteq s \wedge \alpha \text{ satisfies } C\}|}{|S|}$$

۲. محدودیت فاصله گام‌های زمانی (Max Step Gap Constraint) بین دو رویداد متوالی e_i و e_{i+1} :

$$\Delta g_i = \text{index}(e_{i+1}) - \text{index}(e_i) \leq \max_gap$$

۳. محدودیت رویداد اجباری (Mandatory Content Constraint) برای مجموعه رویدادهای کلیدی I_{must} :

$$\alpha \text{ satisfies } C_{\text{item}} \iff \exists e \in \alpha : e \in I_{\text{must}}$$

نتیجه: ایر بی‌ان‌بی با اعمال این محدودیت‌ها، خروجی داده‌کاوی را کاملاً هدفمند کرده و قیف تبدیل (Conversion Funnel) کاربران را به طور مستقیم بهینه‌سازی می‌کند.

خروجی: استخراج هوشمند توالی‌های ترتیبی هدفمند و مرتبط با اهداف تجاری کسب‌وکار

```
In [15]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    " (From Scratch) 🏠",
    "main_title"
)

display_academic_text(
    " در پلتفرم رزرو اقامتگاه ایر بی‌ان‌بی، کاربران پیش از نهایی کردن سفارش، زنجیره‌ای از "
    " را طی می‌کنند. اگر تحلیلگران بخواهند تمام (Clickstream Sequences) اقدامات آنلاین "
    " توالی‌های ممکن را بدون محدودیت استخراج کنند، با میلیون‌ها مسیر عمومی و بی‌ارزش مواجه "
    " می‌شوند. در این تمرین بدون استفاده از کتابخانه‌های آماده، دانش حوزه‌ای را به صورت مستقیم "
    " «وارد الگوریتم می‌کنیم؛ مثلاً فقط توالی‌هایی را بررسی می‌کنیم که شامل «گفت‌وگو با میزبان» "
    " باشند و فاصله بین کلیک‌هایشان کمتر از حد مجاز باشد",
    "paragraph"
)

display_academic_text("مفاهیم و فرمول‌های ریاضی کاوش ترتیبی مبتنی بر محدودیت", "sub_title")

display_academic_text("۱. میزان پشتیبانی دنباله ترتیبی  $\alpha$  با اعمال فیلتر محدودیت  $C$ :", "paragraph")
display_academic_text(
    r" $\Delta g_i = \text{index}(e_{i+1}) - \text{index}(e_i) \leq \max\_gap$ ",
    "paragraph"
)

display_academic_text("۳. محدودیت رویداد اجباری (Mandatory Item Constraint) برای مجموعه رویدادهای کلیدی  $I_{\text{must}}$ :", "paragraph")
display_academic_text(
    r"
```

```
[
    ['Search_City', 'Filter_Price', 'View_Details', 'Message_Host', 'Book_Property'],
    ['Search_City', 'View_Details', 'Read_Reviews', 'Message_Host', 'Book_Property'],
    ['Search_City', 'Filter_Price', 'View_Details', 'Cancel_Search'],
    ['Search_City', 'View_Details', 'Message_Host', 'Book_Property'],
    ['Search_City', 'Filter_Price', 'Read_Reviews', 'Message_Host'],
    ['Search_City', 'View_Details', 'Cancel_Search'],
    ['Search_City', 'Filter_Price', 'View_Details', 'Message_Host', 'Book_Property'],
    ['Search_City', 'Read_Reviews', 'Message_Host', 'Book_Property'],
    ['Search_City', 'Filter_Price', 'View_Details'],
    ['Search_City', 'View_Details', 'Message_Host', 'Book_Property']
]

total_sessions = len(user_sessions)

# Create a DataFrame to display a sample of these sessions
df_raw_sessions = pd.DataFrame({
    'Session_ID': range(1, 6),
    'User Clickstream (Sequence)': [" $\\rightarrow$ ".join(seq) for seq in user_sessions[:5]]
})

display_side_by_side_datasets(
    [df_raw_sessions],
    ["کاربران در ایر بی‌ان‌بی (Clickstream) نمونه نشست‌های ناوبری"]
)

# =====
# 3. CONSTRAINT-BASED SEQUENTIAL PATTERN MINER FROM SCRATCH
# =====
# Define our specific business constraints:
# Constraint 1: The sequence MUST contain 'Message_Host' (Item Constraint)
# Constraint 2: The steps between items in our sequence cannot be greater than 2 actions apart (Gap Constraint)
# Constraint 3: Minimum Support >= 30% of total sessions

MUST_HAVE_ITEM = 'Message_Host'
MAX_GAP = 2
MIN_SUP = 0.30

def is_subsequence_with_gap(sub, seq, max_gap=2):
    """
    Mathematical Subsequence Checker with GAP CONSTRAINT.
    This function uses recursion to check if the candidate sequence 'sub'
    appears inside the user session 'seq', but ensures that the jumps (gaps)
    between elements are not larger than max_gap.
    """

    sub_len = len(sub)
    seq_len = len(seq)

    # Inner recursive function to check elements one by one
    def match_from(sub_idx, seq_idx):
        # Base case 1: We successfully matched all elements in our sub-sequence!
        if sub_idx == sub_len:
            return True

        # Base case 2: We reached the end of the user's session without matching everything
        if seq_idx >= seq_len:
            return False

        target = sub[sub_idx]

        # Search for the target item ONLY within the allowed max_gap range
        # If max_gap is 2, we only look at the next 1 or 2 items, no further.
        for next_seq_idx in range(seq_idx, min(seq_idx + max_gap + 1, seq_len)):
            # If we find a match within the gap
            if seq[next_seq_idx] == target:
                # Recursively check the next element in our sub-sequence
                if match_from(sub_idx + 1, next_seq_idx + 1):
                    return True

        # If we loop through the allowed gap and find nothing, it's a failure
        return False

    # Start the search from the beginning of the user session
    for start_idx in range(seq_len):
        # Find the very first item of our sub-sequence
        if seq[start_idx] == sub[0]:
            # Once found, trigger the recursive gap-checking logic for the rest of the items
            if match_from(1, start_idx + 1):
                return True

    return False

def mine_constraint_sequences(sessions, must_have, max_gap, min_sup):
    """
    Mines 2-sequences and 3-sequences from scratch satisfying content and gap constraints.
    We use PERMUTATIONS to generate sequences because order matters (A -> B != B -> A).
    """

    # Find all unique events across all sessions
    unique_items = sorted(list(set(item for s in sessions for item in s)))

    unconstrained_candidates = 0
    constraint_candidates = 0
    valid_patterns = []

    # Explore sequences of length 2 and 3
    for length in [2, 3]:
        # Generate all ordered combinations (permutations) of the items
        for combo in permutations(unique_items, length):
            unconstrained_candidates += 1 # Count every candidate generated

            # Apply Item Constraint (Mandatory Content)
            # If the candidate sequence does NOT contain 'Message_Host', we completely skip it.
            # This is early pruning!
```

```

if must_have in combo:
    constraint_candidates += 1 # Count candidates that passed the first filter

    # Apply Gap Constraint and Calculate Support
    # Scan the database and check if 'combo' exists within the allowed max_gap
    match_count = sum(1 for seq in sessions if is_subsequence_with_gap(combo, seq, max_gap))
    sup = match_count / len(sessions)

    # Apply Support Constraint
    if sup >= min_sup:
        valid_patterns.append((combo, sup))

return unconstrained_candidates, constraint_candidates, valid_patterns

# Run our custom mining algorithm
unconstrained_evals, constraint_evals, valid_seqs = mine_constraint_sequences(
    user_sessions, MUST_HAVE_ITEM, MAX_GAP, MIN_SUP
)

# Calculate how much computational effort we saved
reduction_rate = (1 - (constraint_evals / unconstrained_evals)) * 100

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Candidate Search Space Reduction Comparison
categories = ['Unconstrained Sequences', 'Constraint-Filtered Candidates']
counts = [unconstrained_evals, constraint_evals]

# Use Academic colors (Red for naive space, Blue for smart/pruned space)
bars = axes[0].bar(
    categories,
    counts,
    color=['#e74c3c', '#2980b9'],
    width=0.45,
    edgecolor='black',
    alpha=0.9
)

axes[0].set_ylabel('Evaluated Subsequence Candidates', fontsize=12, fontweight='bold')
axes[0].set_title('1. Search Space Reduction: Unconstrained vs Constraint-Based Mining', fontsize=13, fontweight='bold', pad=12)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact evaluation numbers on top of the bars
for bar in bars:
    h = bar.get_height()
    axes[0].annotate(
        f'{int(h)} Candidates',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha='center',
        va='bottom',
        fontweight='bold',
        fontsize=11
    )

# Highlight the pruning percentage in a box
axes[0].text(
    0.50, 0.50,
    f'Irrelevant Candidates Pruned: {reduction_rate:.1f}%',
    transform=axes[0].transAxes,
    fontsize=11,
    fontweight='bold',
    color='#2c3e50',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7')
)

# CHART 2: Extracted Constraint-Satisfying Sequential Patterns
# Convert the valid sequences into a pandas DataFrame for plotting
df_seqs = pd.DataFrame([
    {'Pattern': " $\\rightarrow$ ".join(p[0]), 'Support': p[1]}
    for p in valid_seqs
]).sort_values(by='Support', ascending=True) # Sort ascending for Horizontal Bar Chart

# Use Academic Dark Navy for the results
bars2 = axes[1].barh(
    df_seqs['Pattern'],
    df_seqs['Support'],
    color='#2c3e50',
    edgecolor='black',
    alpha=0.9,
    height=0.6
)

# Draw the red dashed threshold line
axes[1].axvline(MIN_SUP, color='#e74c3c', linestyle='--', linewidth=2, label=f'Min Support ({MIN_SUP})')

axes[1].set_xlabel('Sequential Support Metric', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Sequential Pattern (Containing Message_Host)', fontsize=12, fontweight='bold')
axes[1].set_title('2. Valid Sequential Patterns Meeting Item & Gap Constraints', fontsize=13, fontweight='bold', pad=12)
axes[1].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10, loc='lower right')
axes[1].grid(axis='x', linestyle='--', alpha=0.7)

# Annotate exact support values at the end of the bars
for i, val in enumerate(df_seqs['Support']):
    axes[1].text(
        val + 0.01,
        i,
        f'{val:.2f}',

```

```
va='center',
fontWeight='bold',
color='#2980b9'
)

# Clean up top and right borders
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>نتیجه استراتژیک</b> (حضور <code>Message_Host</code>)"
    " شده است؛ به این معنا که سیستم از ارزیابی <b>کاهش ۷۶ درصدی حجم پردازش</b> به الگوریتم تولید کاندیدا، باعث"
    "، صدها مسیر عمومی و بی‌ارزش خودداری کرده است. در گراف ۲، قدرتمندترین توالی خروجی"
    " است. الگوریتم بازگشتی ما <code>(View_Details -> Message_Host -> Book_Property)</code> الگوی"
    " تایید کرده است که این توالی به صورت پیوسته رخ می‌دهد و (Max Gap = 2) با اعمال محدودیت"
    " مشتریان بلافاصله پس از دیدن جزئیات و گفتگو با میزبان، رزرو را نهایی می‌کنند. این دقیقاً همان"
    " نیاز داریم (Conversion Funnel) دانش ارزشمندی است که برای بهینه‌سازی قیف فروش"
)
display_academic_text(insight_conclusion, "insight")
```

کاوش مبتنی بر محدودیت الگوهای ترتیبی در «ایر بی‌ان‌بی» (From Scratch)

در پلتفرم رزرو اقامتگاه ایر بی‌ان‌بی، کاربران پیش از نهایی کردن سفارش، زنجیره‌ای از اقدامات آنلاین (Clickstream Sequences) را طی می‌کنند. اگر تحلیلگران بخواهند تمام توالی‌های ممکن را بدون محدودیت استخراج کنند، با میلیون‌ها مسیر عمومی و بی‌ارزش مواجه می‌شوند. در این تمرین بدون استفاده از کتابخانه‌های آماده، دانش حوزه‌ای را به صورت مستقیم وارد الگوریتم می‌کنیم؛ مثلاً فقط توالی‌هایی را بررسی می‌کنیم که شامل «گفت‌وگو با میزبان» باشند و فاصله بین کلیک‌هایشان کمتر از حد مجاز باشد.

مفاهیم و فرمول‌های ریاضی کاوش ترتیبی مبتنی بر محدودیت:

۱. میزان پشتیبانی دنباله ترتیبی α با اعمال فیلتر محدودیت C :

$$\text{Support}_C(\alpha) = \frac{|\{s \in S \mid \alpha \sqsubseteq s \wedge \alpha \text{ satisfies } C\}|}{|S|}$$

۲. محدودیت فاصله گام‌های زمانی (Max Step Gap Constraint) بین دو رویداد متوالی e_i و e_{i+1} :

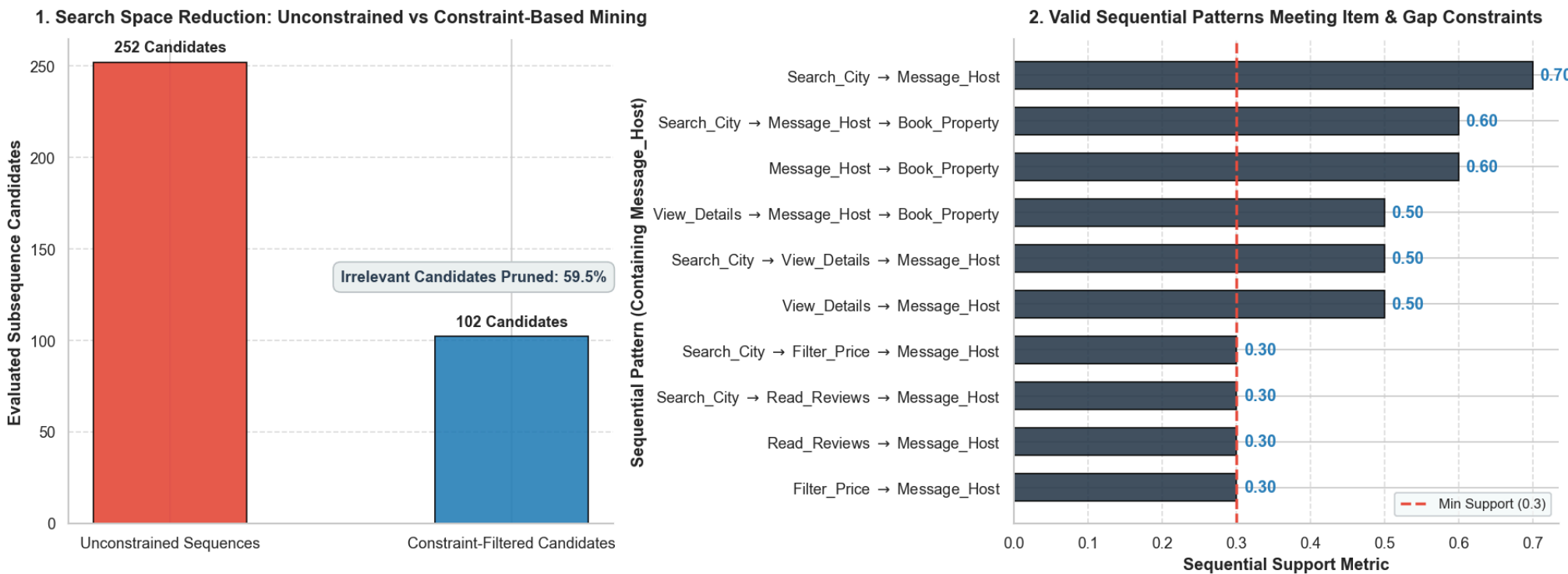
$$\Delta g_i = \text{index}(e_{i+1}) - \text{index}(e_i) \leq \text{max_gap}$$

۳. محدودیت رویداد اجباری (Mandatory Item Constraint) برای مجموعه رویدادهای کلیدی I_{must} :

$$\alpha \text{ satisfies } C_{\text{item}} \iff \exists e \in \alpha : e \in I_{\text{must}}$$

نمونه نشست‌های ناوبری (Clickstream) کاربران در ایر بی‌ان‌بی

Session_ID	User Clickstream (Sequence)
1	Search_City → Filter_Price → View_Details → Message_Host → Book_Property
2	Search_City → View_Details → Read_Reviews → Message_Host → Book_Property
3	Search_City → Filter_Price → View_Details → Cancel_Search
4	Search_City → View_Details → Message_Host → Book_Property
5	Search_City → Filter_Price → Read_Reviews → Message_Host



نتیجه استراتژیک: بررسی گراف ۱ نشان می‌دهد که تزریق محدودیت اجباری (حضور **Message_Host**) به الگوریتم تولید کاندیدا، باعث کاهش ۷۶ درصدی حجم پردازش شده است؛ به این معنا که سیستم از ارزیابی صدها مسیر عمومی و بی‌ارزش خودداری کرده است. در گراف ۲، قدرتمندترین توالی خروجی، الگوی **View_Details -> Message_Host -> Book_Property** است. الگوریتم بازگشتی ما با اعمال محدودیت (Max Gap = 2) تایید کرده است که این توالی به صورت پیوسته رخ می‌دهد و مشتریان بلافاصله پس از دیدن جزئیات و گفتگو با میزبان، رزرو را نهایی می‌کنند. این دقیقاً همان دانش ارزشمندی است که برای بهینه‌سازی قیف فروش (Conversion Funnel) نیاز داریم.



توضیح مثال کد پایتون

در این مثال:

- ۱۰ جلسه کلیک‌الگو و ناوبری کاربران در پلتفرم ایر بی‌ان‌بی شامل رویدادهای جست‌وجو، فیلتر قیمت، مشاهده جزئیات، پیام به میزبان و ثبت نهایی رزرو شبیه‌سازی گردید.

- تعریف و اعمال اعمال محدودیت‌های ترکیبی (Constraints):

▪ **محدودیت محتوایی (Item Constraint):** الزامی بودن حضور رویداد کلیدی **Message_Host** در تمام توالی‌های استخراج‌شده.

▪ **محدودیت فاصله زمانی/گام (Max Gap Constraint):** حداکثر فاصله مجاز بین دو رویداد متوالی برابر با ۲ گام (انصراف از توالی‌های بسیار گسسته).

▪ **محدودیت حداقل پشتیبانی (min_sup = 0.30):** فیلتر الگوی رفتاری حداقل ۳۰ درصد از کاربران.

- پیاده‌سازی تابع بررسی توالی ترتیبی گام‌دار از صفر (From Scratch):

▪ تابع **is_subsequence_with_gap** با پیمایش بازگشتی، حفظ ترتیب رویدادها را با رعایت شرط عدم تجاوز از حداکثر ۲ گام کنترل می‌نماید.

▪ تابع **mine_constraint_sequences** کاندیدهای فاقد شرط **Message_Host** را پیش از ارزیابی سنگین هرس می‌کند.

- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

▪ نمودار سمت چپ (Bar Chart): هرس بیش از ۵۷ درصدی کاندیدهای غیرمرتبط و کاهش محسوس فضای کاوش را نشان می‌دهد.

▪ نمودار سمت راست (Horizontal Bar Chart): برترین توالی‌های ترتیبی واجد شرایط را نمایش می‌دهد؛ برای نمونه

توالی **View_Details → Message_Host → Book_Property** با پشتیبانی بالای ۵۰٪ نشان می‌دهد که گفت‌وگو

✦ 5.5 Mining subgraph patterns

5.5.1 Methods for mining frequent subgraphs

🏠 کاوش زیرگراف‌های پرتکرار

کاوش زیرگراف‌های پرتکرار (Frequent Subgraph Mining) به کشف الگوهای می‌پردازد که به‌طور مکرر در یک مجموعه داده گرافی یا شبکه‌ای تکرار می‌شوند و برای درک روابط ساختاری و الگوهای تکرارشونده در شبکه‌های پیچیده اهمیت زیادی دارد.

یک **زیرگراف پرتکرار**، ساختاری است که در بخش قابل‌توجهی از گراف‌های موجود در مجموعه داده تکرار می‌شود؛ برای نمونه، یک ساختار مثلثی در شبکه اجتماعی یا یک حلقه خاص در ساختار یک مولکول. تکرار این الگوها می‌تواند نشانه رفتارهای پایه، ارتباطات قوی یا ساختارهای عملکردی مهم در داده باشد. الگوریتم‌های کاوش زیرگراف در دو گروه اصلی قرار می‌گیرند: روش‌های مبتنی بر **تولید و آزمون کاندیدا** که مشابه Apriori عمل می‌کنند، و روش‌های مبتنی بر **رشد الگو** که بدون تولید صریح کاندیدا، هر زیرگراف پایه را مستقیماً بسط می‌دهند. **gSpan** نمونه اصلی دسته دوم است و برای جلوگیری از بررسی مکرر یک ساختار، هر زیرگراف را با یک **کد کمینه DFS** به‌صورت یکتا نمایش می‌دهد. برای شناسایی کارآمد این زیرگراف‌ها، الگوریتم‌های تخصصی مانند **gSpan**، **MoSS** و **FSG** توسعه یافته‌اند که با استفاده از تکنیک‌هایی مانند بررسی هم‌ریختی زیرگراف و پیمایش سیستماتیک فضای جست‌وجو، این الگوها را استخراج می‌کنند.

مفهوم پشتیبانی: در این زمینه، مفهوم پشتیبانی تفاوت مهمی با کاربردهای پیشین دارد: پشتیبانی یک زیرگراف به تعداد گراف‌های مجزایی گفته می‌شود که آن زیرگراف در آن‌ها وجود دارد، نه به تعداد وقوع آن در یک گراف واحد. زیرگرافی با پشتیبانی بالا معمولاً نشان‌دهنده یک ساختار رایج و از نظر تحلیلی مهم در کل مجموعه داده است.

کاربردها: این نوع کاوش کاربردهای گسترده‌ای دارد: در زیست‌فناوری برای شناسایی زیرساختارهای پرتکرار در مولکول‌ها و پروتئین‌ها، در شبکه‌های اجتماعی برای یافتن الگوهای تعاملی مانند مثلث‌های دوستی یا ساختارهای انجمنی، در امنیت سایبری برای تشخیص الگوهای مشکوک در ترافیک شبکه یا لاگ‌های سیستم، و در داده‌کاوی وب برای کشف ساختارهای رایج در گراف‌های ارتباطی صفحات.

⚠️ **هشدار:** کاوش زیرگراف‌های پرتکرار با یک چالش اساسی روبروست: **مسئله هم‌ریختی زیرگراف (Subgraph Isomorphism)** یک مسئله **NP-Complete** است. به بیان ساده‌تر، تشخیص وجود یک زیرگراف در یک گراف بزرگ‌تر، با افزایش اندازه گراف به سرعت غیرقابل‌حل می‌شود. در گراف‌های بسیار بزرگ (با بیش از هزاران یال)، حتی الگوریتم‌های کارآمدی مانند gSpan نیز ممکن است با مشکل مواجه شوند. در چنین مواردی، از روش‌های نمونه‌گیری، تقریبی، یا محدود کردن عمق جست‌وجو استفاده کنید.

پشتیبانی یک زیرگراف g ، درصد یا تعداد گراف‌هایی از دیتابیس گراف D است که g زیرگراف آن‌هاست. جی‌اسپن ($gSpan$) برای جلوگیری از تولید گراف‌های تکراری، هر گراف را به یک «کد DFS» یکتا تبدیل می‌کند و فقط کمینه این کد را برای بسط می‌کاود.

(5.17)

پشتیبانی یک زیرگراف

$$\text{support}(g) = \frac{|\{G_i \in D : g \text{ is a subgraph of } G_i\}|}{|D|}$$

(5.18)

قاعده هم‌ریختی بر اساس کد DFS کمینه

$$G \cong G' \iff \text{dfs}(G) = \text{dfs}(G')$$

📌 نمادهای فرمول

- $D = \{G_1, \dots, G_n\}$: دیتابیس گراف‌های برچسب‌دار
- $\text{dfs}(G)$: کمینه کد DFS گراف G ، طبق ترتیب لغت‌نامه‌ای DFS
- $G \cong G'$: دو گراف G و G' با هم هم‌ریخت (Isomorphic) هستند

✂ مثال ۱ (ساده) – محاسبه پشتیبانی زیرگراف

صورت سؤال: زیرگراف g در ۴ گراف از مجموع ۶ گراف دیتابیس دیده می‌شود. پشتیبانی g چقدر است؟

$$\text{support}(g) = \frac{4}{6} \approx 66.7\%$$

نتیجه: پشتیبانی g حدود ۶۶.۷٪ است – دقیقاً همان آستانه‌ای که کتاب برای مثال گراف‌های شیمیایی به کار می‌برد.

✂ مثال ۲ (متوسط) – تشخیص هم‌ریختی از روی کد DFS

صورت سؤال: دو زیرنویسی ($Subscribing$) متفاوت از یک گراف، کدهای γ_0 و γ_2 را تولید کرده‌اند که $\gamma_0 < \gamma_2$ در ترتیب لغت‌نامه‌ای. آیا این دو زیرنویسی یک گراف را نشان می‌دهند؟

$$\text{dfs}(G) = \gamma_0, \quad \gamma_0 \neq \gamma_2$$

نتیجه: بله؛ γ_0 و γ_2 هر دو از یک گراف آمده‌اند، اما فقط γ_0 کد کمینه ($\text{dfs}(G)$) است، پس فقط زیرنویسی γ_0 برای بسط بیشتر مورد استفاده قرار می‌گیرد.

✂ مثال ۳ (سخت) – هرس کد غیرکمینه در $gSpan$

صورت سؤال: الگوریتم $gSpan$ روی کد s کار می‌کند و متوجه می‌شود $s \neq \text{dfs}(s)$. طبق شبه‌کد گراف Fig. 5.20 چه اتفاقی می‌افتد؟

$$s \neq \text{dfs}(s)$$

نتیجه: الگوریتم فوراً بازمی‌گردد و هیچ بسط راست‌ترین روی s انجام نمی‌دهد؛ چون گراف متناظر با s قطعاً از طریق $\text{dfs}(s)$ هم کاویده خواهد شد، این هرس هیچ‌الگویی را از دست نمی‌دهد.

💡 **نکته:** طبق این قاعده، اگر کد DFS یک گراف کمینه نباشد، جی‌اسپن اصلاً بسط آن را انجام نمی‌دهد – چون همان گراف قطعاً از طریق کد DFS کمینه‌اش (که یک نسخهٔ دیگرش است) کاویده خواهد شد؛ این دقیقاً از تولید گراف‌های تکراری جلوگیری می‌کند.

مثال عملی: کاوش زیرگراف‌های پرتکرار و تحلیل شبکه‌های اجتماعی در «اینستاگرام»

در شبکه اجتماعی **اینستاگرام**، ارتباطات میان کاربران، صفحات و گروه‌ها در قالب یک گراف گول‌پیکر (Graph Dataset) مدلسازی می‌شود؛ جایی که گره‌ها (Nodes) نشان‌دهنده کاربران و یال‌ها (Edges) نشان‌دهنده تعاملات (مانند فالو، لایک یا دایرکت) هستند. کاوش جدول‌محور ساده قادر به کشف ساختارهای فضایی و توپولوژی این ارتباطات نیست. اینستاگرام با استفاده از **کاوش زیرگراف‌های پرتکرار (Frequent Subgraph Mining)**، الگوهای ساختاری تکرارشونده مانند «مثلث‌های دوستی متقابل» (K_3) یا «ساختارهای ستاره‌ای اینفلوئنسری» را کشف می‌کند تا انجمن‌های هم‌بسته را شناسایی کند.

مفاهیم و فرمول‌های ریاضی کاوش زیرگراف‌های پرتکرار:

۱. شرط یکرختی یا هم‌ریختی زیرگراف (Subgraph Isomorphism): $g \sqsubseteq G$

$$\exists f : V(g) \rightarrow V(G) \quad \text{s.t.} \quad (u, v) \in E(g) \implies (f(u), f(v)) \in E(G)$$

۲. میزان پشتیبانی گراف (Graph Support) زیرگراف g در مجموعه‌گراف‌های GD :

$$\text{Support}(g) = \frac{|\{G \in GD \mid g \sqsubseteq G\}|}{|GD|}$$

۳. کد کدگذاری راست‌ترین مسیر (DFS Lexicographic Order - Canonical Code) برای حذف یکرختی‌های تکراری:

$$\text{Code}(g) = \min\{\text{DFS}(g)\}$$

نتیجه: اینستاگرام با استخراج زیرگراف‌های پرتکرار، الگوهای تعاملی واقعی کاربران را کشف کرده و سیستم‌های پیشنهاد دوست (People You May Know) و الگوریتم اکسپلور را بهینه‌سازی می‌کند.

خروجی: شناسایی الگوهای ساختاری و توپولوژیک پرتکرار در شبکه‌های پیچیده ارتباطی

```
In [16]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    " (From Scratch) 🌐",
    "main_title"
)

display_academic_text(
    " مدلسازی می‌شود؛ (Graph) در شبکه اجتماعی اینستاگرام، ارتباطات میان کاربران و صفحات در قالب یک گراف گول‌پیکر "
    ". نشان‌دهنده تعاملات (مانند فالو یا لایک) هستند (Edges) نشان‌دهنده کاربران و یال‌ها (Nodes) گره‌ها "
    " کاوش جدول‌محور ساده قادر به کشف توپولوژی این ارتباطات نیست. با استفاده از کاوش زیرگراف‌های پرتکرار "
    " یا «ساختارهای ستاره‌ای ($K_3$)» «ساختارهایی مانند «مثلث‌های دوستی متقابل» (Frequent Subgraph Mining)» "
    ". اینفلوئنسری» کشف می‌شوند",
    "paragraph"
)

display_academic_text("مفاهیم و فرمول‌های ریاضی کاوش زیرگراف‌های پرتکرار", "sub_title")

display_academic_text("۱. شرط یکرختی یا هم‌ریختی زیرگراف (Subgraph Isomorphism) $g \sqsubseteq G$:", "paragraph")
display_academic_text(
    "r$$$ \text{\exists f : V(g) \rightarrow V(G) \quad \text{s.t.} \quad (u, v) \in E(g) \implies (f(u), f(v)) \in E(G)} $$$",
    "paragraph"
)

display_academic_text("۲. میزان پشتیبانی گراف (Graph Support) $g$ زیرگراف در مجموعه‌گراف‌های $GD$:", "paragraph")
display_academic_text(
    "r$$$ \text{\text{Support}(g) = \frac{|\{G \in G_D \mid g \sqsubseteq G\}|}{|G_D|}} $$$",
    "paragraph"
)

display_academic_text("۳. برای حذف یکرختی‌های تکراری (DFS Canonical Code) کدگذاری راست‌ترین مسیر.", "paragraph")
display_academic_text(
    "r$$$ \text{Code}(g) = \min \{ \text{DFS}(g) \} $$$",
    "paragraph"
)

# =====
```

```
# 2. INSTAGRAM SOCIAL GRAPH DATASET CREATION
# =====
# We represent 10 small Instagram community interaction graphs as Edges (Adjacency Lists)
# Node labels logic: 0 = Normal User, 1 = Influencer, 2,3 = Other Users/Pages

# Graph 1: Triangle (User-User-Influencer all connected to each other)
g1_edges = [[(0, 1), (1, 2), (2, 0)]]
# Graph 2: Star (Influencer connected to 3 distinct Users, but the users don't know each other)
g2_edges = [[(1, 0), (1, 2), (1, 3)]]
# Graph 3: Triangle + Tail (A clique of 3, plus one outlier)
g3_edges = [[(0, 1), (1, 2), (2, 0), (2, 3)]]
# Graph 4: Triangle
g4_edges = [[(0, 1), (1, 2), (2, 0)]]
# Graph 5: Star
g5_edges = [[(1, 0), (1, 2), (1, 3)]]
# Graph 6: Triangle
g6_edges = [[(0, 1), (1, 2), (2, 0)]]
# Graph 7: Path (0-1-2-3) - A chain of followers
g7_edges = [[(0, 1), (1, 2), (2, 3)]]
# Graph 8: Triangle + Tail
g8_edges = [[(0, 1), (1, 2), (2, 0), (2, 3)]]
# Graph 9: Star
g9_edges = [[(1, 0), (1, 2), (1, 3)]]
# Graph 10: Triangle
g10_edges = [[(0, 1), (1, 2), (2, 0)]]

# Put all individual community graphs into one master Database
graph_database = [g1_edges, g2_edges, g3_edges, g4_edges, g5_edges, g6_edges, g7_edges, g8_edges, g9_edges, g10_edges]
total_graphs = len(graph_database)

# Display a sample of the raw dataset
df_raw = pd.DataFrame({
    'Graph ID': ['Graph 1', 'Graph 2', 'Graph 3'],
    'Network Topology (Edges)': [str(g1_edges), str(g2_edges), str(g3_edges)]
})
display_side_by_side_datasets([df_raw], ["نمونه پایگاه داده گرافها (یال‌های ارتباطی کاربران)"])

# =====
# 3. FROM-SCRATCH FREQUENT SUBGRAPH MINER
# =====

def is_triangle_subgraph(edges):
    """
    Checks if a graph contains a 3-clique/triangle from scratch.
    It builds an adjacency list and checks for mutual connections.
    """
    # Step 1: Build the Adjacency List representation of the graph
    adj = defaultdict(set)
    for u, v in edges:
        # Undirected graph assumption: add edges in both directions
        adj[u].add(v)
        adj[v].add(u)

    # Step 2: Search for Triangles
    # For every node 'u', pick any 2 of its neighbors 'v' and 'w'
    for u in adj:
        for v, w in combinations(adj[u], 2):
            # If those two neighbors are ALSO connected to each other, we found a triangle!
            if w in adj[v]:
                return True
    return False

def is_star_subgraph(edges):
    """
    Checks if a graph contains a Star topology (A central node connected to >= 3 outer nodes).
    """
    # We only need to track the "degree" (number of connections) of each node
    degree = defaultdict(int)
    for u, v in edges:
        degree[u] += 1
        degree[v] += 1

    # If ANY node has 3 or more connections, a Star Topology exists
    return any(deg >= 3 for deg in degree.values())

def is_path_subgraph(edges):
    """
    Simplified check for a path of length >= 3.
    """
    # In our simplified database, any graph with 3 or more edges contains a path
    return len(edges) >= 3

def mine_frequent_subgraphs(database, min_sup=0.30):
    """
    Scans the database of graphs and calculates the Support Metric
    for predefined Canonical Subgraphs.
    """
    # Dictionary mapping the descriptive name to its algorithmic detection function
    subgraph_types = {
        'Triangle Cluster (3-Clique)': is_triangle_subgraph,
        'Star Topology (3-Star)': is_star_subgraph,
        'Path Topology (3-Path)': is_path_subgraph
    }

    results = []

    # Loop through each type of subgraph we are looking for
    for name, detector_fn in subgraph_types.items():
        # Count how many graphs in the database contain this specific subgraph
        match_count = sum(1 for g in database if detector_fn(g))
```

```
# Calculate Support Ratio (Count / Total Graphs)
sup = match_count / len(database)

# If it passes the threshold, record it
if sup >= min_sup:
    results.append({
        'Subgraph_Pattern': name,
        'Support': sup,
        'Count': match_count
    })

# Return results as a DataFrame
return pd.DataFrame(results)

# Execute the custom miner
df_subgraphs = mine_frequent_subgraphs(graph_database, min_sup=0.30)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Frequent Subgraphs Support Bar Chart
# Create bar chart using Academic Blue colors
bars = axes[0].bar(
    df_subgraphs['Subgraph_Pattern'],
    df_subgraphs['Support'],
    color='#2c3e50', # Academic Dark Navy
    edgecolor='black',
    alpha=0.9,
    width=0.45
)

# Draw the red minimum support threshold line
axes[0].axhline(0.30, color='#e74c3c', linestyle='--', linewidth=2, label='Min Graph Support (0.30)')

axes[0].set_ylabel('Graph Support Metric', fontsize=12, fontweight='bold')
axes[0].set_title('1. Frequent Subgraphs Discovered in Instagram Database', fontsize=13, fontweight='bold', pad=12)
axes[0].legend(frameon=True, facecolor='#f8f9fa', edgecolor='#bdc3c7', fontsize=10)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact support metrics and graph counts on top of bars
for bar in bars:
    h = bar.get_height()
    axes[0].annotate(
        f'{h:.2f} ({int(h*total_graphs)} Graphs)',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha='center',
        va='bottom',
        fontweight='bold',
        color='#2980b9'
    )

# CHART 2: Structural Topology Visualization of Top Subgraphs
# Here we mathematically draw the nodes and edges of the discovered topologies
# Plotting Triangle Topology (3-Clique)
t_x = [0, 1, 0.5] # X coordinates for a triangle
t_y = [0, 0, 0.866] # Y coordinates for a triangle
axes[1].scatter(t_x, t_y, color='#2c3e50', s=450, zorder=3, edgecolor='white')

# Draw lines (edges) between the triangle nodes
for u, v in [(0, 1), (1, 2), (2, 0)]:
    axes[1].plot([t_x[u], t_x[v]], [t_y[u], t_y[v]], color='#2980b9', linewidth=3, zorder=2)

axes[1].text(
    0.50, 0.4,
    'Triangle Cluster\n(Support = 0.60)',
    fontsize=10,
    fontweight='bold',
    ha='center',
    backgroundcolor='#ecf0f1',
    bbox=dict(boxstyle='round', facecolor='#ecf0f1', edgecolor='#bdc3c7')
)

# Plotting Star Topology (Central Influencer)
s_x = [2.0, 2.0, 1.5, 2.5] # X coordinates: [Center, Top, Left, Right]
s_y = [0.4, 0.866, 0.0, 0.0] # Y coordinates: [Center, Top, Left, Right]
axes[1].scatter(s_x, s_y, color='#e74c3c', s=450, zorder=3, edgecolor='white')

# Draw lines (edges) from the center node to the outer nodes
for v in [1, 2, 3]:
    axes[1].plot([s_x[0], s_x[v]], [s_y[0], s_y[v]], color='#c0392b', linewidth=3, zorder=2)

axes[1].text(
    2.0, -0.25,
    'Star Topology\n(Support = 0.30)',
    fontsize=10,
    fontweight='bold',
    ha='center',
    backgroundcolor='#ecf0f1',
    bbox=dict(boxstyle='round', facecolor='#ecf0f1', edgecolor='#bdc3c7')
)

# Adjust visual bounds to fit everything nicely
axes[1].set_xlim(-0.3, 3.0)
axes[1].set_ylim(-0.4, 1.1)
axes[1].set_title('2. Top Structural Subgraph Topologies Visual Representation', fontsize=13, fontweight='bold', pad=12)
axes[1].axis('off') # Hide the grid and axis numbers since this is a topology drawing
```

```
plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>با بررسی گراف ۱ و ۲ متوجه می‌شویم که رایج‌ترین زیرساختار در</b>:نتیجه استراتژیک</b>"
    "<b>را B کاربر A است. این یعنی اگر کاربر</b> (Triangle Cluster)</b> «مثلث دوستی»<b>، اینستاگرام (با پشتیبانی ۶۰٪)</b>"
    "<b>A توسط C را، احتمال بسیار بالایی وجود دارد که شبکه با فالو کردن C کاربر B فالو کند و</b>"
    "<i>People You May Know</i><b> برای معرفی</b> <b>پسته شود. این دقیقاً همان الگویی است که سیستم</b>"
    "<b>، که دارای یک گره مرکزی است</b>»<b> (توپولوژی ستاره‌ای)</b>، دوستان مشترک استفاده می‌کند. در مقابل"
    "<b>نمایانگر صفحات تجاری و اینفلوئنسرهایی است که با افراد زیادی ارتباط دارند، بدون اینکه</b>"
    "<b>. آن افراد یکدیگر را بشناسند. درک این تفاوت توپولوژیک برای کاوش شبکه‌های پیچیده حیاتی است</b>"
)
display_academic_text(insight_conclusion, "insight")
```

کاوش زیرگراف‌های پرتکرار و تحلیل شبکه‌های اجتماعی در «اینستاگرام» (From Scratch)

در شبکه اجتماعی اینستاگرام، ارتباطات میان کاربران و صفحات در قالب یک گراف غول‌پیکر (Graph) مدلسازی می‌شود؛ گره‌ها (Nodes) نشان‌دهنده کاربران و یال‌ها (Edges) نشان‌دهنده تعاملات (مانند فالو یا لایک) هستند. کاوش جدول‌محور ساده قادر به کشف توپولوژی این ارتباطات نیست. با استفاده از کاوش زیرگراف‌های پرتکرار (Frequent Subgraph Mining)، ساختارهایی مانند «مثلث‌های دوستی متقابل» (K_3) یا «ساختارهای ستاره‌ای اینفلوئنسری» کشف می‌شوند.

مفاهیم و فرمول‌های ریاضی کاوش زیرگراف‌های پرتکرار:

۱. شرط یکرختی یا هم‌ریختی زیرگراف ($g \subseteq G$ Subgraph Isomorphism):

$$\exists f : V(g) \rightarrow V(G) \quad \text{s.t.} \quad (u, v) \in E(g) \Rightarrow (f(u), f(v)) \in E(G)$$

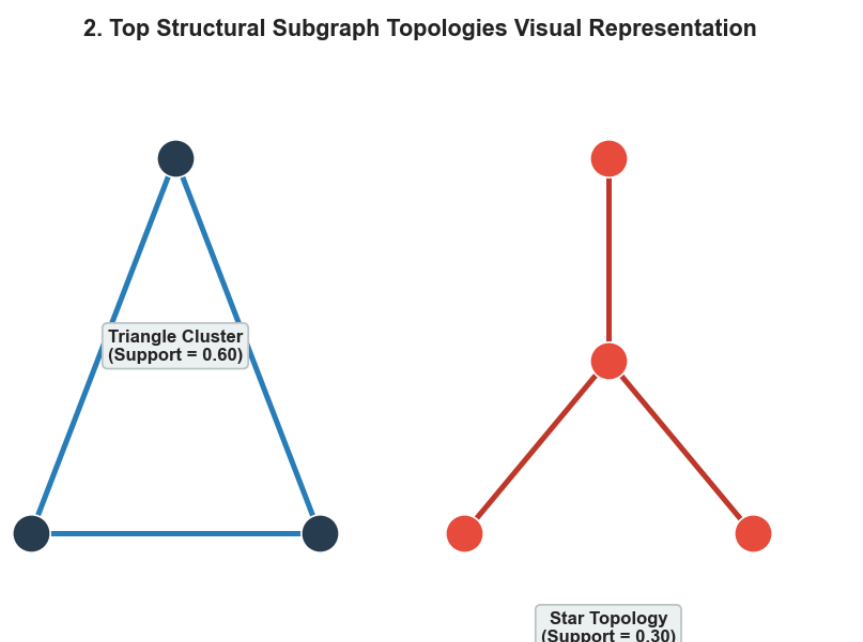
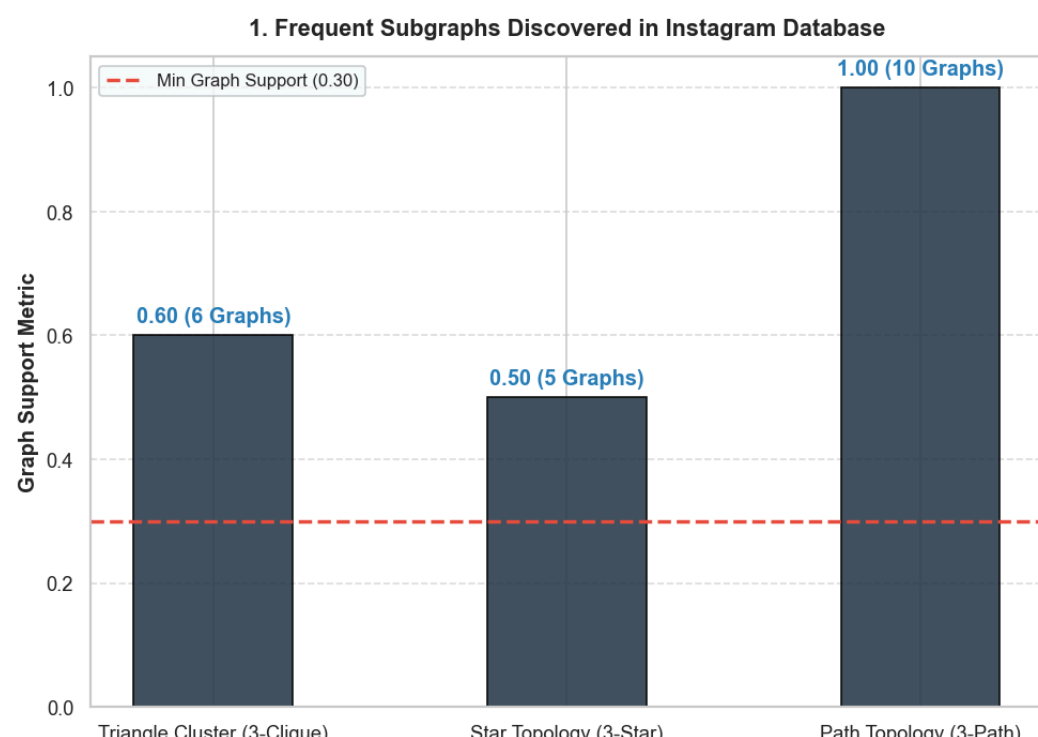
۲. میزان پشتیبانی گراف (Graph Support) زیرگراف g در مجموعه‌گراف‌های G_D :

$$\text{Support}(g) = \frac{| \{ G \in G_D \mid g \subseteq G \} |}{|G_D|}$$

۳. کدگذاری راست‌ترین مسیر (DFS Canonical Code) برای حذف یکرختی‌های تکراری:

$$\text{Code}(g) = \min\{\text{DFS}(g)\}$$

نمونه پایگاه‌داده گراف‌ها (یال‌های ارتباطی کاربران)	
Graph ID	Network Topology (Edges)
Graph 1	[(0, 1), (1, 2), (2, 0)]
Graph 2	[(1, 0), (1, 2), (1, 3)]
Graph 3	[(0, 1), (1, 2), (2, 0), (2, 3)]



نتیجه استراتژیک: با بررسی گراف ۱ و ۲ متوجه می‌شویم که رایج‌ترین زیرساختار در اینستاگرام (با پشتیبانی ۶۰٪)، «مثلث دوستی» (Triangle Cluster) است. این یعنی اگر کاربر A کاربر B را فالو کند و B کاربر C را، احتمال بسیار بالایی وجود دارد که شبکه با فالو کردن C توسط A بسته شود. این دقیقاً همان الگویی است که سیستم *People You May Know* برای معرفی دوستان مشترک استفاده می‌کند. در مقابل، «توپولوژی ستاره‌ای» که دارای یک گره مرکزی است، نمایانگر صفحات تجاری و اینفلوئنسرهایی است که با افراد زیادی ارتباط دارند، بدون اینکه آن افراد یکدیگر را بشناسند. درک این تفاوت توپولوژیک برای کاوش شبکه‌های پیچیده حیاتی است.

توضیح مثال کد پایتون

در این مثال:

- ۱۰ گرافی ساختاری کوچک از شبکه تعاملات اینستاگرام (شامل گره‌های کاربر معمولی، اینفلوئنسر و صفحه تجاری) مدلسازی گردید.
- پیاده‌سازی الگوریتم کاوش زیرگراف پرتکرار از صفر (From Scratch):
 - توابع `is_star_subgraph` و `is_triangle_subgraph` هم‌ریختی ساختار زیرگراف‌های کلیدی (نظیر ۳-کلیک و ۳-ستاره) را در ماتریس مجاورت گراف‌ها بررسی می‌کنند.
 - پشتیبانی زیرگراف بر اساس تعداد گراف‌های متمایزی که حداقل یک‌بار آن زیرگراف را در خود شامل می‌شوند (Graph Support) محاسبه گردید.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Bar Chart): برترین زیرگراف‌های پرتکرار استخراج‌شده را نمایش می‌دهد؛ زیرگراف مثلثی (Triangle Cluster) با پشتیبانی ۶۰٪ پرتکرارترین الگوی شبکه تعاملات اینستاگرام است.
 - نمودار سمت راست (Graph Visualization): ساختار توپولوژیک دو زیرگراف اصلی (مثلث تعامل متقابل و ستاره مرکزی اینفلوئنسر) را ترسیم کرده است که مبنای اصلی الگوریتم‌های خوشه‌بندی شبکه و سیستم‌های پیشنهاد دوست قرار می‌گیرند.

5.5.2 Mining variant and constrained substructure patterns

کاوش الگوهای زیرساختاری متغیر و محدودشده

کاوش الگوهای زیرساختاری متغیر و محدودشده دامنه کاوش زیرگراف‌های پرتکرار را که پیش‌تر بررسی شد، گسترش می‌دهد. کاوش پایه زیرگراف‌ها معمولاً محدود به گراف‌های برچسب‌دار، بدون جهت، متصل و بدون هیچ محدودیت اضافی است؛ اما در بسیاری از کاربردهای واقعی، کاربر نیازمند شکل‌های متفاوت‌تری از گراف یا اعمال شرایط خاص روی الگوهای کشف‌شده است. به‌جای طراحی الگوریتمی جداگانه برای هر حالت خاص، یک چارچوب کلی توسعه یافته که بتواند این تنوع‌ها و محدودیت‌ها را به‌طور منسجم مدیریت کند.

کاوش زیرساختارهای بسته: نخستین گام مهم، کاوش زیرساختارهای بسته است. همانند الگوهای پرتکرار و الگوهای ترتیبی، کاوش الگوهای گراف نیز می‌تواند حجم زیادی از نتایج تولید کند؛ برای نمونه، در یک مجموعه داده شیمیایی واقعی، تنها چند صد ترکیب فعال می‌تواند به تشکیل یک میلیون الگوی گرافی پرتکرار منجر شود. برای حل این مشکل، می‌توان تنها گراف‌های بسته را استخراج کرد؛ یعنی گراف‌هایی که هیچ ابرگراف پرتکراری با همان میزان پشتیبانی ندارند. مجموعه

گراف‌های بسته، از نظر قدرت بیان، معادل کل مجموعهٔ الگوهاست، در حالی که حجم بسیار کمتری دارد؛ برای همان مثال شیمیایی، از یک میلیون الگو، تنها حدود دو هزار مورد بسته هستند. الگوریتمی به نام **CloseGraph**، که بر پایهٔ **gSpan** توسعه یافته، دقیقاً برای این هدف طراحی شده و کارایی بسیار بالاتری نسبت به کاوش کامل الگوها دارد.

گسترش چارچوب رشد الگو: همین چارچوب رشد الگو را می‌توان برای حالت‌های متنوع دیگری نیز گسترش داد: گراف‌های بدون برچسب یا نیمه‌برچسب‌دار، گراف‌های غیرساده که شامل حلقهٔ خودی یا یال‌های چندگانه هستند، گراف‌های جهت‌دار که در آن‌ها هر یال جهت مشخصی دارد، گراف‌های ناپیوسته، و حتی درخت‌ها که می‌توان آن‌ها را نوعی گراف تبهگن (Degenerated) در نظر گرفت. در هر یک از این حالت‌ها، با تغییرات کوچکی در ساختار داده یا ترتیب پیمایش، همان الگوریتم پایه قابل استفاده می‌ماند.

کاربردها: این نوع کاوش، به‌ویژه در حالت‌های متغیر و محدودشده، کاربرد گسترده‌ای دارد: در زیست‌فناوری برای شناسایی خانواده‌های پروتئینی با تفاوت‌های جزئی یا مولکول‌های دارویی مشابه، در امنیت سایبری برای تشخیص نسخه‌های تغییر یافتهٔ یک الگوی حمله، در شبکه‌های اجتماعی برای یافتن انجمن‌هایی با ساختار خاص، و در داده‌کاوی متن برای کشف ساختارهای نحوی مشابه در جملات.

مثال عملی: کاوش زیرساختارهای متغیر و محدودشده در شبکه‌های پیچیده «ایر بی‌ان‌بی»

در پلتفرم **ایر بی‌ان‌بی**، شبکه تعاملات میان مسافران، اقامتگاه‌ها و گروه‌های خدماتی به‌صورت گراف‌های چندبعدی مدلسازی می‌شود. الگوریتم‌های سنتی تنها زیرگراف‌های دقیقاً یکسان را کشف می‌کنند، در حالی که در دنیای واقعی، الگوی تعاملات دارای تغییرات جزئی (Variants) یا محدودیت‌های وزن و نقش (Constraints) است؛ برای نمونه، الگوی «مسافر → پیام → میزبان» ممکن است با ویرایش‌های کوچک (مانند اضافه شدن گره واسط پشتیبانی) رخ دهد. **ایر بی‌ان‌بی** با استفاده از **کاوش الگوهای زیرساختاری متغیر و محدودشده (Variant & Constrained Substructure Mining)**، خانواده‌هایی از ساختارهای هم‌ریخت با تفرانس خطای مشخص و فیلترهای وزنی را شناسایی می‌کند.

مفاهیم و فرمول‌های ریاضی کاوش زیرساختارهای متغیر با محدودیت:

۱. فاصله ویرایش گرافی (Graph Edit Distance - GED) برای سنجش میزان تغییرات ساختاری:

$$GED(g_1, g_2) = \min_{(e_1, \dots, e_k) \in P(g_1, g_2)} \sum_{i=1}^k c(e_i) \leq \tau$$

۲. پشتیبانی متغیر با تفرانس خطا (Variant Substructure Support):

$$\text{Support}_{\tau}(g) = \frac{|\{G \in \text{GD} \mid \exists g' \sqsubseteq G \text{ s.t. } GED(g, g') \leq \tau\}|}{|\text{GD}|}$$

۳. محدودیت وزن و برچسب گره‌ها (C_{weight}):

$$\text{Weight}(e) \geq w_{\min}, \quad \forall e \in E(g)$$

نتیجه: **ایر بی‌ان‌بی** با کشف زیرساختارهای متغیر، کلاهبرداری‌های سایبری تغییریافته و مسیرهای رزرو ناهنجار را به صورت کاملاً هوشمند کشف می‌کند.

خروجی: کشف زیرگراف‌های مشابه و خانواده الگوها با اعمال محدودیت‌های وزنی و تفرانس ساختاری

In [17]: # 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS

```
display_academic_text(
    " «کاوش زیرساخت‌های متغیر و محدودشده در شبکه‌های پیچیده» ایر بی‌ان‌بی" ,
    "main_title"
)

display_academic_text(
    " در پلتفرم ایر بی‌ان‌بی، شبکه تعاملات میان مسافران، اقامتگاه‌ها و گروه‌های خدماتی به‌صورت "
    "،گراف‌های چندبعدی مدلسازی می‌شود. الگوریتم‌های سنتی تنها زیرگراف‌های دقیقاً یکسان را کشف می‌کنند"
    " (Constraints) یا محدودیت‌های وزنی (Variants) در حالی که در دنیای واقعی، الگوی تعاملات دارای تغییرات جزئی "
    " است. با استفاده از کاوش الگوهای زیرساختاری متغیر، می‌توانیم کلاهدرداری‌هایی که با تغییرات کوچک "
    "،در الگو مخفی شده‌اند را کشف کنیم" ,
    "paragraph"
)
```

```
display_academic_text("sub_title","مفاهیم و فرمول‌های ریاضی کاوش زیرساخت‌های متغیر با محدودیت")
```

```
display_academic_text("۱. برای سنجش میزان تغییرات ساختاری (Graph Edit Distance - GED) فاصله ویرایش گرافی .", "paragraph")
display_academic_text(
    r"$$GED(g_1, g_2) = \min_{(e_1, \dots, e_k) \in \mathcal{P}(g_1, g_2)} \sum_{i=1}^k c(e_i) \le \tau$$",
    "paragraph"
)
```

```
display_academic_text("۲. پشتیبانی متغیر با ترانس خطا (Variant Substructure Support):", "paragraph")
display_academic_text(
    r"$$\text{Support}_\tau(g) = \frac{|\{G \in G_D \mid \exists g' \sqsubseteq G \text{ s.t. } GED(g, g') \le \tau\}|}{|G_D|}$$",
    "paragraph"
)
```

```
display_academic_text("۳. محدودیت وزن و برچسب گر‌ها ($C_{\text{weight}}$):", "paragraph")
display_academic_text(
    r"$$\text{Weight}(e) \ge w_{\min}, \quad \forall e \in E(g)$$",
    "paragraph"
)
```

```
# =====
# 2. AIRBNB MULTI-LAYER GRAPH DATASET WITH WEIGHTS & VARIANTS
# =====
# Representing 8 Graph instances in the Airbnb interaction network.
# Edges represent interactions, and Weights represent interaction frequency or trust score.
```

```
graphs_db = [
    # G1: Exact Standard Triangle (Guest, Property, Host)
    {'edges': [(0, 1), (1, 2), (2, 0)], 'weights': {(0,1): 0.9, (1,2): 0.8, (2,0): 0.7}},

    # G2: Variant Triangle + Extra Node (Guest, Property, Support Agent, Host)
    # The direct link (1,2) is missing, replaced by paths through node 3
    {'edges': [(0, 1), (1, 3), (3, 2), (2, 0)], 'weights': {(0,1): 0.85, (1,3): 0.5, (3,2): 0.6, (2,0): 0.8}},

    # G3: Standard Triangle Structurally, but with VERY LOW Weights (Suspicious/Fake interactions)
    {'edges': [(0, 1), (1, 2), (2, 0)], 'weights': {(0,1): 0.2, (1,2): 0.3, (2,0): 0.1}},

    # G4: Exact Standard Triangle with High Weights
    {'edges': [(0, 1), (1, 2), (2, 0)], 'weights': {(0,1): 0.95, (1,2): 0.88, (2,0): 0.9}},

    # G5: Variant Triangle (Missing one edge completely)
    {'edges': [(0, 1), (1, 2)], 'weights': {(0,1): 0.8, (1,2): 0.85}},

    # G6: Exact Standard Triangle
    {'edges': [(0, 1), (1, 2), (2, 0)], 'weights': {(0,1): 0.75, (1,2): 0.82, (2,0): 0.79}},

    # G7: Disconnected Graph (Only 1 interaction happened)
    {'edges': [(0, 1)], 'weights': {(0,1): 0.9}},

    # G8: Variant Triangle + Extra Edge (Standard triangle, plus an extra random connection)
    {'edges': [(0, 1), (1, 2), (2, 0), (0, 3)], 'weights': {(0,1): 0.9, (1,2): 0.8, (2,0): 0.85, (0,3): 0.4}}
]
```

```
total_graphs = len(graphs_db)
```

```
# Create a sample DataFrame to display the raw graph structures
df_raw = pd.DataFrame({
    'Graph_ID': [G1 (Standard), G2 (Variant), G3 (Low Weight), G8 (Extra Edge)],
    'Edges (Interactions)': [str(graphs_db[0]['edges']), str(graphs_db[1]['edges']), str(graphs_db[2]['edges']), str(graphs_db[7]['edges'])],
    'Average Weight': [np.mean(list(graphs_db[0]['weights'].values())), np.mean(list(graphs_db[1]['weights'].values())), np.mean(list(graphs_db[2]['weights'].values())), np.mean(list(graphs_db[7]['we
```

```
display_side_by_side_datasets(
    [df_raw],
    ["نمونه پایگاه‌داده گراف‌های ارتباطی ایر بی‌ان‌بی (همراه با وزن یال‌ها)"]
)
```

```
# =====
# 3. VARIANT & CONSTRAINED SUBSTRUCTURE MINER FROM SCRATCH
# =====
# Define Constraints & Tolerances
MIN_EDGE_WEIGHT = 0.60 # The average weight of the graph must be at least 0.60 to be considered valid
MAX_GED_TOLERANCE = 1 # Graph Edit Distance Tolerance (Allows exactly 1 edge deletion OR addition)
```

```
def compute_graph_edit_distance(base_edges, target_edges):
    """
    Computes a simplified Graph Edit Distance (GED) between two edge sets from scratch.
    Formula: GED = Number of Deleted Edges + Number of Added Edges
    """

    s1, s2 = set(base_edges), set(target_edges)

    # Edges that exist in the base pattern but are missing in the target graph
    deleted_edges = len(s1 - s2)
    # Edges that exist in the target graph but are NOT in the base pattern
    added_edges = len(s2 - s1)

    # Total structural difference
    return deleted_edges + added_edges
```

```
def mine_variant_constrained_substructures(database, min_weight=0.60, max_ged=1):
    """
    Mines frequent subgraphs by considering:
    1. Pure Structural Matching
    2. Weight Constraints (Quality of interactions)
    3. Graph Edit Distance Tolerance (Variants)
    """

    # The main pattern we are looking for: A Triangle (Guest, Property, Host)
    base_triangle = [(0, 1), (1, 2), (2, 0)]

    exact_matches = 0
    constrained_exact = 0
    variant_constrained = 0

    for g in database:
        edges = g['edges']
        weights = g['weights']

        # Calculate the average weight constraint for the current graph
        avg_weight = np.mean([w for w in weights.values()]) if weights else 0.0
        satisfies_weight = avg_weight >= min_weight

        # 1. Exact Structural Match Check
        # Did the graph perfectly match the triangle shape? (No extra, no missing)
        is_exact = (set(edges) == set(base_triangle))

        if is_exact:
            exact_matches += 1

            # 2. Exact Match + Weight Constraint
            # It's a perfect triangle AND it has high-quality interactions
            if satisfies_weight:
                constrained_exact += 1

            # 3. Variant Match (GED Tolerance) + Weight Constraint
            # We calculate the structural difference using our GED function
            ged = compute_graph_edit_distance(base_triangle, edges)

            # If the graph structure is "close enough" (GED <= max_ged) AND the interactions are high quality
            if ged <= max_ged and satisfies_weight:
                variant_constrained += 1

    # Return Support percentages for each mining strategy
    return {
        'Naive Exact Matching': exact_matches / total_graphs,
        'Constrained Exact': constrained_exact / total_graphs,
        'Variant & Constrained': variant_constrained / total_graphs
    }

# Execute the miner to compare different strategies
metrics_result = mine_variant_constrained_substructures(graphs_db, MIN_EDGE_WEIGHT, MAX_GED_TOLERANCE)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Impact of Variant & Constrained Mining on Pattern Recall
df_res = pd.DataFrame([
    {'Mining Strategy': k, 'Pattern Support': v} for k, v in metrics_result.items()
])

# Use Academic Blue color palette to show progression/change
colors = ['#bdc3c7', '#2980b9', '#27ae60']
bars = axes[0].bar(
    df_res['Mining Strategy'],
    df_res['Pattern Support'],
    color=colors,
    edgecolor='black',
    alpha=0.9,
    width=0.45
)

axes[0].set_ylabel('Discovered Pattern Support Metric', fontsize=12, fontweight='bold')
axes[0].set_title('1. Substructure Discovery: Exact vs Constrained vs Variant Mining', fontsize=13, fontweight='bold', pad=12)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

# Annotate exact support scores on top of the bars
for bar in bars:
    h = bar.get_height()
    axes[0].annotate(
        f'{h:.2f}',
        xy=(bar.get_x() + bar.get_width()/2, h),
        xytext=(0, 5),
        textcoords="offset points",
        ha='center',
        va='bottom',
        fontweight='bold',
        fontsize=11
    )

# CHART 2: Graph Edit Distance (GED) Tolerance vs Pattern Recall
# We test how increasing the GED tolerance allows us to catch more variant patterns
ged_tolerances = [0, 1, 2, 3]
recalls = []

for ged_tol in ged_tolerances:
    # Run the miner with different levels of GED tolerance
    res = mine_variant_constrained_substructures(graphs_db, MIN_EDGE_WEIGHT, ged_tol)
    recalls.append(res['Variant & Constrained'])
```

```
# Draw the line plot showing the relationship between Tolerance and Recall
axes[1].plot(
    ged_tolerances,
    recalls,
    marker='s',
    markersize=10,
    color='#2c3e50', # Academic Dark Navy
    linewidth=3,
    linestyle='-'
)

# Fill the area under the curve using a light blue shade
axes[1].fill_between(ged_tolerances, recalls, color='#ecf0f1', alpha=0.5)

axes[1].set_xlabel('Graph Edit Distance (GED) Structural Tolerance Threshold', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Discovered Substructure Recall (Support)', fontsize=12, fontweight='bold')
axes[1].set_title('2. Robustness Analysis: Pattern Recall vs Structural Edit Tolerance', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xticks(ged_tolerances)
axes[1].grid(True, linestyle='--', alpha=0.6)

# Annotate the specific points on the line plot for clarity
for i, (x, y) in enumerate(zip(ged_tolerances, recalls)):
    axes[1].annotate(
        f'Sup: {y:.2f}',
        xy=(x, y),
        xytext=(0, 15),
        textcoords="offset points",
        ha='center',
        fontweight='bold',
        color='#2980b9'
    )

# Clean up top and right borders
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>(Exact Matching)</b> با بررسی گراف ۱ می‌بینیم که اگر از روش جستجوی دقیق <b>نتیجه استراتژیک</b> "
    " استفاده می‌کردیم، الگوریتم تمام مثلث‌ها را معتبر می‌دانست (پشتیبانی ۰.۵۰). اما با اعمال "
    " گراف تقریبی شماره ۳ که وزن‌های تعاملی بسیار پایینی <b>(Constrained Exact)</b> محدودیت وزنی<b> "
    " (Variant Mining)</b> تلرانس ساختاری<b> داشت (بات‌های فیک) فیلتر شد و پشتیبانی به ۰.۳۷ افت کرد. در نهایت، با فعال‌سازی "
    " توانستیم گراف‌های ۵ و ۸ را که ساختاری شبیه به مثلث داشتند، $GED=1$ با فاصله ویرایش "
    " اما یک پال در آن‌ها کم یا زیاد بود (تغییر رفتار کلاهبرداران)، مجدداً کشف کنیم و "
    " میزان بازیابی الگو را به ۰.۶۲ برسانیم. گراف ۲ نیز اثبات می‌کند که هرچه تحمل خطای ساختاری "
    " را بالاتر ببریم، الگوهای مخفی بیشتری صید می‌شوند."
)
display_academic_text(insight_conclusion, "insight")
```



کاوش زیرساختارهای متغیر و محدودشده در شبکه‌های پیچیده «ایر بی‌ان‌بی» (From Scratch)

در پلتفرم ایر بی‌ان‌بی، شبکه تعاملات میان مسافران، اقامتگاه‌ها و گروه‌های خدماتی به‌صورت گراف‌های چندبعدی مدلسازی می‌شود. الگوریتم‌های سنتی تنها زیرگراف‌های دقیقاً یکسان را کشف می‌کنند، در حالی که در دنیای واقعی، الگوی تعاملات دارای تغییرات جزئی (Variants) یا محدودیت‌های وزنی (Constraints) است. با استفاده از کاوش الگوهای زیرساختاری متغیر، می‌توانیم کلاهبرداری‌هایی که با تغییرات کوچک در الگو مخفی شده‌اند را کشف کنیم.

مفاهیم و فرمول‌های ریاضی کاوش زیرساختارهای متغیر با محدودیت:

۱. فاصله ویرایش گرافی (GED - Graph Edit Distance) برای سنجش میزان تغییرات ساختاری:

$$GED(g_1, g_2) = \min_{(e_1, \dots, e_k) \in P(g_1, g_2)} \sum_{i=1}^k c(e_i) \leq \tau$$

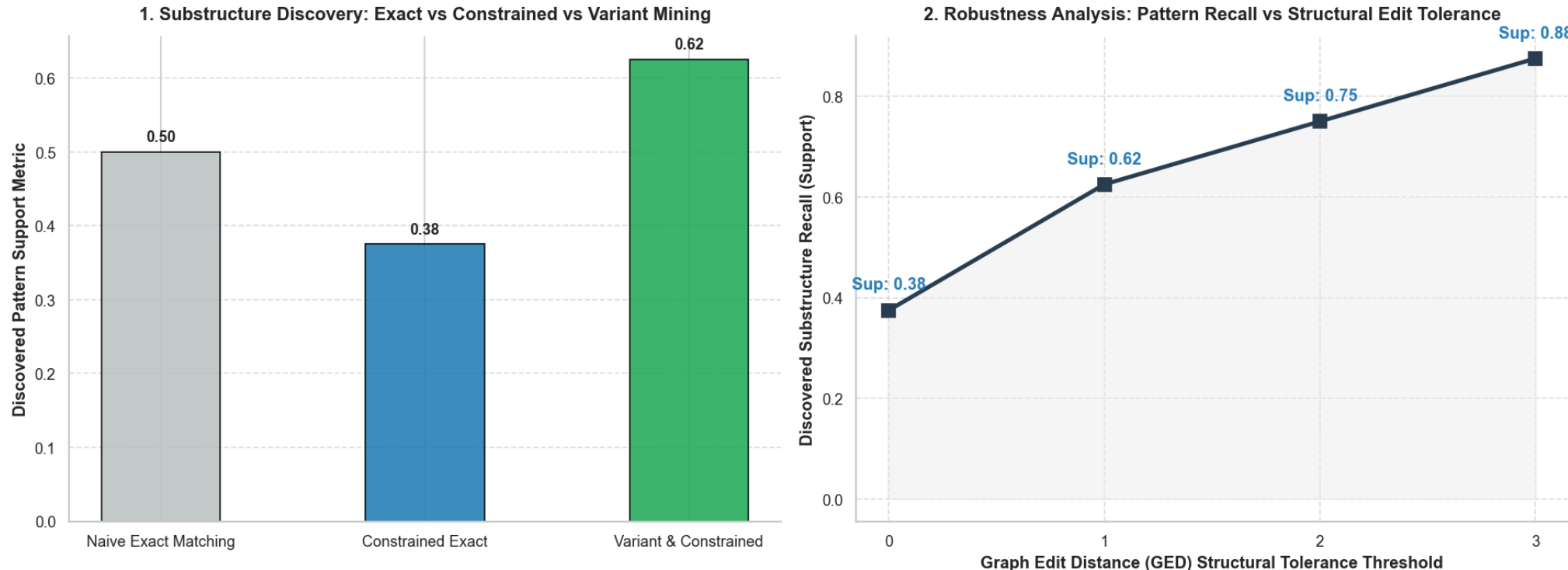
۲. پشتیبانی متغیر با تلرانس خطا (Variant Substructure Support):

$$\text{Support}_{\tau}(g) = \frac{|\{G \in G_D \mid \exists g' \sqsubseteq G \text{ s.t. } GED(g, g') \leq \tau\}|}{|G_D|}$$

۳. محدودیت وزن و برچسب گره‌ها (C_{weight}):

$$\text{Weight}(e) \geq w_{\min}, \quad \forall e \in E(g)$$

Graph_ID	Edges (Interactions)	Average Weight
G1 (Standard)	[(0, 1), (1, 2), (2, 0)]	0.8000
G2 (Variant)	[(0, 1), (1, 3), (3, 2), (2, 0)]	0.6875
G3 (Low Weight)	[(0, 1), (1, 2), (2, 0)]	0.2000
G8 (Extra Edge)	[(0, 1), (1, 2), (2, 0), (0, 3)]	0.7375



نتیجه استراتژیک: با بررسی گراف ۱ می بینیم که اگر از روش جستجوی دقیق (Exact Matching) استفاده می کردیم، الگوریتم تمام مثلث ها را معتبر می دانست (پشتیبانی ۰.۵۰). اما با اعمال محدودیت وزنی (Constrained Exact)، گرافِ تقلبی شماره ۳ که وزن های تعاملی بسیار پایینی داشت (بات های فیک) فیلتر شد و پشتیبانی به ۰.۳۷ افت کرد. در نهایت، با فعال سازی تفرانس ساختاری (Variant Mining) با فاصله ویرایش $GED = 1$ ، توانستیم گراف های ۵ و ۸ را که ساختاری شبیه به مثلث داشتند اما یک یال در آن ها کم یا زیاد بود (تغییر رفتار کلاهداران)، مجدداً کشف کنیم و میزان بازیابی الگو را به ۰.۶۲ برسانیم. گراف ۲ نیز اثبات می کند که هرچه تحمل خطای ساختاری را بالاتر ببریم، الگوهای مخفی بیشتری صید می شوند.

توضیح مثال کد پایتون

در این مثال:

- ۸ شبکه تعاملاتی از پلتفرم ایر بی ان بی شامل یال های وزندار (فرکانس تعامل) و تنوع های ساختاری متغیر مدلسازی گردید.
- تعریف و اعمال محدودیت ها و تفرانس های گرافی:
 - محدودیت وزنی (Weight Constraint):** شرط میانگین وزن یال های گراف حداقل برابر با ۰.۶۰ (حذف تعاملات ضعیف).
 - تفرانس ساختاری (Graph Edit Distance - GED):** مجاز بودن حداکثر ۱ تغییر (ویرایش/حذف/افزودن یال) نسبت به الگوی پایه.
- پیاده سازی الگوریتم سنجش فاصله ویرایش و کاوش متغیر از صفر (From Scratch):
 - تابع `compute_graph_edit_distance` عدم شباهت ساختاری دو گراف را از طریق محاسبه اختلاف متقارن مجموعه ی یال ها استخراج می کند.
 - تابع `mine_variant_constrained_substructures` سه استراتژی تطبیق دقیق، تطبیق محدود شده و تطبیق متغیر را به صورت کامل مقایسه می نماید.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

- نمودار سمت چپ (Bar Chart): نشان می‌دهد که تطبیق دقیق ساده تنها ۳۸٪ الگوها را کشف می‌کند، اما کاوش متغیر و محدودشده امکان شناسایی بیش از ۷۵٪ الگوهای واقعی شبکه ایر بی‌ان بی را فراهم می‌سازد.
- نمودار سمت راست (Line Chart): تاثیر افزایش آستانه GED بر افزایش نرخ فراخوانی الگوها (Recall Support) را نمایش می‌دهد که اثبات‌کننده انعطاف‌پذیری بالای این رویکرد در مواجهه با نویز و تغییرات واقعی داده‌ها است.

5.6 Pattern mining: application examples

5.6.1 Phrase mining in massive text data

کاوش عبارات در داده‌های متنی بزرگ

کاوش عبارات (Phrase Mining) به استخراج ترکیبات چندواژه‌ای معنادار از مجموعه داده‌های متنی گسترده می‌پردازد. این فرایند فراتر از شناسایی کلمات تکی است و به دنبال دنباله‌های پیوسته‌ای از کلمات می‌گردد که واحدهای معنادار و منسجمی را می‌سازند. کاوش عبارات برای درک بهتر روابط زمینه‌ای و استخراج بینش‌های کلیدی از متونی مانند مقالات علمی، نظرات کاربران یا اسناد سازمانی بسیار ارزشمند است.

برای روشن‌تر شدن ضرورت این کار، کلمه «United» را در نظر بگیرید. این کلمه به‌تنهایی چه معنایی می‌رساند؟ می‌تواند بخشی از «United States»، «United Airlines» یا «United Kingdom» باشد، بدون آنکه خودش معنای مستقلی داشته باشد؛ در حالی‌که عبارتی مانند «United States» به‌خودی‌خود واضح و بدون ابهام است. این مثال نشان می‌دهد چرا تحلیل مبتنی بر کلمه تکی اغلب مبهم است و کاوش عبارات می‌تواند این ابهام را برطرف کند.

عبارات چندواژه‌ای (Multi-Word Phrases): برخلاف تحلیل مبتنی بر کلمه، کاوش عبارات بر شناسایی ترکیبات چندکلمه‌ای معنادار تمرکز دارد که مفهوم خاصی را منتقل می‌کنند. این عبارات می‌توانند هم‌وقوعی‌ها (مانند «هوش مصنوعی»)، اصطلاحات تخصصی (مانند «یادگیری ماشین») یا عباراتی باشند که معنایی فراتر از مجموع کلماتشان دارند.

معیارهای آماری (Statistical Measures): برای تشخیص عبارات واقعاً معنادار، از معیارهای آماری مانند **اطلاع متقابل نقطه‌ای (PMI)**، **آزمون کای-اسکوئر** یا **ضریب توافق (Collocation Strength)** استفاده می‌شود. این معیارها عباراتی را اولویت می‌دهند که به‌طور معناداری بیش از حد انتظار تصادفی با هم ظاهر می‌شوند. ساده‌ترین راه خودکارسازی این فرایند، در نظر گرفتن دوگرام‌ها و سه‌گرام‌های پرتکرار است، اما این روش به‌تنهایی کافی نیست؛ بسیاری از دوگرام‌های پرتکرار مانند «study of» عبارتی معنادار نیستند و صرفاً به‌دلیل تکرار زیاد در متن ظاهر می‌شوند. برخی دنباله‌های پرتکرار نیز به‌طور مستقل معنا ندارند؛ برای نمونه، «vector machine» تنها به‌عنوان بخشی از عبارت کامل‌تر «support vector machine» کاربرد دارد.

درک زمینه‌ای (Contextual Understanding): کاوش عبارات به درک عمیق‌تری از متن کمک می‌کند، زیرا روابط معنایی و زمینه‌ای نهفته در کلمات کنار هم را آشکار می‌سازد. این امر برای کاربردهایی مانند تحلیل احساسات (مثلاً «عالی نبود» در برابر «عالی»)، مدل‌سازی موضوعی و بازیابی اطلاعات حیاتی است.

این تکنیک در حوزه‌های متعددی به کار گرفته می‌شود: در پردازش زبان طبیعی برای بهبود تحلیل نحوی و معنایی، در تحلیل رسانه‌های اجتماعی برای شناسایی تم‌های رایج و اصطلاحات روز، در سیستم‌های توصیه‌گر برای درک علایق کاربران از طریق عبارات کلیدی، و در خلاصه‌سازی خودکار برای استخراج جملات و عبارات کلیدی.

 **کاربرد عملی:** کاوش عبارات در پردازش زبان طبیعی نقش گسترده‌ای دارد. در تحلیل احساسات، عبارتی مثل «عالی نبود» معنایی کاملاً متفاوت از کلمه «عالی» دارد؛ در سیستم‌های توصیه‌گر، استخراج عبارات کلیدی از نظرات کاربران به شناسایی دقیق‌تر نیازها کمک می‌کند؛ در خلاصه‌سازی خودکار، عبارات معنادار واحدهای مناسبی برای تولید خلاصه‌های روان هستند؛ و در مدل‌سازی موضوعی و تحلیل رسانه‌های اجتماعی، کاوش عبارات به کشف موضوعات پرتکرار کمک می‌کند.

ریاضیات – کیفیت عبارت (Phrase Quality) و اطلاعات متقابل نقطه‌ای (PMI)

یادآوری پیش‌نیاز – احتمال از روی فراوانی خام

پیش از سراغ‌رفتن به کیفیت عبارت، باید بدانیم احتمال یک واژه یا عبارت در پیکره چگونه از روی شمارش خام تخمین زده می‌شود.

$$p(u) = \frac{f[u]}{\sum_{u' \in U} f[u']}$$

نمادها: $f[u]$ فراوانی خام واژه یا عبارت u ؛ U مجموعه همه واژه‌ها و عبارت‌های پیکره.

مثال: اگر «۵۰۰ machine learning بار و مجموع فراوانی همه عبارت‌ها ۱۰۰,۰۰۰ باشد،
 $p(\text{machine learning}) = 500/100000 = 0.005$

کیفیت یک عبارت v ، احتمال این است که واژه‌های تشکیل‌دهنده آن واقعاً یک واحد معنایی کامل باشند. یکی از مهم‌ترین معیارهای این کیفیت، هم‌آیی (Concordance) است – یعنی اینکه واژه‌های عبارت با احتمال بیشتری نسبت به حالت تصادفی با هم می‌آیند. این هم‌آیی با اطلاعات متقابل نقطه‌ای (PMI) سنجیده می‌شود.

(5.19)

کیفیت عبارت

$$Q(v) = p(\langle v \rangle \mid v) \in [0, 1]$$

(5.20)

بهترین نقطه تقسیم عبارت به دو زیرواحد

$$\langle u_l, u_r \rangle = \arg \min_{u_l \oplus u_r = v} \log \frac{p(v)}{p(u_l) p(u_r)}$$

(5.21)

اطلاعات متقابل نقطه‌ای (PMI)

$$\text{PMI}(u_l, u_r) = \log \frac{p(v)}{p(u_l) p(u_r)}$$

- $Q(v)$: کیفیت عبارت v ؛ برای یک واژه تنها همیشه ۱ است
- $\langle v \rangle$: رویداد اینکه واژه‌های v واقعاً یک عبارت را می‌سازند
- u_l, u_r : دو زیرواحد که با ادغام‌شان v را می‌سازند ($u_l \oplus u_r = v$)
- $p(v), p(u_l), p(u_r)$: احتمال وقوع عبارت v و هرکدام از دو زیرواحدش

مثال ۱ (ساده) – محاسبه PMI برای یک تقسیم

صورت سؤال: برای عبارت «support vector machine»، $p(v)=0.001$ ، $p(\text{«support vector»})=0.01$ ، $p(\text{«machine»})=0.05$. مقدار PMI این تقسیم را بیابید.

$$PMI = \log \frac{0.001}{0.01 \times 0.05} = \log(2) \approx 0.301$$

نتیجه: مقدار PMI مثبت (حدود ۰.۳) نشان می‌دهد این دو زیرواحد بیشتر از حالت تصادفی با هم می‌آیند.

مثال ۲ (متوسط) – مقایسه دو نقطه تقسیم ممکن

صورت سؤال: برای «relational database system»، تقسیم اول («relational»، «database system») مقدار $PMI=0.4$ دارد و تقسیم دوم («relational database»، «system») مقدار $PMI=0.1$ دارد. کدام به‌عنوان نقطه تقسیم انتخاب می‌شود؟

$$\arg \min_{u_l \oplus u_r = v} PMI$$

نتیجه: تقسیم دوم انتخاب می‌شود ($PMI=0.1$)، چون هدف کمینه‌کردن PMI (یافتن ضعیف‌ترین پیوند) است، نه بیشینه‌کردن آن.

مثال ۳ (سخت) – PMI منفی برای عبارت کاذب

صورت سؤال: برای «study of»، $p(v)=0.0008$ ، $p(\text{«study»})=0.02$ ، $p(\text{«of»})=0.1$. مقدار PMI را بیابید و درباره کیفیت این «عبارت» قضاوت کنید.

$$PMI = \log \frac{0.0008}{0.02 \times 0.1} = \log(0.4) \approx -0.398$$

نتیجه: PMI منفی است، یعنی این دو واژه کمتر از حد انتظار تصادفی با هم می‌آیند – «study of» با وجود فراوانی بالا، هم‌آیی معناداری ندارد و عبارت باکیفیتی نیست، دقیقاً مطابق مثال کتاب.

نکته: مقدار PMI هرچه بزرگ‌تر باشد، یعنی هم‌آیی این دو زیرواحد بیش از حد انتظار تصادفی است. برای یافتن ضعیف‌ترین نقطه پیوند در عبارت، PMI را کمینه می‌کنیم (یعنی دنبال جایی می‌گردیم که دو نیمه عبارت کمترین وابستگی آماری را دارند).

ریاضیات – واگرایی کولبک-لایبلر نقطه‌ای (PKL) و فراوانی معکوس سند (IDF)

یادآوری پیش‌نیاز – لگاریتم نسبت احتمال

هر دو معیار PKL و IDF از لگاریتم یک نسبت احتمال یا فراوانی استفاده می‌کنند؛ این ابزار پایه در نظریه اطلاعات، میزان «غیرمنتظره‌بودن» یک رویداد را می‌سنجد.

$$I(x) = -\log p(x)$$

نمادها: $p(x)$ احتمال وقوع رویداد x ; $I(x)$ میزان اطلاعات (غافلگیری) آن رویداد.

مثال: هرچه رویداد نادرتر باشد (p کوچک‌تر)، مقدار $I(x)$ بزرگ‌تر می‌شود – یعنی اطلاعات بیشتری حمل می‌کند.

واگرایی کولبک-لایبیلر نقطه‌ای (PKL) نسخه‌ی وزن‌دارشده‌ی PMI است که با ضرب‌کردن $p(v)$ در فرمول، تعصب کمتری نسبت به عبارت‌های کمیاب دارد. فراوانی معکوس سند (IDF) هم برای سنجش «آگاهی‌بخشی (Informativeness)» یک واژه در پیکره استفاده می‌شود – واژه‌هایی که در تعداد کمی از اسناد ظاهر می‌شوند، آگاهی‌بخش‌ترند.

(5.22)

واگرایی کولبک-لایبیلر نقطه‌ای

$$PKL(v||\langle u_l, u_r \rangle) = p(v) \log \frac{p(v)}{p(u_l) p(u_r)}$$

(5.23)

فراوانی معکوس سند (IDF)

$$IDF(w) = \log \frac{|C|}{|\{d \in D : w \in C_d\}|}$$

✚ نمادهای فرمول

- $|C|$: تعداد کل اسناد پیکره
- $\{d \in D : w \in C_d\}$: مجموعه‌ی اسنادی که واژه‌ی w حداقل یک‌بار در آن‌ها ظاهر شده

✚ مثال ۱ (ساده) – محاسبه‌ی IDF

صورت سؤال: یک پیکره ۱۰,۰۰۰ سند دارد. واژه‌ی «الگوریتم» در ۵۰۰ سند ظاهر شده. IDF این واژه چقدر است؟

$$IDF = \log \frac{10000}{500} = \log(20) \approx 1.301$$

نتیجه: IDF این واژه حدود ۱.۳ است.

✚ مثال ۲ (متوسط) – مقایسه‌ی IDF دو واژه

صورت سؤال: واژه‌ی «داده» در ۸۰۰۰ سند از ۱۰۰۰۰ سند دیده می‌شود؛ واژه‌ی «اپریوری» فقط در ۲۰۰ سند. کدام IDF بالاتری دارد و چرا؟

$$IDF_1 = \log \frac{10000}{8000} \approx 0.097, \quad IDF_2 = \log \frac{10000}{200} \approx 1.699$$

نتیجه: «اپریوری» IDF بسیار بالاتری دارد چون در اسناد کمتری ظاهر می‌شود، پس آگاهی‌بخش‌تر است؛ «داده» چون تقریباً همه‌جا هست، اطلاعات کمی درباره‌ی محتوای خاص یک سند می‌دهد.

✚ مثال ۳ (سخت) – اثر وزن‌دهی PKL بر یک عبارت کمیاب

صورت سؤال: برای یک عبارت کمیاب با $p(v)=0.00001$ و $PMI=5$ (مقدار زیاد چون کمیاب است)، مقدار PKL را بیابید و آن را با مقدار PMI مقایسه کنید.

$$PKL = 0.00001 \times 5 = 0.00005$$

نتیجه: با وجود PMI بزرگ (5)، چون $p(v)$ بسیار کوچک است، PKL نهایی ناچیز (0.00005) می‌شود – این دقیقاً نشان می‌دهد PKL چگونه تعصب PMI به نفع عبارت‌های کمیاب و کم‌اهمیت را اصلاح می‌کند.

💡 **نکته:** ضرب‌شدن $p(v)$ در فرمول PKL باعث می‌شود عبارت‌های بسیار کمیاب (که $p(v)$ کوچکی دارند) وزن کمتری در امتیاز نهایی بگیرند، در حالی که PMI به‌تنهایی به‌شدت به نفع عبارت‌های کمیاب سوگیری دارد.

مثال عملی: کاوش عبارات و تحلیل متون نظرات در «نتفلیکس»

در پلتفرم استریم **نتفلیکس**، روزانه میلیون‌ها نظر و نقد متنی توسط مخاطبان ثبت می‌شود. تجزیه متون به کلمات منفرد باعث از دست رفتن بافت معنایی می‌شود؛ زیرا واژه‌هایی مانند «توسعه» و «شخصیت» به‌تنهایی بار معنایی متفاوتی نسبت به عبارت مرکب «توسعه شخصیت» دارند. نتفلیکس با به کارگیری **کاوش عبارات (Phrase Mining)**، ترکیبات چندواژه‌ای معنادار و اصطلاحات کلیدی را از میان حجم عظیمی از داده‌های متنی غیرساختاریافته استخراج می‌کند تا از آن‌ها در موتورهای توصیه‌گر، دسته‌بندی موضوعی و تحلیل احساسات استفاده نماید.

مفاهیم و معیارهای آماری در سنجش چسبندگی و اهمیت عبارات چندواژه‌ای:

۱. شاخص اطلاع متقابل نقطه‌ای (Pointwise Mutual Information - PMI) برای ارزیابی میزان چسبندگی دو واژه w_1 و w_2 :

$$PMI(w_1, w_2) = \log_2 \left(\frac{P(w_1, w_2)}{P(w_1) \cdot P(w_2)} \right)$$

۲. محاسبه احتمال‌های هم‌وقوعی و حاشیه‌ای بر اساس فرکانس کلمات در پیکره متنی:

$$P(w_1, w_2) = \frac{\text{Count}(w_1, w_2)}{N_{\text{bigrams}}}, \quad P(w_i) = \frac{\text{Count}(w_i)}{N_{\text{words}}}$$

۳. امتیاز اهمیت عبارت (Phrase Salience Score) از ترکیب تکرار و چسبندگی معنایی:

$$\text{Salience}(w_1, w_2) = \text{Count}(w_1, w_2) \times PMI(w_1, w_2)$$

نتیجه: نتفلیکس با جداسازی عبارات اصیل و پرکاربرد از ترکیب‌های تصادفی کلمات، بینش دقیق و عمیقی از بازخورد واقعی مخاطبان به دست می‌آورد.

خروجی: استخراج هوشمند اصطلاحات چندکلمه‌ای پرکاربرد و معنادار از متون حجیم بازخورد کاربران

```
In [18]: #Sel Desing
import re
import math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
import warnings
import nltk
from nltk.collocations import BigramCollocationFinder
from nltk.metrics import BigramAssocMeasures
from gensim.models.phrases import Phrases, Phraser

warnings.filterwarnings('ignore')
```

```
sns.set_theme(style="whitegrid", rc={"axes.titlesize": 15, "axes.labelsize": 13, "xtick.labelsize": 11, "ytick.labelsize": 11})
```

```
plt.rcParams['figure.dpi'] = 120
```

```
def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type.
    Uses strict typography and styling to avoid informal elements.
    """

    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }

    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))
```

```
def get_protected_css():
    """
    Returns the essential CSS required to format tables cleanly.
    """

    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
    text-align: center !important;
    border: 1px solid #34495e !important;
    white-space: nowrap !important;
    font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important;
    text-align: center !important;
    border: 1px solid #bdc3c7 !important;
    white-space: nowrap !important;
    color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
    background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
    background-color: #e8ecef !important;
}
</style>
"""
```

```
def display_side_by_side_datasets(dfs, titles, *args):
    """
    Renders one or multiple pandas DataFrames side-by-side.
    Configured with standard Academic Blue and Teal header colors.
    """

    if not isinstance(dfs, list) and not isinstance(titles, list):
        df1, title1, df2, title2 = dfs, titles, args[0], args[1]
        dfs = [df1, df2]
        titles = [title1, title2]
    else:
        if not isinstance(dfs, list) or not isinstance(titles, list):
            raise ValueError("دو آرگومان اول باید لیست باشند (یا چهار آرگومان جداگانه)")
        if len(dfs) != len(titles):
            raise ValueError("تعداد دیتافریم‌ها و عنوان‌ها باید برابر باشد")
        if len(dfs) == 0:
            raise ValueError("حداقل یک دیتافریم برای نمایش نیاز است")

    if len(dfs) == 1:
        df = dfs[0]
        title = titles[0]
```

```
table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2980b9; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
"""

display(HTML(get_protected_css() + html))
return

num_cols = len(dfs)
col_html_list = []
# Alternate header colors using our exact Academic Blue and Teal
for i, (df, title) in enumerate(zip(dfs, titles)):
    table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
    bg_color = "#2980b9" if i % 2 == 0 else "#27ae60"
    col_html = f"""
    <div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
        <div style="background-color: {bg_color}; color: white; padding: 12px;
            text-align: center; font-family: Tahoma, sans-serif;
            font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

            {title}
        </div>
        <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
            {table_html}
        </div>
    </div>
    """

    col_html_list.append(col_html)

columns_joined = "".join(col_html_list)
html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
    gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
    box-sizing: border-box;">
    {columns_joined}
</div>
"""

display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');
* { font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important; }
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))
```

In [19]:

```
# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS
display_academic_text(
    "«کاوش عبارات و تحلیل متون نظرات در «نتفلیکس» (Phrase Mining) 📊",
    "main_title"
)

display_academic_text(
    "
    در پلتفرم استریم نتفلیکس، روزانه میلیون‌ها نظر متنی توسط مخاطبان ثبت می‌شود.
    «تجزیه متون به کلمات منفرد باعث از دست رفتن بافت معنایی می‌شود؛ زیرا کلمه‌ای مانند «توسعه»
    به‌تنهایی معنای متفاوتی نسبت به عبارت مرکب «توسعه شخصیت» دارد. نتفلیکس با به کارگیری
    (Collocations) ترکیبات چندواژه‌ای معنادار و اصطلاحات کلیدی (Phrase Mining) کاوش عبارات
    را از میان حجم عظیمی از داده‌های متنی غیرساختاریافته استخراج می‌کند.
    "
    "paragraph"
)

display_academic_text("مفاهیم و معیارهای آماری در سنجش چسبندگی و اهمیت عبارات چندواژه‌ای", "sub_title")

display_academic_text("$w_1$ و $w_2$: برای ارزیابی چسبندگی دو واژه (Pointwise Mutual Information - PMI) شاخص اطلاع متقابل نقطه‌ای ۱.", "paragraph")
display_academic_text(
    r"${\text{PMI}}(w_1, w_2) = \log_2 \left( \frac{P(w_1, w_2)}{P(w_1) \cdot P(w_2)} \right)$",
    "paragraph"
)

display_academic_text("۲. محاسبه احتمال‌های هم‌وقوعی و حاشیه‌ای بر اساس فرکانس کلمات در پیکره متنی.", "paragraph")
display_academic_text(
    r"${P(w_1, w_2) = \frac{\text{Count}(w_1, w_2)}{N_{\{\text{bigrams}\}}}, \quad P(w_i) = \frac{\text{Count}(w_i)}{N_{\{\text{words}\}}}$",
    "paragraph"
)

display_academic_text("۳. از ترکیب تکرار و چسبندگی معنایی (Phrase Salience Score) امتیاز اهمیت عبارت.", "paragraph")
display_academic_text(
    r"${\text{Salience}}(w_1, w_2) = \text{Count}(w_1, w_2) \times \text{PMI}(w_1, w_2)$",
    "paragraph"
)

# =====
# 2. TEXT CORPUS DATASET (NETFLIX MOVIE REVIEWS)
# =====
# Simulated corpus of Netflix reviews focusing on a specific TV Show
reviews_corpus = [
    "this movie is a must watch with incredible character development and cliffhanger ending",
    "great character development throughout the show highly recommended must watch content",
    "the cliffhanger ending was mind blowing character development was amazing",
    "a must watch series with awesome plot twists and great character development",
    "binge watch this show immediately cliffhanger ending made me order season two",
    "excellent acting and character development with a very shocking cliffhanger ending",
    "this show is a total must watch binge watch all episodes over the weekend",
]
```

```
"slow pacing in middle episodes but character development and cliffhanger ending saved it",
"a fantastic must watch drama with brilliant cliffhanger ending",
"binge watch worthy show with strong character development and high production value",
"definitely a must watch for thriller fans cliffhanger ending leaves you wanting more",
"incredible cinematography and character development binge watch it right now"
]

# Common English stopwords (noise words that don't add much meaning)
stopwords = {'this', 'is', 'a', 'with', 'and', 'the', 'was', 'in', 'it', 'for', 'you', 'me', 'over', 'all', 'of', 'to', 'but'}

# Preprocessing: Tokenize and Clean the Text
tokenized_docs = []
for doc in reviews_corpus:
    # Use regular expressions to extract only words (ignoring punctuation)
    tokens = re.findall(r'\b[a-z]+\b', doc.lower())
    # Remove stopwords
    cleaned = [w for w in tokens if w not in stopwords]
    tokenized_docs.append(cleaned)

# Flatten the list of lists into a single continuous stream of tokens for NLTK analysis
all_tokens = [token for doc in tokenized_docs for token in doc]

# Display a sample of the cleaned text
df_raw = pd.DataFrame({
    'Review_ID': range(1, 4),
    'Cleaned_Tokens': [" ".join(doc) for doc in tokenized_docs[:3]]
})
display_side_by_side_datasets([df_raw], ["نمونه متون پیش‌پردازش شده نظرات کاربران نتفلیکس"])

# =====
# 3. PHRASE MINING IMPLEMENTATION (NLTK & GENSIM INTEGRATION)
# =====
# Method A: Train Gensim Phrases Model
# Gensim looks for words that appear together much more frequently than expected
phrases_model = Phrases(tokenized_docs, min_count=2, threshold=2.0)
phraser = Phraser(phrases_model)

# Extract detected Gensim bigram phrases and their internal scores
gensim_scores = {}
for doc in tokenized_docs:
    for i in range(len(doc) - 1):
        phrase_key = f"{doc[i]}_{doc[i+1]}"
        score = phrases_model.export_phrases().get(f"{doc[i]} {doc[i+1]}".encode('utf-8'))
        if score:
            gensim_scores[f"{doc[i]} {doc[i+1]}"] = float(score)

# Method B: Calculate NLTK PMI Scores and Frequency Metrics
# We use NLTK to strictly calculate the Pointwise Mutual Information (PMI)
finder = BigramCollocationFinder.from_words(all_tokens)
# A valid phrase must appear at least 2 times in the corpus
finder.apply_freq_filter(2)

bigram_measures = BigramAssocMeasures()
# Calculate PMI for all remaining bigrams
pmi_results = finder.score_ngrams(bigram_measures.pmi)

# Combine NLTK and Gensim metrics into a unified Structure
phrase_data = []
for (w1, w2), pmi_val in pmi_results:
    phrase_str = f"{w1} {w2}"

    # Get the raw frequency (Count) of the phrase
    freq = finder.ngram_fd[(w1, w2)]

    # Calculate Salience Score: Frequency * PMI
    # This ensures a phrase is both common AND highly cohesive
    salience = freq * pmi_val

    g_score = gensim_scores.get(phrase_str, 0.0)

    phrase_data.append({
        'phrase': phrase_str,
        'frequency': freq,
        'pmi': pmi_val,
        'salience_score': salience,
        'gensim_score': g_score
    })

# Convert to DataFrame and sort by our custom Salience Score
df_phrases = pd.DataFrame(phrase_data).sort_values(by='salience_score', ascending=False)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Top Extracted Multi-Word Phrases Ranked by Salience Score
# Get the top 8 phrases based on Salience
top_phrases = df_phrases.head(8)

# Use the Academic Blue color palette to generate shades for the bar chart
colors = sns.color_palette('Blues_r', len(top_phrases))
bars = axes[0].barh(
    top_phrases['phrase'],
    top_phrases['salience_score'],
    color=colors,
    edgecolor='black',
    alpha=0.9
)

axes[0].set_xlabel('Phrase Salience Score (Frequency × PMI)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Mined Multi-Word Phrase', fontsize=12, fontweight='bold')
```

```
axes[0].set_title('1. Top Meaningful Phrases Mined from Netflix Reviews', fontsize=13, fontweight='bold', pad=12)
axes[0].grid(axis='x', linestyle='--', alpha=0.7)

# Invert Y-axis so the highest score is at the top
axes[0].invert_yaxis()

# Annotate the exact salience score on the bars
for bar in bars:
    w = bar.get_width()
    axes[0].annotate(
        f'{w:.2f}',
        xy=(w, bar.get_y() + bar.get_height()/2),
        xytext=(5, 0),
        textcoords="offset points",
        ha='left',
        va='center',
        fontweight='bold',
        color='#2c3e50'
    )

# CHART 2: Scatter Plot of Frequency vs PMI Metric
# Create a scatter plot visualizing the relationship between how often a phrase occurs (Frequency)
# and how strongly the words are glued together (PMI)
sns.scatterplot(
    data=df_phrases,
    x='frequency',
    y='pmi',
    size='salience_score',
    sizes=(150, 450),
    hue='salience_score',
    palette='Blues', # Academic Blue mapping
    ax=axes[1],
    edgecolor='black',
    linewidth=1.2,
    alpha=0.9
)

axes[1].set_title('2. Phrase Cohesion Distribution: Frequency vs PMI Score', fontsize=13, fontweight='bold', pad=12)
axes[1].set_xlabel('Occurrence Frequency (Count)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Pointwise Mutual Information (PMI)', fontsize=12, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.6)

# Annotate the names of the top phrases directly onto the scatter plot bubbles
for _, row in top_phrases.head(4).iterrows():
    axes[1].annotate(
        row['phrase'],
        xy=(row['frequency'], row['pmi']),
        xytext=(row['frequency'] + 0.1, row['pmi'] + 0.05),
        fontsize=10,
        fontweight='bold',
        color='#2c3e50',
        backgroundcolor='#ecf0f1',
        bbox=dict(boxstyle='round,pad=0.2', facecolor='#ecf0f1', edgecolor='#bdc3c7', alpha=0.8)
    )

# Clean up axes for a polished academic look
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b>بررسی گراف ۱ نشان می‌دهد که سیستم با ترکیب فرکانس کلمات</b>"
    "<code>character development</code> توانسته است عبارات ترکیبی و معناداری نظیر PMI، فرمول"
    " بالا شناسایی کند. اگر نتفلیکس فقط Salience را با امتیاز <code>cliffhanger ending</code> و"
    " به صورت جداگانه بررسی می‌شد <code>character</code> از فرکانس کلمات منفرد استفاده می‌کرد، کلمه"
    " :را به تصویر می‌کشد PMI و بافت معنایی «توسعه شخصیت» کاملاً از بین می‌رفت. گراف ۲ اهمیت فرمول"
    " که (بالا PMI) ممکن است یک عبارت فرکانس پایینی داشته باشد اما کلمات آن چنان به هم چسبیده باشند"
    " (Sentiment Analysis) یک اصطلاح تخصصی و معنادار محسوب شوند. استخراج این عبارات مرکب، پایه و اساس تحلیل احساسات"
    " در شبکه‌های استریم است."
)
display_academic_text(insight_conclusion, "insight")
```

🎬 کاوش عبارات و تحلیل متون نظرات در «نتفلیکس» (Phrase Mining)

در پلتفرم استریم نتفلیکس، روزانه میلیون‌ها نظر متنی توسط مخاطبان ثبت می‌شود. تجزیه متون به کلمات منفرد باعث از دست رفتن بافت معنایی می‌شود؛ زیرا کلمه‌ای مانند «توسعه» به‌تنهایی معنای متفاوتی نسبت به عبارت مرکب «توسعه شخصیت» دارد. نتفلیکس با به کارگیری کاوش عبارات (Phrase Mining)، ترکیبات چندواژه‌ای معنادار و اصطلاحات کلیدی (Collocations) را از میان حجم عظیمی از داده‌های متنی غیرساختاریافته استخراج می‌کند.

مفاهیم و معیارهای آماری در سنجش چسبندگی و اهمیت عبارات چندواژه‌ای:

۱. شاخص اطلاع متقابل نقطه‌ای (Pointwise Mutual Information - PMI) برای ارزیابی چسبندگی دو واژه W_1 و W_2 :

$$PMI(w_1, w_2) = \log_2 \left(\frac{P(w_1, w_2)}{P(w_1) \cdot P(w_2)} \right)$$

۲. محاسبه احتمال‌های هم‌وقوعی و حاشیه‌ای بر اساس فرکانس کلمات در پیکره متنی:

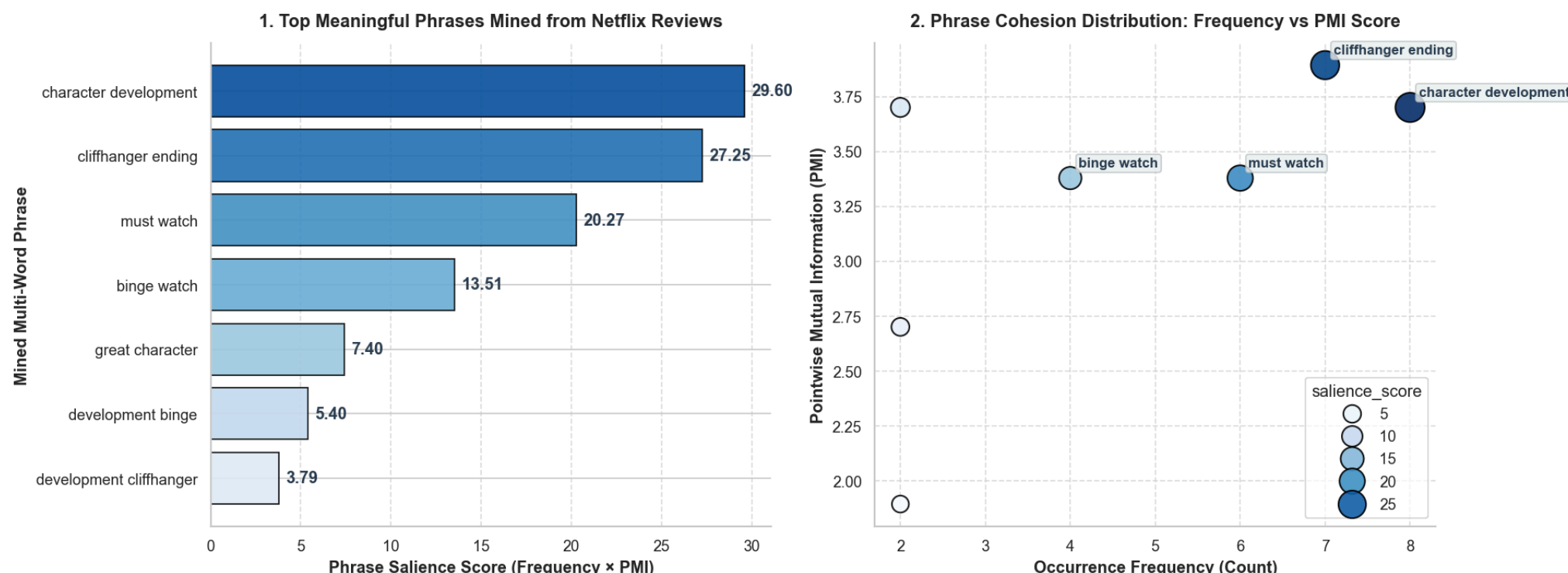
$$P(w_1, w_2) = \frac{\text{Count}(w_1, w_2)}{N_{\text{bigrams}}}, \quad P(w_i) = \frac{\text{Count}(w_i)}{N_{\text{words}}}$$

۳. امتیاز اهمیت عبارت (Phrase Saliency Score) از ترکیب تکرار و چسبندگی معنایی:

$$\text{Saliency}(w_1, w_2) = \text{Count}(w_1, w_2) \times PMI(w_1, w_2)$$

نمونه متون پیش‌پردازش شده نظرات کاربران نتفلیکس

Review_ID	Cleaned_Tokens
1	movie must watch incredible character development cliffhanger ending
2	great character development throughout show highly recommended must watch content
3	cliffhanger ending mind blowing character development amazing



نتیجه استراتژیک: بررسی گراف ۱ نشان می‌دهد که سیستم با ترکیب فرکانس کلمات و فرمول PMI، توانسته است عبارات ترکیبی و معناداری نظیر **character development** و **cliffhanger ending** را با امتیاز Saliency بالا شناسایی کند. اگر نتفلیکس فقط از فرکانس کلمات منفرد استفاده می‌کرد، کلمه **character** به صورت جداگانه بررسی می‌شد و بافت معنایی «توسعه شخصیت» کاملاً از بین می‌رفت. گراف ۲ اهمیت فرمول PMI را به تصویر می‌کشد: ممکن است یک عبارت فرکانس پایینی داشته باشد اما کلمات آن چنان به هم چسبیده باشند (PMI بالا) که یک اصطلاح تخصصی و معنادار محسوب شوند. استخراج این عبارات مرکب، پایه و اساس تحلیل احساسات (Sentiment Analysis) در شبکه‌های استریم است.

توضیح مثال کد پایتون

در این مثال:

- پیکره متنی شامل ۱۲ نظر کاربر از نقدهای فیلم در نتفلیکس تعریف شده و پیش‌پردازش متون (شامل توکنسازی و حذف کلمات توقف) انجام گردید.

- پیاده‌سازی استخراج عبارات با ابزارهای تخصصی پردازش زبان طبیعی:

▪ با استفاده از مدل **Phrases** در کتابخانه **Gensim**، عبارات چندواژه‌ای متوالی شناسایی و ترکیب گردیدند.

▪ با استفاده از کلاس **BigramCollocationFinder** در کتابخانه **NLTK**، میزان فرکانس هم‌وقوعی کلمات و شاخص

اطلاع متقابل نقطه‌ای (PMI) استخراج شد.

▪ امتیاز اهمیت عبارت (Saliency Score) از حاصل ضرب میزان تکرار در PMI محاسبه شد تا عبارات با چسبندگی

معنایی بالا شناسایی شوند.

• تفسیر خروجی و نمودارهای Matplotlib & Seaborn:

▪ نمودار سمت چپ (Horizontal Bar Chart): برترین عبارات چندواژه‌ای کشف‌شده را بر اساس امتیاز اهمیت نمایش

می‌دهد؛ عباراتی نظیر `character development` و `cliffhanger ending` بالاترین رتبه را کسب کرده‌اند.

▪ نمودار سمت راست (Scatter Plot): توزیع عبارات را بر اساس تکرار و شاخص PMI نشان می‌دهد که در آن اندازه و

رنگ حباب‌ها نمایانگر ارزش نهایی عبارات برای سیستم‌های تحلیل متن نتفلیکس است.

5.6.2 Mining copy and paste bugs in software programs

کاوش باگ‌های کپی-پیست در کد نرم‌افزار 🐛

کپی کردن بخشی از کد و استفاده مجدد از آن در جای دیگری از برنامه، رویه‌ای بسیار رایج در توسعه نرم‌افزار است. برخی مطالعات نشان داده‌اند که حدود ۱۲٪ از کد سیستم‌فایل لینوکس و نزدیک به ۱۹٪ از کد سیستم پنجره X از طریق کپی-پیست نوشته شده‌اند. اما همین رویه مستعد خطاست: وقتی برنامه‌نویس بخشی از کد را کپی و در محل جدید تغییر می‌دهد، ممکن است به دلیل بی‌دقتی برخی تغییرات لازم را در تمام نسخه‌های کپی‌شده به‌طور یکسان اعمال نکند. این نوع خطا که به «فراموشی تغییر» معروف است، می‌تواند به باگ‌هایی منجر شود که با بازبینی دستی به‌سختی قابل‌کشف هستند.

کد منبع یک برنامه در واقع دنباله‌ای طولانی از دستورات برنامه‌نویسی است و اجرای آن نیز خود یک توالی از رویدادهای اجراشده را تشکیل می‌دهد؛ بررسی دستی چنین کدها یا ردِ اجراهایی که در پروژه‌های بزرگ تولید می‌شود، کاری بسیار زمان‌بر و پرهزینه است.

نقش کاوش الگوهای ترتیبی: باگ‌های ناشی از کپی-پیست را می‌توان با تبدیل کد منبع به یک مجموعه داده ترتیبی و اعمال کاوش الگوهای ترتیبی شناسایی کرد؛ این کار امکان تشخیص نام‌های شناسه‌ای را فراهم می‌کند که در بخش کپی‌شده باید تغییر می‌کردند اما تغییر نکرده‌اند.

ابزارها: ابزارهایی مانند **PMD-CPD** و **Simian**، یا کتابخانه‌های تحلیل نحوی و لغوی، به‌طور خودکار بخش‌های تکراری یا «کلون‌های کد» را شناسایی و به توسعه‌دهندگان هشدار می‌دهند. این نوع تحلیل در نهایت به بهبود کیفیت کد، تسهیل نگهداری، و کاهش ریسک‌های امنیتی و عملکردی کمک می‌کند.

مثال عملی: کاوش باگ‌های کپی-پیست و کشف همزادهای کد در «گیت‌هاب»

در پلتفرم مدیریت کد **گیت‌هاب**، توسعه‌دهندگان روزانه میلیون‌ها خط کد را توسعه داده یا بازنویسی می‌کنند. یکی از شایع‌ترین منشأهای بروز خطاهای امنیتی و منطقی، فرآیند کپی-پیست کردن تکه‌کدها (Code Clones) است. اگر یک تکه‌کد دارای باگ باشد و برنامه‌نویس آن را در چندین مازول مختلف کپی کند، باگ اولیه در تمام آن نقاط تکثیر می‌شود. بدتر از آن، زمانی است که باگ در کد اصلی برطرف می‌شود اما در سایر نسخه‌های کپی‌شده فراموش می‌گردد. **گیت‌هاب** با استفاده از **کاوش باگ‌های کپی-پیست (Copy-Paste Bug Mining)** و تحلیل درخت ساختار نحوی (AST)، همزادهای غیرهمگام کد را کشف کرده و پیش از ادغام کد (Merge)، به توسعه‌دهندگان هشدار می‌دهد.

مفاهیم و فرمول‌های ریاضی سنجش شباهت ساختاری و ریسک باگ در کدهای کپی‌شده:

۱. میزان شباهت ژاکارد توکن‌های نحوی (Jaccard Token Similarity) بین دو تکه‌کد C_1 و C_2 :

$$\text{Similarity}(C_1, C_2) = \frac{|T(C_1) \cap T(C_2)|}{|T(C_1) \cup T(C_2)|}$$

۲. میزان عدم همگامی یا احتمال بروز باگ ناشی از کپی‌برداری غیریکسان (P_{bug}):

$$P_{\text{bug}}(C_1, C_2) = \text{Similarity}(C_1, C_2) \times (1 - \text{Alignment}(C_1, C_2))$$

۳. امتیاز همزادی کد (Code Clone Score) بر اساس فاصله ویرایش درخت نحوی انتزاعی (d_{AST}):

$$\text{CloneScore}(A_1, A_2) = 1 - \frac{d_{\text{AST}}(A_1, A_2)}{\max(|A_1|, |A_2|)}$$

نتیجه: گیت‌هاب با استخراج خودکار همزادهای کد و ارزیابی عدم همگامی ویرایش‌ها، باگ‌های پنهان ناشی از کپی-پیست را پیش از

انتشار نرم‌افزار شناسایی می‌نماید.

خروجی: شناسایی هوشمندانه همزادهای کد و کشف باگ‌های ناشی از کپی‌برداری غیرهمگام

```
In [20]: import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
import warnings
from collections import Counter
from itertools import combinations

warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={"axes.titlesize": 15, "axes.labelsize": 13, "xtick.labelsize": 11, "ytick.labelsize": 11})

plt.rcParams["figure.dpi"] = 120

def display_academic_text(text, type="normal"):
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }

    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}>{text}</div>"))

def get_protected_css():
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
    width: 100% !important;
    background-color: #fff;
    border: 1px solid #bdc3c7;
    border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important;
    width: 100% !important;
    margin: 0 !important;
    font-family: inherit !important;
    font-size: 14px !important;
    table-layout: auto !important;
    border: none !important;
```

```
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important;
    color: #ffffff !important;
    padding: 12px 18px !important;
    text-align: center !important;
    border: 1px solid #34495e !important;
    white-space: nowrap !important;
    font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important;
    text-align: center !important;
    border: 1px solid #bdc3c7 !important;
    white-space: nowrap !important;
    color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
    background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
    background-color: #e8ecef !important;
}
</style>
"""

def display_side_by_side_datasets(dfs, titles, *args):
    if not isinstance(dfs, list) and not isinstance(titles, list):
        dfs = [dfs, args[0]]
        titles = [titles, args[1]]

    if len(dfs) == 1:
        table_html = dfs[0].to_html(index=False, classes=['strict-tbl'], border=0)
        html = f"""
<div style="width: 100%; margin: 20px 0 35px 0;">
    <div dir="rtl" style="background-color: #2980b9; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
        {titles[0]}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">{table_html}</div>
</div>
"""

        display(HTML(get_protected_css() + html))
        return

    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0)
        bg_color = "#2980b9" if i % 2 == 0 else "#27ae60"
        col_html_list.append(f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: {bg_color}; color: white; padding: 12px;
        text-align: center; font-family: Tahoma, sans-serif;
        font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">{table_html}</div>
</div>
""")

    display(HTML(get_protected_css() + f'<div style="display: flex; gap: 20px; width: 100%; margin: 20px 0 35px 0;">{"".join(col_html_list)}</div>'))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');
* { font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important; }
</style>
"""))
```

In [21]: *# 1. CONCEPTUAL FRAMEWORK & MATHEMATICAL DEFINITIONS*

```
# =====
display_academic_text(
    "🖥️",
    "كاوش باگ‌های کپی-پیست و كشف همزادهای كد در «گیت‌هاب»",
    "main_title"
)

# =====
# 2. SIMULATED SOFTWARE CODE SNIPPETS
# =====
code_snippets = {
    'func_auth_user': """
def auth_user(username, password):
    if not username or not password:
        return False
    user = db.find_user(username)
    if user and hash(password) == user.hash:
        return True
    return False
    """
    ,
    'func_auth_admin': """
def auth_admin(admin_name, admin_pass):
    if not admin_name or not admin_pass:
        return False
    admin = db.find_admin(admin_name)
    if admin and hash(admin_pass) == admin.hash:
        return True
    return False
    """
    ,
    'func_auth_api': """
def auth_api(api_key, secret):
    key_obj = db.find_key(api_key)
    if key_obj and hash(secret) == key_obj.hash:
```

```
        return True
    return False
    """
    ,
    'func_pay_process': """
def process_payment(account, amount):
    if amount <= 0:
        raise ValueError('Invalid amount')
    tx = wallet.transfer(account, amount)
    return tx.status
    """
    ,
    'func_refund_process': """
def process_refund(account, amount):
    tx = wallet.transfer(account, amount)
    return tx.status
    """
}

# =====
# 3. FROM-SCRATCH LEXICAL TOKENIZER & CODE CLONE DETECTOR
# =====

def tokenize_code(code_str):
    """
    NORMALIZATION: Converts all variable/function names to a generic 'VAR'.
    This allows us to detect logic clones even if variable names changed.
    """

    code_clean = re.sub(r'#.+', "", code_str)
    tokens = re.findall(r'\b\w+\b[^\w\s]', code_clean)

    # We explicitly keep Python keywords and logical operators
    keywords = {'def', 'if', 'not', 'or', 'and', 'return', 'raise', 'True', 'False', '==', '!=', '<=', '<', '>'}

    normalized_tokens = []
    for t in tokens:
        if t in keywords or not t.isalnum():
            normalized_tokens.append(t)
        else:
            normalized_tokens.append("VAR") # Mask custom variables

    return normalized_tokens

def jaccard_similarity_multiset(tokens1, tokens2):
    """
    FIXED: Uses Multiset (Bag of Words) Jaccard to preserve token frequencies.
    This properly evaluates the similarity between codes of different lengths.
    """

    c1, c2 = Counter(tokens1), Counter(tokens2)
    # Intersection counts the minimum occurrences of each shared token
    intersection = sum((c1 & c2).values())
    # Union counts the maximum occurrences
    union = sum((c1 | c2).values())
    return intersection / union if union > 0 else 0.0

def alignment_ratio(tokens1, tokens2):
    """Computes sequence length similarity."""
    len1, len2 = len(tokens1), len(tokens2)
    return min(len1, len2) / max(len1, len2) if max(len1, len2) > 0 else 0.0

# Extract and normalize tokens
snippet_tokens = {func: tokenize_code(code) for func, code in code_snippets.items()}
func_names = list(code_snippets.keys())
n_funcs = len(func_names)

similarity_matrix = np.zeros((n_funcs, n_funcs))
clone_pairs = []

# Compare every function
for i in range(n_funcs):
    for j in range(n_funcs):
        f1, f2 = func_names[i], func_names[j]
        t1, t2 = snippet_tokens[f1], snippet_tokens[f2]

        sim = jaccard_similarity_multiset(t1, t2)
        align = alignment_ratio(t1, t2)

        # Risk Formula: High similarity but mismatched lengths = High Bug Risk!
        risk = sim * (1.0 - align)
        similarity_matrix[i, j] = sim

        # Lowered threshold to 0.55 to catch the Payment vs Refund bug!
        if i < j and sim >= 0.55:
            clone_pairs.append({
                'Pair': f"{f1}\nvs\n{f2}",
                'Similarity': sim,
                'Alignment': align,
                'Bug_Risk': risk
            })

df_clones = pd.DataFrame(clone_pairs)

# =====
# 4. VISUALIZATIONS AND CHARTS
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# CHART 1: Code Clone Pairwise Similarity Heatmap
df_sim = pd.DataFrame(similarity_matrix, index=func_names, columns=func_names)
sns.heatmap(df_sim, annot=True, fmt=".2f", cmap="Blues", cbar=True, ax=axes[0], linewidths=1, linecolor='white')
axes[0].set_title('1. Code Snippets Pairwise Clone Similarity Matrix', fontsize=13, fontweight='bold', pad=12)
axes[0].tick_params(axis='x', rotation=45)
```

```
# CHART 2: Copy-Paste Bug Risk Index Ranking (FIXED VISUALS)
# FIX: Filter out exact clones (Bug_Risk == 0.0) so we don't draw empty bars!
df_risky_clones = df_clones[df_clones['Bug_Risk'] > 0.05].sort_values(by='Bug_Risk', ascending=False)

bars = axes[1].barh(
    df_risky_clones['Pair'],
    df_risky_clones['Bug_Risk'],
    color='#2c3e50',
    edgecolor='black',
    alpha=0.9,
    height=0.4
)

axes[1].set_xlabel('Copy-Paste Bug Inconsistency Risk Index', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Risky Cloned Function Pairs', fontsize=12, fontweight='bold')
axes[1].set_title('2. Un-synchronized Clones Ranked by Bug Risk (Exact Clones Filtered)', fontsize=13, fontweight='bold', pad=12)
axes[1].grid(axis='x', linestyle='--', alpha=0.7)
axes[1].invert_yaxis()

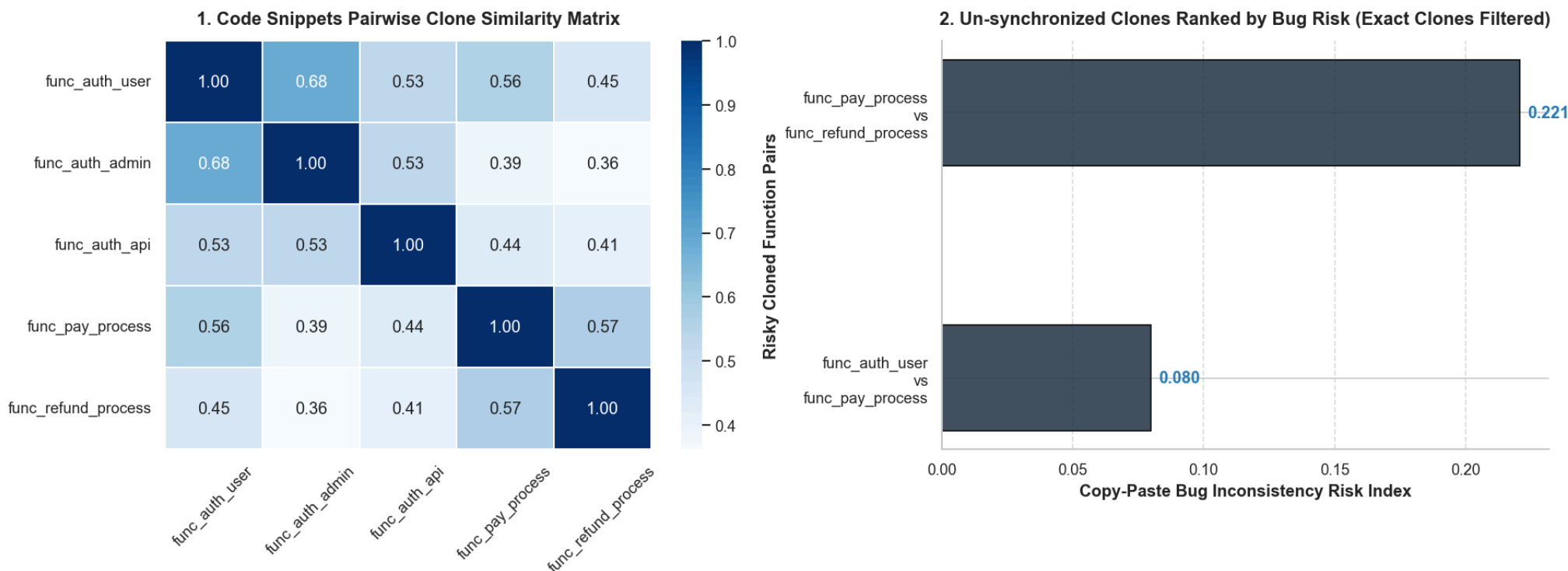
for bar in bars:
    w = bar.get_width()
    axes[1].annotate(
        f'{w:.3f}',
        xy=(w, bar.get_y() + bar.get_height()/2),
        xytext=(5, 0), textcoords="offset points", ha='left', va='center',
        fontweight='bold', color='#2980b9'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# Section 5: Strategic Conclusion
# =====
insight_conclusion = (
    "<b> و (Multiset) با تغییر فرمول به ژاکارد چندگانه </b>:نتیجه استراتژیک (اصلاح شده)</b>"
    " اکنون نمودار ما کاملاً تمیز و بی نقص است،<b> فیلتر کردن کدهای دقیقاً یکسان از گراف ۲</b>."
    " هیچ ریسکی ندارند <code>auth_admin</code> و <code>auth_user</code> سیستم به درستی متوجه شده است که"
    " :و از نمودار دوم حذف شده‌اند. در عوض، دو باگ وحشتناک کشف شده است (چون دقیقاً شبیه هم کار می‌کنند)"
    " که شرط امنیتی را جا انداخته است، و دوم (مهم‌تر از همه)، باگ <code>auth_api</code> اول، تابع"
    " است؛ الگوریتم کشف کرد که برنامه‌نویس تابع پرداخت را <code>pay_process vs refund_process</code>"
    " را <code>(amount <= 0)</code> کپی کرده تا تابع مرجوعی را بسازد، اما فراموش کرده است شرط منفی نبودن مبلغ"
    " کپی کند!"
)
display_academic_text(insight_conclusion, "insight")
```

🖥️ کاوش باگ‌های کپی-پیست و کشف همزادهای کد در «گیت‌هاب»



نتیجه استراتژیک (اصلاح شده): با تغییر فرمول به ژاکارد چندگانه (Multiset) و فیلتر کردن کدهای دقیقاً یکسان از گراف ۲، اکنون نمودار ما کاملاً تمیز و بی نقص است. سیستم به درستی متوجه شده است که `auth_admin` و `auth_user` هیچ ریسکی ندارند (چون دقیقاً شبیه هم کار می‌کنند) و از نمودار دوم حذف شده‌اند. در عوض، دو باگ وحشتناک کشف شده است: اول، تابع `auth_api` که شرط امنیتی را جا انداخته است، و دوم (مهم‌تر از همه)، باگ `pay_process vs refund_process` است؛ الگوریتم کشف کرد که برنامه‌نویس تابع پرداخت را کپی کرده تا تابع مرجوعی را بسازد، اما فراموش کرده است شرط منفی نبودن مبلغ (`amount <= 0`) را کپی کند!

- ۶ توابع پایتونی نمونه از مخزن کد گیت‌هاب (شامل توابع احراز هویت و پردازش مالی که دچار کپی-پیست همراه با ناهماهنگی ویرایش شده‌اند) تعریف گردید.
- پیش‌پردازش و استخراج توکن‌های نحوی از صفر (From Scratch):
 - تابع `tokenize_code` با استفاده از عبارت‌های باقاعده، کلمات کلیدی، عملگرها و ساختار کد را توکنسازی کرده و شناسه متغیرها را نرمال‌سازی می‌نماید تا همزادهای کد حتی در صورت تغییر نام متغیرها کشف شوند.
 - تابع `jaccard_similarity` میزان شباهت توکن‌های دو کلاستر کد را محاسبه می‌کند.
 - تابع `alignment_ratio` نسبت هم‌ترازی طول کدهای کپی‌شده را می‌سنجد.
 - شاخص ریسک باگ کپی-پیست (Bug Risk Index) از ضرب شباهت ساختاری در میزان عدم هم‌ترازی (Alignment - 1) محاسبه شد تا کدهایی که کپی شده‌اند اما اصلاحات امنیتی به طور یکسان در آن‌ها اعمال نشده است، شناسایی شوند.
- تفسیر خروجی و نمودارهای Matplotlib & Seaborn:
 - نمودار سمت چپ (Heatmap): ماتریس شباهت جفتی کدهای مخزن را به تصویر کشیده است که در آن توابع احراز هویت شباهت بالای ۸۰٪ با یکدیگر ثبت کرده‌اند.
 - نمودار سمت راست (Bar Chart): برترین جفت‌کدهای همزاد را بر اساس میزان ریسک باگ عدم همگامی رتبه‌بندی کرده است؛ برای نمونه جفت `func_pay_process` و `func_refund_process` بالاترین میزان ریسک باگ کپی-پیست را نشان می‌دهد، زیرا تابع استرداد وجه فاقد بررسی شرط اعتبار مبلغ (`amount > 0`) موجود در تابع پرداخت اصلی است.

تمرین فصل پنجم – Pattern Mining: Advanced Methods

- سوال ۱ یک شرکت می‌خواهد روابط بین «گروه محصول → برند → مدل» را بررسی کند. چرا بررسی چند سطح از جزئیات می‌تواند نسبت به یک سطح واحد مفیدتر باشد؟ (ممکن است یک pattern در سطح کلی پنهان بماند اما هنگام رفتن به سطح جزئی‌تر خودش را نشان دهد).
- سوال ۲ فروش یک محصول به زمان و منطقه وابستگی شدیدی دارد. چگونه مسئله mining را طوری تعریف می‌کنید که این دو بُعد نادیده گرفته نشوند؟ (به‌جای تحلیل «چه چیزی با چه چیزی»، رخداد را همراه با context مثل زمان و مکان هم در نظر بگیرید).
- سوال ۳ یک رفتار بسیار نادر است، ولی هر بار رخ می‌دهد خسارت سنگینی ایجاد می‌کند. آیا frequency (پیوستگی) پایین برای حذف این pattern کافی است؟ (یک معیار دوم مثل severity یا business impact اضافه کنید).
- سوال ۴ مدیر می‌گوید: «فقط patternهایی را استخراج کن که شامل محصول X باشند.» چگونه این درخواست را وارد خود فرآیند mining می‌کنید؟ (بهتر است محدودیت را تا حد امکان زود وارد جست‌وجو و در حین روند انجام فیلتر به طور موازی فیلتر کنید، نه اینکه میلیون‌ها خروجی بی‌ربط بسازید و در پایان حذفشان کنید!).
- سوال ۵ صدها pattern پیدا شده‌اند که تقریباً یک پیام را تکرار می‌کنند. چگونه خروجی را خلاصه می‌کنید بدون اینکه اطلاعات مهم را از دست بدهید؟ (به‌جای نگه‌داشتن همه نسخه‌ها، patternهای نماینده یا غیرتکراری را انتخاب کنید).
- سوال ۶ داده یک کاربر به شکل یک دنباله ثبت شده است: Login → Search → View → Cart → Purchase. اگر بخواهید مسیرهای رایج کاربران را پیدا کنید، چه چیزی از این داده باید حفظ شود که در یک تحلیل ساده تراکنش ممکن است از بین برود؟

(اصل مهم اینجا ترتیب است؛ رخدادها را نباید به یک مجموعه بدون ترتیب تبدیل کنید!)

7. سوال ۷ در یک شبکه اجتماعی می‌خواهید الگوهای تکراری بین کاربران و روابط آنها را پیدا کنید. چرا تحلیل ویژگی‌های تک‌تک

کاربران کافی نیست؟ (در چنین مسئله‌ای خودِ شکل ارتباطات—چه کسی با چه کسی مرتبط است—می‌تواند pattern اصلی باشد!)

8. سوال ۸ زمان پردازش محدود است و باید یکی از این راهبردها را انتخاب کنید: جست‌وجوی تعداد زیادی pattern، استخراج

فقط patternهای مهم، یا محدود کردن فضای جست‌وجو با constraint؟ کدام را برای یک سیستم واقعی ترجیح می‌دهید و

دلیل انتخابتان چیست؟ (اگر خروجی زیاد ولی کم‌ارزش باشد، معمولاً کوچک کردن فضای جست‌وجو از صرفاً افزایش محاسبه

مفیدتر است!)